This document is for the sole use of Kun Wang of Seas

Jasper™ Formal Fundamentals

Course Version 2403

Lecture Manual **Revision 1.0**

cadence®

This document is for the sole use of Kun Wang of Seas

© 1990-2024 Cadence Design Systems, Inc. All rights reserved.

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication subject to the following conditions:

- The publication may be used solely for personal, informational, and noncommercial purposes;
- The publication may not be modified in any way;
- Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement; and
- Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only by Cadence customers in accordance with, a written agreement between Cadence and the customer.

Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor

This document is for the sole use of Kun Wang of Seas

Table of Contents

Jasper Formal Fundamentals

Module 1 About This Course 2

Module 2 Introduction to Formal Analysis 11

Module 3 Formal Friendly SVA Coding 58

Lab 3-1 Simple Assertions to Verify FSM

Module 4 Property Development Introduction 97

Module 5 Overview of Formal Analysis Use Models 117

Module 6 Jasper Apps Usage: Formal Analysis Flow (DLFA) 142

Submodule: Jasper Apps Usage: Phases in Formal Analysis Flow 158

Lab 6-1 Benefits of Formal Analysis Flow

Module 7 Visualize 215

Module 8 Jasper Expert System 257

Module 9 Interface Property Development 267

Lab 9-1 Developing Blackbox Protocol Properties

Module 10 Basic Complexity Handling 331

Submodule: Complexity Flowchart 355

Submodule: Generic Complexity Reduction Methods 362

Submodule: Complexity Evaluation Tools 385

Lab 10-1 Basic Complexity Handling

Module 11 Complexity Reduction Methods 410

Submodule: Abstraction (Underconstraint) 416

Submodule: Overconstraint 434

Lab 11-1 Complexity Reduction Techniques

This document is for the sole use of Kun Wang of Seas

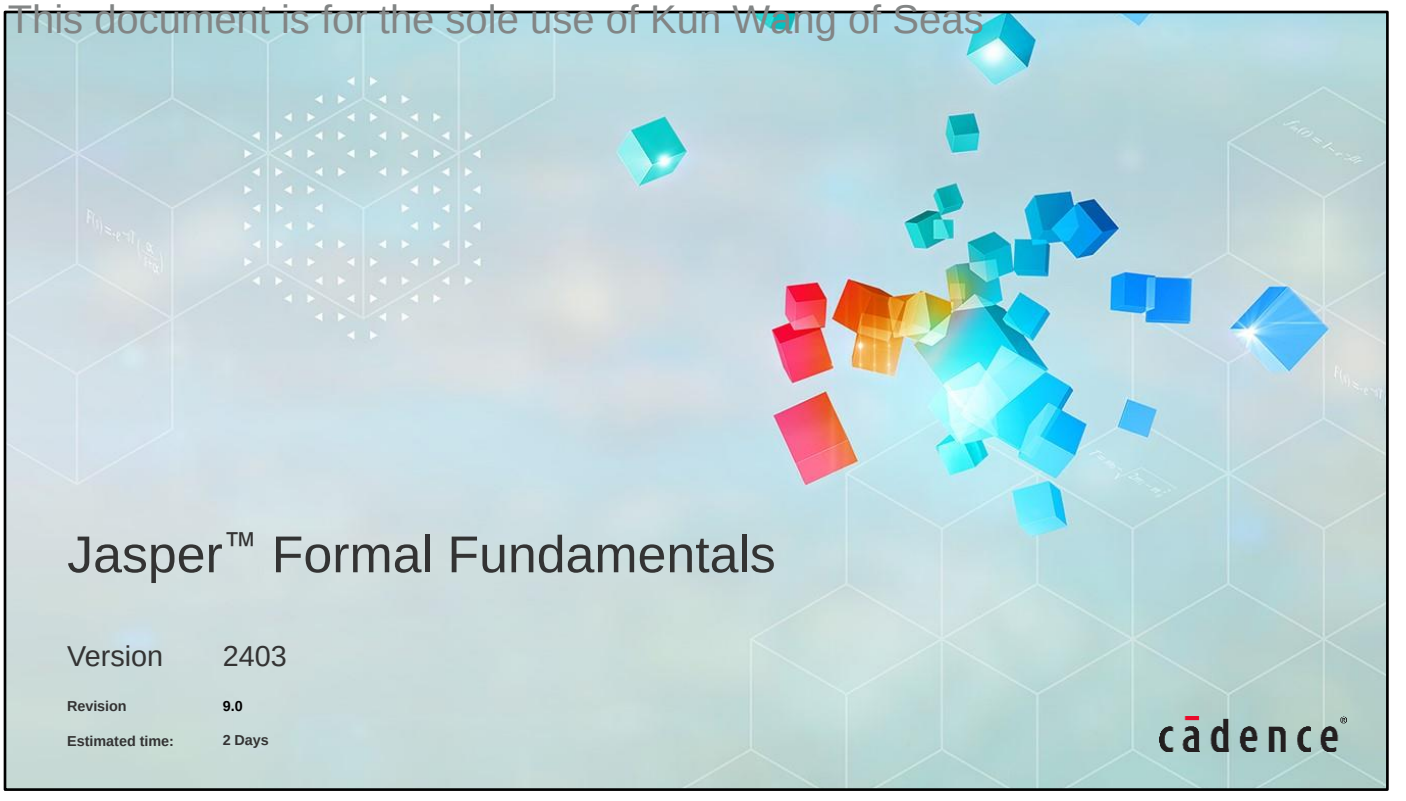
Module 12 The Verification Completeness Problem..... 445

Module 13 Coverage in Formal ……..... 466

Module 14 Course Conclusions 492

Module 15 Next Steps..... 497

This document is for the sole use of Kun Wang of Seas



Welcome to the Jasper Formal Fundamentals course. This course will take you anywhere between 16 and 20 hours of your own time. The course will be a mixture of lectures, lab exercises and tool demonstrations.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 1

About This Course

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Course Prerequisites

Before taking this course, you need to

- Have previous experience with System Verilog Assertions
- If you do not know any SVA, then it is recommended first to do the System Verilog Assertions course: https://www.cadence.com/en_US/home/training/all-courses/82165.html
- **This course does not cover basic SVA syntax and principles**

© Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Course Objectives

In this course, you will learn how to:

- Define reusable, functionally correct SVA properties that are efficient for the formal tool
  - Make use of abstract auxiliary code to simplify descriptions,
  - Make code maintenance easier,
  - Reduce debug time and reduce tool proof runtime
- Set up, run and analyze results from Formal Analysis
- Identify designs upon which formal is likely to be successful, while understanding formal complexity issues and how to identify and overcome them
- Identify the symptoms of over-constraining, debug the causes, and solve the issue
- Identify situations where Abstractions would be appropriate and how to define them
- Use a systematic process of property development to approach a completely new verification problem
- Describe different formal verification use models of Jasper™ Apps

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This course aims to learn how to define reusable, functionally correct SVA properties that are efficient for formal verification. So that's one of the first sections we talk about. Just writing the correct property is not good enough. It also needs to be efficient for the tool. Things you'll need that you probably wouldn't have used if you'd only used SVA in simulation before are things like abstractions and auxiliary code to simplify the descriptions to the formal tool, and this has the added benefit that it makes it much easier to maintain the code, reduce the amount of time it takes to debug any property that fails, and also makes the tool run quicker. You get your results faster, and your verification throughput increases. So, to many people attending this course, you would never have used Jasper before or even seen it and possibly never use formal analysis before formal verification. So we'll first guide you through all the steps you need to know to set up the tool, run it and analyze the results, and you'll be able to do this yourself on your designs. On completing this course, we also need to understand where we can apply formal and where we're likely to get good results because there are specific problems that formal could do better with lots of expertise and time. We also deal with a very common problem in formal verification: over constraining. So we need to understand what that is, how it occurs, how we know it's happened, and how to fix it.

We'll also describe all the other things you can do with formal. So many people won't be aware of all the different formal use models. So, for each different kind of use model, Jasper has something called an app, which uses a tool in a way that is easy for users to solve specific problems,

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Setting Your Expectations: What the Course Does Intend to Do

This course is intended to:

- Introduce formal to people with little prior experience and make them relatively proficient with Jasper at an intermediate user level.
  - Provides a shortcut to experience.
- Show which problems you are likely to encounter and how to solve them.
- Given only a specification, determine what properties they need to verify specified functionality.
- Understand how to interpret results.
- Understand some of the other problems that formal can help with:
  - Datapath, full SoC connectivity, etc.

Following the course...

- Recommend that you contact your local Jasper AE and organize an on-site visit to:
  - Give a more detailed overview of all the Jasper Apps.
  - Give detailed workshops on the Apps that are interesting to you.
  - Help you apply the Apps to your designs.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Setting Your Expectations: What the Course Does Not Intend to Do

This course does not:

- Try to teach everything about Jasper or Formal
  - But enough to be aware of the issues and start using Jasper effectively
- Describe all of the Jasper Apps in detail
- Take intermediate users to an “advanced” level
- Replace the help
- Remove the need for you to consult with your local Jasper AE to become an advanced user

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Course Agenda

- Introduction to Formal Analysis
- Formal Friendly SVA Coding
  - Lab: Simple Assertions to Verify FSM
- Property Development Introduction
- Overview of Formal Analysis Use Models
- Jasper Apps Usage: Formal Analysis Flow (DLFA)
  - Submodule: Jasper Apps Usage: Phases in Formal Analysis Flow
    - Lab: Benefits of Formal Analysis Flow
- Visualize
- Jasper Expert System
- Interface Property Development
  - Lab: Developing Blackbox Protocol Properties
- Basic Complexity Handling
  - Submodule: Complexity Flowchart
  - Submodule: Generic Complexity Reduction Methods
  - Submodule: Complexity Evaluation Tools
    - Lab: Basic Complexity Handling
- Complexity Reduction Methods
  - Submodule: Abstraction (Underconstraint)
  - Submodule: Overconstraint
    - Lab: Complexity Reduction Techniques
- The Verification Completeness Problem
- Coverage in Formal
- Course Conclusions

 © Cadence Design Systems, Inc. All rights reserved.

cadence

We'll first discuss the Formal introduction and then SVA coding. So the expectation is that if you're doing this course, you already know SVA; this will not teach you SVA basics. But what's essential to learn if you're using formal is how to write your code in a way that will be efficient for the tool. So, this is called formal friendly SVA.

Some of the things are unique to Jasper. So, for example, Jasper expert system and visualize are features of the tool itself. Another thing is methodology. So, things like complexity and complexity reduction apply to whatever formal tool you're using.

So it's a mixture of methods and just telling you what the tool does. These are the kinds of things the methodology is important background in understanding how the tool works and how you can verify something. This does not have to do with actually driving the tool. So these are more methodology things, which are mostly the parts with which people have the most problems.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Software and License Prerequisites

The course was developed with Jasper 2403-p001 and VIPCAT 11.30.095

It should also be compatible with later versions.

Jasper Apps		
Product ID	Product Name	License Feature Name(s)
JGFVBASE	Jasper Formal Verification Platform	jasper_papp jasper_pcov jasper_pint
VIPCAT 11.3	VIP Catalogue 11.30.095	No licenses required for APB VIP

8 © Cadence Design Systems, Inc. All rights reserved.

cadence

The licenses you need for this course are listed here. This course was developed for Jasper version 2403, but it will also work with earlier or later versions. This should be very few problems with compatibility between versions, so it's preferred but optional that you use the 2403 version. You'll also need the VIP catalog of VIPCAT 11.30.095 as well because one of the labs uses the some verification IP. From that, the VIP catalog needs to be installed, but it doesn't require a separate license so you won't need any extra license. So that concludes the introduction to this course. So, continue to the next section.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Become Cadence Certified by Earning a Digital Badge

Digital badges indicate mastery in a certain technology or skill and give managers and potential employers a way to validate your expertise.

- Cadence Training Services offers digital badges for our popular training courses.
- Your digital badge can be added to your email signature or social media platforms like LinkedIn or Facebook.

Benefits of Cadence Certified Digital Badges

- Validate expertise
  - Expand career opportunities
- Professional credibility
  - Stand apart from your peers
- For more information, go to www.cadence.com/training or email es_digitalbadge@cadence.com.

How do I register to take the exam?

- Log in to our Learning Management System, click on the course in your transcript, and go to the Content tab to locate the exam.

How long will it take to complete the exam?

- Most exams take 45 to 90 minutes to complete. You may retake the exam multiple times to pass the exam.

How do I access and use the digital badge?

- After you pass the exam, you get a digital badge and instructions on how to place it on social media sites.

How is the digital badge validated?

- Credly validates the digital badge as issued to you by Cadence and includes the details of the criteria you completed to earn the badge.

© Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Icons Used in This Class



Best Practice



Language/
Command Syntax



Concept/
Glossary



Frequently Asked
Questions/
Quiz



Error Message



Problem & Solution



Quick Reference



GUI and Command



How To



Lab List

Throughout this class, we use icons to draw your attention to certain kinds of information. Here are the icons we use, and what they mean.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 2

Introduction to Formal Analysis

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Module Objectives

In this module, you will be able to,

- Define specific terminology used in Formal Analysis
- Evaluate factors that affect the quality of results
- List out the limitations of Formal Verification tools
- Identify issues related to initialization
- Identify complexity and challenges in Formal verifications

 © Cadence Design Systems, Inc. All rights reserved.

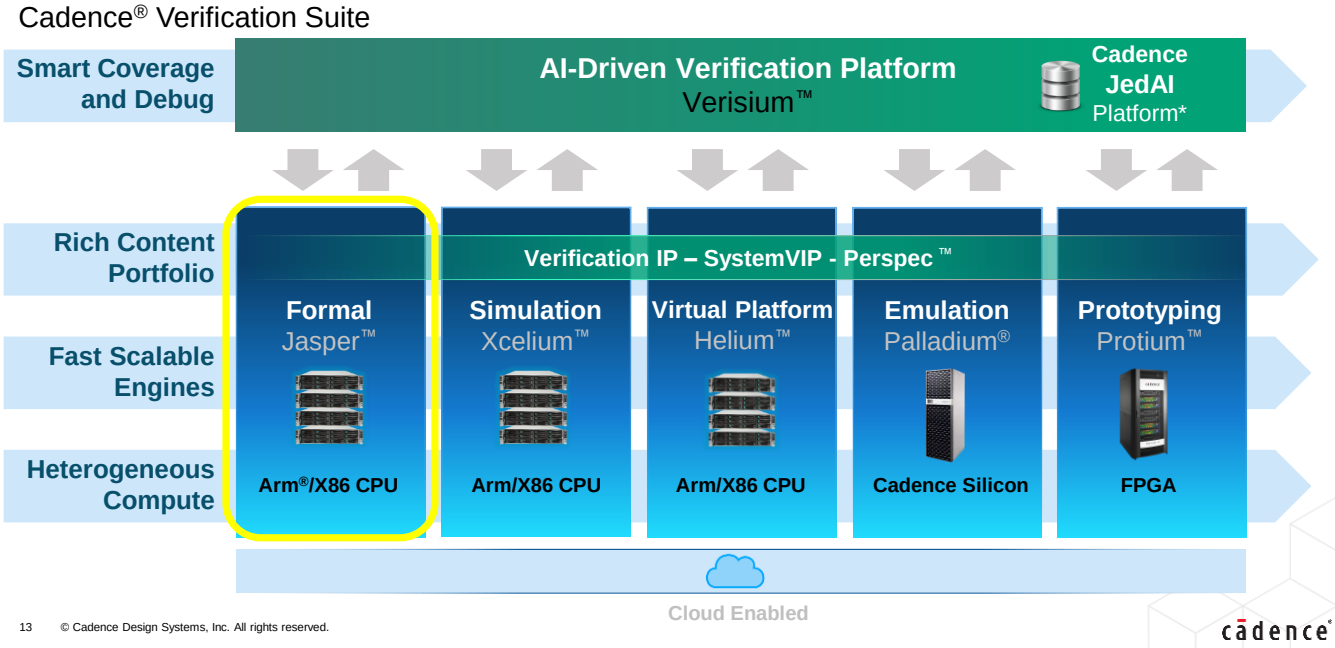
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Cadence Verification Suite for Formal: Jasper



13 © Cadence Design Systems, Inc. All rights reserved.

Purpose of this slide is to show that formal analysis is just part of the solution to a verification problem. It does not directly replace any other methodology or tool, but it can make significant savings, for example use of more can drastically reduce the amount of simulation which needs to be done.

It is another tool which the verifcation engineer has to solve todays complex verification problems.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

What Is Formal Verification?

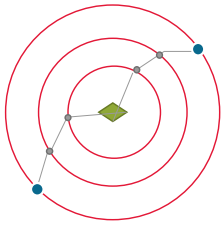
Why is it called `formal`?

- Mathematical analysis of the space of possible behaviors of a design

Reasoning based on manipulation of formulas:

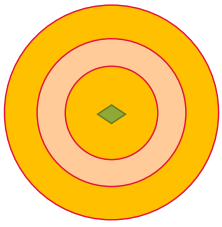
- As opposed to intuition
  - Simulation and random stimulus generation only test a tiny sampling of possible behaviors

`formal` will look at the full space of possible simulation, rather than trying specific values



Simulation





Formal Verification

 © Cadence Design Systems, Inc. All rights reserved.

Why is it called formal. So, this is derived from formal logic based upon formula manipulation. So basically, it's just algebra. And this is as opposed to simulation in which we are generating random stimulus which will test tiny sampling behaviors. So, you can't possibly test everything in simulation, because there are too many different cases to test in any reasonable kind of time frame.

formal will look at the full space of possible simulation, rather than trying specific values

As we can see in these diagrams.

Formal techniques have been around for decades.

It has lately become a mainstream verification method for these reasons:

- Industry standard languages for defining properties.
- Improvements in algorithms can yields results in practical timeframes.
- Software tools have become much more user friendly.
- CPUs become much more powerful and less expensive – and memory too.
- Widespread availability of commercial Distributed Resource Management software, e.g., LSF, Sun/Oracle Grid Engine, etc.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Verification Practical Issues

Writing formal (symbolic) specifications is difficult

How do I know whether I've specified:

- What do I want?
- Everything I want?
- The environment correctly?

Tools and recommended methodologies can help

These are general verification problems

- Not limited to Formal Analysis

© Cadence Design Systems, Inc. All rights reserved.

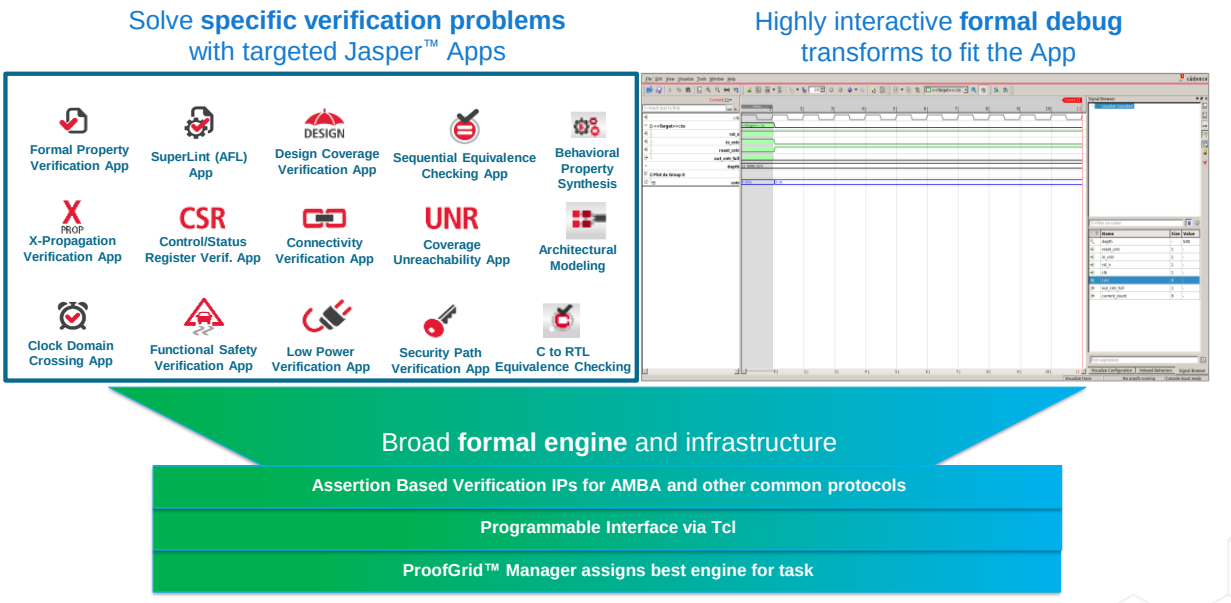
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Jasper Formal Apps



- This is the overview of Jasper Formal Apps, which are different apps for different requirements.
- Jasper is not just intended for formal experts. It's also usable by verification engineers with little experience and, designers.
- You don't need a complete design, which is one of the advantages of formal, So you can test out your ideas as you're going through, get rid of easy bugs, and also have those assertions there for later in the verification cycle, without the need for a test bench. You can do things much quicker and eliminate the bugs earlier on.
- So formal analysis verifies the correctness of RTL designs using assertions without needing a testbench.
- These different apps are intended for different functionalities
- Some of these apps will be briefly introduced in a later module.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Terminology Introduction

Boolean

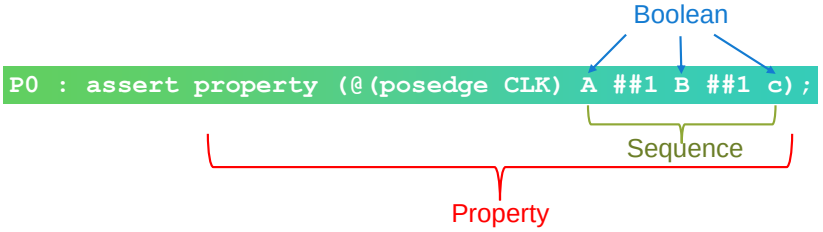
- A Boolean can only have one of two possible values: 0 or 1
- A Boolean expression is any algebraic expression involving Boolean values.
  - A, B and C are Boolean

Sequence

- A series of Boolean conditions evaluated over time.
- For example, if `A` is high in one cycle, then `B` must be high in the next cycle, and `C` must be high cycle after that.

Property

- A design behavior defined as logical and temporal relationships between Boolean and sequential expressions.



17 © Cadence Design Systems, Inc. All rights reserved.

cadence

- A Boolean condition can take any one of the values 0 or 1.
- A Sequence is a series of Boolean conditions evaluated over a time from one cycle to the next. In formal there is no concept of glitches or delays or delta cycles. It's all based upon cycles. So saying something like a is high in one cycle, B must be high in the next cycle, and then C must be high. All it is Boolean conditions separated by different cycles.
- A property is a definition of design behavior by logical and temporal relationships,
- This is demonstration of uses of Boolean, sequence and Property in an assertion and the important thing here is to understand that a property and an assertion are not the same thing.
- A property is a definition of the behavior. An assertion is an example. What's known as a verification directive. A verification directive is an instruction to the EDA tool to tell it what the tool should do with that property.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

SVA Verification Directives

Property

Expression of design behavior.

- A design behavior defined as logical and temporal relationships between Boolean and sequential expressions.

Verification Directive Keywords

What the tool should do with the expressed design behavior.

- assert – An assertion
  - The tool must check the property is always true.
- assume – An assumption/constraint
  - In formal – The tool must only allow the Design under verification (DUV) and inputs to behave in ways that never violate the property.
  - In Simulation – Has no meaning in this context, so it is treated as an assertion.
- cover – A cover expression
  - In formal – Demonstrate that it is possible for the expression to be true.
  - In Simulation – Count how many times the expression is true.

The stated design behaviors in the SVA properties do not care if we are a simulation or formal user.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

- A property is a definition of behavior.
- The verification directives we have in SVA are
- Assert, which means the tool must check that the properties are always true.

- An assumption is a constraint in formal, what that means is the tool is only allowed to drive the inputs in ways that don't violate that property.
  - In simulation it has no meaning because you know the testbench determines the stimulus.
  - So, what happens in simulators?
    In the cadence Xcelium simulator is that these will be automatically converted into assertions.

- The cover in formal is an example of something being true.
  - Normally when you write a cover, you're writing it on a sequence. So in formal when you say cover a sequence, you're saying to the tool, show me an example of that sequence occurring.
  - In simulation, it means counting how many times the expression is true.
    So if the cover was on a sequence, you would count how many times you observed that sequence during the simulation you did.

- One thing to notice is that none of these verification directives are specific for formal or simulation. What happens depends upon the tool you're using. Still, there is no definition in the language reference manual of whether you're doing a simulation or using formal.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Property Checking

Properties are created from the design specification and implementation decisions:

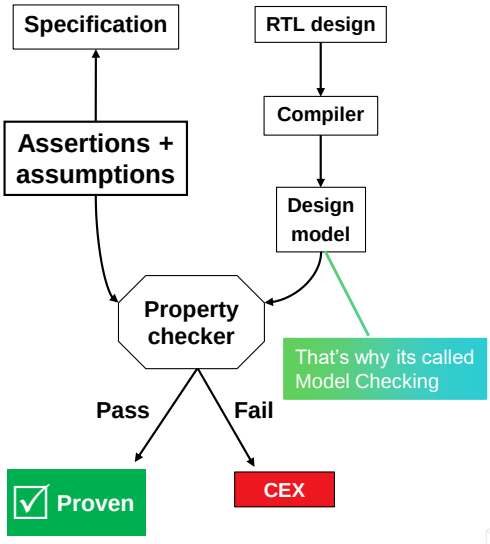
- *Assertions* are properties that the tool must prove.
- *Assumptions (a.k.a Constraints)* are properties that inform the tool how the design inputs can behave.

Formal engines verify whether the assertions hold under all input conditions allowed by constraints.

- There is no testbench driving input stimulus.
- On a failure, a formal engine should provide vectors to demonstrate the failure.
  - Known as a Counter Example (CEX)

Properties can be defined with SVA, PSL or OVL.

- Although the vast majority (98%+) of people use SVA.



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

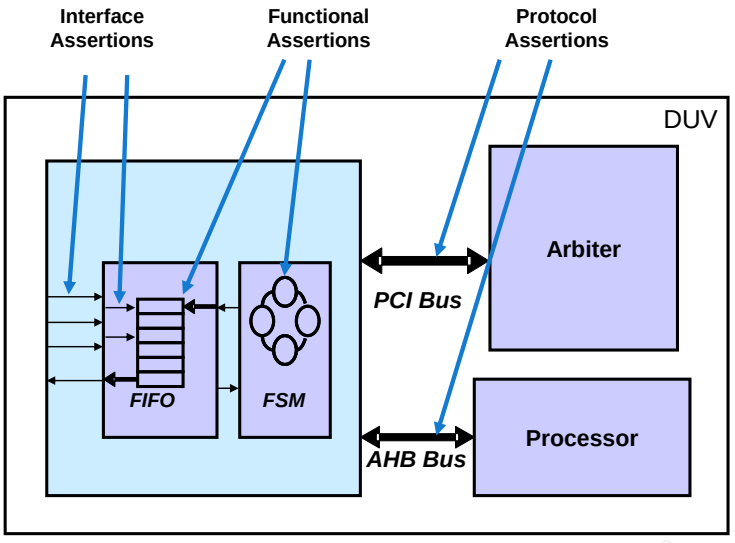
This document is for the sole use of Kun Wang of Seas

Dynamic Verification (Simulation)

Assertions monitor and report

- Expected behavior
- Forbidden behavior
- Signal protocols

Assertions are dependent on the effectiveness of simulation vectors.



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

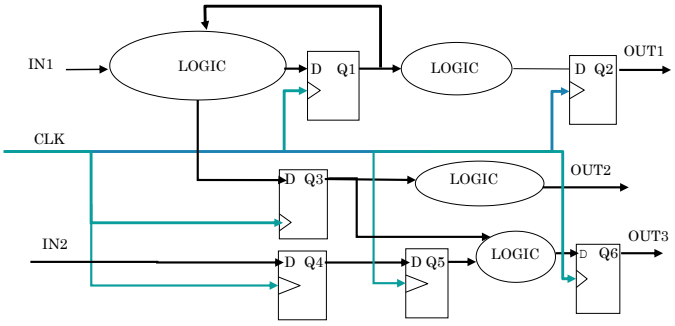
This document is for the sole use of Kun Wang of Seas

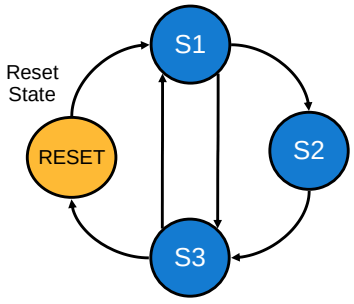
What Is Your Design?

- One big state machine
- Output of memory elements determines the state of the design

State on the next clock depends *only* on:

- Inputs
- Current state
- The design





State	Q1	Q2	Q3	Q4	Q5	Q6
Reset	1	0	0	1	1	1
S1	0	1	0	1	0	0
S2	1	0	1	0	0	1
S3	1	0	0	1	1	0

21 © Cadence Design Systems, Inc. All rights reserved.

cadence

- You can view your entire design as being one large state machine. Each memory element, either a latch or a flip-flop, remembers a value.
- If you take an aggregate of all of the memory elements in a the design, then this represents the state that the design is in.
- In a design with only flip-flops, the state of the entire design can change on every active clock edge.
- The state that the design goes into next is determined by only three things:
  - The state that you are currently in,
  - The values of the inputs,
  - The design itself.
- The combinational logic is always going to produce the same output, given the same inputs. Outputs change in response to inputs immediately as far as we are concern for functional verification. Remember, Formal Analysis is about functionality not timing.

© Cadence Design Systems, Inc. All rights reserved.

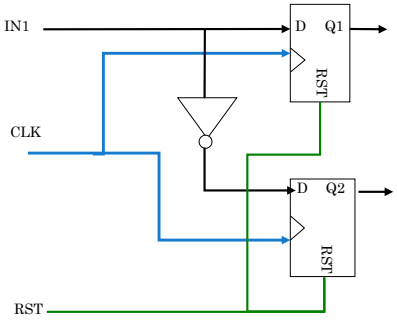
This document is for the sole use of Kun Wang of Seas

How Many States?

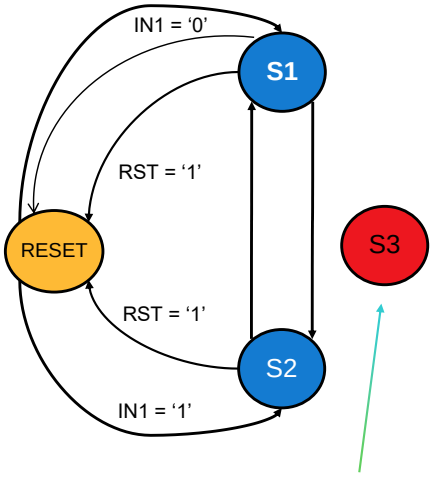
Most probably << $2^{(\text{Number of FFs})}$

Depends on the design

Some states may never be reached



State	Q1	Q2
RESET	0	0
S1	0	1
S2	1	0
S3	1	1



S3 is always unreachable
If you start from S1, S2 or RESET

22 © Cadence Design Systems, Inc. All rights reserved.

cadence

- It is highly unlikely, though possible, that the number of states that your design can be in, is two raised to the power of the number of FFs in your design.
- Your design itself determines how many states that it can be in.
- In the example, Q1 can never equal Q2 after the first active clock edge has passed, unless RST is active.
- The state where Q1 = '1' and Q2 = '1' can only occur if you initialize the FFs to those values. In verification, that can be by way of initial signal assignments. In hardware, this can be if your specific technology has FFs that can power up with the Q output set to '1'.
- After an active RST or clock edge occurs, that state is never be reached, again.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Unreachable States

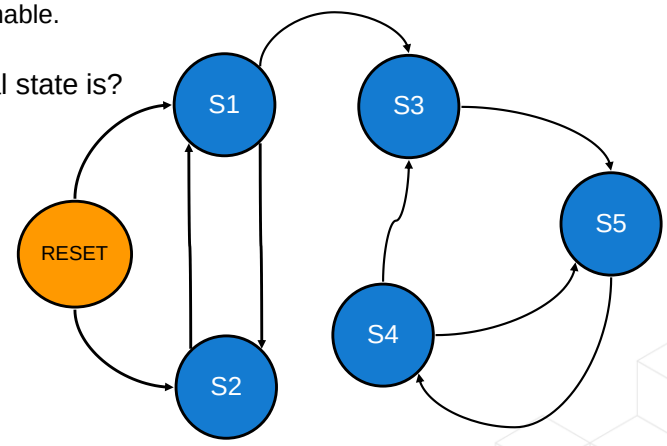
Which states are unreachable may depend on the initial state

Which state do you decide to start from?

- If initial state is S3, S4, S5 then S1, S2, RESET is unreachable.
- If the initial state is RESET, then all states are reachable.

How does the verification tool know what the initial state is?

We need to inform Jasper what the initial state is?



23 © Cadence Design Systems, Inc. All rights reserved.

- It is not necessarily true that a state is always unreachable. It can depend upon where you start from.
- In the example, if the design ever enters state S3, then it never goes to state RESET, S1 and S2, again.
- If you initial state for verification is S3, then this means that RESET, S1 and S2 are never reached during that verification run.
- However, if you start from state RESET, then all states are reachable during that verification run.
- The verification engineer has to inform the verification tool of the initial state of the design for a particular verification run.
- *<<states, reachable>> <<states, unreachable>>*

© Cadence Design Systems, Inc. All rights reserved.

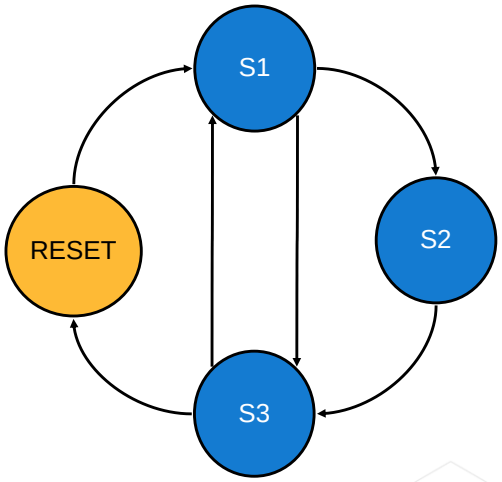
This document is for the sole use of Kun Wang of Seas

Changes of State

For every state, a finite number of states can be reached on the next cycle

In this example:

- All states are reachable
- From state S2, only S3 can be reached on the next cycle



24 © Cadence Design Systems, Inc. All rights reserved.

cadence

- In most cases only a finite number of states can be reached on the next clock edge.
- The example shows all states are reachable.
- This does not necessarily have to be true. There can be unreachable states.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

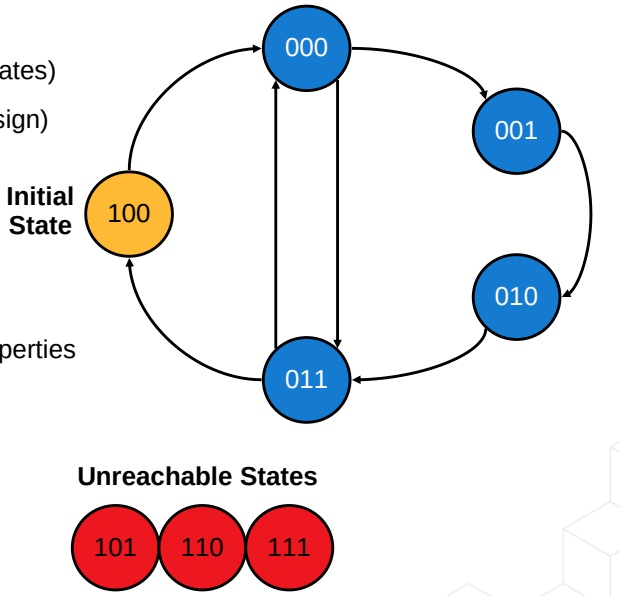
Terminology: State Space

Key terms

- State space = 8 (As there are 3 state elements, 2^3 total states)
- Reachable state space = 5 (5 being determined by the design)
- Diameter = 3
  - How many steps from the initial state to all reachable states

Observations

- Reachability and diameter depending on the initial state
- Verification complexity depends on the design *and* the properties
- Assertions pass if they are true in every *reachable* state



 © Cadence Design Systems, Inc. All rights reserved.

State space. How many possible states that the design can be in. Determined by the number of storage elements in your design.

Effectively, you can view your entire design as being one big state machine.

Reachable state space is how many of the possible states our design can ever be in.

Some states may be unreachable by design, that is, the number of states that your design can ever be in is less then two state space states.

Diameter definition: number of steps it takes to reach all the states in the system starting from the initial set of states.

A good estimate of the diameter of a design is the largest value a counter in the design can count to.

Get to initial state by constraints you give the Formal Analysis tool.

For example, typically, the design is put into the RESET state in a tool specific way.

There is another design methodology called hybrid verification, where the initial state of the design considered for Formal Analysis is achieved by running some simulation vectors. This is useful in situations where describing constraints for a static design is extremely difficult.

<<state space>> <<diameter>>

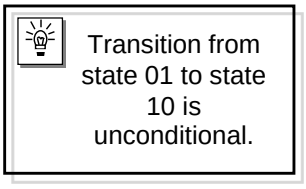
© Cadence Design Systems, Inc. All rights reserved.

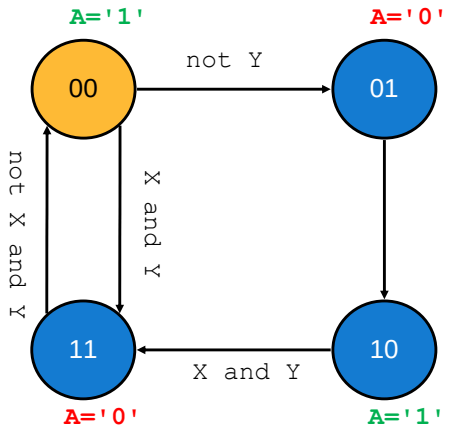
This document is for the sole use of Kun Wang of Seas

Checking a Single Cycle Assertion

A Single Cycle assertion must be satisfied at every (reachable) state.

It essentially checks an instantaneous Boolean condition in every state.






```
P1 : assert property (@(posedge CLK) (A));
P2 : assert property ( @(posedge CLK) (STATE == 2'b10) |-> A );
```

26 © Cadence Design Systems, Inc. All rights reserved.

This check can be verified by simulation, if the design ever enters the state with the applied test vectors, or during Formal Analysis.

For property P1, the assertion fails because A can be 0 in the reachable states “01” and “11”.

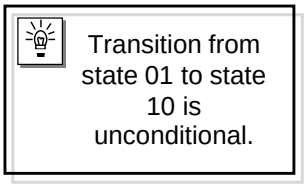
The assertion of property P2 passes because from the state “10”, A is always ‘1’.

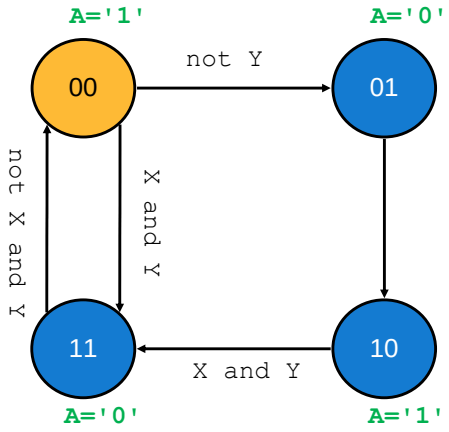
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Checking a Multi Cycle Assertion

As name suggest Multi Cycle assertions reason about relationships of Boolean expressions over multiple evaluation cycles






```
P4: assert property ( @(posedge CLK)   (STATE == 2'b01) |=> (STATE == 2'b10 ) ); ☑
P5: assert property ( @(posedge CLK)   (STATE == 2'b00) && !Y |-> ##2 A  );      ☑
```

27 © Cadence Design Systems, Inc. All rights reserved.

cadence

This check can be verified by simulation if the design ever enters the state with the applied test vectors or during Formal Analysis.

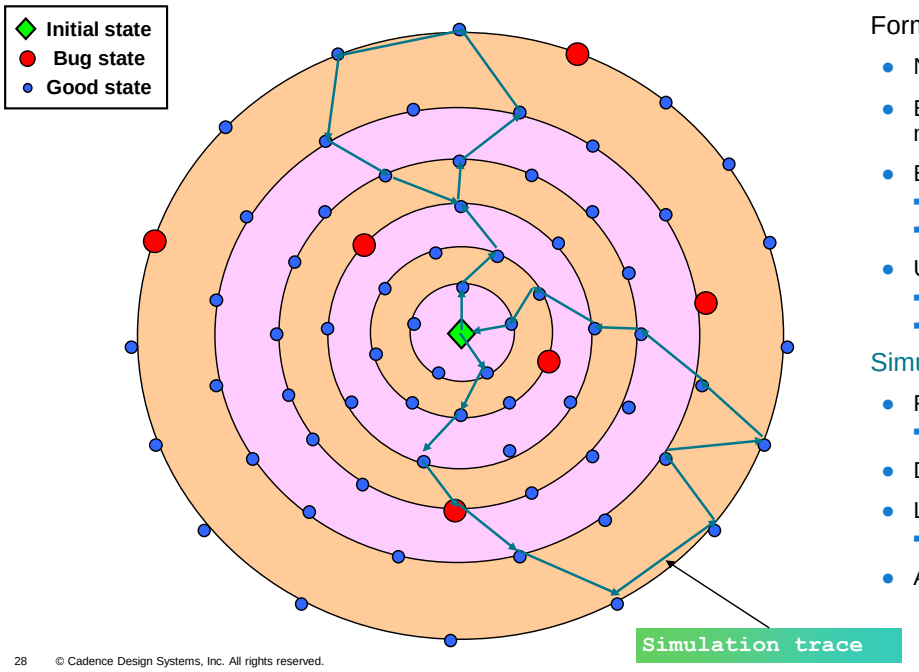
The assertion of property P4 passes because the only reachable state one cycle after “01” is state “10”. As the transition from state “01” to state “10” is unconditional we cannot stay in the same state.

The assertion of property P5 passes because from state “00” and Y = ‘0’, the only reachable state in two cycles time is state “10”. In state “10”, A can only have the value ‘1’.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Analysis Versus Simulation



Formal Analysis

- No testbench required
- Even small diameters equivalent to millions of simulation vectors
- Exhaustive
  - Uncovers all possible bugs
  - Reached max depth/diameter
- Undetermined
  - Inconclusive result
  - Reports verification depth

Simulation

- Follows specific paths
  - Might not hit a bug
- Depends on testbench quality
- Limited controllability
  - Difficult to hit all states
- Applicable later in the design cycle

© Cadence Design Systems, Inc. All rights reserved.

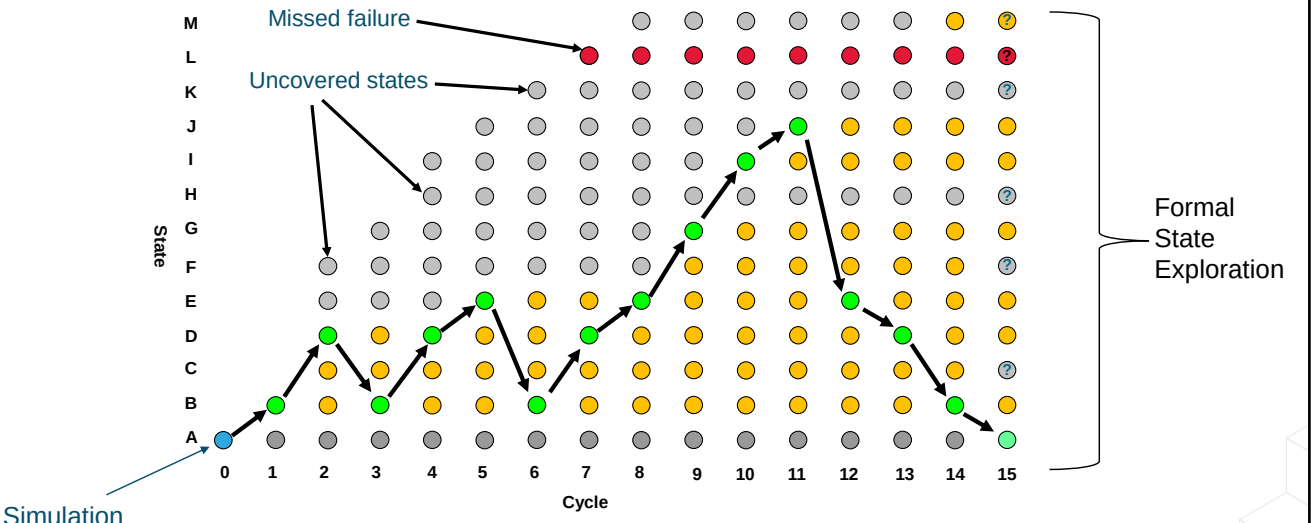
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Compared with Simulation



- Misses many reachable states, for example, (C, F, H, K, L, and M)
- Covers many states multiple times, for example, (A, B, D, and E)

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Simulation Versus Formal in Practice

Concept	Simulation	Formal
Values Checked • How To Target • Constraining Models	**Specific test values** • Describe the journey • Limit values driven	**Universe of possible values** • Describe the destination • Behavioral assumptions
Debugging • "What-if" questions • Waveform Length • Noise in waveforms	**Fixed waveform** • Modify/rerun test • Long- arbitrary activity on path • Hard to isolate relevant behavior	**Modifiable waveform** • Add live constraint & replot • Short- tools aim for minimal • Tool can Quiet Trace
Validation Efficiency • General ROI • Corner Case Bugs	**Good** • If you write & run lots of tests • Rare: Depends on specific tests	**Best** • Each run covers full space • Common: Covers all legal behaviors under constraints
Capacity	**High- run at cluster or full chip** • Linear cost in model size	**Lower- aim at unit, or targeted logic** • Worst-case exponential
Design/RTL Styles	**General System Verilog**	**Synthesizable Logic** **Complex Data Transforms hard…**

30 © Cadence Design Systems, Inc. All rights reserved.

cadence

Let's see the difference between simulation and formal in practice.

- Regarding value checked, Formal can take a universe of possible values compared to simulation where specific test values are given.
- While debugging in formal you will get a much more modifiable waveform that can be modified on the go to check more behaviors, on the other hand in the simulation you only get a fixed waveform which makes it hard to isolate relevant behavior.
- In terms of validation Efficiency, Formal is more efficient than simulation, because formal covers full space in each design run while simulation generates a random trace.
- It is not like formal has an advantage in every scenario, In terms of capacity simulation can handle much larger designs like full chip or cluster compared to formal which can handle smaller units or targeted logic, because of the exponential nature of formal, and complexity is a big concern in formal.
- In terms of design styles simulation can be done using a general system Verilog, on the other hand, formal requires synthesizable logic and sometimes complex modeling to get the job done.

As per the result formal has an advantage over simulation in some cases and vice versa

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Assumptions in Formal

Without assumptions, a FV tool checks all possible input combinations of a Design Under Verification (DUV):

- Likely to get false negatives.

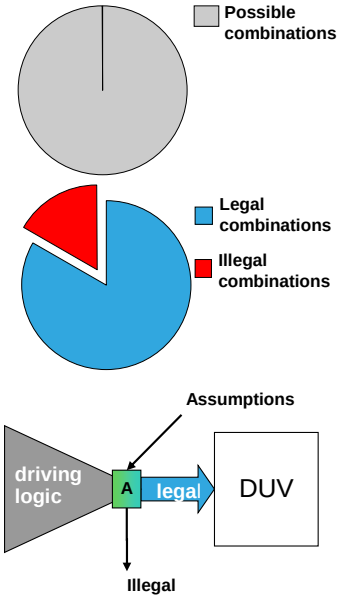
Assumptions specify the legal input combinations of a DUV:

- Model the verification environment.

Assumptions exclude illegal input combinations from the verification.

Assumptions intentionally reduce the state space.

Formal tools will **NEVER** violate assumptions.



 © Cadence Design Systems, Inc. All rights reserved.

Legal inputs are those that we expect to see during normal operation. It is not realistic to expect your design to behave in the correct way when all possible input combinations are being applied, unless you explicitly define every possible set of input combinations that your design can theoretically see.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Example: Assumptions

We want to formally verify DUV.

We know (can assume) that FULL and EMPTY can never be '1' at the same time.

- For example, the FIFO block has already been formally proved.

Assertion on FIFO outputs is an assumption on DUV inputs and vice versa.

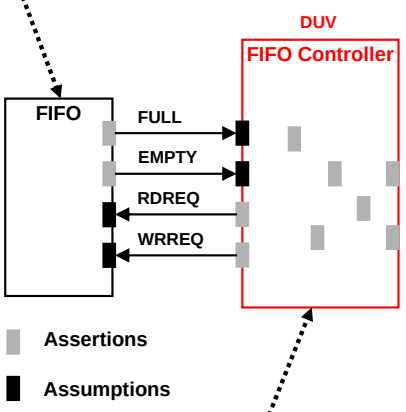
- We can leave assertions and assumptions in place when simulating.

If we let Formal Analysis consider FULL and EMPTY to be 1 at the same time, we may get a DUV assertion to fail.

- But we learn nothing.
- Inputs will never behave like this in real life.

Assumptions stop this false negative.

```
AST_FIFO: assert property (!( FULL && EMPTY ) );
```




```
ASM_FIFO_CTRL: assume property ( !( FULL && EMPTY ) );
```


© Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

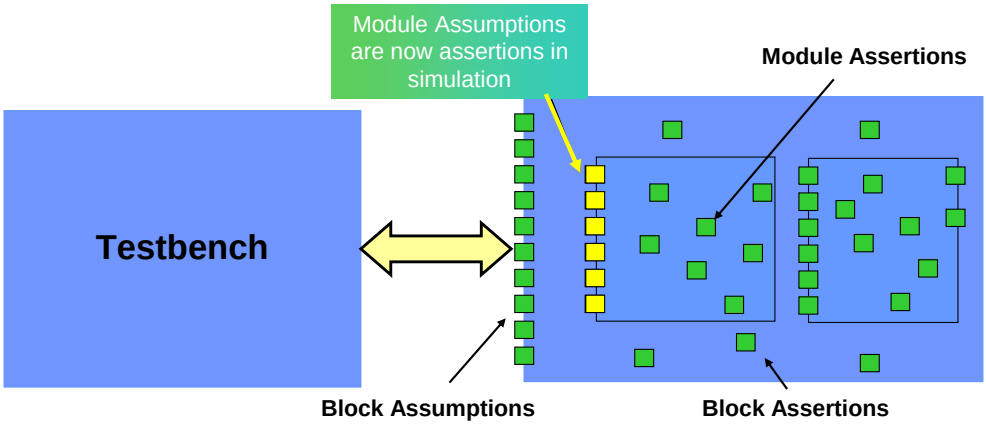
This document is for the sole use of Kun Wang of Seas

Simulation of a Block

Both assertions and assumptions are verified.

Assumptions are treated as Assertions in the simulation environment.

- Allows us to check the validity of assumptions we use in Formal Analysis.



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Cases Only Applicable to Formal: Overconstraining

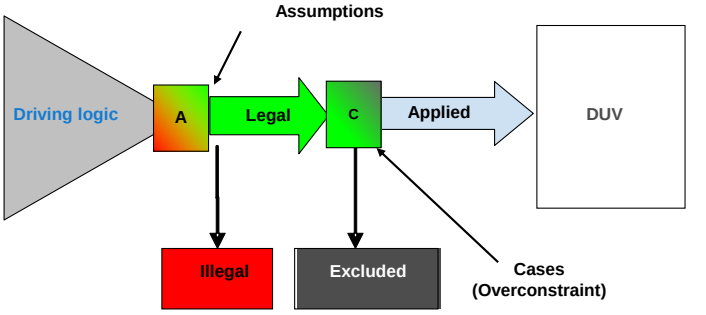
While verifying critical design, sometimes we wanted to focus on some particular cases.

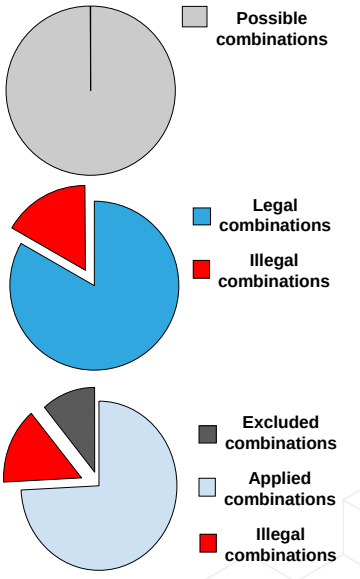
You can add assumptions that exclude illegal and some legal input combinations.

They over-constrain a DUV, intentionally and temporarily.

With overconstraining, the tool applies a subset of the legal input combinations:

- Useful to partition the verification
- Simplify the verification problem
- Start with simple properties





 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Example: Overconstraining

We want to formally verify some *write* functionality of the FIFO.

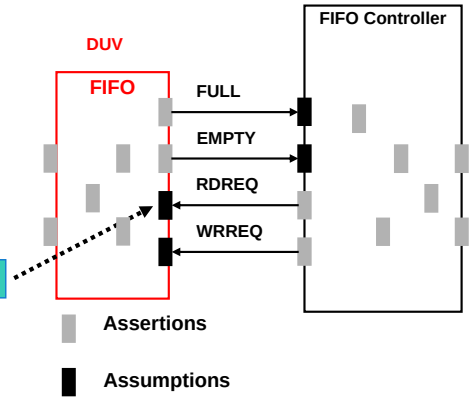
To simplify the verification task, add `pin constraints`:

- For example, hold `RDREQ` at `'0'`.
- Effectively eliminates some legal input combinations.

Reduces the amount of state space that needs to be explored.

- Improves tool runtime for this particular verification task.

```
ASM_NO_RD: assume property @(posedge CLK)(!RDREQ);
```



 © Cadence Design Systems, Inc. All rights reserved.

Clearly we don't expect RDREQ to be '0' all of the time.

In a simulation it is highly probable that this property fails, as most simulations are going to cause a read to occur.

We only did this as a temporary measure while we verified some write functionality because that way we can make better progress.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Example: Overconstraining (continued)

We want to formally verify some *read* functionality of the FIFO.

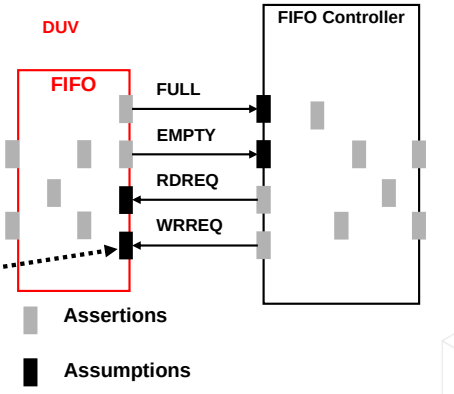
Constrain `WR_REQ` to `'0'` instead of `RD_REQ`.

It is not safe to assume there are no bugs if cases 1 and 2 show all properties pass.

- Bugs can still be related to simultaneous read/write activity.

Understanding what can go wrong in your design is vital.

```
ASM_NO_WR: assume property @(posedge CLK)(!WRREQ);
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Review: Overconstraining

In the previous examples, we said that there were either:

- No writes
- No reads

Clearly, we expect our design to both read and write.

It is important to understand that Cases are deliberately over-constraining.

They are temporary artificial properties used to speed up verification.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

In SVA the restrict keyword was added in the 2009 IEEE-1800 LRM.

It has the same behaviour as assume but allows a distinction to be made by humans and tools where the assumption is constraining the legal state space (assume) or a temporary constraint to make things easier for the tool and enable verification in a shorter time (restrict).

In PSL the keyword restrict has a radically different meaning. See IEEE 1850 for an explanation

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Over-Constraining Too Much ….

Assumptions reduce the state space to a legal state space.

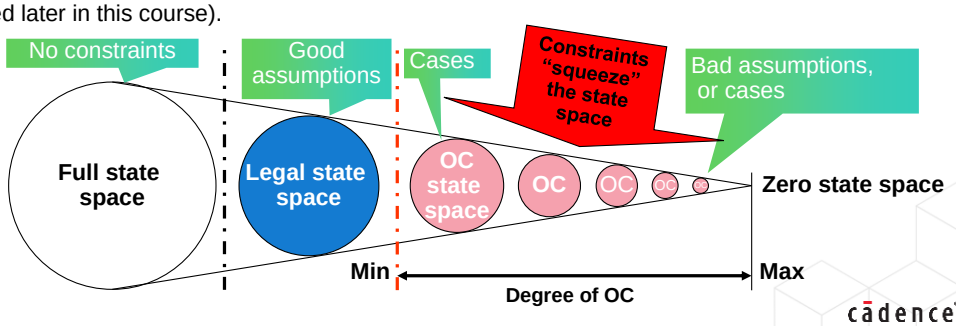
- The intention is to remove false negatives or to define “cases”.

Over-constraining **too much** will reduce state space below *legal and desired* state space:

- The difficult part is knowing when you reduce state space more than intended.
- For getting confidence in one design part overconstraint can be helpful.

The worst case is when state space is reduced to 0:

- No possible assertion failures.
- Nothing is reachable (discussed later in this course).
- Jasper will report an error.



 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Analysis Is Question and Answer

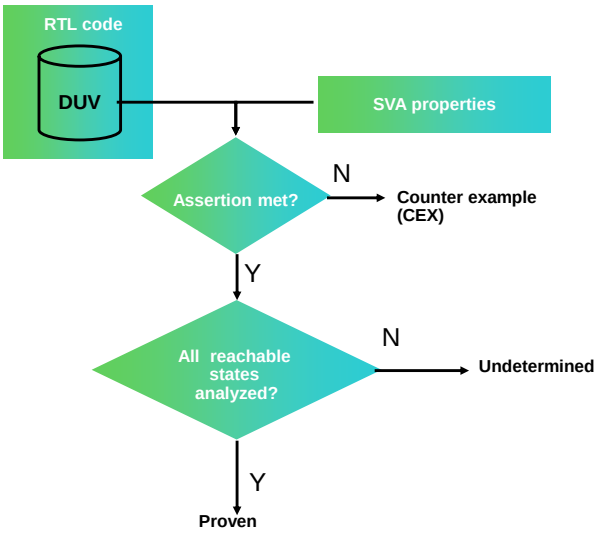
Properties are statements of behavior

- An assertion is a directive for a tool to check that the property always holds
- An assumption is a directive to the tool stating how the inputs can be driven

You ask a question in the form of assertions

The FA tool provides an answer in the context of the following:

- Constraints
- The RTL design



© Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Proof Results

Proven Unreachable
(Full Proof)

	Type	Bound
✓	Assert	Infinite
✗	Cover	Infinite

- Formal analyzed all reachable states
- Impossible to violate assertion
- Impossible to hit cover

Undetermined

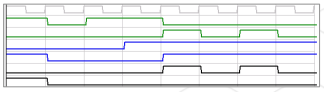
	Type	Bound
?	Assert	7 -
?	Cover	7 -

- Formal analyzed a subset of all reachable states
- Assertion does not fail in these states
- Cover is not hit in these states

CEX Covered

	Type	Bound
✗	Assert	12
✓	Cover	49

- Formal found a state that hits a property
- Assertion failure
- Cover hit
- Waveform available



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Understanding Proven and CEX

Defining `Proven` and `CEX` is not as easy as you may think:

- The reasons why may not seem intuitive to users of simulators
- When using FV, we define constraints
- These constraints make some states unreachable
- It might or might not be our intention to make a specific state unreachable
- We might accidentally overconstrain an environment

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Definition of CEX in Formal Verification

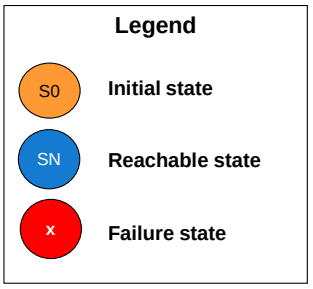
A violation of the assertion

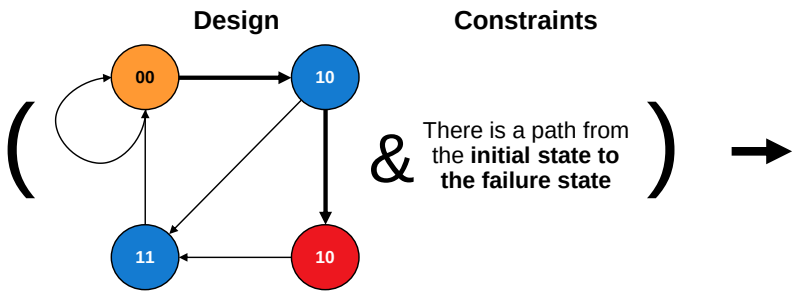
A CEX needs to satisfy both the design and the constraints

A failure state needs to be reachable based upon

- The design
- The constraints

If there is no path to the failure state, then there is no counter example!





 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Definition of a Proven in Formal Analysis

Explored the reachable state space fully and found no CEXs

- Put simply, the absence of CEX assertions is proven ← This is absolutely fundamental

The crucial factor is the reachable state space

- This is a function of the design, the initialization, and the constraints

If the state space is 0, then

- All assertions pass
  - There cannot be any CEXs
  - No path exists to the failure state
- All cover statements, precondition covers, triggers, and dead code checks are unreachable
  - Nothing is reachable

© Cadence Design Systems, Inc. All rights reserved.

cadence

Do not believe a tool that passes all assertions in a very short time.

It is highly likely that this indicates that there is an over constrained condition.

Luckily for us, if we have the foresight to buy a FV tool with built-in automatic checks, then we see this because covers, triggers, and dead code checks show our verification is not complete.

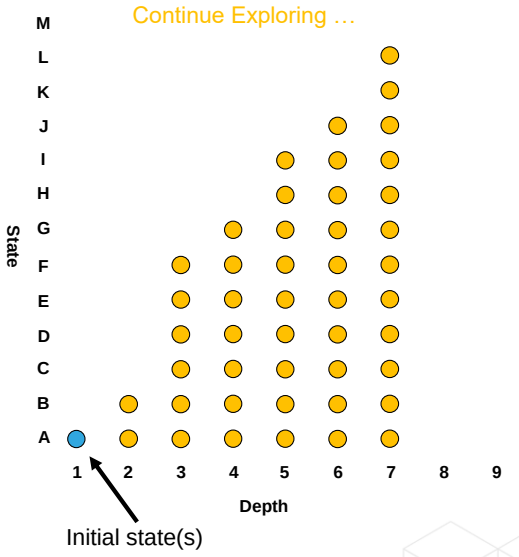
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal State Space Exploration

Formal visits DUT states as it walks through stimulus

- State = value of all registers/inputs



© Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

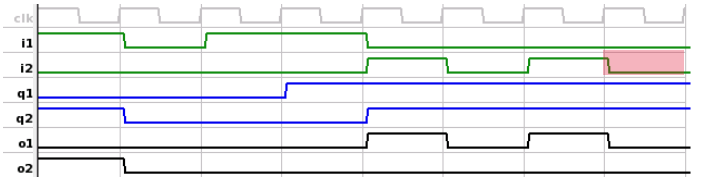
Formal State Space Exploration (continued)

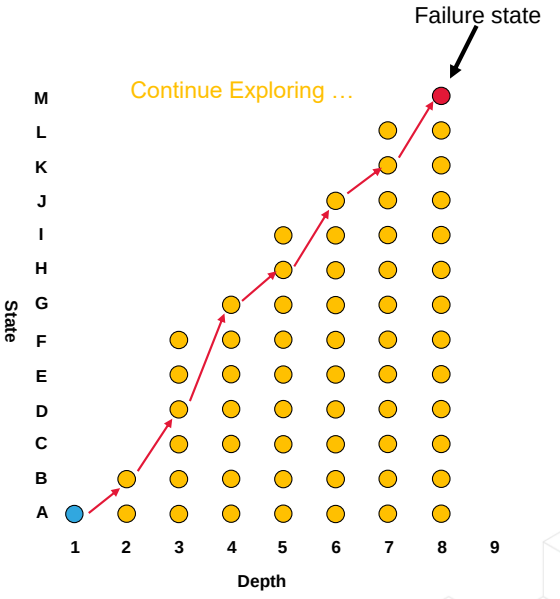
Formal visits DUT states as it walks through stimulus

- State = value of all registers/inputs

Formal may reach states that hit properties

- Assertion failures, cover hits
- Converted to waveforms





45 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal State Space Exploration (continued)

Formal visits DUT states as it walks through stimulus

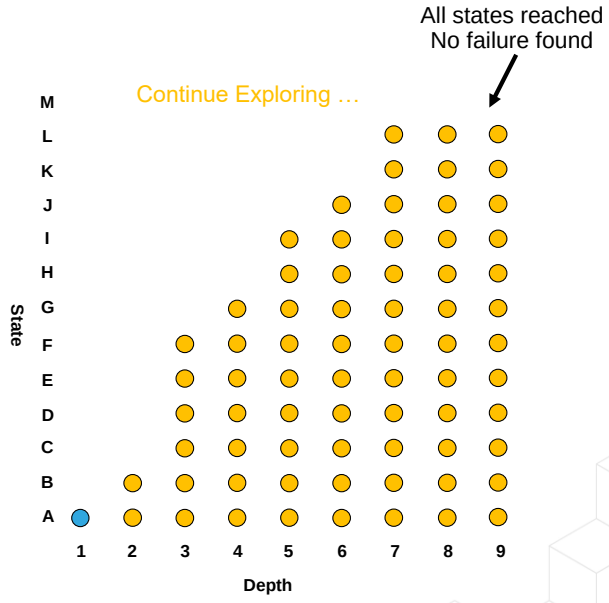
- State = value of all registers/inputs

Formal may reach states that hit properties

- Assertion failures, cover hits
- Converted to waveforms

It may also explore all reachable states

- If no properties are hit, then they will never be hit!
- "Full Proof"



46 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal State Space Exploration (continued)

Formal visits DUT states as it walks through stimulus

- State = value of all registers/inputs

Formal may reach states that hit properties

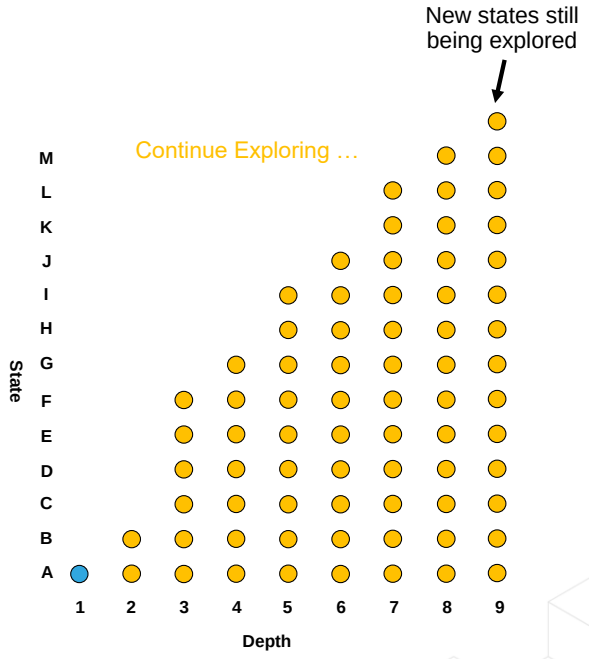
- Assertion failures, cover hits
- Converted to waveforms

It may also explore all reachable states

- If no properties are hit, then they will never be hit
- "Full Proof"

Or it may keep exploring new states until timeout

- Undetermined result



© Cadence Design Systems, Inc. All rights reserved.

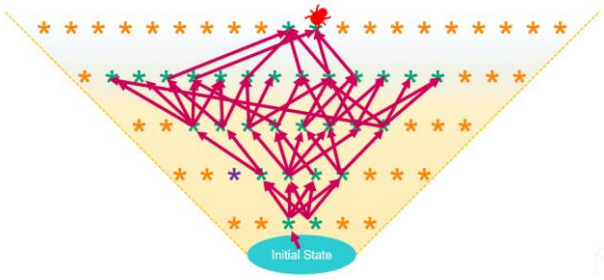
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

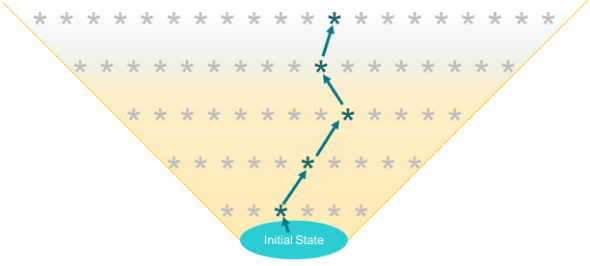
State Space Exploration: Formal Versus Simulation

Formal



Formal uses smart mathematical techniques to consider all relevant states.

Simulation



Simulation only covers some of the DUT states.

48 © Cadence Design Systems, Inc. All rights reserved.
cadence

When we look side by side at these images representing the formal state space exploration versus the simulation state space exploration, we can see how powerful formal is.

This isn't a fair comparison of course. The simulation would be run with many seeds over many iterations while collecting coverage. But there is still no guarantee that all states will be reached in a reasonable timeframe

But we can start to see that formal is working hard to explore all relevant reachable states in every cycle.

This is part of the math problem formal is presented with, and this is where complexity problems can occur.

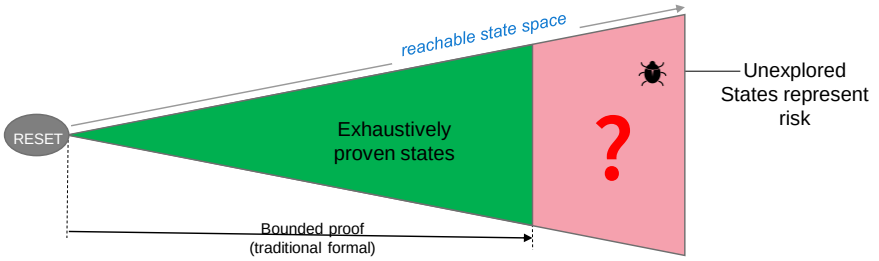
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Deep Bug Hunting

Goal

- Find new counterexamples and cover traces by exploring design states beyond what can be reached using traditional formal analysis.



Benefits

- Can provide a way to identify bugs even in the presence of complexity.
- Reduce the risk represented by undetermined properties.
- Close coverage for sign-off.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Deep Bug Hunting In formal
It is a powerful semi-formal method that can be used to find bugs that are challenging to find by traditional formal methods.

The goal is to find bugs beyond the formal reach.

sometimes, due to complexity, it is hard for formal engines to give full proof, tool will provide us with bounded proof in such a scenario.

Still, at this point, it is undetermine there is a chance of having bug, this can be found by using bug hunting.

some benefits of bug hunting are.
Even after complexity, it can provide a way to that bug.

Closes coverage for sign-off

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Coverage in Formal Analysis

We have seen that:

- Proven and CEX are not as intuitive as we would wish
- Constraints make some states unreachable

Cover statements express the desired reachability of states of the DUV and the environment

- In formal with a cover statement, we are asking, “Is this cover statement possible to observe”
- Enables us to detect an unintentionally over-constrained environment

In the simulation, the same cover statement states, “Count how many times this cover statement was observed”

As users of simulation are aware:

- Checks (assertions) without coverage are of little value

Coverage is as vital in formal as in simulation

 © Cadence Design Systems, Inc. All rights reserved.

cadence

It is important that constraints and properties considered during Formal Analysis are complete and correct. Check for over-constraining or under constraining. Check that all inputs and outputs are covered by properties, that is, that there are no holes in the coverage.

It is highly recommended to have more then one person creating the properties and constraints. It is also highly recommended that sanity checks be added during the verification.

Jasper has several tools to automatically check validity of the user input: Vacuity checks find contradictions in the design or properties and find inputs and outputs that are not covered in formal proofs. This identifies holes in verification coverage. Incisive Formal Verifier looks at cover directives and say if the sequence that is the subject of the cover directive will never happen.

If the sequence can never occur, then the sequence, constraints or the design must be incorrect.

<<complexity>>

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Challenges for Formal Analysis

Complexity

- Size
  - Usually correlated to state space and inputs
- Diameter (#cycles after reset to reach all states)
  - Counters have large state space diameter
    - 32-bit counter has 2^{32} states
- May not be able to explore entire state space in a realistic time frame
  - For example, with a 3GHz clock, it would take over 194 years to see all values of a 64-bit counter
- Design Type
  - Formal is more suitable for control logic, but can be used on data transport and data transform design type

Results depend heavily on the user input:

- Need complete and correct property specification
- Need valid environmental constraints
- Overconstrained need to be intentional and well-understood

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Cone of Influence (COI)

Each assertion has its own COI

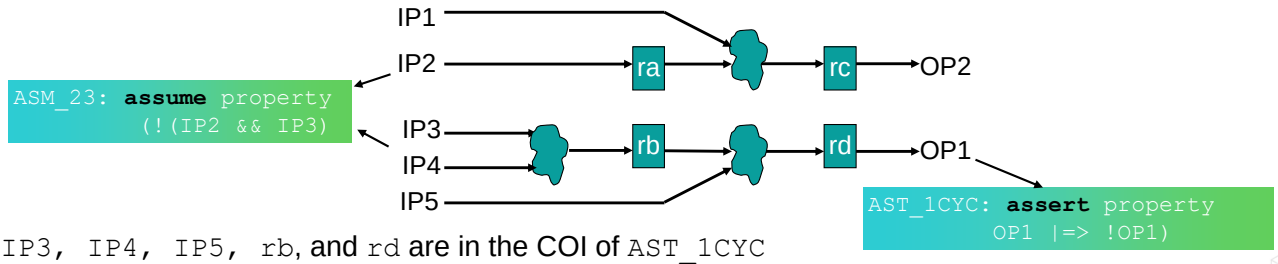
COI is all things that can structurally affect a particular assertion

- State elements, combinational logic, and inputs

State elements remember things, for example, Registers, Memories, Latches

COI size is not related to the design size

- It depends only upon the assertions and constraints



`IP3`, `IP4`, `IP5`, `rb`, and `rd` are in the COI of `AST_1CYC`

If we add assumption ASM_23, we add IP2 to the COI

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

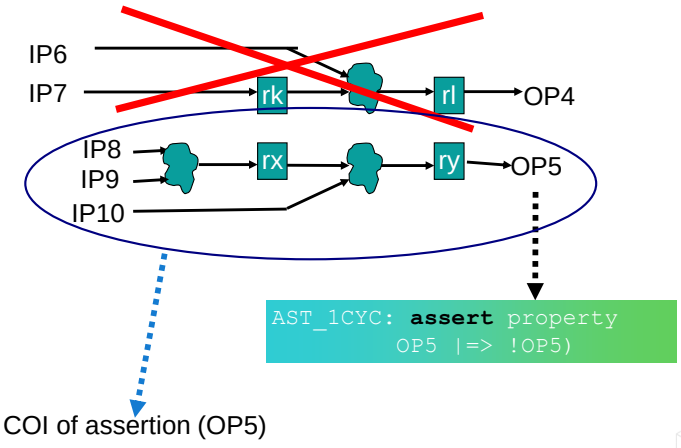
Complexity and Size of a Problem

Biggest factors are:

- Size of the COI
- The property and constraints

What does a formal tool do?

- Computes COI of each assertion
- Analyze that COI to find if violations are possible
- Thus, even in large/complex designs, some assertions may be provable



53 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Amenability Table

Design Type	Sequential Depth	Interface Type	Block Specification	Criticality & Risk	Block Size
Control Logic	10s of cycles	Protocol Handshake	Well Documented	High	Small (<100k flops)
Data Transport	100s of cycles	Pipelined/ Parallel	Partially Documented	Medium	Medium (100K-500K)
Data Transform	1000s of cycles	Serial	Minimally Documented	Low	Large (500K+)

FPV Easiest → FPV Hardest

Mainly Signoff, some Bug Hunting → Mainly Bug Hunting, some Signoff → Bug Hunting only

54 © Cadence Design Systems, Inc. All rights reserved.

cadence

- This Table Demonstrates the complexity of different design types while using formal Property verification.
- Control logic is best suited for FPV because of the less sequential depth, the focus here is on formal signoff.
- Some parts of data transport design can be signed off by FPV with assisting bug hunting.
- In data transform designs only bug hunting is recommended, because of complexity and data manipulation, there are other techniques in formal like C to RTL which can be used to verify data transform design.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Preparing for Formal

Need to put design into a known initial state

- Determines and eliminates unreachable states

Otherwise, likely to get false negatives

Need to specify how design inputs are constrained

- Using assume directives

Pass some form of linting check prior to creating a mathematical model:

- Synthesis check
- Good HDL coding for synthesis enforced

Specify resets and clocks:

- + relationships between multiple clocks

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Activity Goals: A Non-Exhaustive List

Goal	Goal Characteristics
Block bring-up, sanity checking of block	Increase confidence in basic operation of block. Find bugs on new RTL. Do not fully verify.
Increasing confidence in verification of feature or block	Increase confidence in feature functionality by proving some assertions or exploring assertions to sufficient depth.
Finding bugs	Activity focused on finding bugs. Not necessary to prove assertions. Focus is not to find all bugs; easier to debug early in flow. Easier to find corner cases.
Fully verifying an interface	Fully verify an interface. Focus is not to verify block functionality.
Fully verifying a feature or block	Get conclusive results with FA for all features check and coverage items.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Summary

- Formal is a very different paradigm from simulation
- One needs to be prepared for a complete change in mindset
- Even `proven/found CEX` are not straightforward to describe
- Design size (in FFs/memory) does not correlate with how hard the problem is to prove for formal – it depends upon many factors
- Formal tools need to be initialized for the results to make sense
- The initial state is where you decide to start verification from
- This is different from simulation, where one generally starts from the reset state

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 3

Formal Friendly SVA Coding

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Module Objectives

In this module, you will learn how to

- Prepare for Complexity
- Identify SVA which is likely to be inefficient
- Create Auxiliary Code (HDL helper code) to assist when writing properties
- General RTL design with FV complexity prevention in mind
- Identify the problems that cannot be solved by SVA alone
- Implement a combination of simple properties and auxiliary code to replace inefficient SVA
- Evaluate the potential difficulty of specific properties or coverage points for formal tool

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Why Prepare for Complexity?

Theoretically impossible for Jasper™ (or any FV tool) to prevent

- "NP-Complete" problem: mathematically proven hard
  - Any algorithm takes exponentially long for some case
- This is why Jasper tries many "engines" when proving
  - Each engine is good at some problems
  - Each engine is bad at some problems
  - We can give general guidelines but not guaranteed predictions

Take action at the design stage and verification planning

- Good RTL design ➔ complexity is easier to deal with
- Small amount of up-front work ➔ FPV easier during crunch periods
- Formal Design Bring-up can also benefit design

60 © Cadence Design Systems, Inc. All rights reserved.

cadence

- Complexity is a big concern in formal verification. So while writing SVA if the properties itself is complex then it is hard for any formal tool to converge it.
  - To deal with such a scenario, write properties with complexity prevention in mind.
  - Write simpler more efficient properties than complex ones, which we are going to discuss later in this module.

- Taking action at early stages can reduce complexity which can be writing a good parametrized design that can help reduce future work if the goal is to verify the design completely.

- We will discuss more about complexity and how to reduce it in a later module.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

What Does Inefficient SVA Mean?

SVA efficiency is a consideration in both simulation and formal.

- There is some overlap between simulation and formal concerns:
  - Example: An assertion that launches lots of potential evaluation threads is bad for both.
- In simulation, it's easier to use profiling tools to find out exactly which SVAs are consuming time or memory.
- In formal, this is hard, as SVAs are part of the full formal model and are hard to separate out.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

- What's meant when we say efficient and inefficient SVA?
- Now, if you're an SVA user and you've only used it in simulation before, then generally, you only spend a little bit of time thinking about how efficient it is because, in simulation, what you see is what you get.
- If you've done something so inefficient, it rises to the top of any kind of profiling report. So, every simulator has a way of profiling what time has been spent on or what memory allocation is being spent on.
- In formal this is Hard as SVAs are part of the full formal model & hard to separate out.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Inefficient SVA in Simulation: Example

In the simulation, the request goes high, and we must wait for 1000 cycles before a grant

- How many overlapping copies of the assertion does the simulator need to evaluate?
- How many grants are needed to satisfy each one of those 1000 overlapping properties?
- How would we change the property so that only one copy of the property was being evaluated at any time?

```
P_BAD: assert property ( @(posedge CLK)
                req  |-> s_eventually gnt );
```

Creates a requirement for every cycle which request is true (1000 of them), I eventually get a (single) grant

```
P_GOOD: assert property ( @(posedge CLK)
   $rose(req)  |-> s_eventually gnt );
```

It is easy to make mistakes

Small things can have large effects

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Can I Use the Same Properties in Simulation and Formal?

In principle and in general, yes

- Behavior described in SVA transcends tools and technology
- However, our focus is efficiency here

In formal, the evaluation of properties is a very different story

- Each assertion is evaluated in isolation using a handful of relevant constraints
- Every possible way the property can fail needs to be exhaustively explored

If we write inefficient assertions, then we are penalized once

- Assertion proof
- Debug

If we write inefficient assumptions, we are penalized multiple times

- Assumptions are in the fanin cone of other assertions and add to proof complexity

 © Cadence Design Systems, Inc. All rights reserved.

cadence

There are some exceptions where a property which is good in formal is meaningless in simulation, for example, properties using Non-Deterministic Constants

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

What Are the Symptoms of Inefficient SVA in Formal?

- Undetermined Properties
- Long run times
- High Memory Consumption

Problems are not limited to tools and technology

Human element: Hard for the user to understand

- Did I capture in SVA what I actually intended to verify?
- Property is difficult to understand and interpret if the CEX is correct or not
- Did I unintentionally create overlapping properties? Will this affect my run?
  - Does this then change what I thought I was checking?

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Considerations for Efficient SVA

Writing functionally correct SVA is not good enough

It also needs to be easily understood by users and efficient for the tool

Similar to RTL Synthesis:

- Functionally correct RTL code is not good enough
- It needs to be efficient RTL for Synthesis

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Why Is Simplifying SVA Useful?
Simple Example

Requirement: If sig_a |-> sig_b ##5 sig_c ##3 sig_d

```
P1: assert property ( sig_a |-> sib_b ##5 sig_c ##3 sig_d );
```

- Property “P1” meets the requirement
- But combines many possible errors in one line
- If any part is hard to prove, the whole assertion doesn't prove
- If it fails, it takes debug effort to find whether sig_b, sig_c, or sig_d is responsible

Q1) What should be done?

Just divide the Properties into simple ones

```
P1: assert property (
 sig_a |-> sig_b );

P2: assert property (
 sig_c |-> ##5 sig_c );

P3: assert property (
 sig_c |-> ##3 sig_d );
```

Q2) Now is it simpler?

Yes, it is easier to run simpler properties for the tool; if any property fails, it is easier to debug.

66 © Cadence Design Systems, Inc. All rights reserved.

cadence

Let’s understand why simplifying SVA can be useful.

The requirement is that, If signal A is High then signal b should be high on the same cycle after 5 cycles signal C and then again after 3 cycles signal d should be high.

Property P1 meets the requirement but, comes with possible errors, if any part is Hard to Prove, the whole assertion will be undermined, If it fails, it will take more debug effort to find whether signal b, signal c, or signal d is responsible.

What should be done ?
Simple, divide the properties into simple ones.

The property P1 is divided into three simpler properties,

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Why Is Simplifying SVA Useful?
Complex Example

Requirement: If a request goes high, then it must be high up to and including the cycle in which an acknowledge is observed

```
P1: assert property (
 $rose(req) |-> req[*] ##0 ack );
```

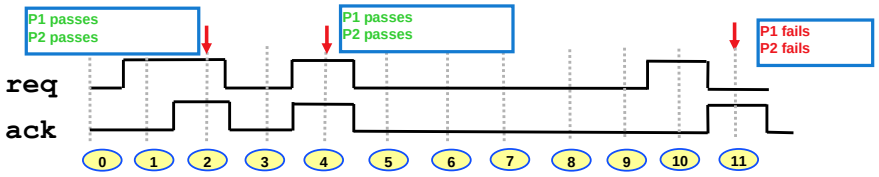
```
P2: assert property (
 $rose(req) |-> req[+] ##0 ack );
```

Q1) Are these properties equivalent?

Yes. `req[*0] ##0 ack` cannot create a match. Hence for `P1` writing `req[*] ##0 ack` is **exactly** the same as writing `req[+] ##0 ack`

Q2) Which of these properties, if any, meet the requirement? **Both**

Q3) Are these both liveness properties, i.e., there may exist an infinite length CEX **Yes**



Q4) Would these properties fail if `req` went high, stayed high forever, but `ack` never occurred

It depends! As per the IEEE-2005 SV LRM they would fail
As per the 2009 or 2012 SV LRM, the properties would pass
This change is due to introduction of strong/weak concepts and operators in 2009 LRM

This simple problem seems harder than one would expect
What can I do to help myself and the tool?

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Simplifying SVA: Example

A reminder of our requirement:

If a request goes high, then it must be high up to and including the cycle an acknowledge is observed

- We saw that P1 meets our requirement but doesn't seem easy to understand for humans or to evaluate for the tool

```
P2: assert property (
  $rose(req)  |-> req[+] ##0 ack );
```

- Is there an alternative?

```
A_REQACK_SIMPLE: assert property (
    ( req && !ack)  |=> req );
```

Q1) Does this meet the requirement? **Yes**

Q2) Is this easier to understand? **Yes**

Q3) How long could a CEX be?

Every CEX would require consideration of only 2 cycles

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Note that we didn't state that there was a requirement for ack to ever occur.

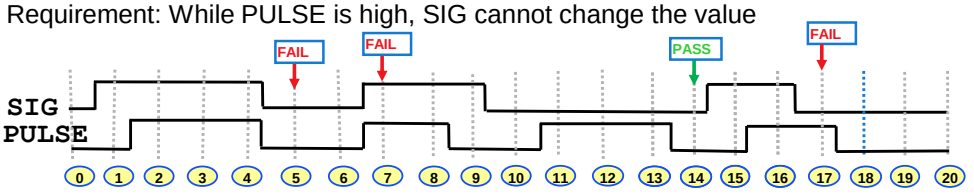
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

What If Things Seem Complicated?

Then you are often doing something wrong

Requirement: While PULSE is high, SIG cannot change the value



Should we use the `throughout` operator?

- The issue then is to detect the beginning and end of the pulse

```
P20: assert property (
 $rose(PULSE) |-> $stable(SIG) throughout (##[1:$] $fell(PULSE)) );
```

```
P21: assert property (
 $rose(PULSE) |-> $stable(SIG) throughout (!PULSE[->1]) );
```

Q1) Are the properties exactly equivalent? **Yes**

NO. For example, at cycle 7 the assertions fail as SIG is not the same value it was on cycle 6

Q2) Do they meet the requirement?

NO. For example, at cycle 5 SIG is not stable. We are checking outside of the PULSE being high at both ends

Q3) How would we recode the property?

69 © Cadence Design Systems, Inc. All rights reserved.

cadence

Always look for the easiest solution. If it takes more than 5 minutes to think of a solution then this could mean:

- You are using the wrong approach. Draw out the requirement visually, e.g. pen and paper. Then it is much easier to see the solution.
- The problem needs to be divided into more than one property.

Although the “throughout” operator may at first glance seem like what we are looking for, one needs to caution against choosing a solution that is looking for a problem.

The simpler the property the better for simulation and especially formal.

Just because keywords like throughout, intersect, within, etc exist, then that does not mean we are obliged to use them.

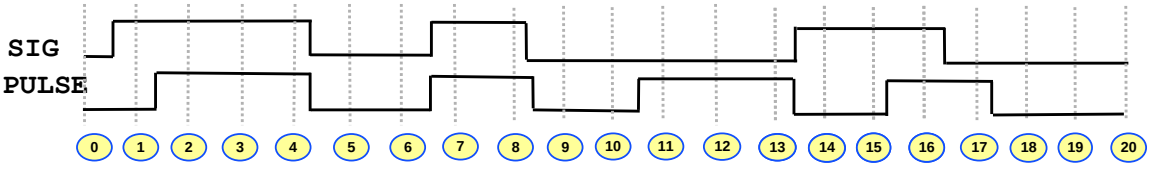
Almost every problem can be solved without such operators, with simple sequences or Booleans, and maybe some Auxiliary HDL code as well.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Pick the Simplest Way of Expressing the Requirement

Requirement: While PULSE is high, SIG cannot change the value




```
P2: assert property ( PULSE |=> !PULSE || $stable(SIG) );
```

Q1) Does this property cover all cases?

Yes

© Cadence Design Systems, Inc. All rights reserved.

cadence

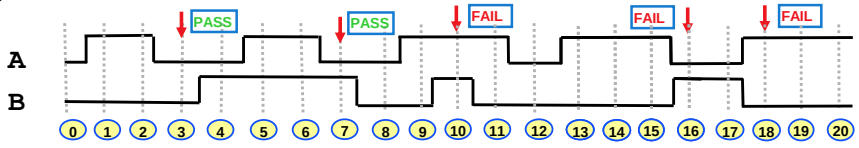
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Keep It Simple!

Requirement: B must be stable when A is '1' and remain stable until and including the cycle where A changes to '0'




```
P2: assert property (
 A |=> strong( $stable(B)[*0:$] ##1 ($stable(B) && !A) ) );
```

```
P1: assert property (
 A |-> ( $stable(B) s_until_with !A );
```

Q1) Are the properties all exactly equivalent?
Q2) Do they meet the requirement?

Q1) Yes
Q2) Yes

Q3) Did I create infinite length sequences?
Q4) Could I have unintentionally created overlapping properties?
Q5) Is the solution hard to understand?

Yes to all

Q6) Could the overlapping affect the result?

No

© Cadence Design Systems, Inc. All rights reserved.

cadence

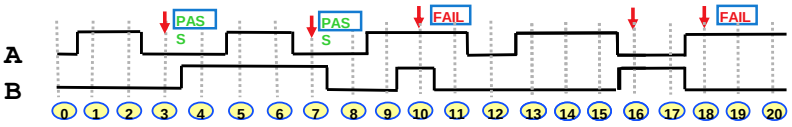
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Simplification: Divide and Conquer

Requirement: B has to be stable when A is ‘1’ and remain stable until and including the cycle where A changes to ‘0’




```
P4: assert property (  A         |=> $stable(B) );
P5: assert property (  $fell(A) |-> $stable(B) );
```

The combination of P4 and P5 checks all required behavior

- Both are trivial to understand, same cycle implication properties

Do not try and state everything in a single complex property. Instead, split the requirement into multiple properties:

- It’s easy to understand and debug
- Formal evaluates each assertion in isolation
  - A simpler property is easier for the tool to prove
  - We can parallelize proofs and improve productivity

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Human Readability

- Understanding how to state clearly what is required is essential for human understanding
- Mistakes and incorrect expectations can be avoided if understanding of the spec is clear

For example, take these 4 statements:

1. If it is sunny, then I must have my sunglasses
2. It's undesirable for it to be sunny when I don't have my sunglasses
3. If I don't have my sunglasses, then it's not sunny
4. I must have my sunglasses, or it won't be sunny

```
P1: assert property (
  sunny |-> sunglasses );
```

```
P2: assert property (
  not (sunny && !sunglasses) );
```

```
P3: assert property (
  !sunglasses |-> !sunny  );
```

```
P4: assert property (
  sunglasses || !sunny );
```

Q1) Are these properties all equivalent?

Yes

Q2) Which would you say best conveys the intent?

If we were having a human conversation then the most likely inference would be P1

73 © Cadence Design Systems, Inc. All rights reserved.

cadence

One could go back to Greece in 600BC and hear the same proposition logic argument.

……and yes even the sunglasses: http://www.glasseshistory.com/glasses-history/history-of-sunglasses/

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Useful Things to Remember

All these properties are equivalent

A	B	Result
0	0	PASS
0	1	PASS
1	0	FAIL
1	1	PASS

```
P1: assert property (  A |-> B      );
P2: assert property (  !(A && !B)  );
P3: assert property (  !B |-> !A   );
P4: assert property (  B || !A     );
```

Hence, the only disallowed condition is `A && !B`

There are several considerations that affect the best direction for an implication.

- The LHS should be a rare condition, that triggers interesting behaviors. (interesting_stuff -> consequence):

```
P_GOOD: assert property (GRANT |-> REQ );
```

- Is better than (!consequence -> !interesting_stuff):

```
P_SAME_BUT_BAD: assert property (  !REQ |-> !GNT );
```

- The meaningful trigger allows a better-automated precondition check in formal, and triggering on the rarer condition also can be much more efficient for simulation.
- The natural-language meaning is a good guideline, but a secondary consideration.

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Recommended *Not* to Be Used in Formal

A non-exhaustive list

It is not a case of Jasper™ not supporting these constructs

It's a question of efficiency

This refers to the sequence OR/AND operators – it's fine to use ||, &&, etc. on Boolean expressions.

```
intersect                Local variables
throughout               Immediate
or                       assertions
and                      if property else
within                   property
expect                   sequence.triggered
first_match
```

With `$past(expr, N)` keep N very small refers

```
$past(expr, 2) //OK, probably
$past(expr, 15)//BAD
```

Sequence repetitions

- `SEQ[*N]` – May create infinite length sequences
- `SEQ[->N]` – Will always create infinite length sequences
- `SEQ[=N]` – Will always create infinite length sequences

```
SEQ[*5]   //OK
SEQ[*200] //BAD
```

Cycle repetitions

- ##[<RANGE>]
- Remember, during the delay `##N`

  Nothing is being checked!

```
##3          //OK, probably
##[2:1000]//BAD
##[2:$]     //Probably much easier for
            // tool than ##[2:1000]
```


© Cadence Design Systems, Inc. All rights reserved.

cadence

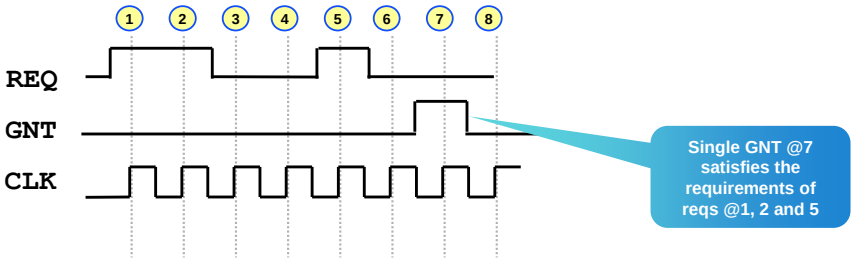
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Overlapping REQ/GNT: Bad Implementation

- This can be challenging to describe in SVA
- A single occurrence of GNT can satisfy multiple requests
- You need to model using Auxiliary HDL code (Aux code)
  - This will also be much more efficient for formal




```
A1: assert property(@(posedge clk) REQ |-> ##[2:8] GNT));
```


© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Example: An Overlapping Behavior Requirement

Required request/grant interface behavior:

- If you have a REQ, then you must have a GNT within 2 to 8 cycles
- Requests can overlap
  - For example, you can have another request before the first grant received
- Each REQ must have a corresponding GNT
- No GNT without a REQ

This initially seems easy in SVA

Why won't it work as planned?

```
A1: assert property(@(posedge clk) REQ |-> ##[2:8] GNT));
```

77 © Cadence Design Systems, Inc. All rights reserved.

cadence

You have a different situation, which looks easy to solve with SVA.

The cause of the problem is the overlapping nature of the REQ/GNT handshake behavior.

This makes it impossible to discern from handshake from the other, because you need to consider things in a global sense.

If you can guarantee that there are no overlapping transactions then this is easy.

You can even write an assertion to check that there were no overlapping transactions, for example:

```
REQ |=> !REQ && !GNT[*0:$] ##1 GNT && !REQ
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

What Is Auxiliary Code?

Auxiliary code has 3 main purposes

- Make it easier to write properties
- Make the problem easier for the tool to evaluate
- Do things that assertion languages cannot

Possible location of Aux code

- In Verification IP /Checkers
- Normal Verilog modules, which have only inputs
  - VIP is merely a container for Aux code, assertions, assumptions, and cover statements, i.e., only verification code
  - Connected to the DUV, even legacy code, using the SV `bind` statement
  - Just instance and connect them where you need them
    - Just like UVCs in a UVM testbench
- Embedded in RTL
  - Surrounded with synthesis pragmas or `` `ifdef ``
  - Not recommended

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

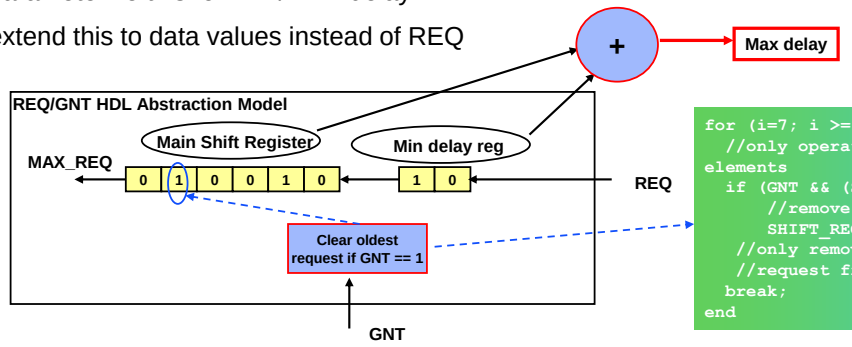
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

REQ/GNT Abstraction Model

Requirement of the Aux code model:

- If you have a request, you must receive a GNT within 2-8 cycles
- 2 represents the minimum delay for the GNT response
- 8 represents the maximum delay for the GNT response
- Can parameterize this for MIN/MAX delay
- Can extend this to data values instead of REQ




```
for (i=7; i >= 2; i--)
  //only operate on MAIN_SHIFT_REG
elements
  if (GNT && (SHIFT_REG[i] == 1'b1))
      //remove oldest request
      SHIFT_REG[i] <= 1'b0;
   //only remove a maximum of one
   //request from shift reg per clock
  break;
end
```

```
A1: assert property ( MAX_REQ |-> GNT);
A2: assert property ( GNT |-> $countones(MAIN_SHIFT_REG) != 0);
```


© Cadence Design Systems, Inc. All rights reserved.

cadence

Note that for this to work in the detailed implementation, SHIFT_REG[0] and the REQ signal would have to be assigned at the same clock, for example:

Always @(posedge CLK) begin

  REQ <= NEXT_REQ;

  SHIFT_REG[0] <= NEXT_REQ;

end

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Overlapping REQ/ACK Handshake: Requirement

Every REQ must have an ACK sometime later

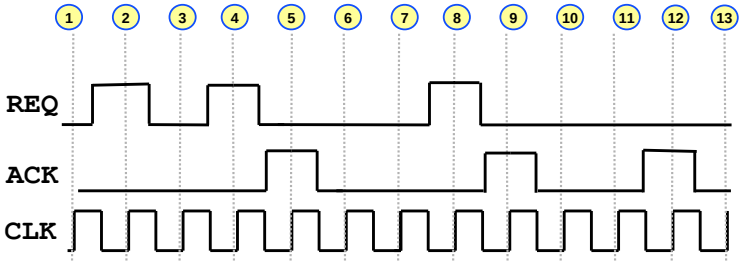
Maximum of 5 outstanding requests allowed

The ACK can occur at any time

- There does not have to be an ACK before another REQ
- No limit on the number of cycles from REQ to ACK

No ACK without REQ

How can you write this with SVA?



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

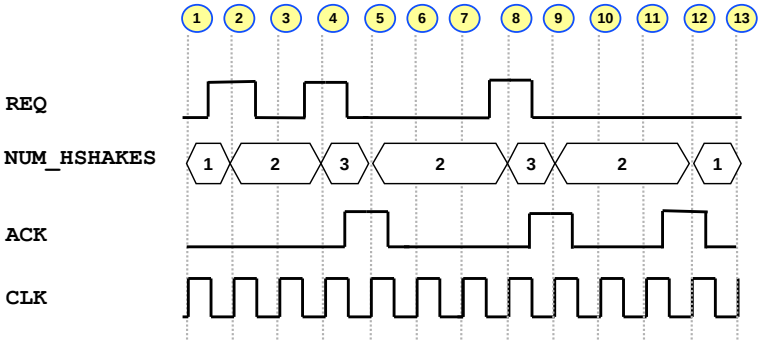
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Overlapping REQ/ACK Handshake Solution

No way to write this in just SVA

You need an Aux code to do the *global counting*



Use the smallest counter size possible, not an integer (32 bits)

Start unsigned count at 1

```
reg[2:0]  NUM_HSHAKES;
always @(posedge CLK or posedge
RESET)
 begin
  if  (RESET)
    NUM_HSHAKES <= 1;
  else
   begin
    if (REQ & !ACK)
       NUM_HSHAKES<= NUM_HSHAKES+ 1;
    if  (ACK & !REQ)
       NUM_HSHAKES<= NUM_HSHAKES- 1;
   end
end
```

AUX Code

```
LESS_THAN_6_REQS: assert property ( ;
 @(posedge CLK)   ( NUM_HSHAKES !=7) );
endproperty

NEVER_MORE_ACKS_THAN_REQS : assert property (
@(posedge CLK) (NUM_HSHAKES != 0) );

NEVER_ACK_WITHOUT_PREV_REQ : assert property (
@(posedge CLK)
  (REQ && ACK |=> NUM_HSHAKES != 1) );
```

Q1) What would be good cover statements to write?

```
C1: cover property (
@(posedge CLK)
   (NUM_HSHAKES ==6) ##[+] (NUM_HSHAKES ==1) ) ;
```


© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Ack Response Within a Window

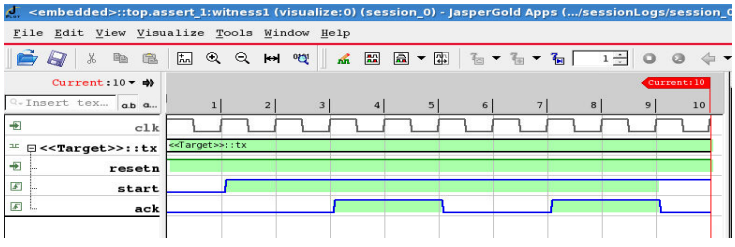
Requirement:

- If I observe the rise of `start`, then I must observe between 1 and 3 `ack`'s within a 10 cycle window
- This 10 cycle window begins in the same cycle in which I detect a rise of `start`
- The `ack`'s need to occur in consecutive cycles but at any time in the 10 cycle window

```
S_ACK_WIN_10: assert property (
          @(posedge CLK) $rose(start) |-> ack[*1:3] within 1'b1[*10] );
```

Looks like a sensible property at first glance

However, if I ask Jasper to cover the property (`witness`), then this is what I see:



It is not expected design behavior to see 4 acks in the 10 cycle window. Why does Jasper show the design behaves like this?

 © Cadence Design Systems, Inc. All rights reserved.

cadence

By default Jasper will cover the left hand side of implication only, namely $rose(start) in our S_ACK_WIN_10.

If we wish to observe a cover of the entire property passing, known as a "witness", then we need to use the elaborate command option:

elaborate –create_related_covers witness

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Why the `within` Property Doesn't Work?

The LRM states that these 2 properties are equivalent:

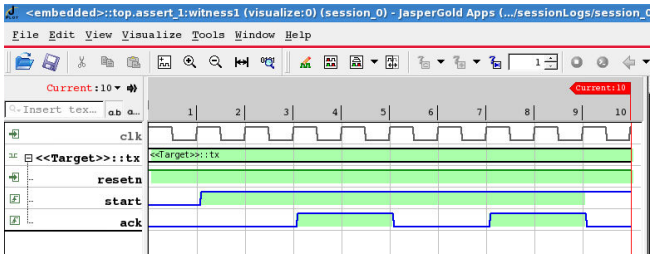
```
S_ACK_WIN_10: assert property (
          @(posedge CLK) $rose(start) |-> ack[*1:3] within 1'b1[*10] );
```

Hence, a single `ack` anywhere in the window satisfies all requirements, and `ack` can have any other behavior in the remaining 9 cycles of the window

Easy to make mistakes while trying to understand what `intersect` and `within` do

Auxiliary code is easier to understand for such "`count occurrences in a window`" type properties

- It's also easier for the tool



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Limit on Number of Occurrences in a Given Window

A different requirement of the Aux code model:

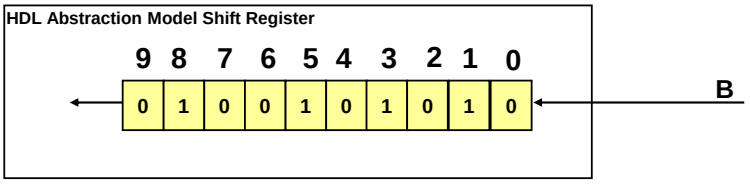
- No more than 3 occurrences of B in any 10 cycles

```
A1: assert property (not strong( (1 [*10]) intersect (B[=4]) ) );
```

Property A1 meets the requirement, but …

- Is not particularly easy to conceive or understand
- Is not that easy for the tool to evaluate

Remodel with a simple shift register and a simple Boolean property




```
A2: assert property ($countones(my_10_elem_shift_reg) != 4);
```

 © Cadence Design Systems, Inc. All rights reserved.

Could make these properties assumptions for formal

- REQ and GNT will be driven by formal tool
- Following rules of properties and HDL model

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

SVA Alone Cannot Solve All Problems

One-shot properties – Properties that we only want to check once

- Unlike PSL, SVA does not have simple semantics for this

Requirement:

- From the beginning of verification, we require A to be high up to and including the cycle B occurs
- After B occurs, we don't care about what either A or B is doing

```
ONESHOT_A_UNTIL_B: assert property (
   @(posedge CLK) ( !B_SEEN |-> A) );
```

```
reg B_SEEN;
always @(posedge CLK or negedge RSTn)
 begin
  if  (!RSTn)
     B_SEEN <= 0;
  else if (B)
     B_SEEN <= 1'b1;
  end
```

AUX Code

© Cadence Design Systems, Inc. All rights reserved.

cadence

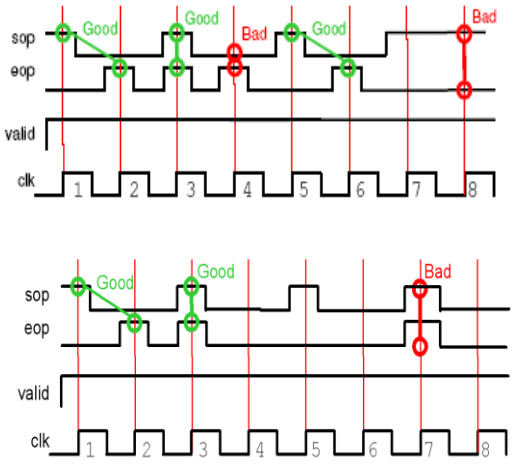
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Framing Protocol Example: Requirement

- We have start of packet and end of packet framing signals `sop` and `eop`
- They should be "`pair-wise`"
- There can be single cycle packet transfers, i.e., `sop` and `eop` pair on the same cycle
- No `eop` without a corresponding `sop`
- There should not be a second `sop` until after the corresponding paired `eop` is observed



© Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Avoid Complex Sequences

Let us concentrate on just one requirement

- No `eop` without a corresponding `sop`

```
AST_NO_EOP_WO_SOP  : assert property ( @(posedge clk)  eop |=> !eop[*] ##1 sop);
```

Q1) Does the property meet the requirement? **No**

Q2) Does the property check from the very start of verification that there is not an `eop` without a prior `sop`? **No**

Q3) Is `sop` ever required to occur and does it matter?

2005 SV LRM: `sop` must occur or the property fails. There could be an infinite length CEX.
2009/2012 SV LRM: `sop` never required to occur. All CEXs will be finite length.

```
AST_NO_EOP_WO_SOP_S  :
assert property ( @(posedge clk)  eop |=> strong(!eop[*] ##1 sop) );
```

I could use SV 2009/2012 operator **`strong`** if I cared about `sop` occurring. However, that was not part of the requirement.

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Tackle It with Aux Code

Let us concentrate on the same requirement

- No `eop` without a corresponding `sop`

```
reg sop_without_eop; //Use as a flag
 always @(posedge clk or negedge rst_n)
   if (!rst_n)
     sop_without_eop <= 1'b0; //Clear flag
   else begin
    if (sop && !eop)
      sop_without_eop <= 1'b1; //Set flag
    else if (eop)
      sop_without_eop <= 1'b0; //Clear flag
   end

NO_EOP_WO_SOP_W_AUX_CODE:
  assert property ( @(posedge clk)
    (eop && !sop) |-> sop_without_eop       );
```

Q1) Does the property and Aux code meet the requirement? **Yes**

Q2) Do I need to care about `strong/weak` operators or which SV LRM version I'm using? **No**

Q3) Does the very first `eop` get checked to ensure there was a paired `sop`? **Yes**

Q4) Did I create infinite length sequences?
Q5) Could I have unintentionally created overlapping properties?
Q6) Is the solution hard to understand?

No to all

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

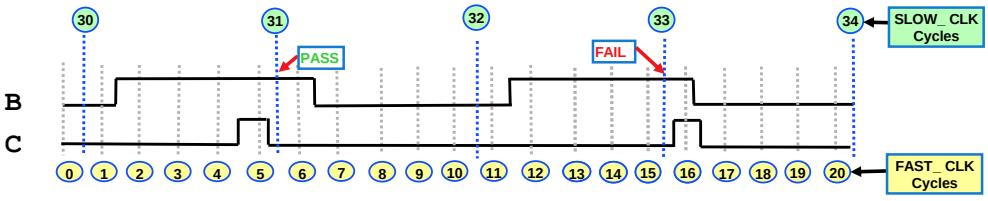
This document is for the sole use of Kun Wang of Seas

Properties Which Require Multiple Clocks

Rule #1: Don't use properties with more than one clock unless there is absolutely no other way

Requirement for a design with a fast clock and a slow clock:

- `B` is clocked by the slow clock and `C` by the fast clock
- If `B` rises, then `C` should have been `'1'` on the previous fast clock




```
P3: assert property (@(posedge SLOW_CLK) $rose(B)  |-> @(posedge FAST_CLK)  $past(C));
```

This works but there are many concerns and self-doubt

- Does it really express what I want to check? Could I have overlooked something?
- Does the relationship of the clocks affect the result? How do I state that to the tool?
- What is meant by "same cycle" when I use `|->` in this context?

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

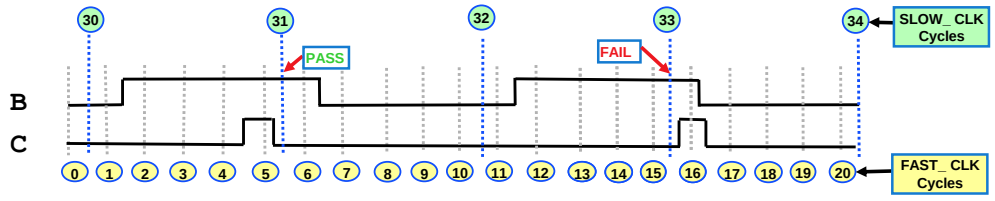
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Aux Code Makes the Problem Easier

Requirement for a design with a fast clock and a slow clock:

- `B` is clocked by the slow clock and `C` by the fast clock
- If `B` rises, then `C` should have been `'1'` on the previous fast clock




```
P3: assert property ( @(posedge SLOW_CLK) $rose(B) |-> C_PREV_CYCLE));
```

```
reg C_PREV_CYCLE;
always @(posedge FAST_CLK or negedge rst_n)
  begin: SAVE_DELAYED_C
   if (!rst_n)
     C_PREV_CYCLE <= 0;
   else
     C_PREV_CYCLE <= C;
  end
```


© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Aux Code Versus SVA

A common question for beginners to formal is, “What is the point of SVA if I have to write more RTL?”

- Aux code will be more work

SVA is a very concise way to describe complex behavior

That is a source of strength and weakness, a double-edged sword

- Even with as few as 6 characters, one can create an SVA property that takes a long time to understand

This module was about efficient SVA coding for Formal tools, and Aux code is an essential part of that

This is a naive view:

- Try, and describe everything that needs checking in a single assertion
- SVA encompasses all your needs

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Are There Any Common Rules from the Examples?

If you see properties like this in Formal, then alarm bells should be ringing

```
P1: assert property
( A ##[0:$] B |->
    C throughout( D[=4] ##1 E));
```

For efficiency in Formal try to aim for this:

- Modeling the problem in an abstract way using Aux code without duplicating RTL
- Properties with simple implications without complex sequences
- Properties based on the Aux code signals rather than the direct use of DUV signals

```
AST_SAME_CYC: assert
property ( A |-> B);

AST_NEXT_CYC: assert
property ( C |=> D);
```

This has been a feature of all examples which we have seen in this module

All Formal Verification IP (ABVIP), Proof Accelerators, etc., are written like this because it's the most efficient way for the tool

© Cadence Design Systems, Inc. All rights reserved.

cadence

It is important to understand that it is the verification problem which needs solving is abstracted and not the design.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Putting What We Have Learned in Context

We are not stating that all SVA code should be replaced by Aux code

Clearly, SVA is more concise, and that is desirable

However, we cannot ignore:

- Practical limitations
- Time is taken to express in Aux code and SVA rather than just SVA

What factors influence our decision?

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Reflection Upon What Formal Tells Us

Formal does not have a single “one size fits all” use model

Exhaustive proofs on every single assertion are unrealistic

Let’s review what results formal can give us:

- Reachable Cover
  - One single example, out of a potentially infinite number, of an occurrence of a particular scenario
- Unreachable Cover
  - Every possibility of a particular scenario occurring has been exhaustively ruled out
- CEX
  - One single example, out of a potentially infinite number, of an assertion violation
- Proven
  - Every possibility, out of a potentially infinite number of possibilities, has been checked, and no CEX exists
  - Liveness assertion
    - CEXs may be of infinite length
  - Safety Assertion
    - All CEXs are finite length
  - Bounded proof
    - There are CEXs within X cycles (the bound) of the initial state
    - It is a human decision on what is a “good enough” bound
- Undetermined
  - The tool ran into a user-defined time limit while still working on one of the above

© Cadence Design Systems, Inc. All rights reserved.

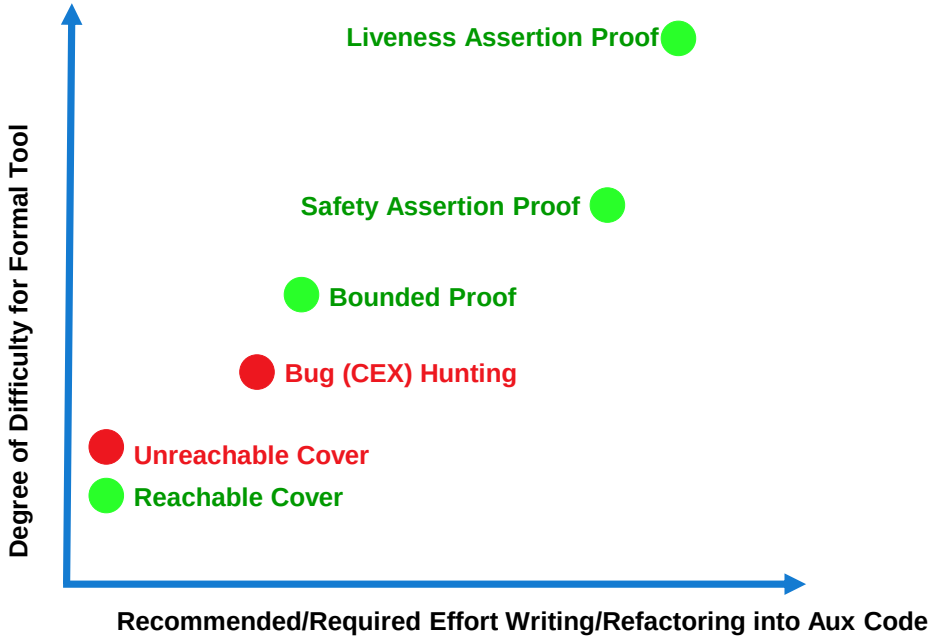
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Generalities in Degrees of Difficulty for Formal



 © Cadence Design Systems, Inc. All rights reserved.

cadence

Of course this graph is a sweeping generalism, but those can be useful at times.

One would always be able to point to an exception.

The graph has no units on the axes, it is merely a conceptual observation where the axes are not specifically linear or not.

But in general:

- For covers and bug (CEX) hunting Aux code may not give a significant improvement, i.e., just using SVA, even the non recommended constructs, will *probably* be fine.
- For exhaustive assertion proofs, then this is likely to be hard for the tool and the use of Aux code is likely to significantly improve how deep the tool can get in any given time.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Summary

Writing complex SVA is often counter-productive

- It's hard for the tool to evaluate
- It's hard for other engineers to understand

Almost every problem will require an Aux code

- Allows the problem to be abstracted
- Does not create overlapping copies of itself
- More verbose but easy to understand
- Typically checks more behavior than the intended complex SVA
- Easier to write covers and even allows the tool to infer covers automatically

We need to consider how hard the problem is likely to be for the Formal tool

- Judge the ROI in coding/refactoring into Aux code + SVA
- If only interested in covers and bug hunting, then even the non-recommended constructs are probably OK

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 4

Property Development Introduction

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Module Objectives

In this module, you will learn how to

- Define motivations for writing properties
- Choose appropriate goals for Formal Verification
- Identify different property categories in order to simplify verification
- Define properties categories in the correct sequence to maximize chances of success

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Who Writes Properties and Why?

Who	Type of Assertion	Reason	Example
System Architects	Abstract Property	Ensures implementation meets functional spec	An asynchronous FIFO does not indicate full and empty at the same time
RTL Designer	Functional Property	Implementation has properties the RTL designer intended it to have	When Read and Write pointers are both 0, the FIFO indicates empty
Verification Engineer	Functional Coverage	Ensures test stimulus exercises the corner cases as required by the test plan	Did we read from full FIFO, write to the last empty FIFO location, etc.,?
IP Designer	Correct Use of IP	Indicates to IP user that the IP is not being used correctly	FIFO was read when it was empty

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Property Development Challenges

- Blank-page syndrome
  - What should I test, and where should I start?
- Overall function of the chip might be easy to state
  - Whereas the implementation of the chip is very complex
- Block-level specifications are often not well formulated
- Designer intuitively understands a block but often has trouble coming up with meaningful properties
- Not sure what the goals are

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

General Guidelines

- Keep it simple
- **Simplicity:**
  - Decreases your ramp-up time with assertions
  - Often ends in better results with formal analysis
- Most intended behavior can be expressed with simple constructs
- Better to write multiple simple properties than complex one
- Leverage auxiliary HDL code
  - It often makes things simpler and more readable
- Build your properties and FV environment in a gradual manner
- Simplify complexity and build confidence in your FV environment

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Guidelines for Writing Properties

Start simple

- Easier to write and debug properties
- Often better for FV tool
- Mutexes and one-hots are good places to start.

Use more simple properties rather than fewer, more complex ones

Properties should be created from the *specification*, *not* the *implementation*

Look at the problem from multiple perspectives:

- Try to write assertions that pull together several disparate parts of the code and tie them together

```
        // Avoid Copying RTL HDL
if (ADD)
  C <= A + B;  // Add A & B

CHECK_ADD: assert property ( ADD |=> (C == $past(A+B)) );
```

This assertion is a very low value because effectively we just translated a single RTL statement into SVA

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Abstract Properties

Make abstract assertions:

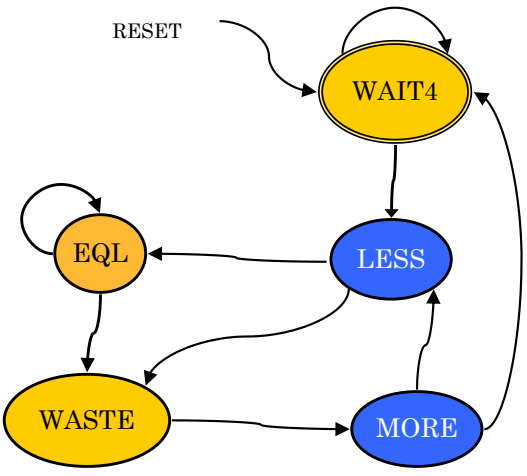
- For example, about expected FSM behavior

You do not need to understand the design to make the assertion:

- “If I am in state LESS, eventually I should get to state MORE”

Formal tools can show you a counter-example of how this property fails

The formal tool helps you to learn the design




```
LESS2MORE: assert property (
    @(posedge CLK) disable iff(!RST_N)
        (STATE == LESS) |=> s_eventually(STATE == MORE) );
```

 © Cadence Design Systems, Inc. All rights reserved.

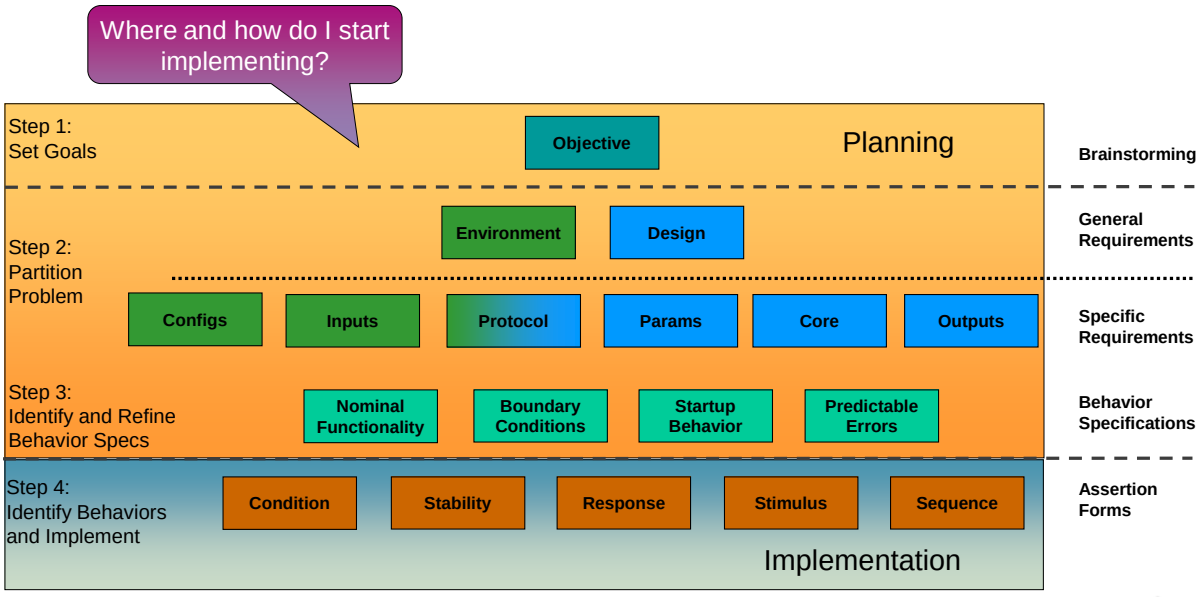
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Complete Assertion Development Process



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Assertion Development Process

Step 1: Define your goals

Step 2: Property categories

Step 3: Property development flow

Step 4: Property implementation

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Step 1: Define Your Goals

Potential goals include:

- Bring-up test
- New block cleanup
- Corner-case exploration
- Checking simulation coverage holes
- Interface verification (Completeness required)
- Full functional verification (Completeness required)



Completeness of properties is not required for every goal

Aggressive goals assume appropriate focus area selection

- The more aggressive your goals, the more time you need to invest

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Step 2: Property Categories

Blackbox Properties

- Function of input and outputs only
- Written by both the designer and verification engineer
- Most often can be derived from the specification
- 2 types:
  - Protocol Properties
  - Functional Properties

Whitebox Properties

- Function of inputs, outputs, and internal signals
- Design specific behavior that is not directly visible to the outside
- Written typically by designers as they have design knowledge

 © Cadence Design Systems, Inc. All rights reserved.

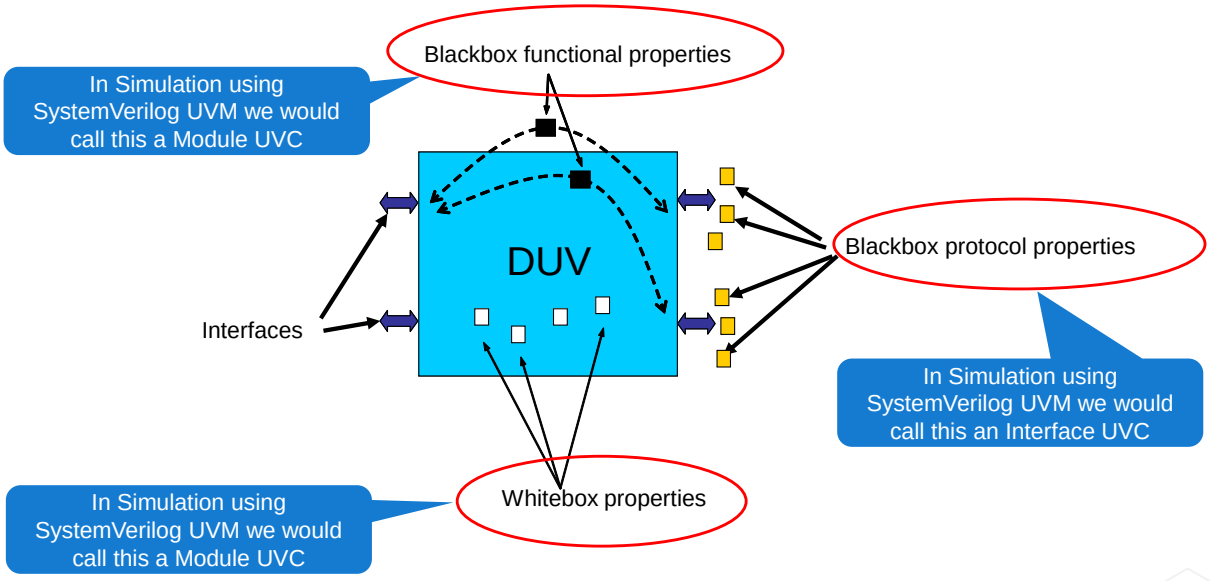
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Categories of Properties



108 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Blackbox Functional Properties

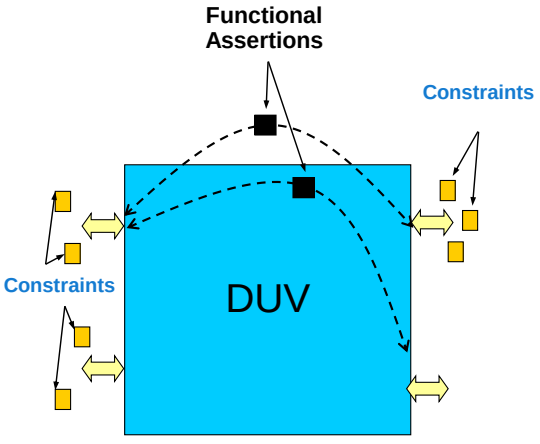
Describe the functionality of the block as it relates to the outside world

Typically, cover most block functionality:

- Represent end-to-end checks

Large part of the design is exercised:

- Tend to be quite complex
- Potentially significant constraint development required
- The properties are typically very simple to state
  - For example, data that goes in eventually comes out



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

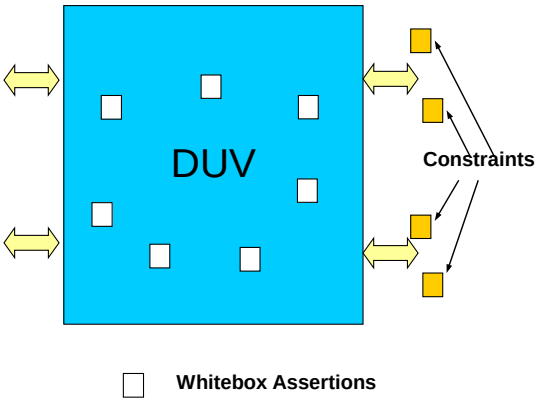
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Whitebox Properties

Tend to be localized

- High ROI:
  - Easy to express
  - Smaller COI (Cone of Influence) means good FV results
  - Less constraints required
  - Closer to bugs means easier to debug
- These properties don't verify overall functionality
- Can miss block functional bug



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Step 3: Property Development Flow

Assuming all 3 categories are needed, the following order should be followed:

- Blackbox protocol properties
  - Some of the protocol properties serve as constraints
  - Constraints are a prerequisite for almost all subsequent assertions
  - In FV, if any blackbox protocol assertions fail, one cannot rely upon results from any functional properties
  - Blackbox protocol properties are essential for FV, hence should be developed first
- Whitebox properties
  - Local, hence smaller, COI
  - Easier to debug than blackbox functional properties
- Blackbox functional properties
  - Full end-to-end check
    - Only block functional bugs should be left by this time

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

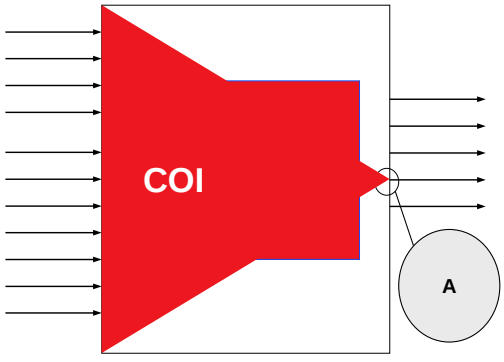
Blackbox Functional Property Death Spiral

End-to-end checks prove a lot of functionality

Typically have a large COI, hence tend to:

- Have long run times
- Hard to understand CEXs
- The death spiral goes like this:
  1. Long proof finds a CEX
  2. The CEX is complex and takes a long time to debug
  3. The bug gets fixed
  4. Long proof finds a different CEX
  5. Go to 2; slightly more disheartened than last time

It is generally counter-productive to rush to end-to-end checks



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Analysis of Categories

Group the three categories into two domains:



- Proper interface functionality required to test core functionality
- If the block cannot communicate correctly, then it does not matter what is inside

- The purpose of the block
- Tendency is to rush to verify this domain

 © Cadence Design Systems, Inc. All rights reserved.

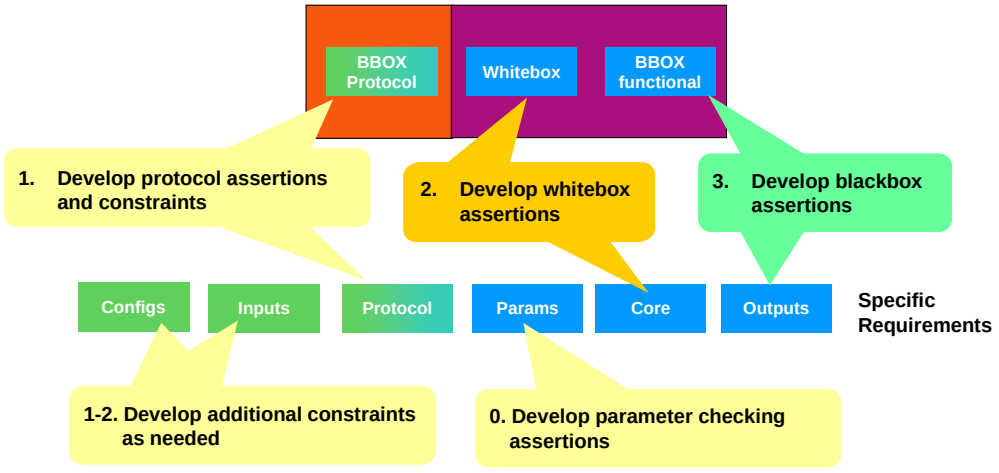
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Tying It All Together



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Step 4: Property Implementation

Capture the property description in a natural language (English):

- For example: *"do not overflow FIFO"*

Determine the property type within the context of the module: assertion or assumption

Define the property in SVA

```
output_no_overflow: assert property ( (write && full) |-> read));
```

Ensure there are adequate covers to detect over-constraining

By default, Jasper™ will automatically create this:

```
output_no_overflow_precondition1: cover property ( write && full);
```

To observe the entire property passing, as opposed to only LHS of an implication

```
% elaborate ... -create_related_covers witness ...
```

Results in the automatically created cover

```
output_no_overflow_witness1: cover property ( write && full && read);
```

115 © Cadence Design Systems, Inc. All rights reserved.

cadence

Using –create_related_covers means we can have a complete example of the property passing. This might reveal over-constraining which could be hidden if we rely upon the default setting for "–create_related_cover" which is "precondition".

This of course would probably be at the expense of longer run times for the cover.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Summary

- There is no single job function that writes SVA properties
- Formal has many use models, and these require different goals
- Defining properties categories in the correct sequence can maximize chances of success
- Natural language descriptions of required properties aid in writing and maintaining properties

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 5

Overview of Formal Analysis Use Models

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Module Objectives

In this module, you will learn how to state how to

- Explain how Formal Verification differs from Simulation
- Use Formal Analysis for design exploration
- Use Formal Analysis to prove user-defined functional properties
- Use Formal Analysis to perform Deep Bug Hunting (DBH)
- Apply each of the different Jasper™ Apps to obtain verification closure

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Property Verification: The Core Jasper App

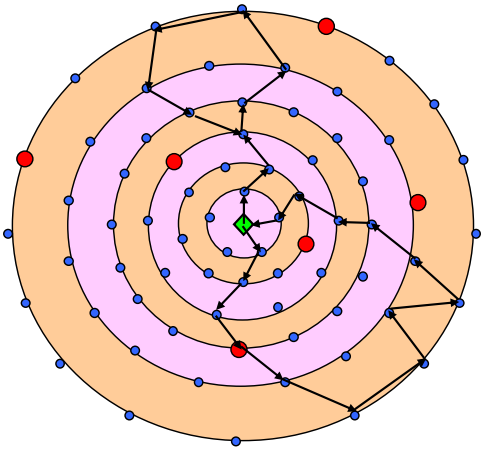
FPV is a Jasper App and is what most people think of when they hear the term “Formal Verification.”

Simplistically, FPV is this:

- Write user-defined SVA functional properties and covers
- Use Jasper to find full proofs, bounded proofs, and counterexamples for assertion and cover properties
- Refine counterexamples by adding appropriate assumptions, or report as real design bugs
- Evaluate coverage to prevent validation holes
- This FPV use model is known as “Classic Formal”

Other apps are built on top of FPV

- Many apps auto-generate properties in some domain
- Apps simplify the interface to hide FPV details



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

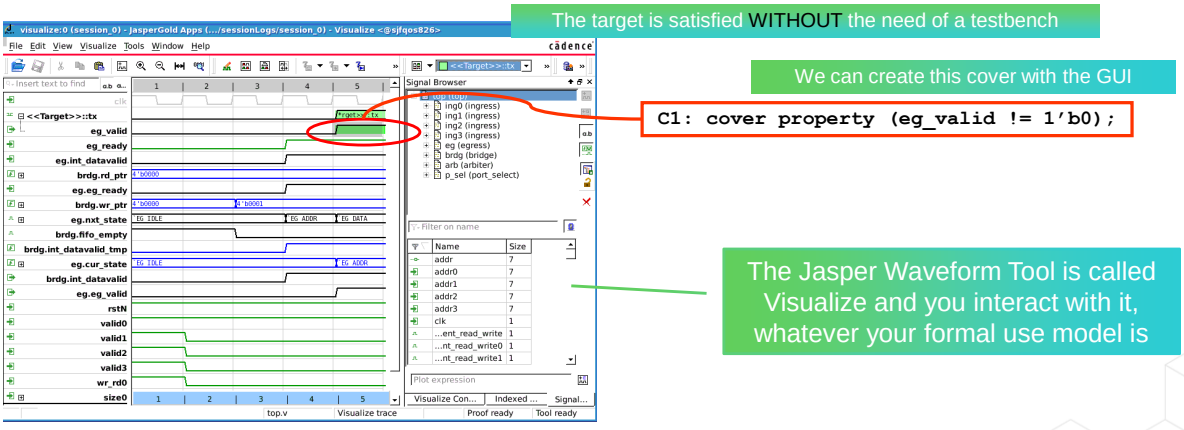
This document is for the sole use of Kun Wang of Seas

Design Exploration: The Simplest Use of FPV

We don't even need to write any properties for Jasper to be useful

A cover in formal means "show me an example of this happening"

One does not need to create a testbench or think about what stimulus is needed for the cover to be hit



 © Cadence Design Systems, Inc. All rights reserved.

cadence

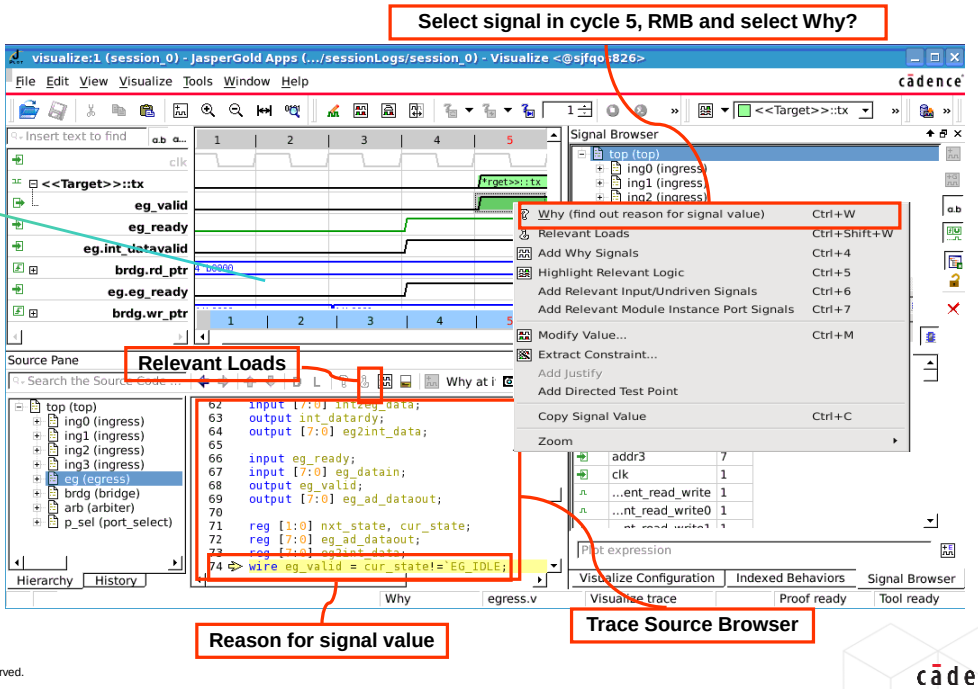
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Features: Why?

One can even modify the waveform observed by dragging signal values with a mouse pointer



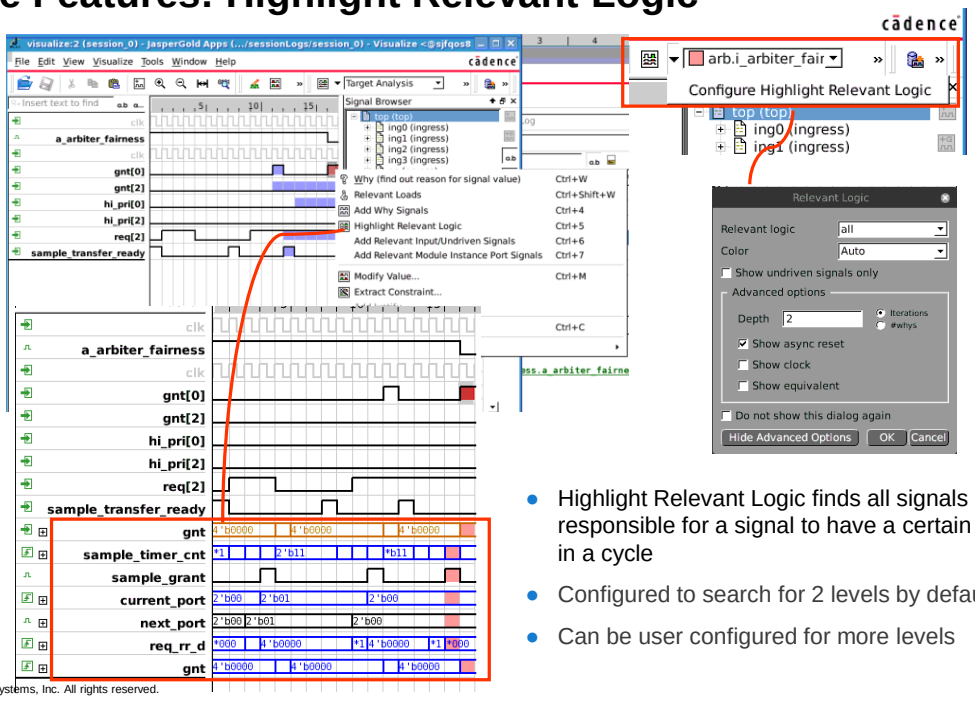
 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Features: Highlight Relevant Logic



- Highlight Relevant Logic finds all signals responsible for a signal to have a certain value in a cycle
- Configured to search for 2 levels by default
- Can be user configured for more levels

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

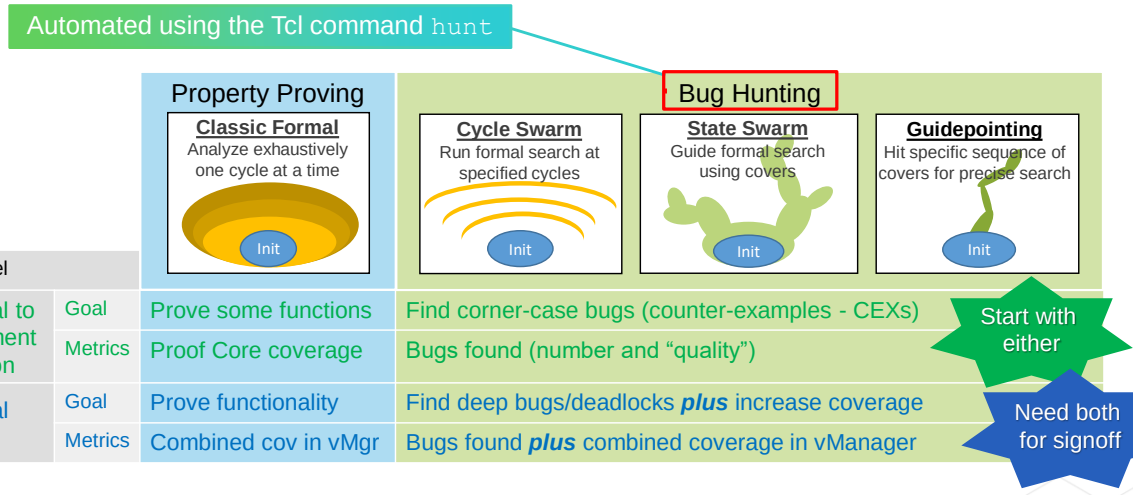
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Jasper FPV Use Models: Bug Hunting

Property Proving is exhaustive

Bug Hunting is not exhaustive – a.k.a. Semi-Formal – One can find bugs but not prove properties



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

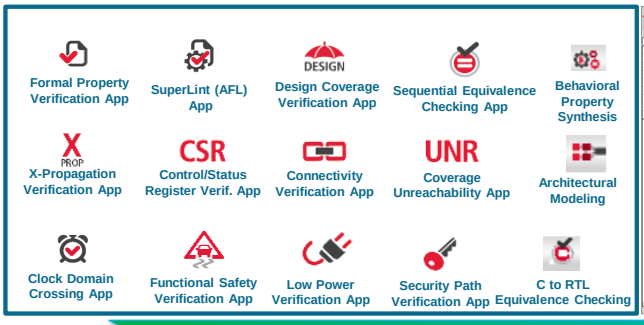
© Cadence Design Systems, Inc. All rights reserved.

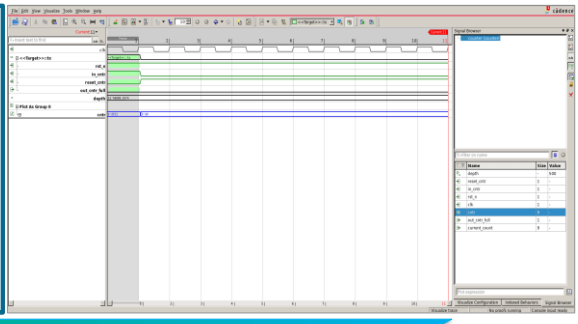
This document is for the sole use of Kun Wang of Seas

Jasper Formal Verification Apps
Automating Common Verification Tasks

Solve **specific verification problems** with targeted Jasper Apps

Highly interactive **formal debug** transforms to fit the App





Broad **formal engine** and infrastructure

- Assertion Based Verification IPs for AMBA and other common protocols
- Programmable Interface via Tcl
- ProofGrid™ Manager assigns best engine for task

 © Cadence Design Systems, Inc. All rights reserved.

cadence

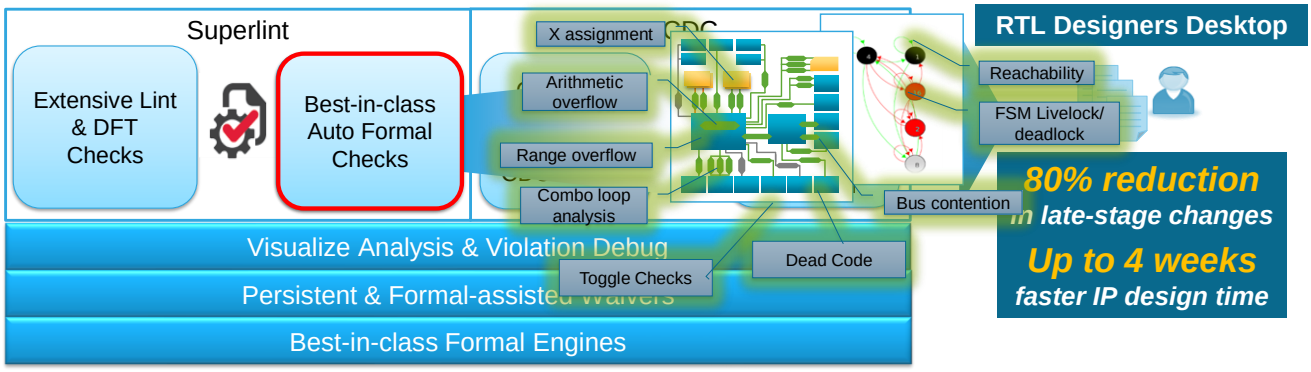
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Jasper RTL Designer Apps: Superlint and CDC

Shift-left: remove bugs before handoff to verification and implementation



- Common Jasper front-end with leading System Verilog support & capacity
- Leverages formal technology to reduce noise and automate waivers

125 © Cadence Design Systems, Inc. All rights reserved.

cadence

80% reduction – endorsement from Arm when we launched.

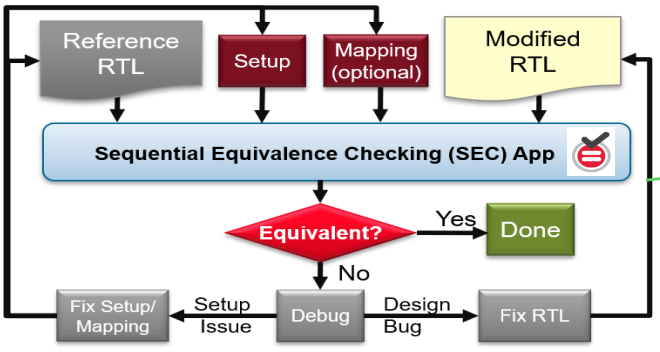
Up to 4 weeks per IP – endorsement from STMicro.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Sequential Equivalence Checking App

Accelerates design convergence



Answer the question: Are these two versions of the design functionally equivalent?

Sweet-spot use cases:

- Clock Gating Optimization
- Pipeline Retiming
- Incremental verification, so after a small change the whole verification suite doesn't need to be re-run

 © Cadence Design Systems, Inc. All rights reserved.

cadence

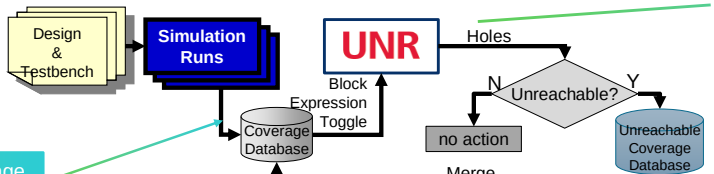
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

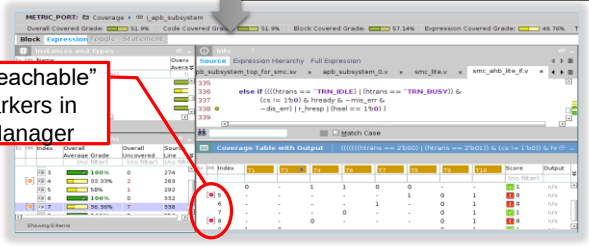
Coverage Unreachability App

Saves simulation users weeks of time and effort for verification closure



Answer the question: Do I have simulation coverage holes because my testbench doesn't exercise the design or because the coverage is unreachable, ever?

Inputs: simulation coverage database and RTL



"Unreachable" markers in vManager

Output: Unreachable cover points database

Integrated with vManager ™ to clearly show unreachable coverage points

Resilient compilation with Xcelium

Supports all Xcelium™ modeling languages and setup

 © Cadence Design Systems, Inc. All rights reserved.

cadence

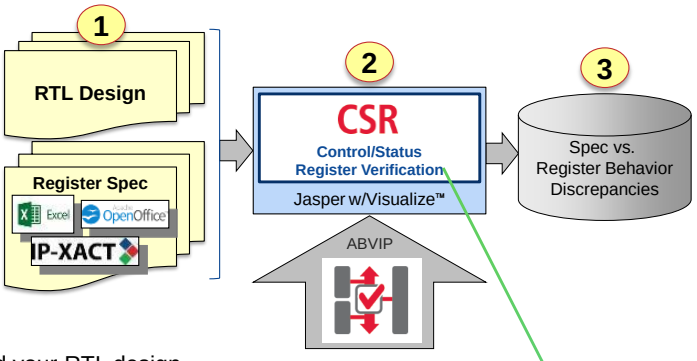
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

CSR Verification App

Exhaustive verification with reduced set up & run times



1. Inputs: register spec and your RTL design
2. Run Jasper Control/Status Register Verification App
   - App automatically derives & generates all properties
   - Automatically runs formal engines to prove all properties
3. Output: CEXs show discrepancies between spec and RTL

Answer the question: Do all of my register accesses always obey the access policies in my spec?

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

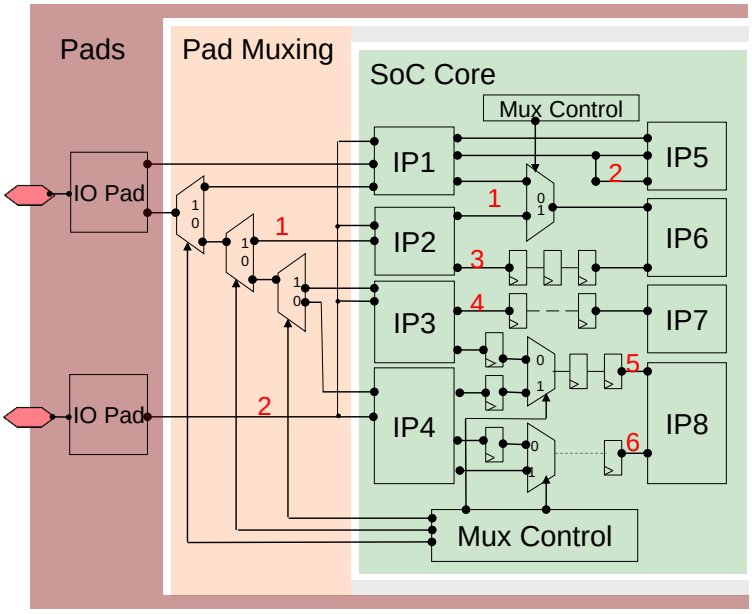
This document is for the sole use of Kun Wang of Seas

Connectivity Problem: A Tangled Web

Connection Types

Combinational
1. Direct
2. Conditional

Pipelined
3. Fixed latency
4. Variable latency
5. Fixed latency and multiplexing logic
6. Variable latency and multiplexing logic



129 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

The Key: The 3 Cs

Proving all connections in the chip are correct

- In a typical chip there may be 10,000+ connections to verify

Traditional simulation methods lack efficient way to exercise the connections

- Significant manual effort to apply stimulus → **Controllability**

Very difficult to assess coverage in a simulation environment

- Have you really exercised all combinations? → **Coverage**

Finally, it's a big challenge to identify whether all connections are tested by the testbench

- Do you have checks for all connections? → **Completeness**

 © Cadence Design Systems, Inc. All rights reserved.

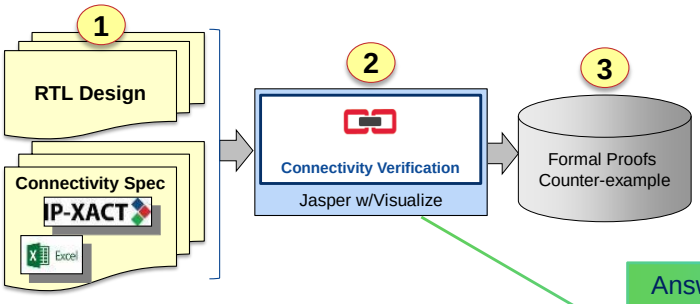
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Connectivity Verification App



Answers the question: Do all of my connections match my specification, and did I specify all connections?

Fast, exhaustive verification with lower setup & maintenance

1. Inputs: static & temporal connectivity spec and SoC RTL
2. Run Jasper Connectivity Verification App
   - App automatically derives & generates all properties
   - Automatically runs formal engines under-the-hood to prove all properties
3. Output: any CEXs are connectivity errors

 © Cadence Design Systems, Inc. All rights reserved.

cadence

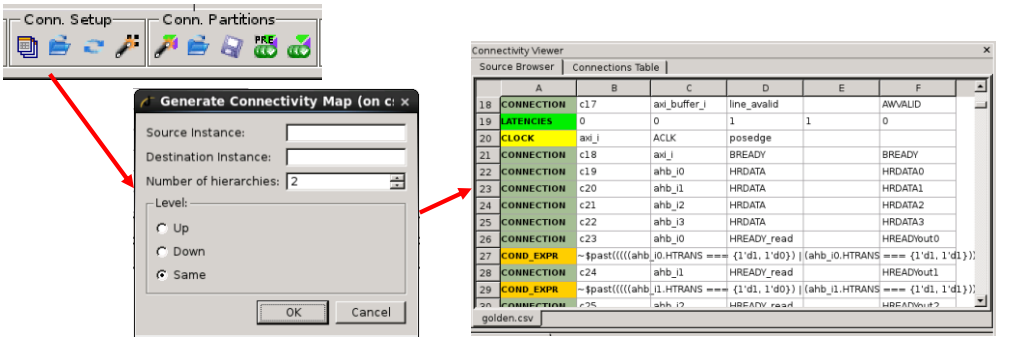
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Built-In “Reverse Connectivity” Capability

Generate a connectivity map from a golden RTL and re-verify it in a revised RTL



(Add other configurations accessible from Tcl)

 © Cadence Design Systems, Inc. All rights reserved.

cadence

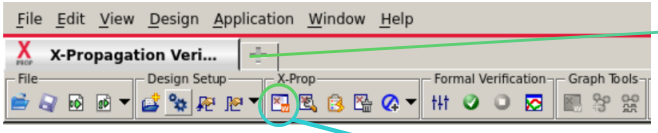
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

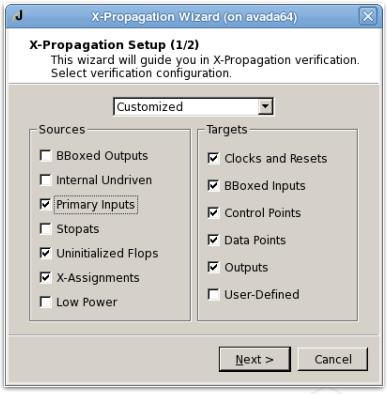
This document is for the sole use of Kun Wang of Seas

X-Propagation Verification App

Automatic formal checks with X-aware analysis & debug



Answer the question: Can Xs reach destinations in my design where I don't want them?



Analysis reflects actual silicon behavior

- No missed bugs due to either X-optimism or X-pessimism

Customized GUI and advanced reporting features enable rapid X-propagation fault analysis

Requires no knowledge of formal or SVA

 © Cadence Design Systems, Inc. All rights reserved.

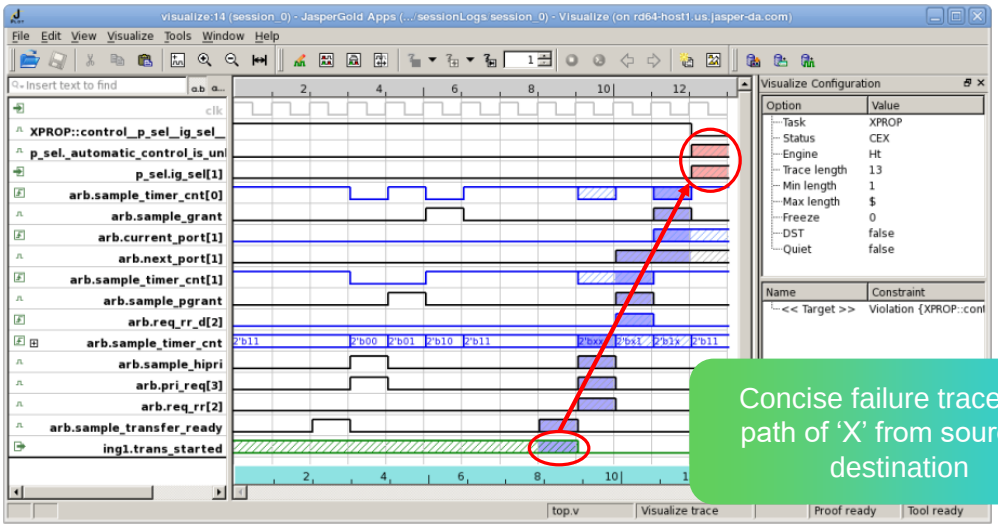
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

XPROP CEX Debug Using Visualize



 © Cadence Design Systems, Inc. All rights reserved.

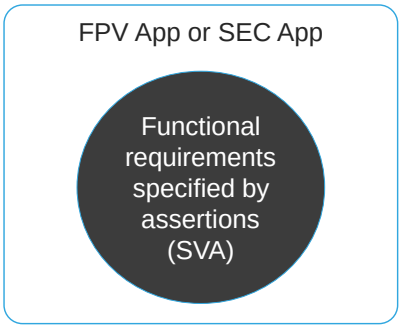
cadence

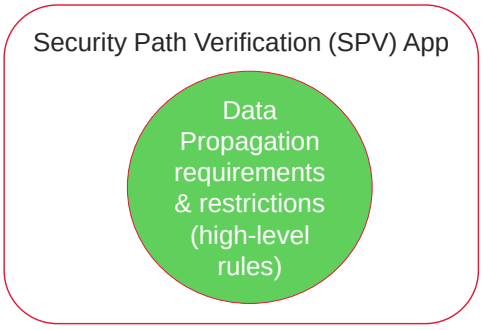
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Security: Functional Versus Data Propagation Requirements





Examples

- CPU must not be interrupted if running secure code
- System must be reset if an environment monitor trips
- FSM must never transition to SECURE after reaching TEST or DEBUG states

Examples

- Data in secure area must not be visible by CPU if it is not in secure mode
- Secure register must not be written by non-secure agent

Other Jasper apps relevant to security:

- CSR app verifies the integrity of register access policies
- FSV app models direct attacks on internal HW circuitry (laser, radiation, electrical, thermal, overclocking…)

 © Cadence Design Systems, Inc. All rights reserved.

cadence

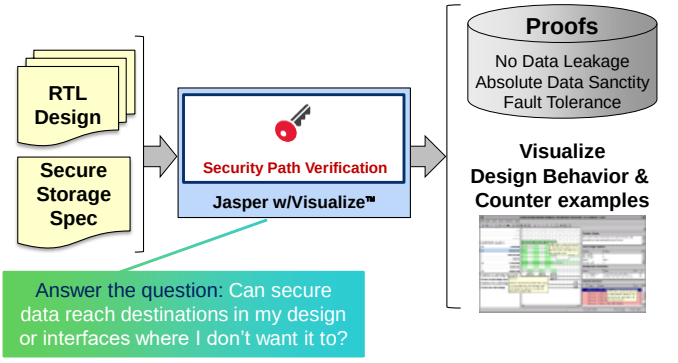
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Security Path Verification App

Formally Prove Secure Data Cannot Leak



1. Inputs: RTL and spec. of the secure storage element
2. Run Jasper Security Path Verification app
   - App automatically derives & generates all properties
   - Automatically runs special path analysis, optimized formal engine under-the-hood
3. Output: CEXs show data leakage, violations of data sanctity, or vulnerabilities to tampering/faults

 © Cadence Design Systems, Inc. All rights reserved.

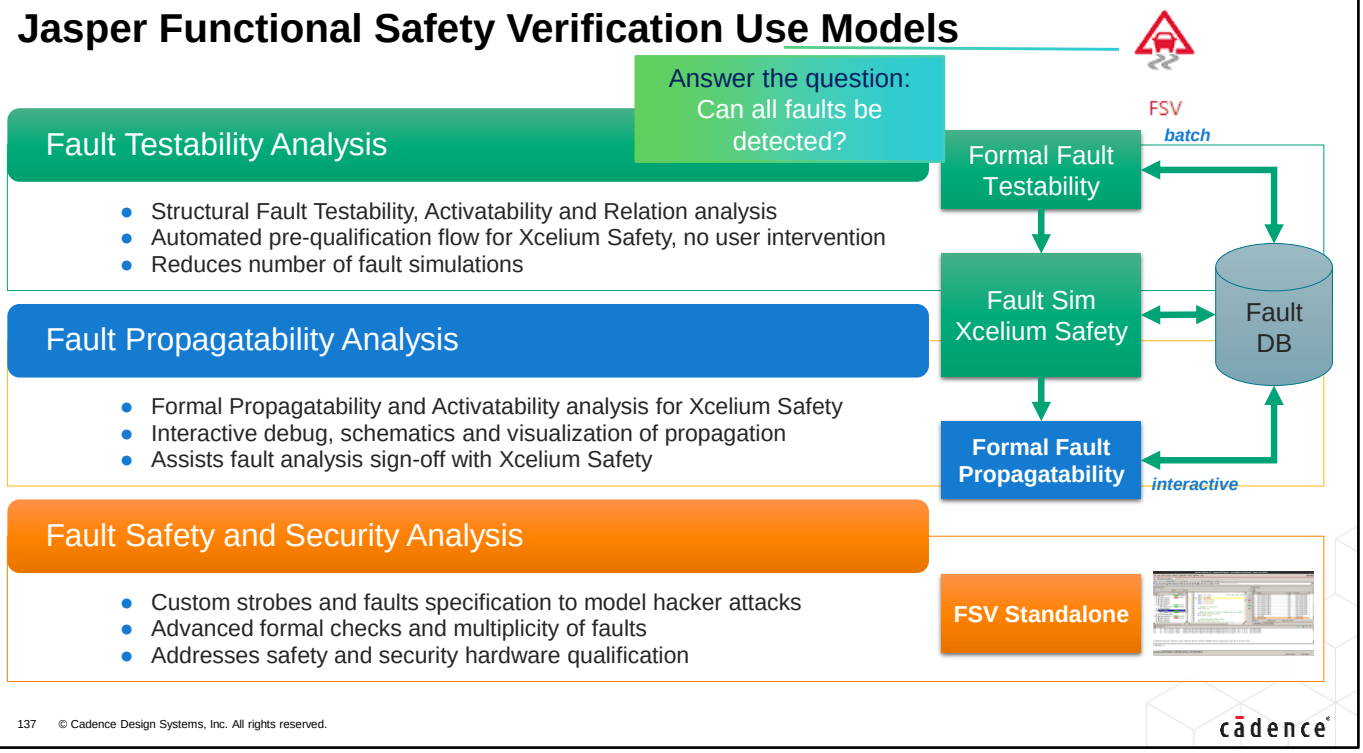
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Jasper Functional Safety Verification Use Models



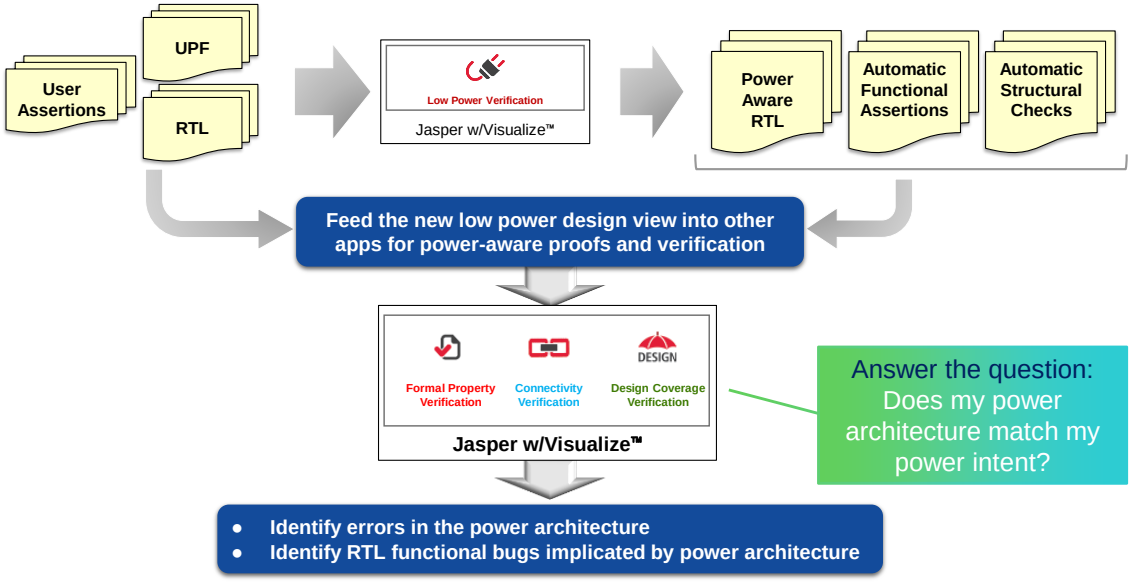
137 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Low-Power Verification



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

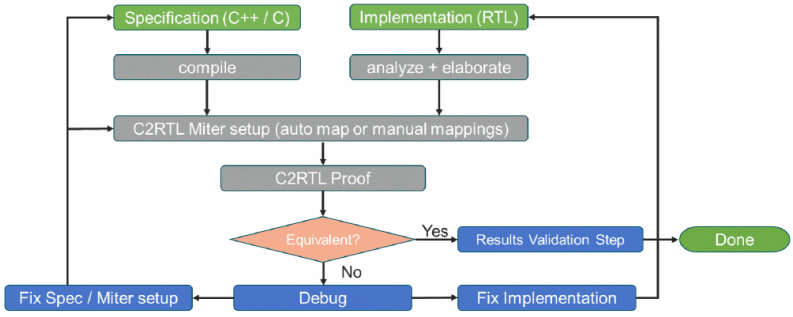
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

C2RTL Verification

C2RTL App has a variety of use models. Any functional comparison activity involving a C++ model can be considered a general C2RTL use case:

- **RTL design** – **Datapath verification:** This use model aims to ensure that the data manipulations and datapath control are both functionally correct. A C++ model is the golden (reference) model against which we verify an RTL implementation.



139 © Cadence Design Systems, Inc. All rights reserved.

C to RTL is useful for Datapath verification where manipulation of data happened.

Its basically a sequential equivalence checking but with the C++ golden reference model instead of RTL golden model.

As FPV is inefficient when verifying Data path which contains data manipulations, C2RTL more suited

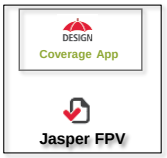
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Jasper Coverage App
What Formal-Specific Questions Need to Be Answered?

What code or functionality has been “exercised” by the formal testbench?

- During classic formal or bug hunting
- Covers may be reached, undetermined, unreachable



Answer the question: Is every part of my design exercised meaningfully by an assertion or a cover?

Is my formal checking complete?

- Properties may be proven, but how much of the design do they cover?
- Complete checks, but not be fully proven – have the engines achieved sufficient design coverage to detect a bug?

Use jg -cov with FPV (or other apps) to answer these questions!

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Summary

- Formal is a very different paradigm from simulation
- One needs to be prepared for a complete change in mindset
- Formal does not have a simple “one size fits all” use model
- New use models are always being created or modified as the technology progresses
- Apps exist to fix common design and verification problems
- Many Apps are designed with the intent to minimize the amount of SVA and Formal expertise required
- Each separate App is a specialist in its own right
  - No one knows all of them in detail
  - They have their own Rapid Adoption Kits (RAKs) on http://support.cadence.com/Jasper
  - Digital badges are available for many apps

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 6

Jasper™ Apps Usage: Formal Analysis Flow (DLFA)

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Module Objectives

By end of this module, you will be able to:

- Set up and operate Jasper™ for verifying user-defined properties through Formal Property Verification (FPV)
- Demonstrate the general flow of Formal analysis
- Identify problems that are well-suited for formal analysis success
- Identify and debug overconstrained environments
- Locate all available Jasper help resources

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Expectations of This Module

- The module aims to efficiently cover a broad subject area.
- It includes both highly technical methodology slides requiring detailed explanations and screenshot slides best demonstrated live by the instructor to highlight key points.
- The lecture will blend demonstrations with slide presentations where appropriate.
- Due to the module's length, there will be a scheduled break halfway through.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Agenda

- Context of Jasper in the Overall Verification Process
- Introduction to Formal Analysis (FA)
- Focus areas
- FA flow phases
  - Phase I: Setup and Initialisation
  - Phase II: Design Bring-Up Overview
  - Phase III: Formal Property Verification (FPV) & Debugging
- Recommendations
- Constraint Management
- Where to get help

 © Cadence Design Systems, Inc. All rights reserved.

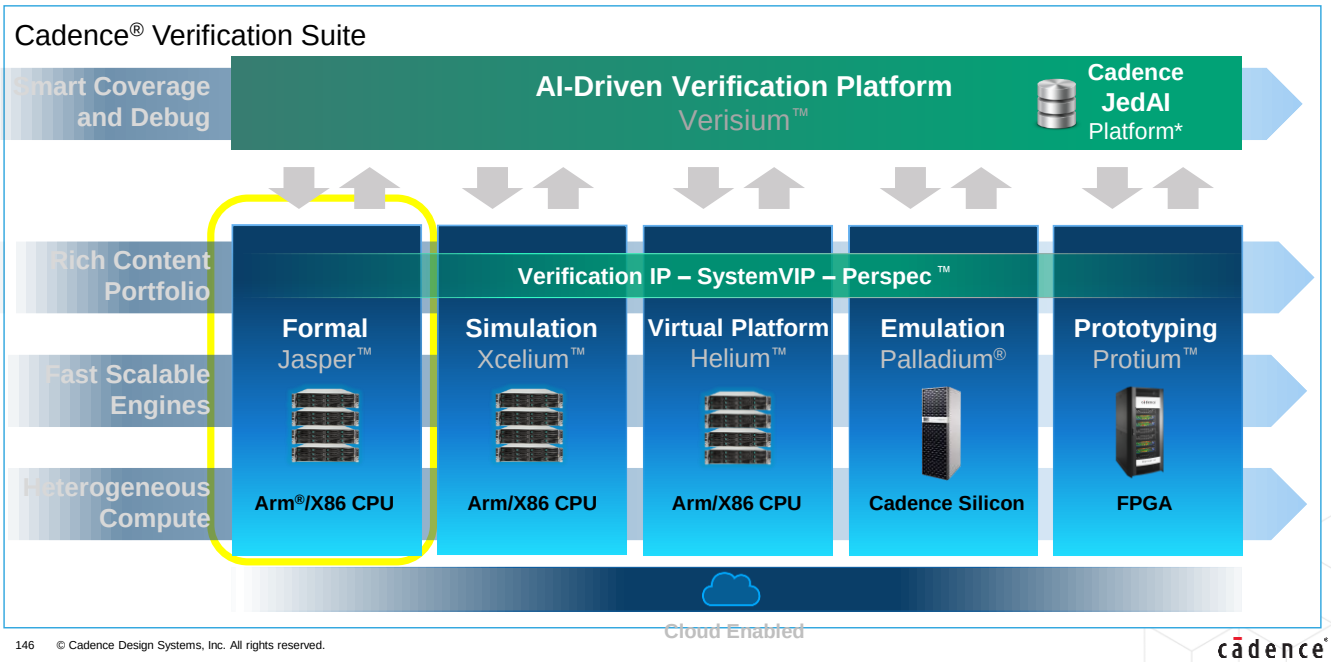
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Jasper Is Not a Point Tool



146 © Cadence Design Systems, Inc. All rights reserved.

Purpose of this slide is to show that formal analysis is just part of the solution to a verification problem. It does not directly replace any other methodology or tool, but it can make significant savings, for example use of more can drastically reduce the amount of simulation which needs to be done.

It is another tool which the verifcation engineer has to solve todays complex verification problems.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Analysis (FA) in the ABV Flow

Designers write assertions along with RTL

FA used to start verification before the testbench is ready

Using Jasper, designers get immediate rewards for writing assertions

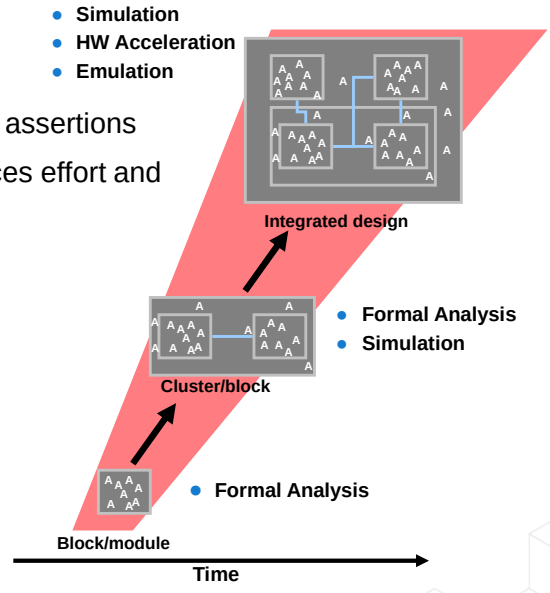
Integration of clean blocks and modules significantly reduces effort and debug time

Same assertions are used in:

- Simulation
- Acceleration/emulation
- Cluster and full chip level

Unified ABV flow results in:

- Improved productivity and quality
- Overall reduction in verification time/effort



147 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

What Is Formal Analysis?

- Maximizes productivity when using FV
- Guides users on what actions to take and what to avoid
- Helps to achieve optimal ROI for FV and overall verification effort
- It is an important part of the overall ABV project methodology

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Typical Formal Analysis Use Model

Target users are designers working on RTL

Main steps in flow:

- Write RTL
- Identify the focus areas for FA based on expected ROI
- Write assertions and assumptions
- Run the FA tool
- Debug failures:
  - Fix the spec, the RTL, and/or properties
- Iterate the process

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

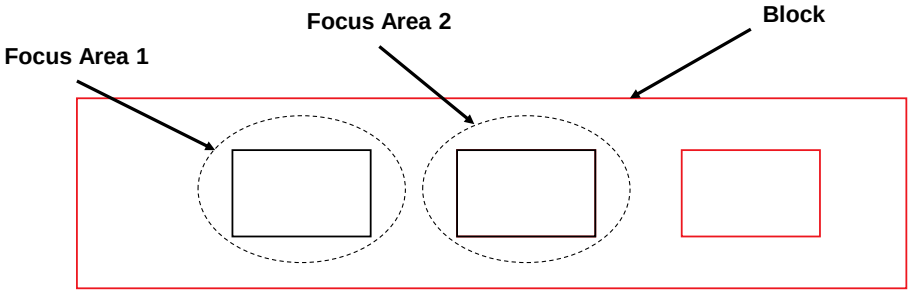
What Is a Focus Area?

Not all design blocks and types are suitable for Formal Analysis

Focus area selection is a process of identifying suitable design blocks that will provide an ROI

- How much of the RTL we run FA upon depends on the verification goals

Within a block of interest, there can be multiple focus areas



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

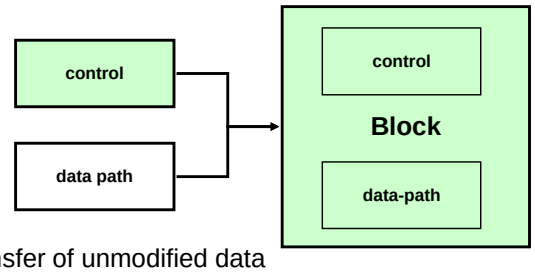
Selecting a Focus Area

Identify:

- Likelihood of finding bugs in DUV
- Size and structure of DUV
- Probability of bugs continuing to appear in the DUV as RTL evolves
- Simplicity of interface to the DUV
- Difficulties achieving good ROI using simulation

Spend your effort wisely

- When verifying at the module level, pick the control module:
  - Higher ROI can be achieved on control paths
  - Many data path errors are caused by control path errors
- Some datapath FPV may be efficient if it is checking the transfer of unmodified data
  - See Jasper Scoreboard RAK



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Good Focus Area Design Types

- Non-exhaustive list
- These design types typically score high as good focus areas
- A good preliminary indicator
- Analysis confirms the specific block as a good focus area

Memory Controller	Arbiter
Generic complex state machine	Cache coherency block
Non-packet-based interface (for example, PCI, AHB)	DMA controller
Interrupt controller	Bus bridge which does not transform data

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Typically, Good Focus Areas

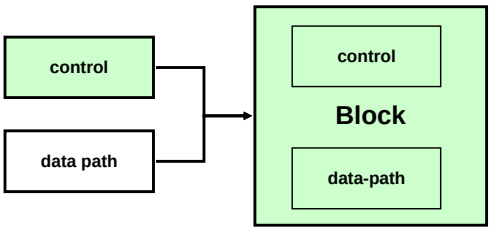
Control path is best suited to Formal Analysis

Data path where no data transformation occurs

- Look for a Datapath that does not:
  - Transform data
  - Perform mathematical operations on the data
- FA is more likely to be independent of data width
  - Can analyze one bit of the bus at a time

Not all data transformations are bad focus areas

- For example, an ECC block is a good focus area
  - ECC is hard (very time-consuming) to verify in simulation



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Example: Arbiter Focus Area

Arbiters are typically good focus areas

Interface complexity is low:

- Simple req/ack protocol

High chance of having bugs

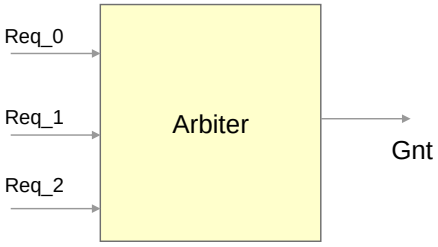
- Complex logic

Low formal complexity

- Low FF count

Hard to fully verify with dynamic simulation

- Fairness and liveness properties



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Typical Poor Focus Areas

Datapath where data transformation occurs

Does not allow the datapath to be scaled down in size

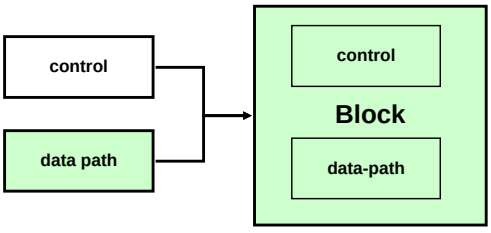
Performs the mathematical operation on data

This is known to provide high formal complexity, for example:

- Multipliers
- Mpeg decoders

Packet-based protocols

- Normally cannot be scaled down in size
- Takes many cycles to get a single data item into the DUT



 © Cadence Design Systems, Inc. All rights reserved.

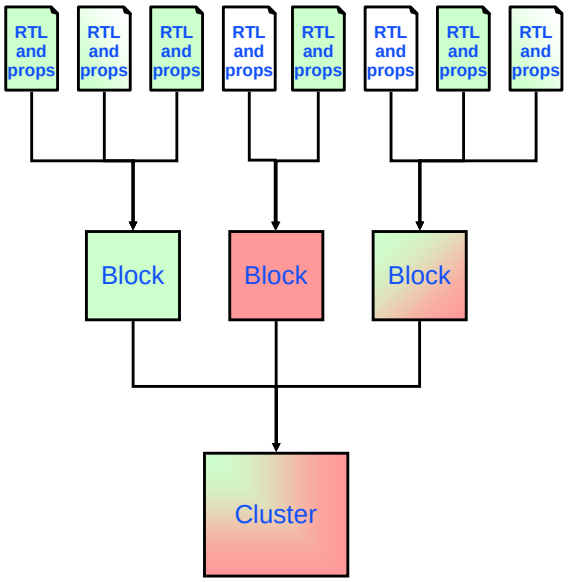
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Analysis Application Illustration



- Apply formal analysis at the module level.
- Apply formal analysis on focus areas.
  - Not on everything
- Apply formal analysis and simulation on block level.
- Apply simulation on a cluster level and above.

Legend

Formal analysis is applied.
Simulation is applied.
Both tools are applied.
Neither tool is applied.

156 © Cadence Design Systems, Inc. All rights reserved.

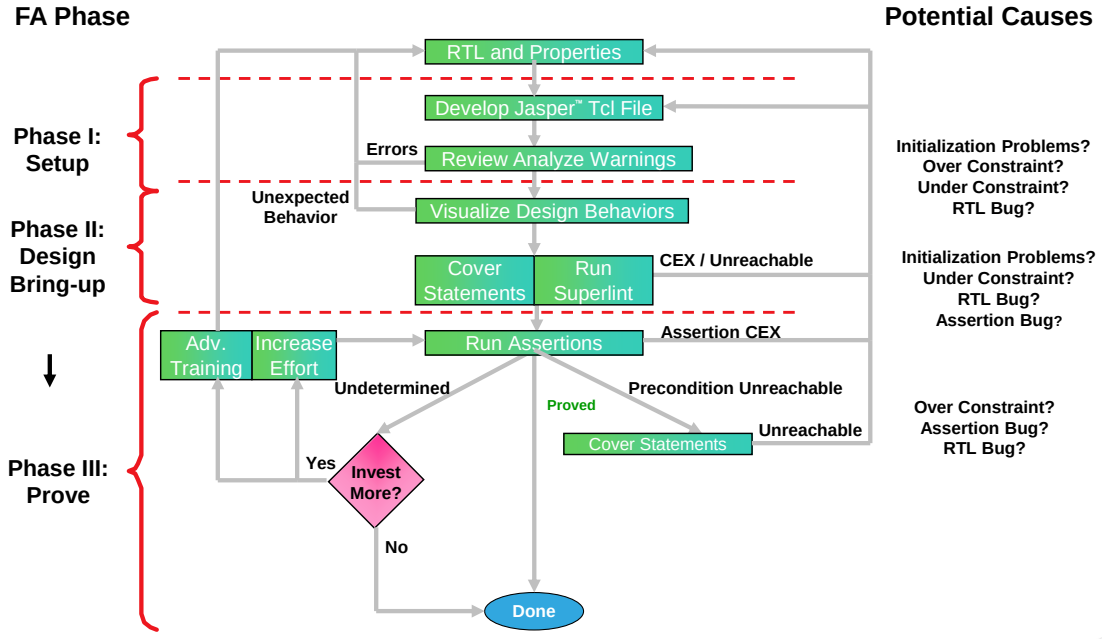
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Flow Using Jasper



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Submodule 6.1

Jasper™ Apps Usage: Phases in Formal Analysis Flow

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Analysis Flow Overview

This is a simplification of how Jasper™ is used within a FV methodology

In practice, there are more loops and iterations

These are customized on a per-customer and per-application basis

Phase III is running the assertions

- This is the primary focus here

The diagram shows running multiple steps in Phase I and II prior to running Phase III

Typically, after Phase I and Phase II are clean one doesn‘t need to do these steps again

- Unless there have been changes to the existing RTL and/or constraints

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase I: Setup Overview

Goals

- Develop Jasper Tcl file
  - Make sure RTL and properties can be parsed and elaborated
- Review RTL checks

Inputs

- Jasper Tcl file
  - Location of RTL
  - Location of additional assertion files
  - Location of an assertion library
  - Assertions and assumptions

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase I: Create a Jasper Tcl File

- All Jasper related commands can be put in a Tcl file

```
% jg test.tcl
```

  - Specify multiple design files in a file list

```
% analyze -sv09 -f design.f
```

- Jasper Tcl file consists of
  - Verilog files, libraries, etc.
  - Include directories
  - Property files
  - Other options intended to be used in every run
- Jasper Tcl file advantages
  - Invocation flexibilities, organization, save options
- Naming convention: `jg_<top level>_<run configuration>.f`

design.f

```
// RTL files:
../rtl/top.v
../rtl/cpu.v
../rtl/reg.v
-y ./rtl
+incdir+rtl
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

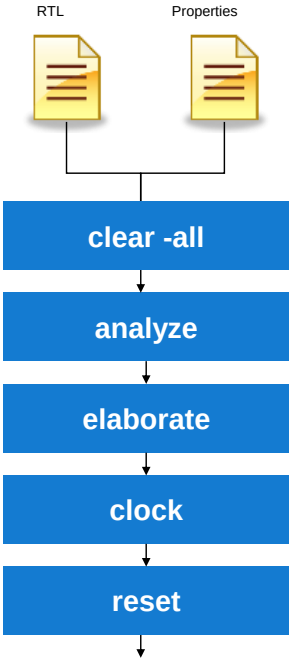
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase I: Jasper Tcl Command Flow

Basic command flow in Jasper is as follows

- Step 1: `clear -all`
  - `Configure -default` can be added with clear -all
  - Rehash your verification session
- Step 2: `analyze`
  - Parse your source code and properties
- Step 3: `elaborate`
  - Synthesize your design
- Step 4: `clock`
  - Define Clocks in your design
- Step 5: `reset`
  - Define reset pin or expressions in your design



162 © Cadence Design Systems, Inc. All rights reserved.

Jasper uses the term synthesize to mean the elaboration and creation of a mathematical model of your design.

Properties are then checked against this model.

In this context the term synthesize does not mean make a gate level model from a technology cell library.

However, there are almost identical restricts on the RTL coding styles which are required in order to build a mathematical model for formal and synthesis in the sense that a design model using cell library components is created with the intent to place and route in a physical device.

Hence why the term synthesize is used.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase I: Analyzing Design and Property Files

`analyze` is the compilation step in Jasper

- Analyzes design files and property files for Verilog, SystemVerilog, VHDL, SVA
- Analyzes requirement files with embedded SVA
- Checks for syntax errors in design and property files
- Examples

```
% analyze -verilog filename.v
% analyze -vhdl filename.vhd
% analyze -vhdl {mylib::/work/my_design.vhd}
% analyze -sv09 filename.sv
% analyze -v2k -psl my_vunit.v.psl
```

- Do `help analyze` to see complete help and more options

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase I: Elaboration

`elaborate` is the synthesis step in Jasper

- If the option `-top` is not used, `elaborate` automatically selects a top-level
- Examples

```
% elaborate
% elaborate -top top_level_module
% elaborate -vhdl -top entity(architecture)
```

- The top level decides the language flavor on the command line for entering expressions
- Automatic mixed language support

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase I: Elaboration (continued)

- Blackboxing
  - `-bbox true` automatic blackboxing for undefined modules
  - `-bbox_m/-bbox_i` blackboxing modules and instances
  - Tool will automatically blackbox certain elements – large arrays / memories – complex arithmetic (mult, div, etc.)
    - This can be overridden – see manual for options – but not recommended since these are costly for FV engines.“

```
% elaborate -bbox_m *ram* -top mytop
```

- Parameter
  - Define Verilog parameter and VHDL generic values on the top level

```
% elaborate -parameter {width} {32} -parameter {depth} {16}
```

- Do `help elaborate` to see complete help and more options

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase I: Single Clock Specification

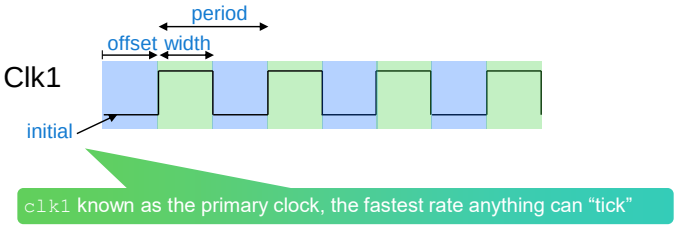
Tcl **`clock`** command specifies the characteristics of how the clock is driven during a formal analysis run

It can be viewed as a clock waveform constraint defined by the phase and the factor

If no clock sequence is specified, then Jasper uses the default values

Single clock can be added by

```
clock clk1
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

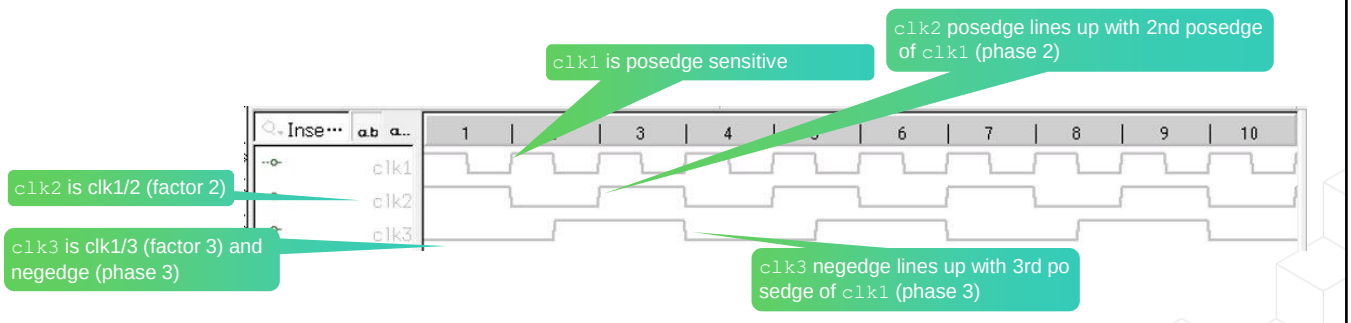
This document is for the sole use of Kun Wang of Seas

Phase I: Multi Clock Specification

If design is having multiple clock than it can assign with different factors

Multi clock in jasper

```
clock clk1
clock clk2  -factor 2 -phase 2
clock ~clk3 -factor 3 -phase 3
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase I: Reset Specification

Why reset before proof?

- Brings the design to a known state
- Avoids unreachable failure states

Command reset defines the reset condition in Jasper

- The command is applied globally and not cumulative

Reset Specification

- Single reset

```
% reset !rstN
```

  - During reset analysis `rstN` will be active (`1'b0`)
  - A simulation is run until no signals change anymore
    - i.e., we know that reset has propagated everywhere
  - During **all** of the formal verification, `rstnN` will be inactive (`1'b1`)
- Multiple resets

```
% reset !rst1N !rst2N
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Reset analysis is a simulation under the hood.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase I: Reset Specification

Other possibilities to define the reset

- Reset sequence in a file

```
% reset -seq reset.seq
```

- SHM/VCD/FSDB file with reset values for flops

```
% reset -seq reset.vcd
% reset -seq reset.fsdb
% reset -sequence -shm reset.shm \
  -start_time 5 -end_time 8000 -time_scale ns
```

- Reset file with reset values for a specific set of flops

```
% reset -init_state init.seq
```

```
// reset.seq file format

// <signal> <value>
// Number of clocks

rstN 1'b0
sigA 4'b1111
sigB 2'b01
sigC 1'b1
2
rstN 1'b1
```

```
// init.seq file format

// Individual Declaration
    sigA 4'b0
    sigB 4'b0
    sigC 4'b0

// Wildcard Declaration
    sig* 4'b0

// All Signals Declaration
    * 0
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase I: Reset Analysis

State of the design out of reset can be obtained using **get_reset_info**

```
% prove -all
....
INFO (IPF036): Starting proof on task: "<embedded>", 75 properties to prove with ...
INFO (IRS029): Starting reset analysis: phase 1 of 4.
INFO (IRS030): Running reset analysis phase 2 of 4.
INFO (IRS031): Running reset analysis phase 3 of 4.
INFO (IRS020): Starting the reset analysis simulation with a limit of 100 iterations ....
INFO (IRS024): Reset iterations 0 to 4 analyzed.
INFO (IRS018): Reset analysis simulation executed for 3 iterations.
Assigned values for 814 of 950 design flops, 0 of 0 design latches, 92 of 92 internal
elements.List of Registers: 60
....
% get_reset_info
List of Registers: 71
-----------------
arb.current_port
  Value at reset: 2'b0
arb.gnt
  Value at reset: 4'b0
...

% get_reset_info -value_x
List of Registers: 5
-----------------
eg.eg_ad_dataout
  Value at reset: 8'bXXXXXXXX
ing0.wdata_reorder
  Value at reset: 32'bXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
ing1.wdata_reorder……
....
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

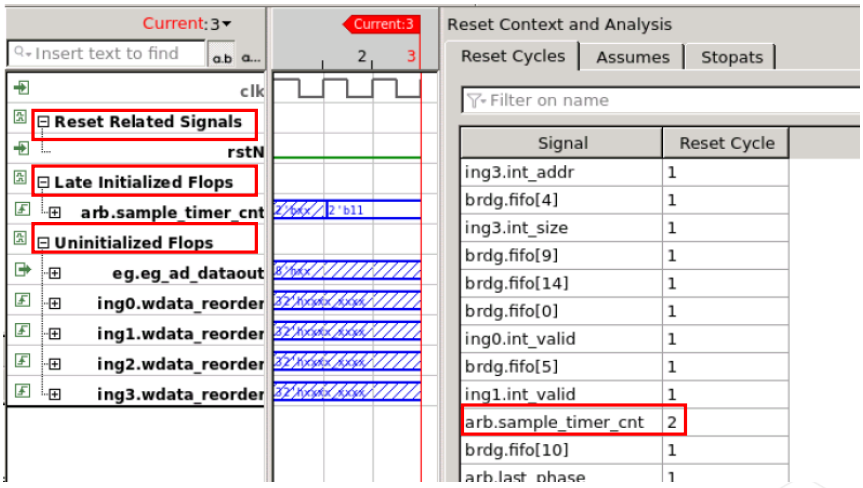
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase I: Reset Analysis Window

Debugging reset cycles is done using `visualize -reset`

- By default, the following is displayed:
  - Clocks for uninitialized flops
  - Reset related signals
  - Late initialized flops
  - Uninitialized flops



171 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Basic Tie-off Constraints

Common to have to tie off certain pins/registers as part of the setup

Scan mode, test mode, etc.

Use **assume** command to apply constraints

Use **-env** switch to apply constraint globally to all tasks and reset analysis

- Use **stopat** command to cut the driving logic of the signal you need to constrain (e.g., config register)

Example

Global assumption (across all tasks and during reset analysis) – apply the constant value

% assume –env {scan_mode == 1'b0}

Or task-based one – apply the constant value

% assume {func_mode == 2'b01}

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase I: Compilation Messages

Compilation Messages are a large collection of automatic checks:

- Independent of properties
- Range from simple lint to complex structural checks

- GUI Support:
  - Description, severity, and detailed explanation
  - Cross-reference to source code
- Message Customization
  - Messages, severity levels, parameters
  - checks can be included, excluded, and more

ROI is very high if the problem can be detected by Lint Messages

- Problems found this way are typically simpler and easier to debug
- We recommend analyzing Lint Messages before analyzing properties

We will discuss Superlint later

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase I: Log Files

- User messages generated by the tool

  `<cwd>/jgproject/jg.log`

- Console process log capturing tool events

  `<cwd>/jgproject/jg_console.log`

- Tcl commands used during the session

  `<cwd>/jgproject/jg.cmd`

- Create or use a different project directory at startup

  ```
  jg -proj <my_proj>
  ```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase II: Design Bring-Up Overview

Goals

- Run automatic checks
- Run cover statements
- Visualize design behavior

Inputs

- Parsed and elaborated design from the setup phase
- Tcl file

At this point, be aware that we would typically run these automated checks before proving our user-defined functional properties

- Utilize automated Jasper Apps where appropriate for your problems.
  - XPROP, SUPERLINT, CSR, CDC and, CONN
  - We have separate training for each app; go to support.cadence.com and search Jasper Apps

 © Cadence Design Systems, Inc. All rights reserved.

cadence

So, the goal of doing a design bring-up is to bring your design to a state where you've run these automatic checks and visualize that your design can do the behaviors you expect it to do. So, we can run our user-defined checks, which is phase three.

Utilize automated Jasper Apps where appropriate for your problems.

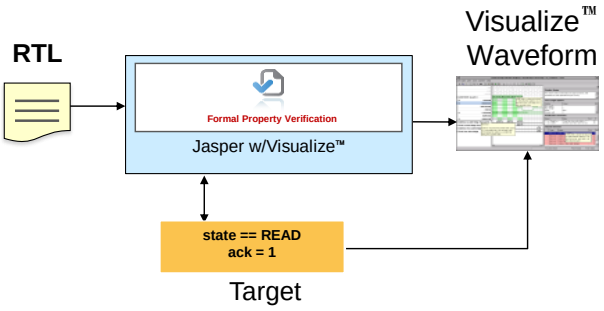
These are the apps XPROP, SUPRELINT, CSR, CDC, and CONN.

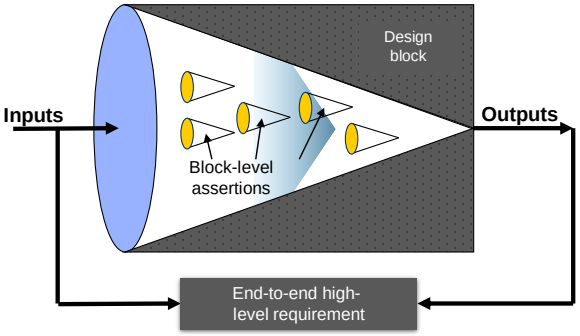
We have separate training for each app. Go to support.cadence.com and search Jasper Apps.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase III: Formal Property Verification (FPV)





- Specify the target and let the formal engines automatically generate the stimulus (“output driven” method)
- Interactively add constraints and extract properties from waveforms
- End-to-end properties provide the highest return on investment
- These end-to-end properties are easier if Block-Level Assertions (helpers) are proven first
  - As asserts are proven, they can be used as assumptions

 © Cadence Design Systems, Inc. All rights reserved.

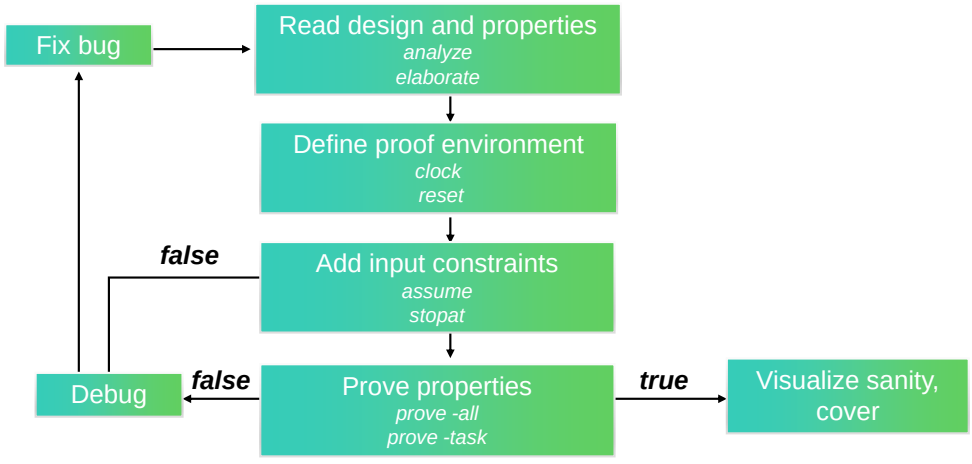
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase III: FPV Use Model Overview



 © Cadence Design Systems, Inc. All rights reserved.

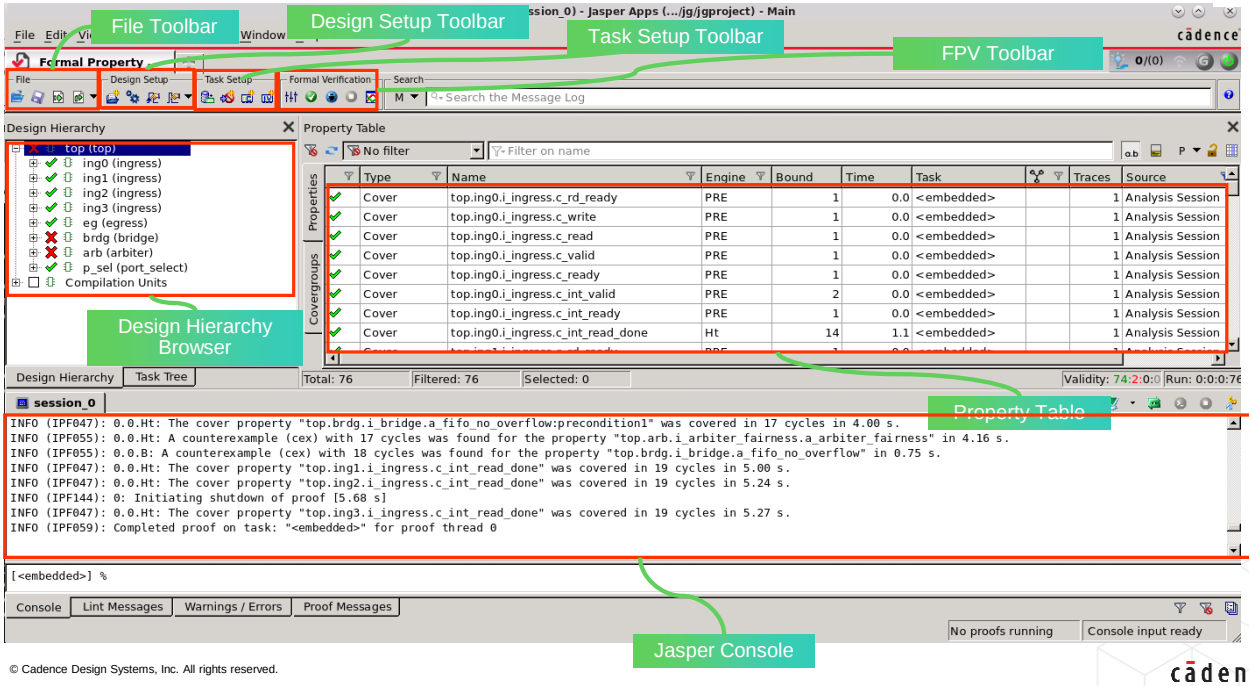
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase III: FPV GUI



178 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase III: Changing Property Designation

Right-click on a property

Select **Enable** or **Disable** to manage properties

- The tool issues corresponding commands:

```
assert -disable my_prop_1
```




```
assert -enable  my prop_1
```



Go from assert to assume directly (or vice versa)

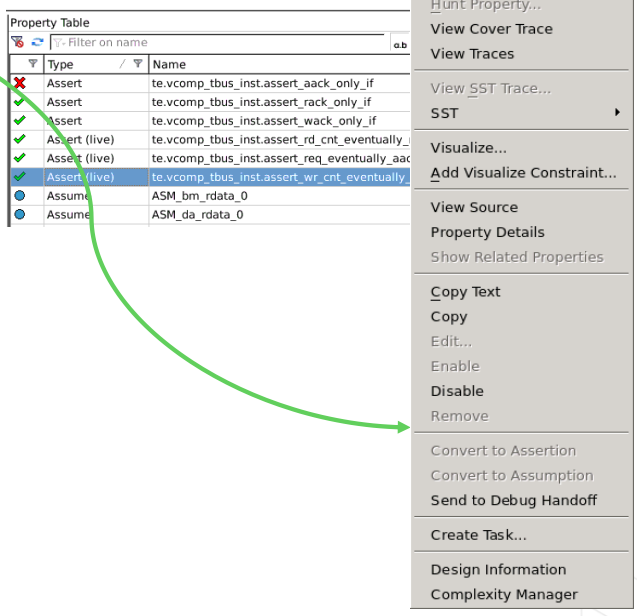
- The tool issues corresponding commands:

```
assume -from_assert my_prop_1
```




```
assert -from_assume my_prop_2
```





179 © Cadence Design Systems, Inc. All rights reserved.

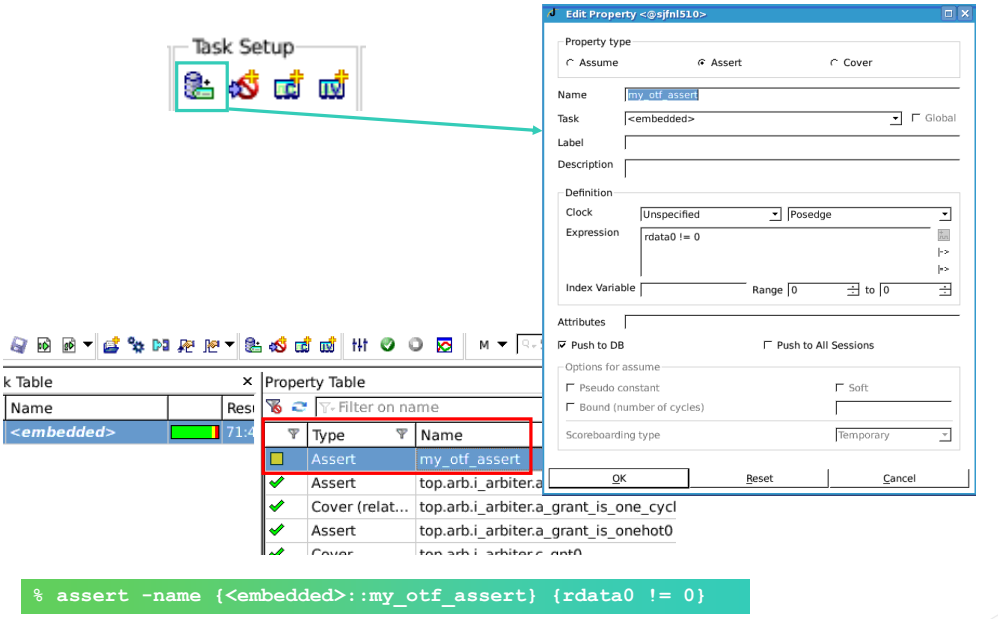
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase III: Adding “On the Fly” Properties




```
% assert -name {<embedded>::my_otf_assert} {rdata0 != 0}
```

 © Cadence Design Systems, Inc. All rights reserved.

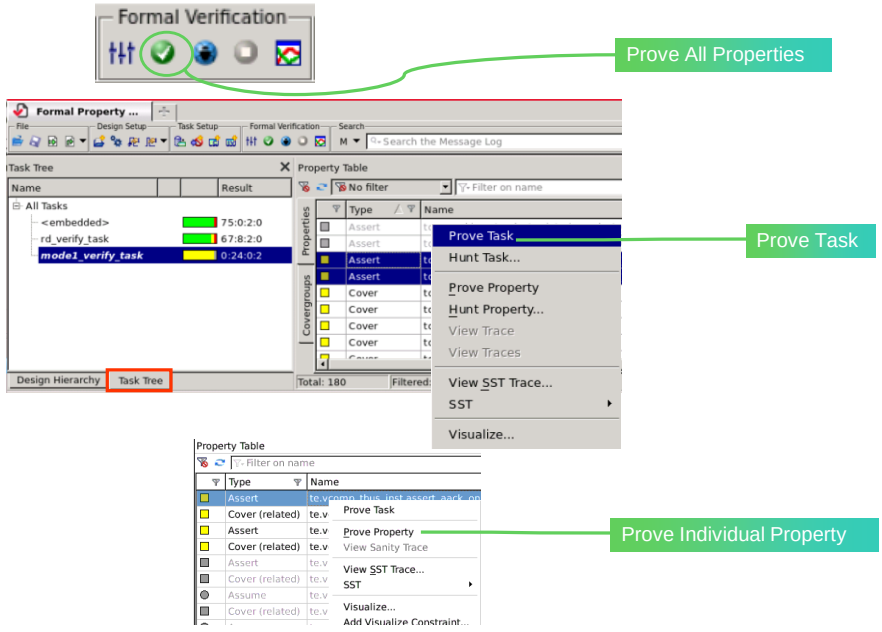
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase III: Proving Properties



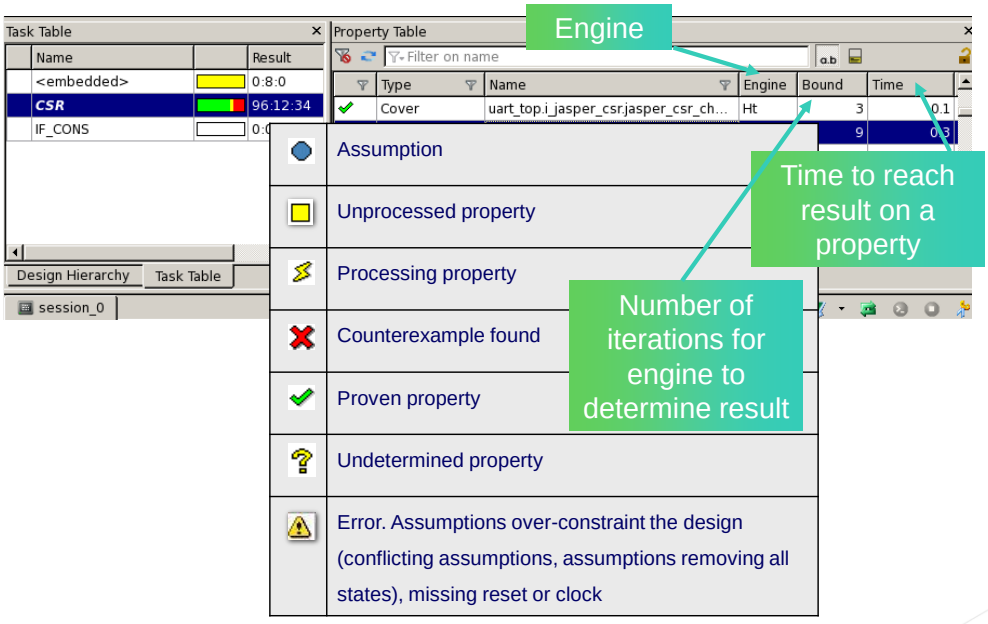
 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase III: Understanding Proof Results



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase III: Results Reporting

- Generate a report of the results of proof in the console pane

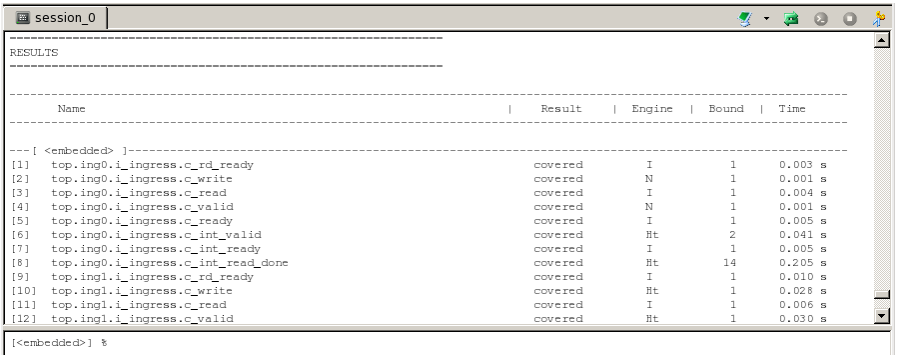
```
% report
```

- Summary report includes basic property list with results

```
% report -summary
```

- Detailed report includes all proof information for every property

```
% report -detailed
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase III: Assertion Precondition

Assertion passes for two cases:

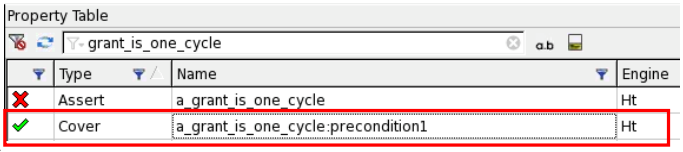
- A is always 0
- If A is 1 and B == 1

```
assert property (A |-> B);
```

Inability to show a path to a CEX state is a PROVED

Incorrect constraints or a design bug can make An unreachable

Very important to understand that a PROVED means little if the precondition for the assertion is unreachable



An automated cover is known as a “related cover”

```
cover property (A);
```

To observe the entire property passing, as opposed to only LHS of an implication

```
% elaborate ... -create_related_covers witness ...
```

Results in the automatically created cover

```
cover property ( A && B);
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Why Should We Choose to See a Witness?

Given this assert, Jasper *by default* creates this cover

```
AST: assert property (A |=> B ##1 C);
```

```
default_rel_cov: cover property (A);
```

`elaborate -create_related_covers witness` creates this cover

```
witness_rel_cov: cover property (A ##1 B ##1 C);
```

Now consider we have this bad assumption

```
BAD_ASM: assume property (A |-> ##1 (A && !A) )
```

The assert is proven, there is no CEX

- `default_rel_cov` is covered
- `witness_rel_cov` is unreachable; hence we know there is a problem
- BTW, check_assumptions - live fails (but I better check that – need test case)

Why isn't `-create_related_covers witness` the default?

- Because it takes more effort

(A && !A) can never be true. Hence the assumption has a hard conflict 1 cycle after A occurs. This means no further state space can be explored 1 cycle after A.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase III: Debugging Preconditions

✗	Cover	ASM_S_HRESP_two_cycle_resp:precondition1	PRE	Infinite
✓	Cover	COV_S_HRESP_cfg_okay	Ht	1

If the precondition can be reached, then:

- Reachable and has a Visualize trace

Cover statements can be used in debugging failing trigger checks

Example

```
output_foo : assert property ((a ##1 b ##1 c ##1 d) |=> e);
```

If the precondition is unreachable, interesting cover statements might be:

```
output_foo1 : cover property (a);
output_foo2 : cover property (a ##1 b);
output_foo3 : cover property (a ##1 b ##1 c);
```

 © Cadence Design Systems, Inc. All rights reserved.

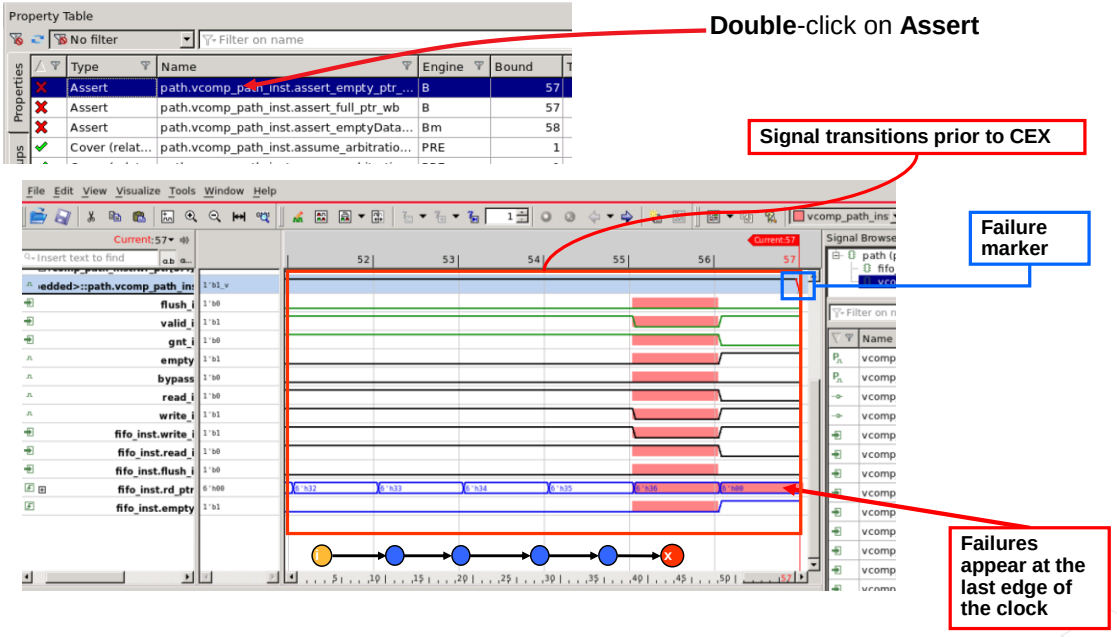
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase III: Debugging CEX



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

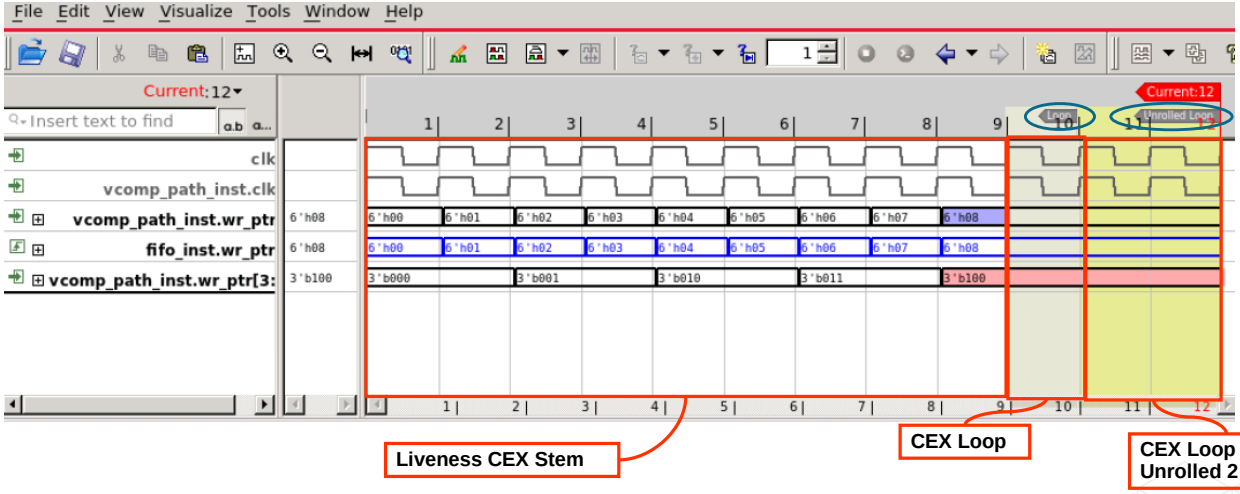
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase III: Debugging Liveness Failures

```
P1 : assert property ( wr_ptr == 8 |-> s_eventually wr_ptr==6 );
```

A CEX to liveness must show how something can *never* happen.
This requires the CEX to display an infinite length trace, e.g., `wr_ptr==6` never occurring



188 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

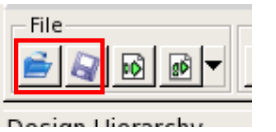
Phase III: Save/Restore Database

- A Jasper database stores the design setup and the results
  - Save database with the existing setup script

```
% save -jdb db.jdb -import_setup setup.tcl
```

  - Save the database with all commands executed

```
% save -jdb db.jdb -capture_session_data
```



- To restore the database
  - Start Jasper with the database
  - Load from Jasper

```
% restore -jdb /home/jasper/demos/Fifo_PA/db.jdb
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Debug Handoff Overview

Problem

- Sharing traces with screenshots, waveform files, or a complete database is often inefficient or complicated
- Users do not want to set up the testcase from scratch and run a formal proof again
- Users want to keep adding debugging information and have access to Visualize features when handing off a trace for investigation

Solution

- Debug Handoff bundles one or more traces into an easy-to-handle database
- The database captures necessary design files, setup, and Visualize information

Benefits

- Debugging work can be stored, shared, and continued later by the same or different person – easier collaboration
- Easy to restore the database – no need to set up the design or formal environment or rerun the formal proof – saves time
- One file captures everything – easy to share and store

 © Cadence Design Systems, Inc. All rights reserved.

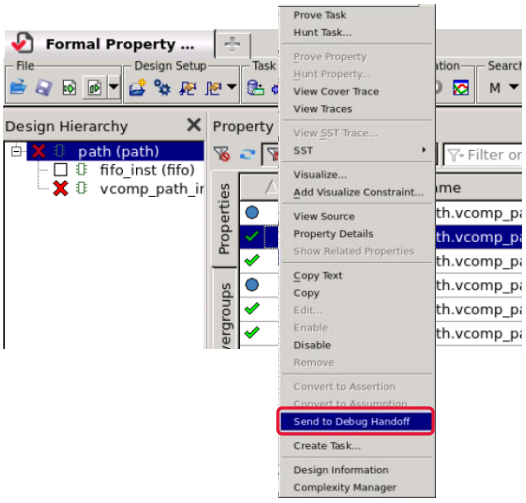
cadence

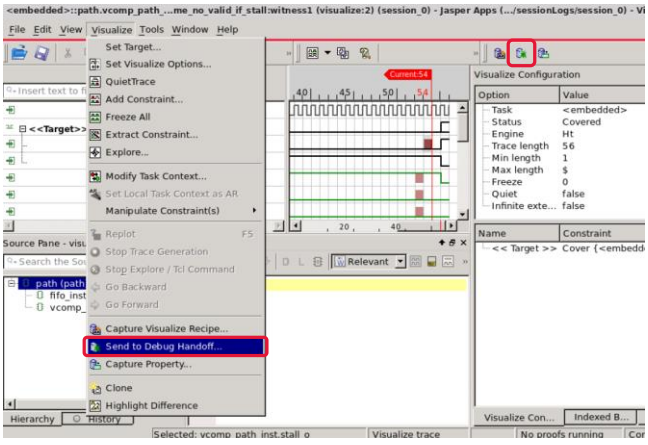
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Creating and Adding Traces to a Database via GUI





 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Debug Handoff Tcl Interface

Traces are stored in the database with:

- `debug_handoff -save -property <property_name_list> [<database>]`
- `debug_handoff -save -visualize_window <visualize_window_name> [<database>]`

Note: To save traces in the database, the traces must have been generated (prove, hunt, …)

- Debug Handoff does not trigger formal engines

The database file must be specified in the first save operation

For subsequent saves to the same database, specifying the name is optional

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

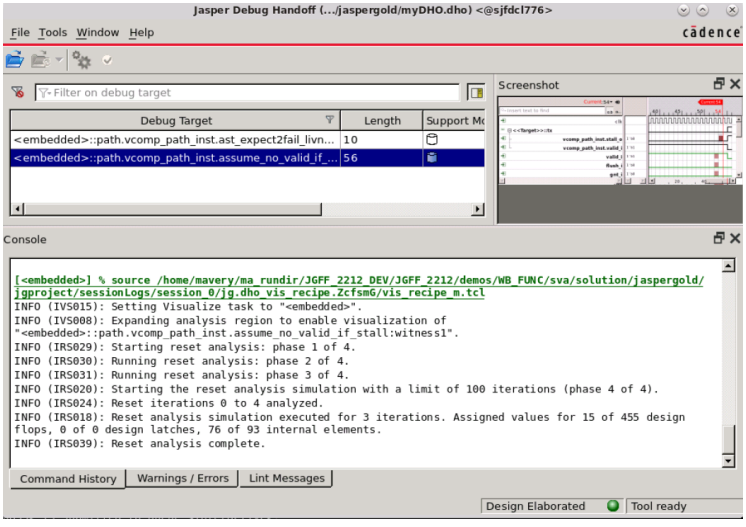
Loading Debug Handoff Database

Invoke Jasper with the database as an argument

- `jg -debug_handoff <db_name>`

In the GUI:

- Preview waveforms with saved screenshots by selecting a trace from the list
- **Double**-clicking opens a saved trace from the database in Visualize
- First opening of a saved trace requires elaboration



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Agenda

Context of Jasper in the Overall Verification Process

Introduction to Formal Analysis (FA)

Focus areas

FA flow phases

- Phase I: Setup and Initialisation
- Phase II: Design Bring-Up Overview
- Phase III: Formal Property Verification (FPV) & Debugging

Recommendations

Constraint Management

Where to get help

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Initial Design Bring-Up

The flow described in this section describes a process to ensure a safe and efficient bring-up of your design and assertions.

- Lint checks, XPROP, Superlint, cover statements, etc., are all helpful in creating a proper formal analysis environment

During this initial bring-up, if you received no CEX then something is most likely wrong

- Very unlikely to have no bugs in newly developed RTL and/or properties

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Setting Up a Productive FA Environment

An FA environment is productive when

- Results can be achieved quickly
  - Addressed primarily by following a FA methodology
    - Picking appropriate focus areas
    - Building an FA environment gradually
- Results are correct/complete
  - Essentially making sure your environment is not overconstrained
  - Addressed by following a FA methodology in which assumptions are verified at the next higher hierarchy level either by FA or simulation
  - Also addressed by
    - Building an FA environment gradually
    - Sanity checks

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Building an FV Environment Gradually

Break problems into pieces (by function, protocol, etc.) and develop assertions and assumptions on a per piece basis.

Constraints

- Try and identify 75-80% of all the necessary constraints before running assertions for the piece of the problem you are solving:
  - Running formal analysis with no constraints leads to a lot of false negatives
  - Overplanning your constraints can be dangerous because it can hide bugs
  - Add additional constraints as needed
- Initially place additional [over-]constraints (that is, cases) to simplify the problem or break the problem further into pieces
  - After assertions are working, start removing cases to increase the scope of verification

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Utilizing Assertion Based VIP (FVIP)

FVIPs can reduce bring-up time and increase productivity and quality significantly.

Features

- Protocol correctness properties and coverage points
- Pre-verified: No time spent on debugging assertions, only design
- Work in simulation, formal analysis, and emulation

Available Cadence FVIP

- Interface protocols: OCP,
  - AHB, APB, ATB, AXI-3, AXI-4, AXI4-Stream, AMBA 4 ACE, AMBA 5 CHI
  - OCP, PIPE, I2C, SPI, SPMI, CXL.Mem, CXL.Cache
- Memory controller protocols: SDRAM, DDR/DDR2/DDR3, LPDDR/LPDDR2/LPDDR3, DFI, DFI3/DFI4/DFI5

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Sanity Checks

Sanity checks are feedback mechanisms to ensure your environment is sane.

- If you never get an assertion CEX, there might be something wrong in your environment.
  - For example, it is over-constrained
- Use cover statements and witnesses to visually check the operation of the DUV and assess the sanity of the environment.
  - Specify valid sequences that should be reachable
  - If a cover statement is unreachable, then the environment is not set up correctly

Example

```
op_MT2full2MT: cover sequence (empty ##[+] full ##[+] empty);
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Confirming Correct Ranges

Confirm ranges by turning expected Proven into CEX by changing the timing:

- For example: Reduce range to confirm properties and interpretation are correct:
  - Specification: “The maximum latency of *gnt* is 3 cycles”

```
output_reqThenGnt: assert property (req |-> ([*0: 3] ##1 gnt));
```

copy and paste

```
output_reqThenGnt_test: assert property (req |-> ([*0: 2] ##1 gnt));
```

- By reducing the range to 2, one expects the second property to have a CEX
- If the second property still remains Proven, it might indicate an incorrect understanding of the design or environment

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Agenda

Context of Jasper in the Overall Verification Process

Introduction to Formal Analysis (FA)

Focus areas

FA flow phases

- Phase I: Setup and Initialisation
- Phase II: Design Bring-Up Overview
- Phase III: Formal Property Verification (FPV) & Debugging

Recommendations

Constraint Management

Where to get help

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Plan to Verify Assumptions

In formal, the assumptions have to be taken as axioms

Jasper cannot verify that they are correct

If the assumptions are incorrect, then every assertion result is invalid.

Some powerful ways in which we can check the correctness of assumptions

- Use Assume-Guarantee (AG) reasoning
  - Assert the property and prove it before using the property as an assumption
  - Jasper provides optional automation for this – see Proof Structure class
- Use the assumptions as assertions in Simulation and ensure there are no failures
  - Xcelium simulator will convert assumptions to assertions automatically

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Assumptions are instructions which you give about the environment under which the design is verified.

If the assumptions are wrong, then every assertion result is invalid.

Clearly validation of the assumptions via some other means is vital.

This requires a means to track and integrate results from multiple tools and measure that against a verification plan.

This is what Verisium Manager does.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Assumptions and Overconstraints

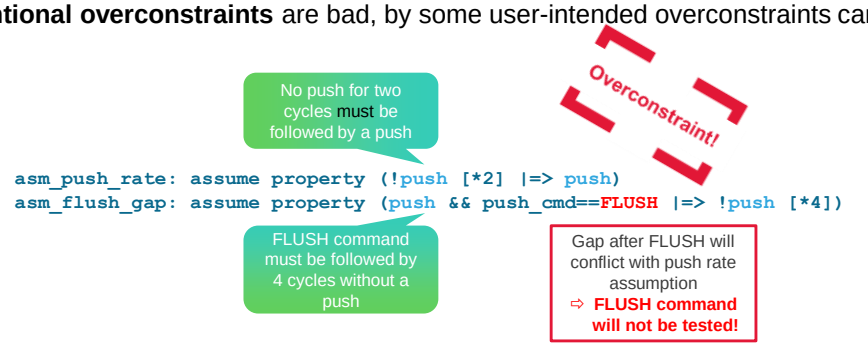
All assumptions in a formal environment must be respected by proof engines

- All waveforms will show assumptions as true in all cycles

Mistakes in assumption coding can lead to overconstraining

- Input sequences and behaviors that you want to test are not covered because assumptions are “too strong”
- **Unintentional overconstraints** are bad, by some user-intended overconstraints can be good

Example



 © Cadence Design Systems, Inc. All rights reserved.

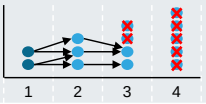
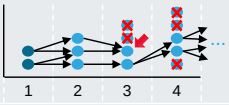
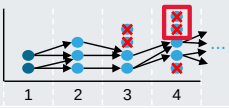
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Types of Overconstraints

	Conflict	Dead-End	Behavior Removal
Description	Assumptions prevent **anything** from being driven ⇒ Easy to detect and fix	Assumptions prevent anything from being driven **from a particular state** (the dead-end state) ⇒ Usually harmless ⇒ May be hard to detect	Assumptions prevent relevant behaviors from being tested ⇒ Very dangerous ⇒ Hard to detect
Illustration ● = State ✕ = State which violates an assumption	Conflict at cycle 4:  	Dead-end state at cycle 3:  	Behaviors are missed at cycle 4: 
How to Detect	• `check_assumptions -conflict` • Related covers	• `check_assumptions -dead_end`	• Related covers • User-defined covers

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

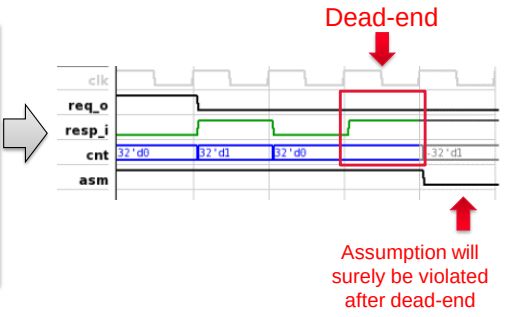
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Dead-End Example

```
// Count number of requests
integer cnt;
always @(posedge clk)
  if (rst) cnt <= 0;
  else cnt <= cnt + req_o - resp_i;

// Assume that counter is never negative
asm: assume property (cnt >= 0);
```



- Combination of `cnt=0`, `resp_i=1`, and `req_o=0` is a dead-end.
  - `cnt` will surely go negative in the next cycle, violating `asm`
- Dead ends (unlike conflicts) may be acceptable. However, it is often best to avoid them since they can hurt FPV performance and make debugging more difficult.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Detecting Overconstraints – Related Cover Status

Definition of assertion pass

1. Given the assumptions and initial state, every time assertion triggers the consequence is true
2. Assertion cannot be triggered (a.k.a. “vacuous pass”)



Unreachable related covers may point to overconstraints

- But it could also be a design bug or mistake in how the assertion was coded

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

check_assumptions Tcl Command

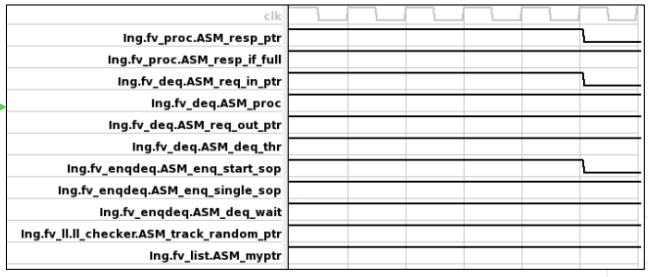
- Jasper has two built-in assertions to flag conflict and dead-end overconstraints
  - `:noConflict` and `:noDeadEnd`
  - Show them with `check_assumptions -show [dead_end]` (they are hidden by default)
  - Prove with `check_assumptions (-all | -task)`
- A CEX for any of these two assertions means that an overconstraint may exist
  - Bring up waveform from GUI or with `check_assumptions -trace (-conflict | -dead_end)`

Example

No conflict for 190 cycles

	Type	Name	Engin	Bound
?	Assert(noConflict)	:noConflict	Q3	190 -
✖	Assert(noDeadEnd)	:noDeadEnd	Q4	6

Dead-end found at cycle 6



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Liveness Conflict Checking

Every task has an additional built-in cover "`:live`" property – not shown by default

`:live` is unreachable if there is a liveness conflict

The property automatically displays in the property table when the tool detects a conflict

In the case of vacuity, an indicator displays all proven liveness properties

Explicit check and root causing with

- `check_assumptions -show -live`
- `check_assumptions -live [-from]`
- `check_assumptions -live -minimize`

Benefits

- The tool has implicit and explicit checks for fairness vacuity
- Indicator makes it easy for the user to recognize fairness vacuity

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

get_needed_assumptionsTcl Command

Often useful to narrow down the source of overconstraint to a small set of assumptions

Command `get_needed_assumptions` determine which assumptions are required to maintain the status for a property

Most common usage is to debug unreachable covers

- Which assumptions are needed to make this cover unreachable?

```
% get_needed_assumptions -property <cover name> -status unreachable
```

Can also be used through `check_assumptions -minimize`

- Which assumptions are needed to hit the conflict?

```
% check_assumptions -conflict -minimize
```

Tip: Use switches `-include_reset`, `-include_clock`, and `-include_env_assumes` to include global configurations

- By default, only task configurations are included

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Agenda

Context of Jasper in the Overall Verification Process

Introduction to Formal Analysis (FA)

Focus areas

FA flow phases

- Phase I: Setup and Initialisation
- Phase II: Design Bring-Up Overview
- Phase III: Formal Property Verification (FPV) & Debugging

Recommendations

Constraint Management

Where to get help

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Getting Help in Jasper

Help Menu and Quick Help wizard button

Help Command

- Display help on a specific command

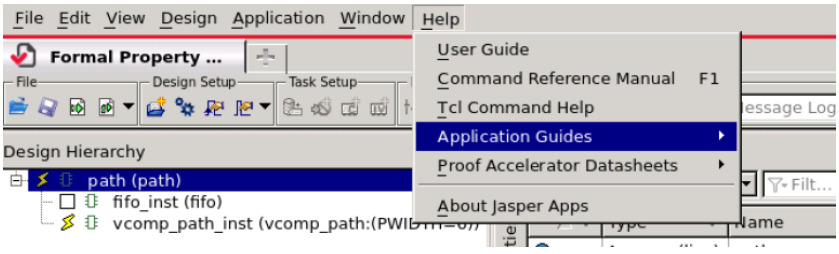
```
% help <command_name>
```

- Display help on specific info, warning, and error messages

```
% help -message <msg_number>
```

Examples

```
% help
% help -message
% help view
% help -message EAS007
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

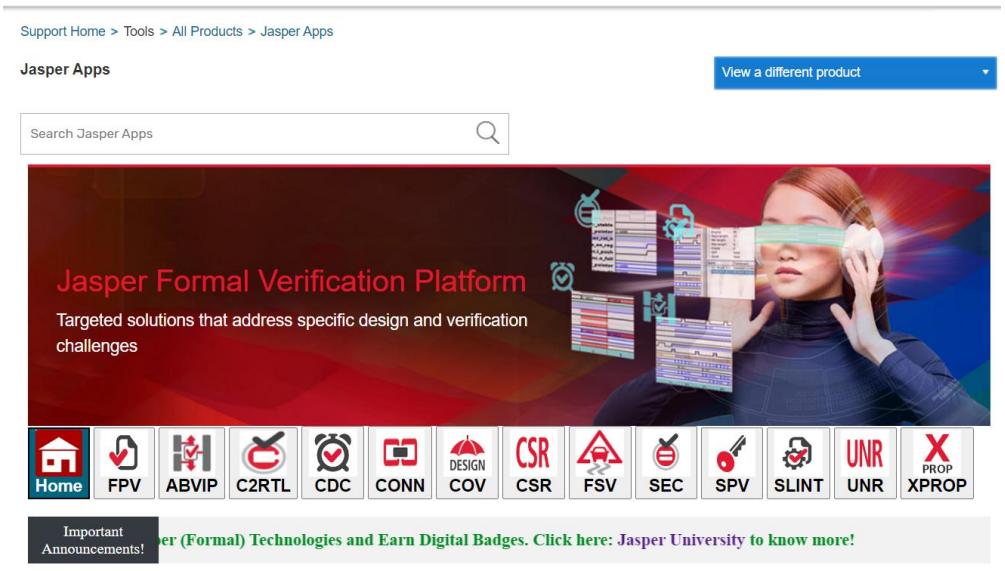
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Cadence Learning and Support Product Landing Page

https://support.cadence.com/Jasper



212 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

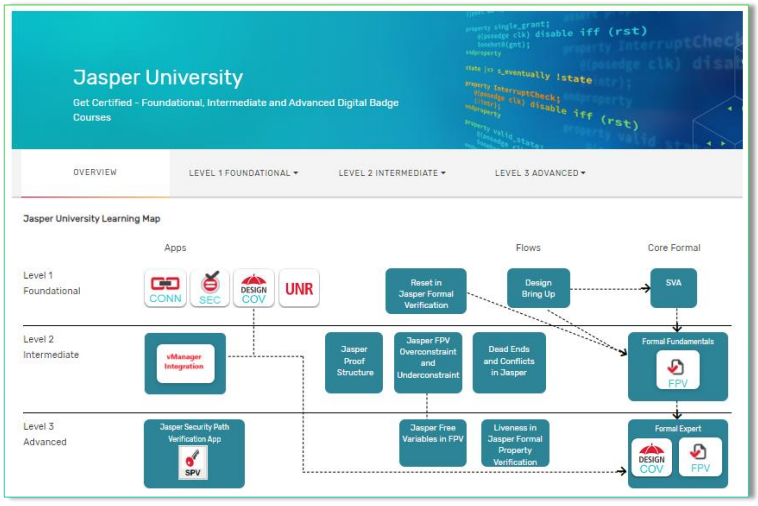
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Jasper University

Free online training, Rapid Adoption Kits (RAKs), digital badges and much more

https://support.cadence.com/jasperuniversity



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Summary

Verification Engineers new to Formal or Designers can get the best ROI out of Jasper by following the recommended FA flow

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 7

Visualize

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Module Objectives

In this module, you will learn how to use the features of Visualize™ to

- Debug CEXs
- Simplify Waveforms
- Perform What-if exploration
- Perform RTL Design Exploration
- Capture reusable behaviors

These slides are provided as a reference of some of the main features of Visualize in a single place. If this course is presented to you by an instructor, either live or in an online course, it is more effective for the instructor to demonstrate those features instead of showing every single slide.

 © Cadence Design Systems, Inc. All rights reserved.

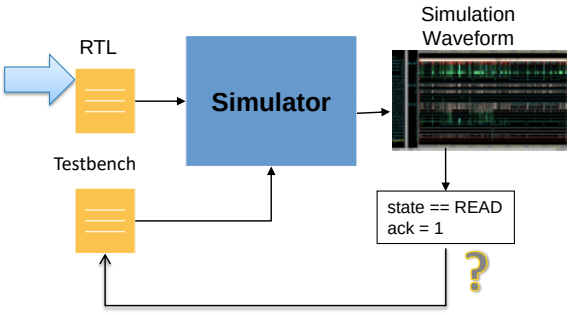
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

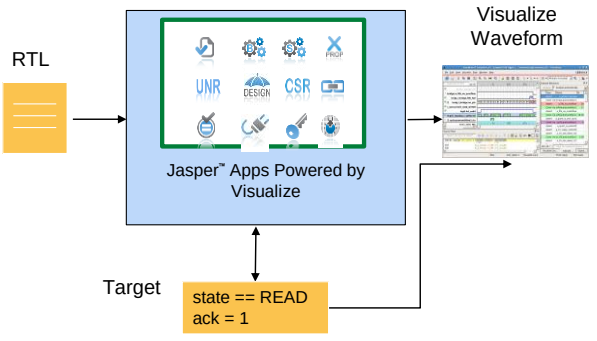
Benefits of the Visualize Approach



Simulation

Pain Points

- "Input driven" trial & error approach to produce the desired behaviors
- May not exercise desired states
- Need all elements of the design or must manually create stubs



Visualize Flow

Benefits

- Specify the target and let the formal engines automatically generate the stimulus ("output driven" method)
- Proof (no trace possible) is exhaustive
- Interactively add constraints to construct desired waveform(s)

217 © Cadence Design Systems, Inc. All rights reserved.

cadence

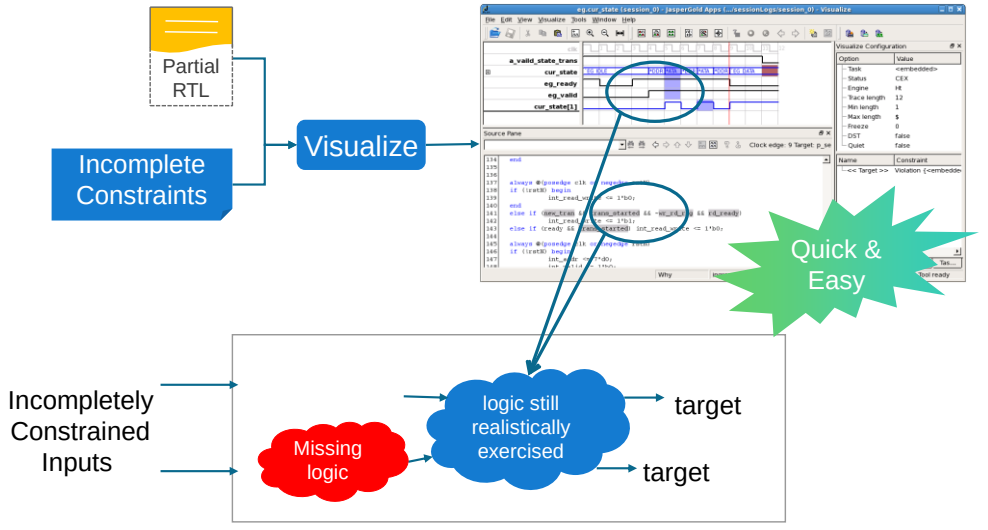
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Partial Designs Can Be Verified

Incomplete RTL or constraints is not a show-stopper for visualize



 © Cadence Design Systems, Inc. All rights reserved.

cadence

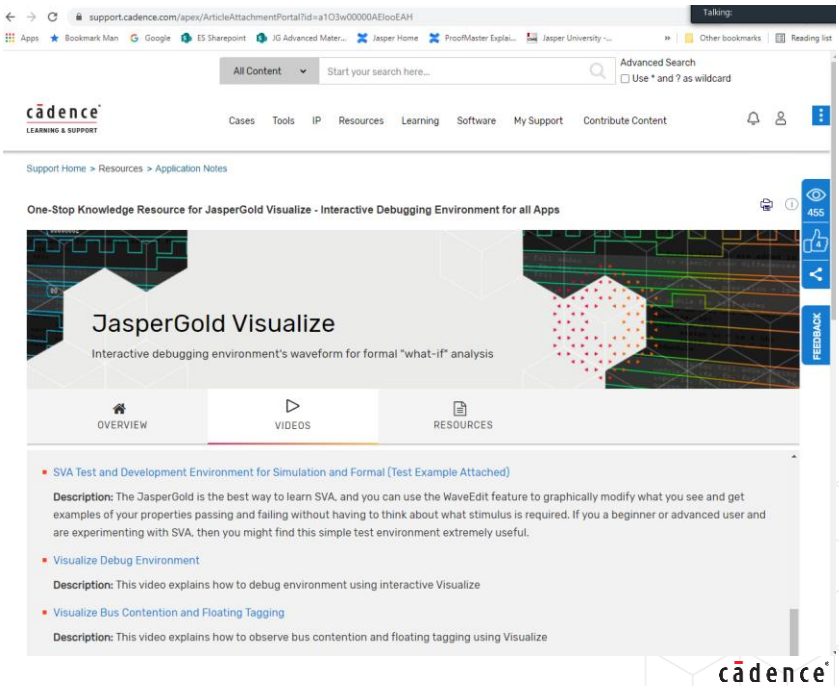
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Landing Page

- http://support.cadence.com/visualize
- Many videos and other resources showing visualize features



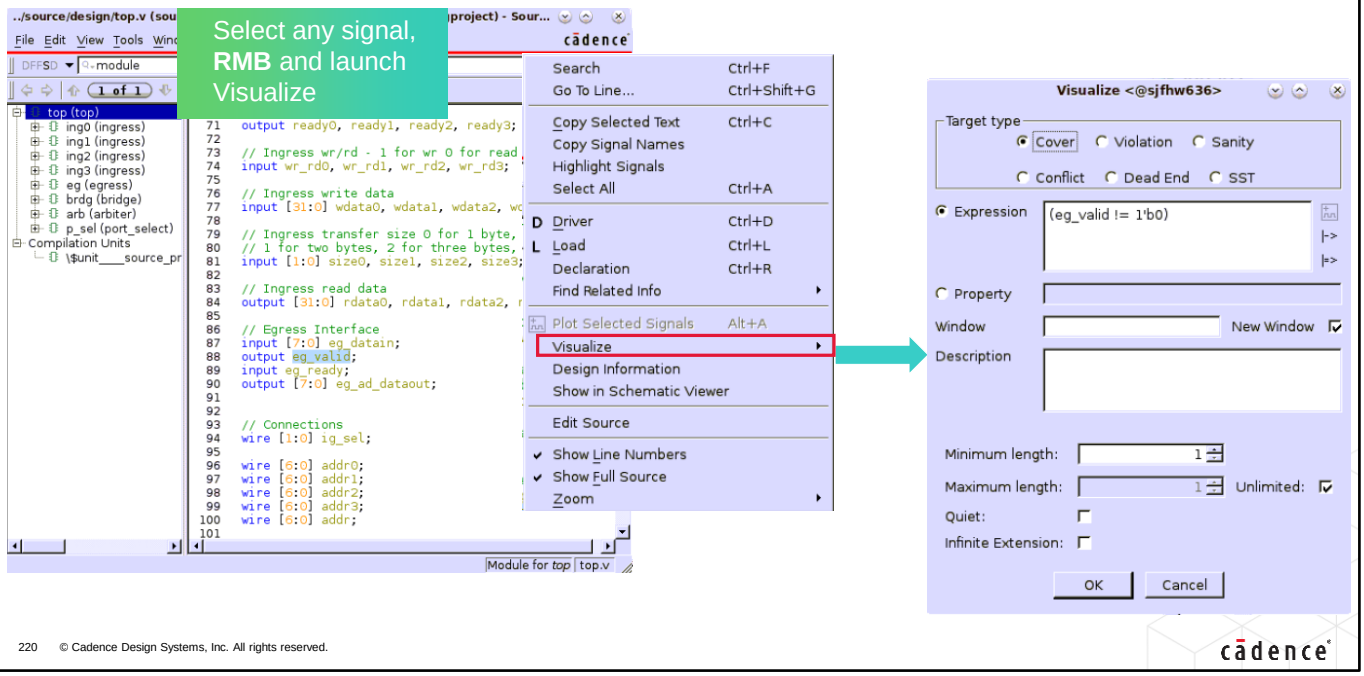
219 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Flow: Target Any Signal



 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

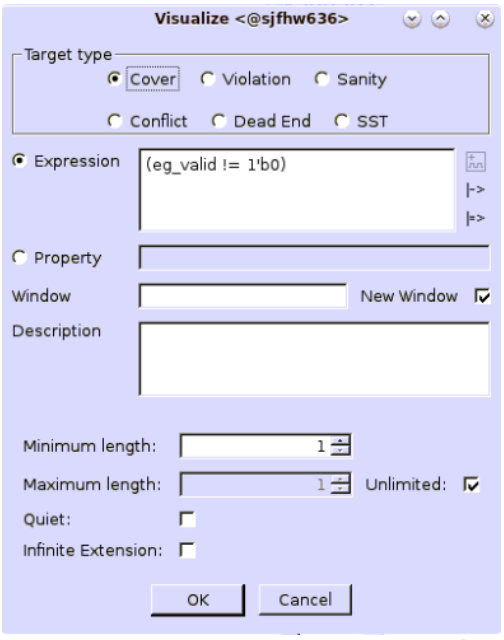
Visualize Flow: Targets and Types

- Targets
  - The patterns of behavior you want to see in the trace
  - Examples

```
eg_valid == 1'b1
eg_valid == 1'b1 &&
  eg.cur_state != `EG_IDLE
eg_valid ##1 !eg_valid ##1 eg_valid
eg_valid |-> eg_ad_dataout != 8'h00
eg_valid |=> eg_ad_dataout != 8'h00
```

- Types
  - Cover – At least once TRUE
  - Violation – At least once FALSE

Tcl Command can be used :
visualize -new_window; visualize -cover {(eg_valid != 0'b0)} -bg



 © Cadence Design Systems, Inc. All rights reserved.

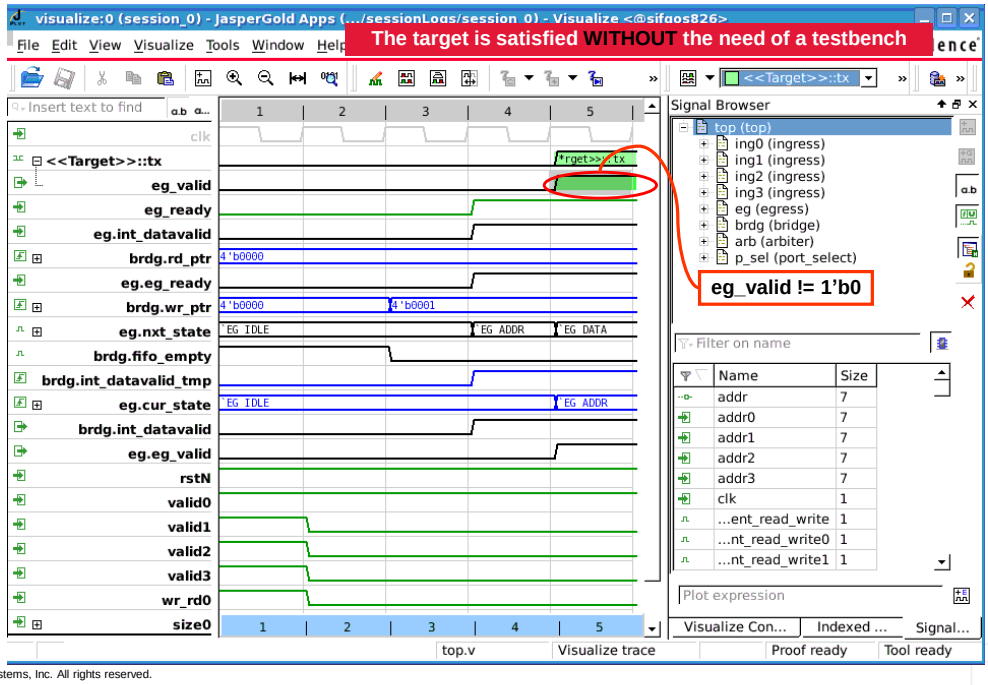
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Flow: Trace



222 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Flow: Tips

- Use `cover` as type
- Use output ports or internal signals as targets
- Use simple expressions you are interested in
- Jasper™ will, by default, produce the shortest possible trace

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Agenda

- What is Visualize?
- Visualize Flow
- Visualize GUI
- Visualize Features
- Summary

 © Cadence Design Systems, Inc. All rights reserved.

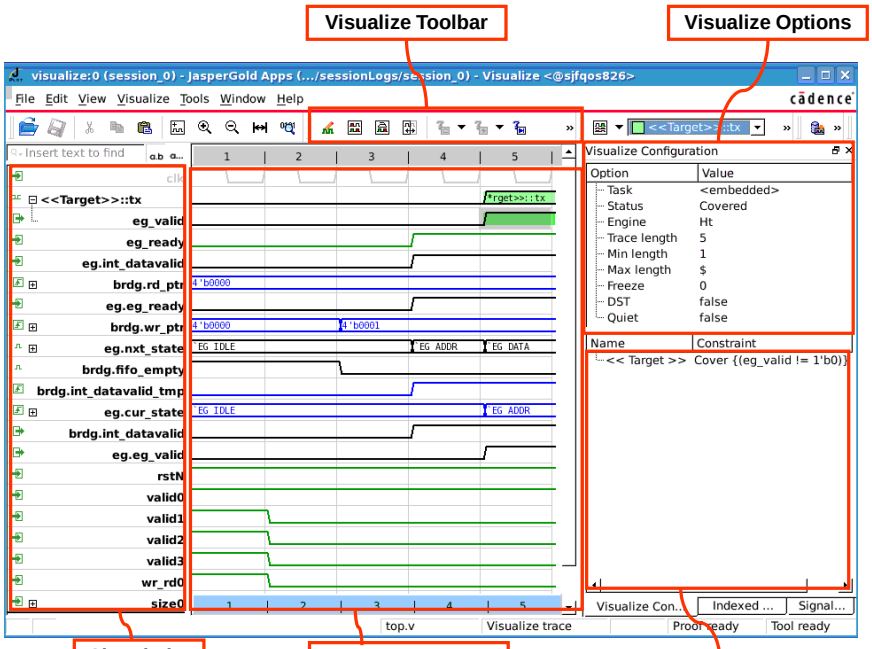
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize GUI



225 © Cadence Design Systems, Inc. All rights reserved.

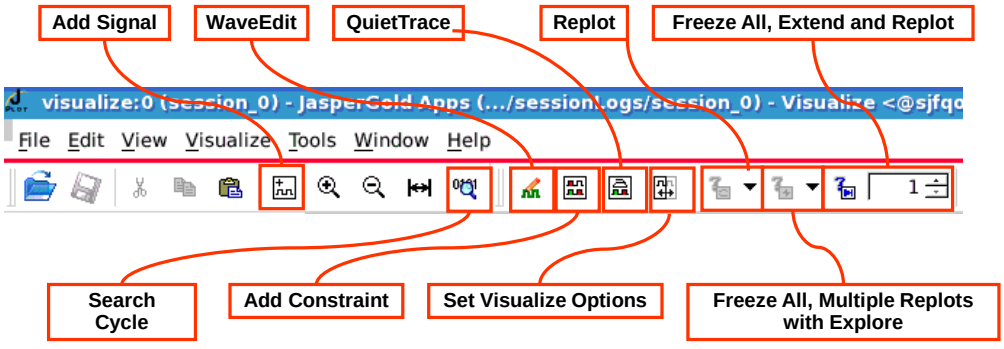
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize GUI: Toolbar

Most Visualize features are a click away and are present in the toolbar



 © Cadence Design Systems, Inc. All rights reserved.

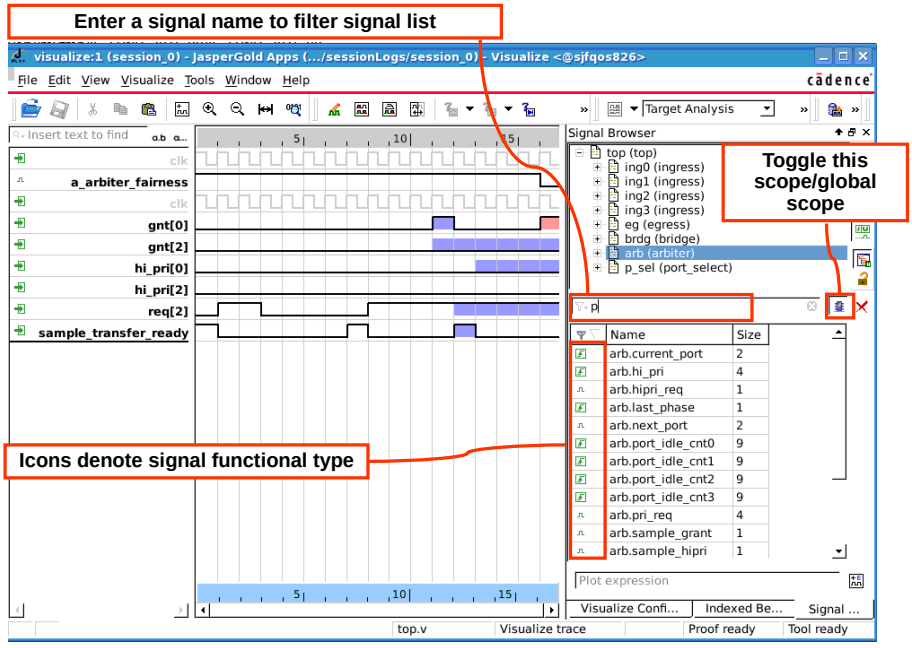
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize GUI: Signal Browser



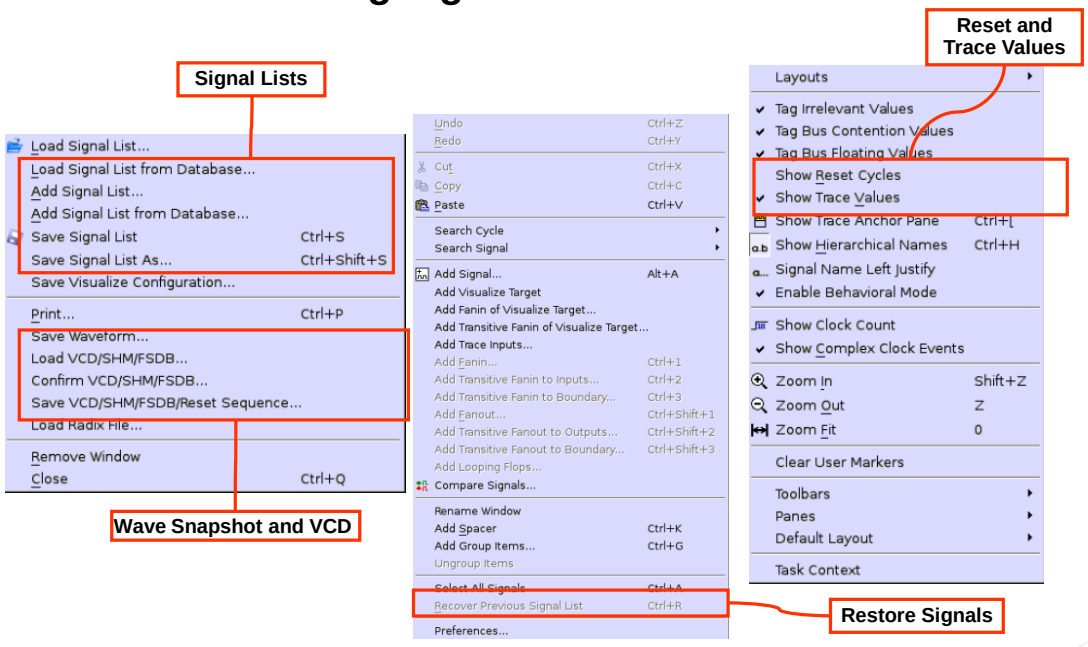
227 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize GUI: Menu Highlights



 © Cadence Design Systems, Inc. All rights reserved.

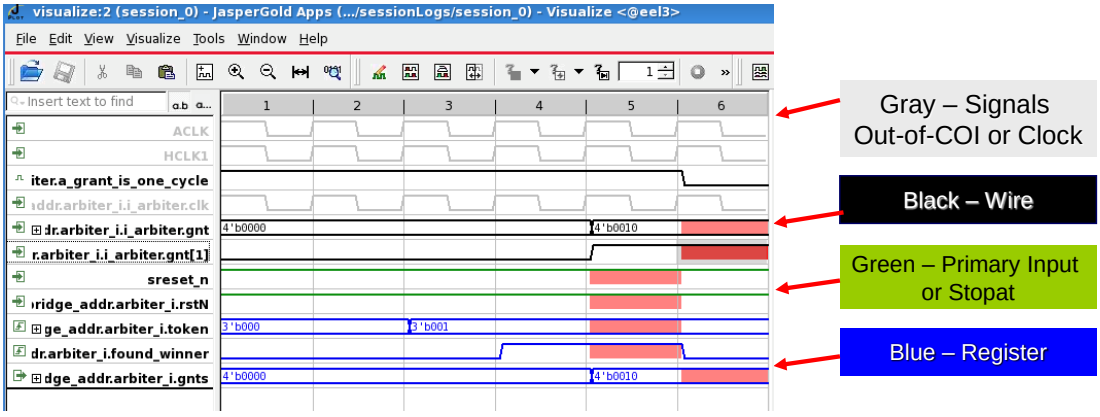
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize GUI: Signal Color Coding



- Signals are color coded based on their type
- Gray waveforms represent either signals clock or outside the cone of influence (COI) of the target property
- Additional colors appear when using advanced features such as counter abstractions

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

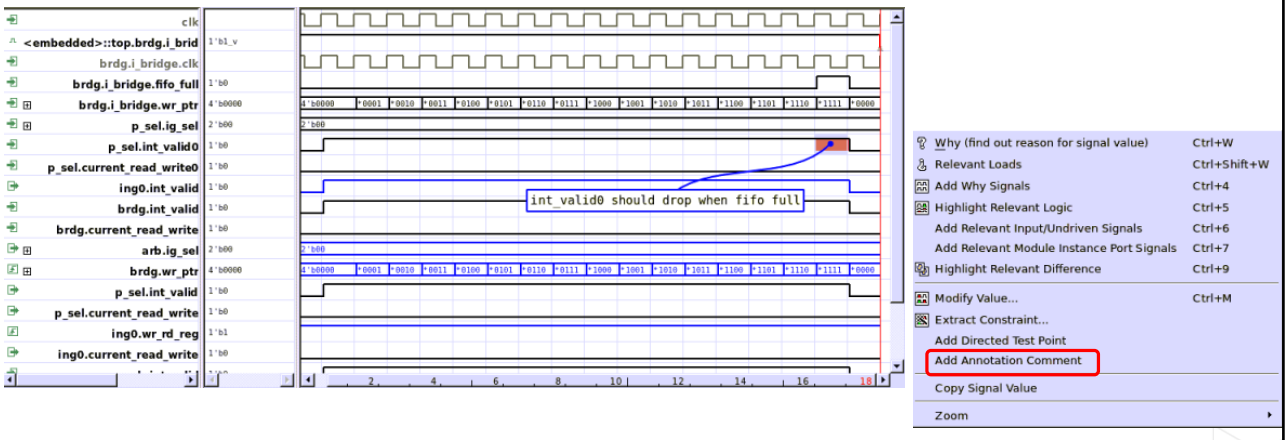
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Annotate Signals

To assist with debug handoff (which we discuss later) we can add comments to the waveform

Click RMB on waveform where annotation required



 © Cadence Design Systems, Inc. All rights reserved.

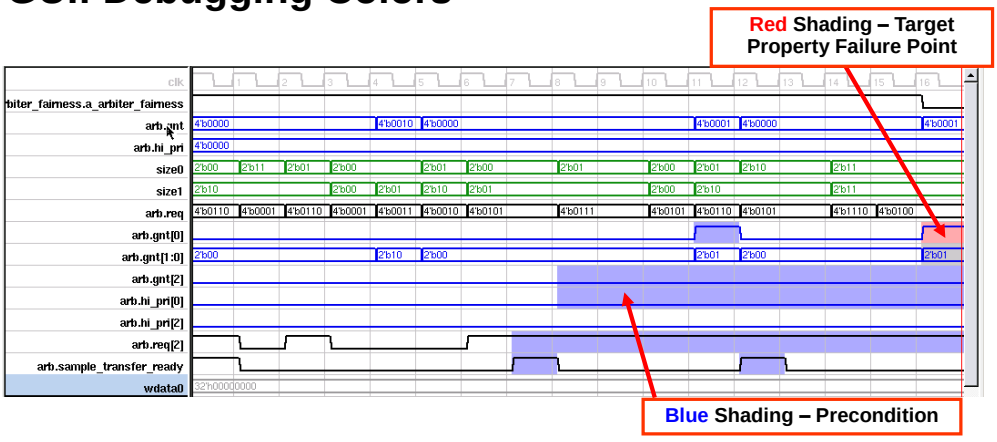
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize GUI: Debugging Colors



- Advanced Temporal Debugging (ATD) helps you debug complex sequential expressions for PSL/SVA properties
  - Blue shading shows when precondition (antecedent expression) triggers, this leading to a CEX or Covered expression
  - Red shading shows when the target property actually fails, highlighting failure points as candidates for the `Why?` feature

 © Cadence Design Systems, Inc. All rights reserved.

cadence

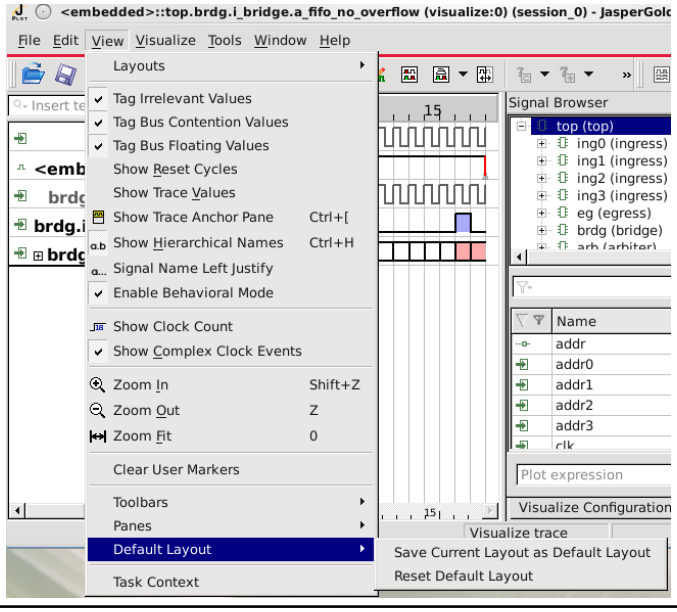
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize GUI: Save Your Layout

You can customize your layout and save it as the default layout for future visualize sessions



232 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Agenda

- What is Visualize?
- Visualize Flow
- Visualize GUI
- Visualize Features
- Summary

 © Cadence Design Systems, Inc. All rights reserved.

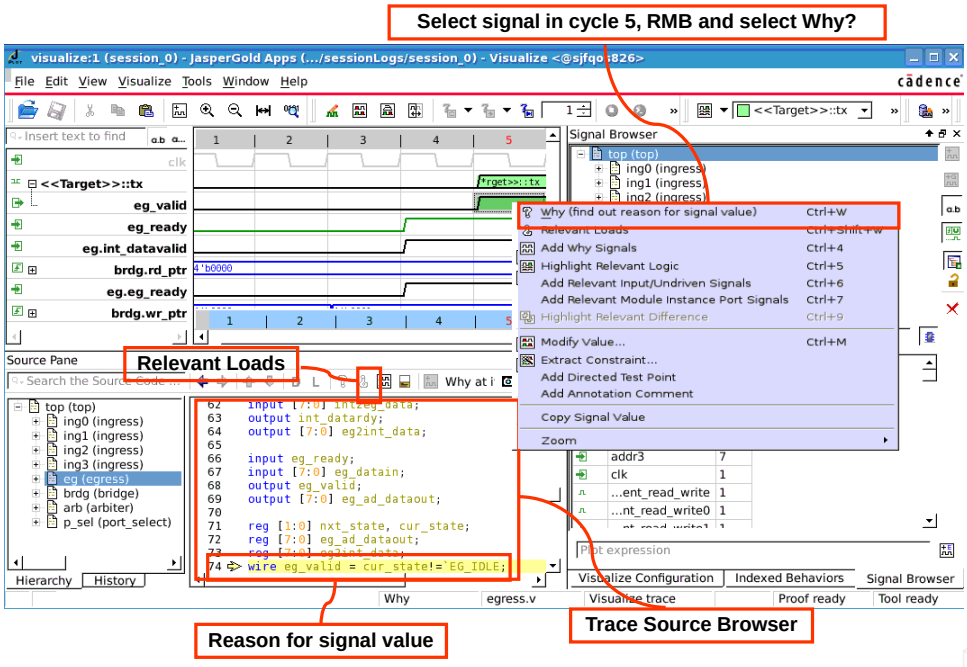
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Features: Why?



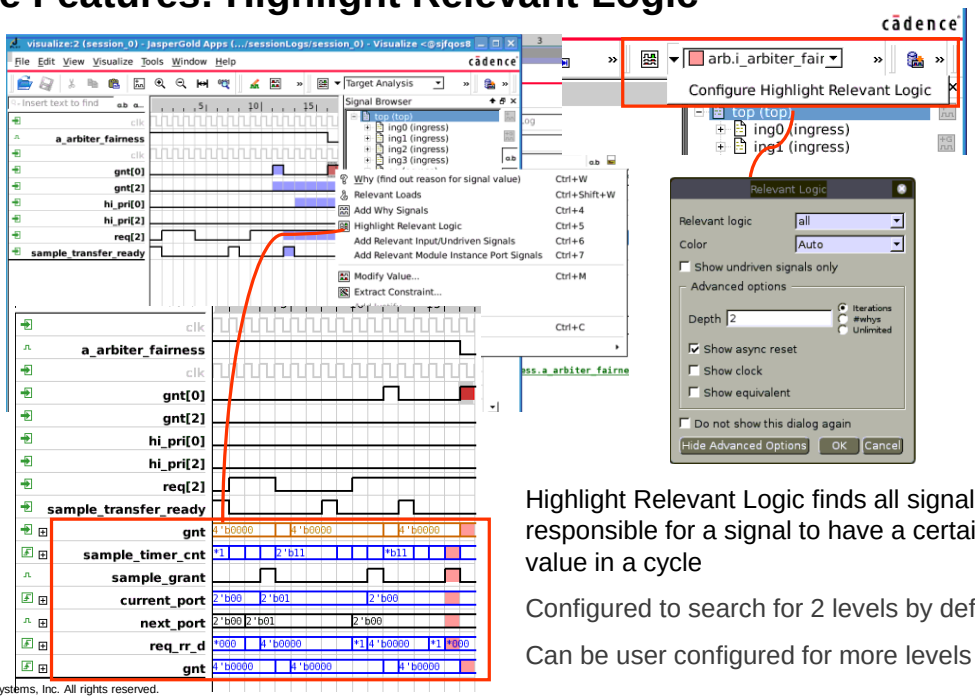
234 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Features: Highlight Relevant Logic



Highlight Relevant Logic finds all signals responsible for a signal to have a certain value in a cycle

Configured to search for 2 levels by default

Can be user configured for more levels

 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Features: Add Constraints

Force

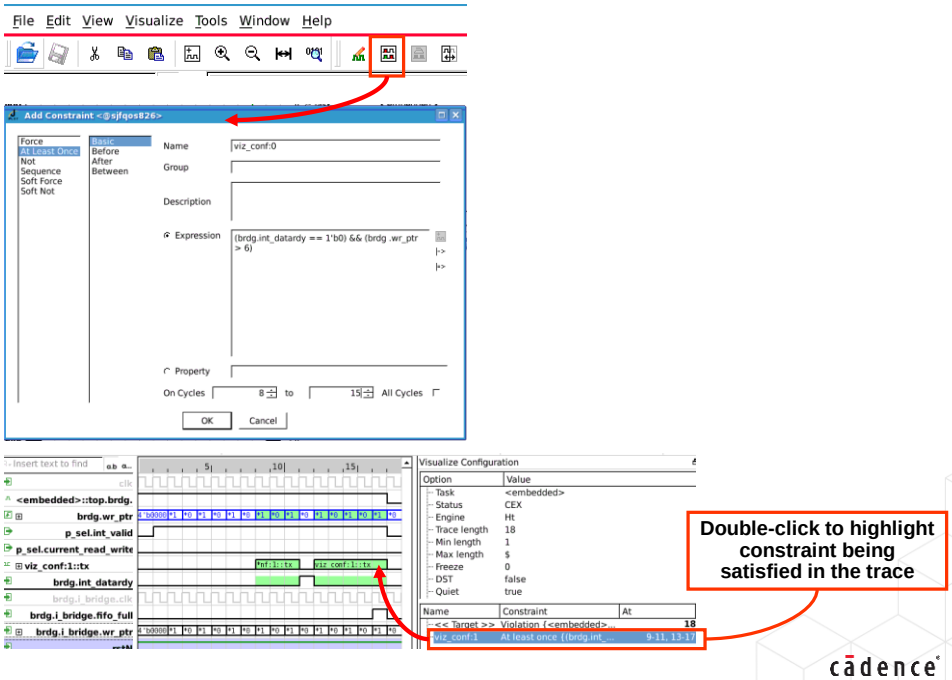
- Specify the value for the duration of the given cycles

At Least Once

- Specified value holds ‘at least once’ during the duration of the specified cycles

Soft

- Create a ‘soft’ constraint that can be overridden by other constraints if there is a conflict



 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

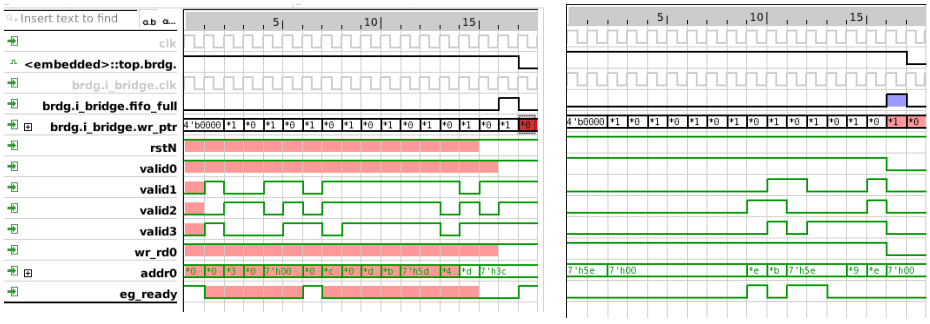
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Features: QuietTrace

- QuietTrace minimizes signal toggle while maintaining the same visualization target
- Makes debug more clear and concise
- QuietTrace can be run on a subset of cycles

```
% visualize -quiet true 3:9; visualize -replot
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

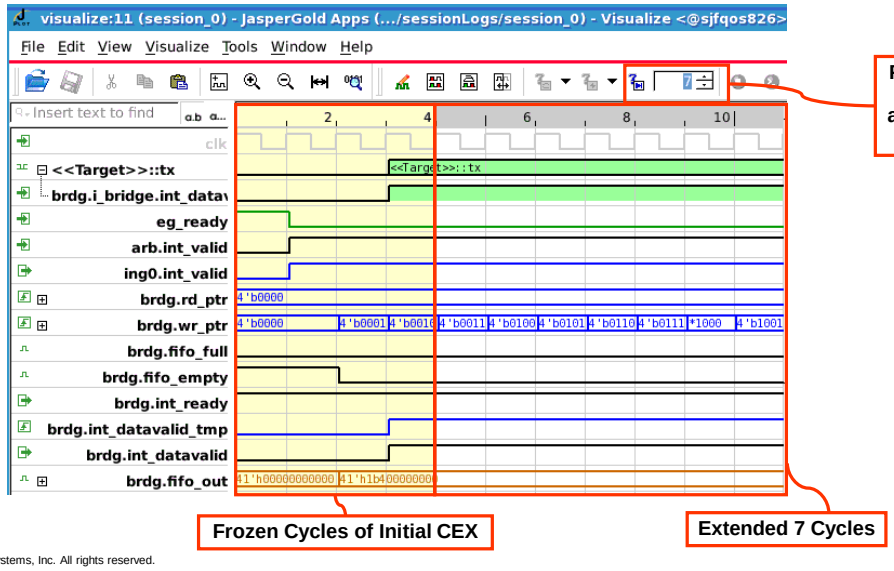
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Features: Freeze All, Extend and Replot

Freeze allows you to preserve a given number of cycles and extrapolate further cycles from there on to observe design behavior in the future



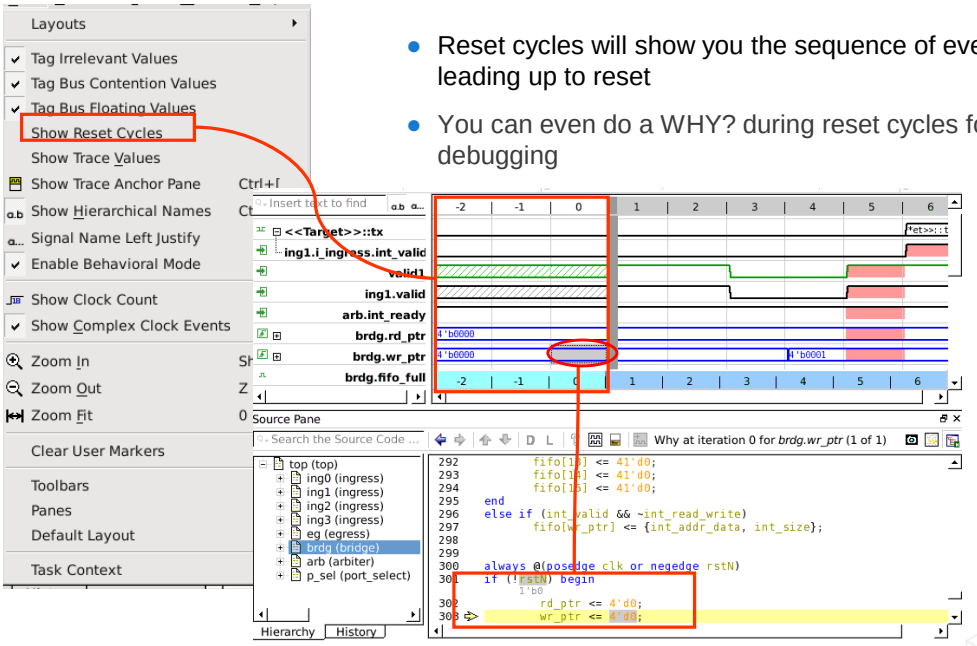
238 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Features: Reset Cycles



- Reset cycles will show you the sequence of events leading up to reset
- You can even do a WHY? during reset cycles for debugging

239 © Cadence Design Systems, Inc. All rights reserved.

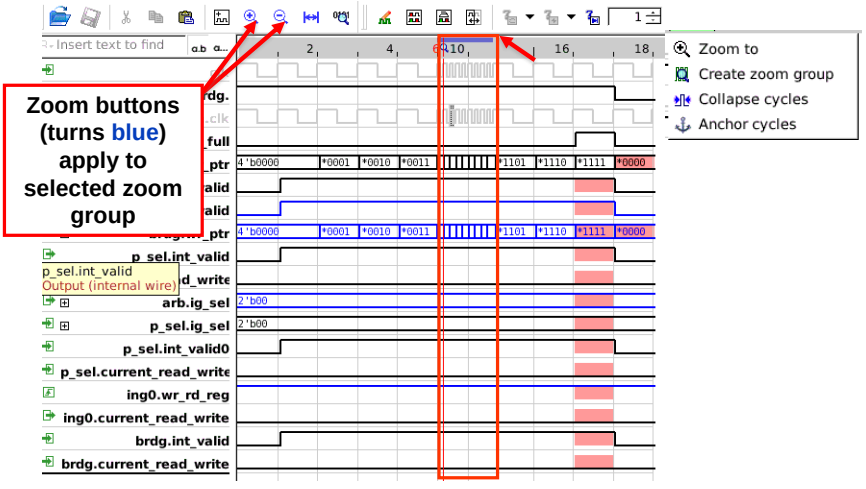
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Features: Zoom Group

- Use the zoom group feature to specify and easily go back to cycles of interest
- After you specify a cycle interval as a zoom group, the zoom buttons turn blue, and all zoom operations apply to the group, i.e., the zoom operations have no effect on cycles outside the group



 © Cadence Design Systems, Inc. All rights reserved.

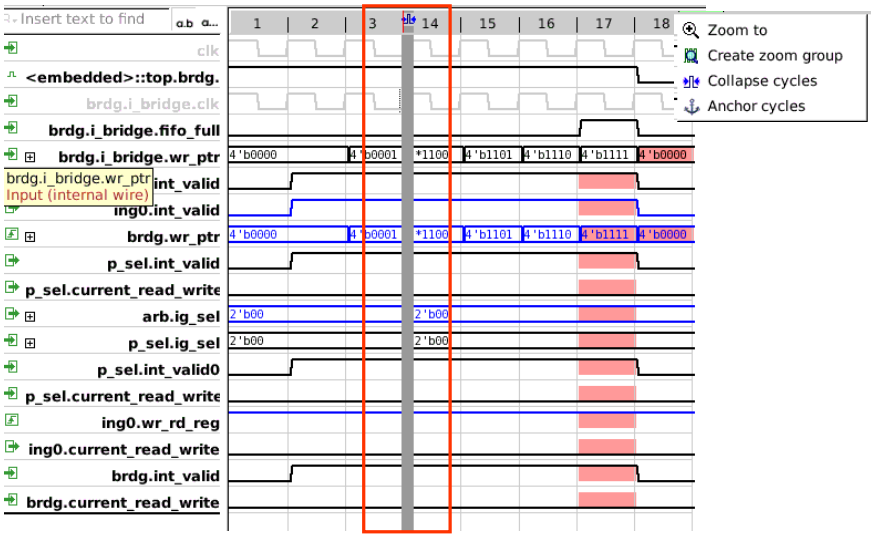
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Features: Collapse Cycles

This feature provides better access to cycles of interest by collapsing the cycles you do not care to see



241 © Cadence Design Systems, Inc. All rights reserved.

cadence

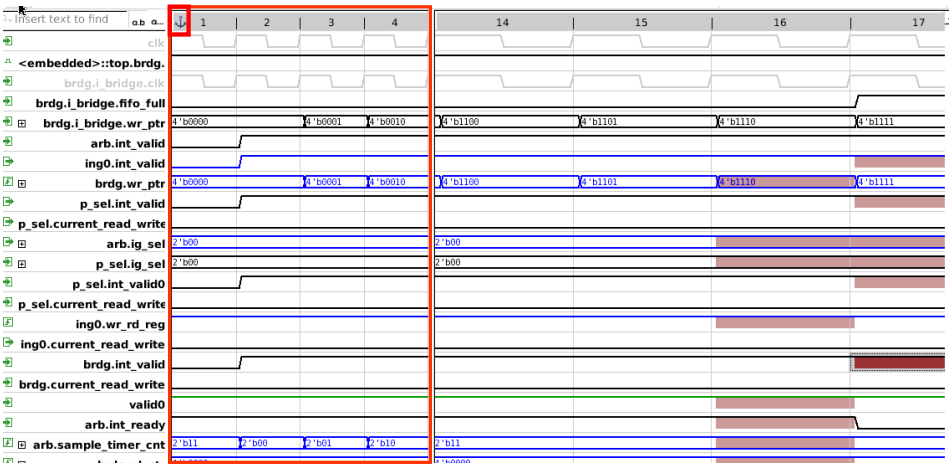
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Features: Anchor Cycles

- Create a split view with an interval that is visible at all times with the anchored cycles at the left-hand side of the waveform.
- You can scroll the remaining cycles in and out of view as usual. The anchored cycles sync to vertical scrolling in the un-anchored portion of the waveform.



 © Cadence Design Systems, Inc. All rights reserved.

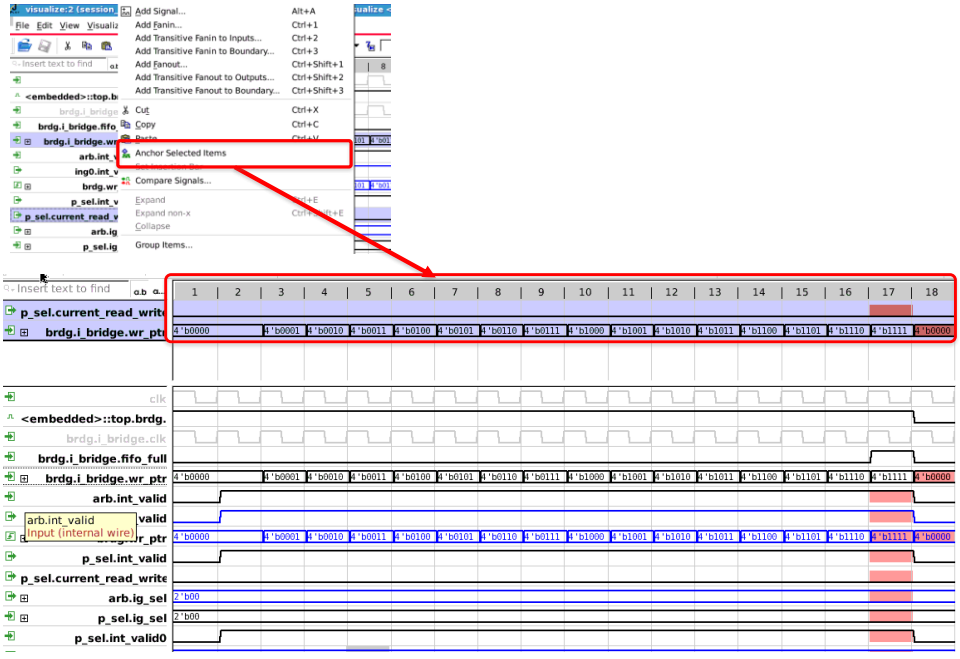
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Features: Row Anchoring



 © Cadence Design Systems, Inc. All rights reserved.

cadence

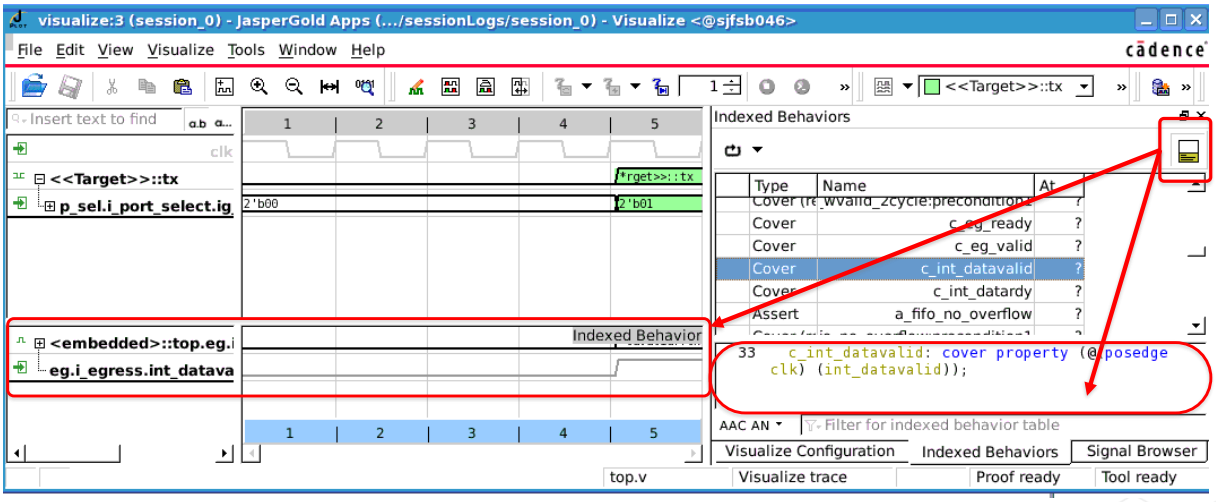
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Features: Preview Indexed Behaviors

- The *Indexed Behaviors* pane includes a button for *Preview Waveform* and Source
- Use this button to preview the source code and waveform for the selected property



 © Cadence Design Systems, Inc. All rights reserved.

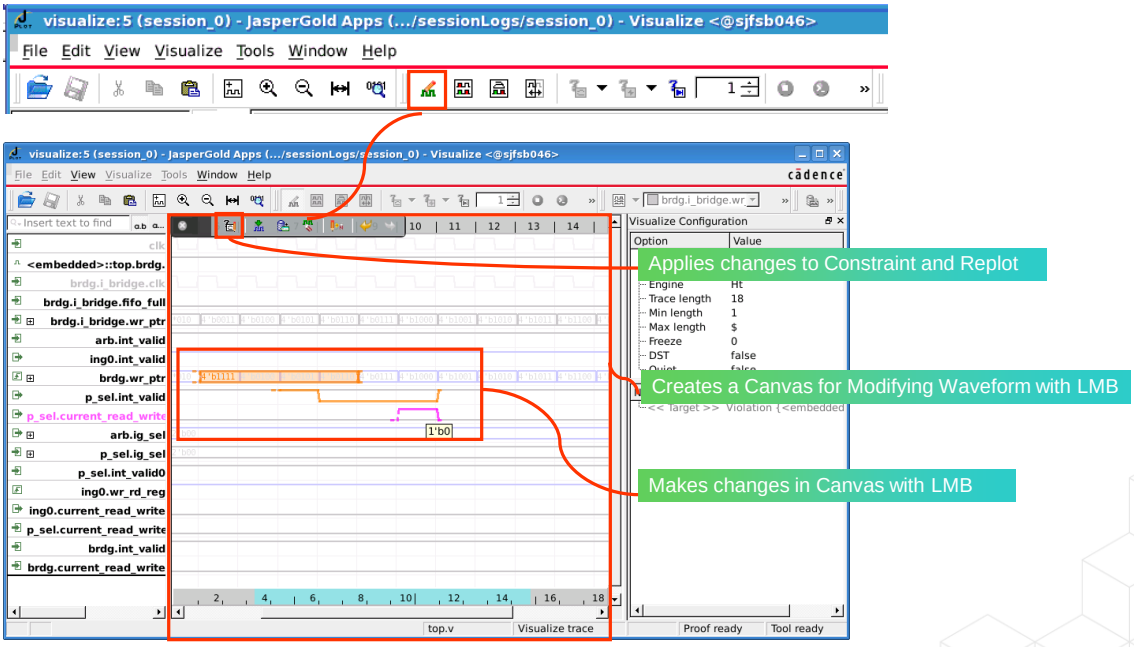
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Features: WaveEdit



 © Cadence Design Systems, Inc. All rights reserved.

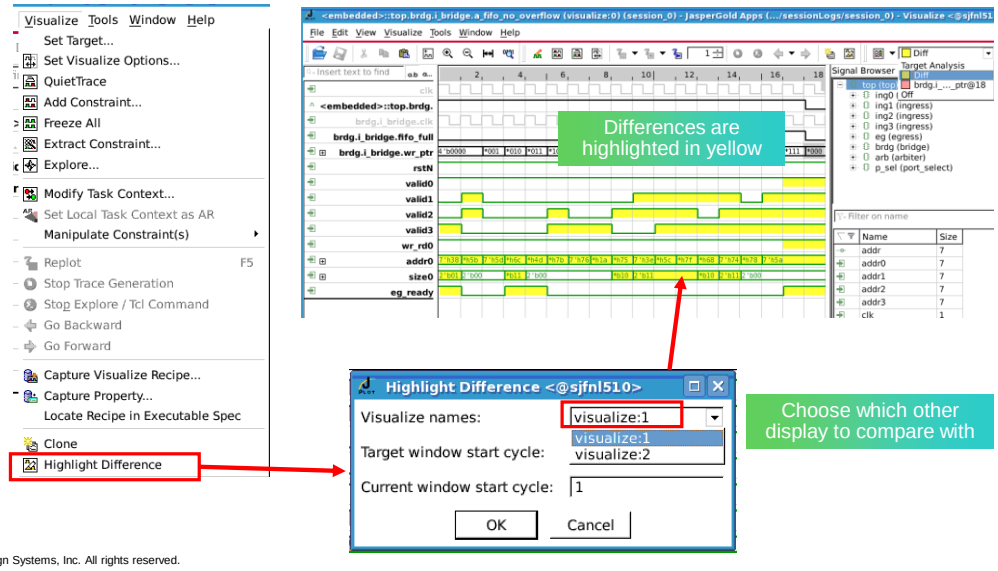
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Features: Diff Traces

What if we added constraints, used quiet trace, used WaveEdit and so on, and we want to see what difference it made?



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Highlight Relevant Difference Motivation

Related features:

- **Highlight relevant logic:** It enables users to see which signals are relevant to a given event occurring in a Visualize trace
- **Highlight diff:** Performs diff between different traces of the same signal (CEX vs. witness, for example) to see how they are different

Goal

- Provide the best of both worlds, allowing the user to highlight signals that are **both** relevant in one trace or the other, **and** different between the two traces
- By doing so, the user is presented with a very focused “root cause analysis” of what happened in the CEX trace that caused it to **not** be a witness trace (for example)

Other uses include:

- **Fairness debugging:** Highlight relevant differences between liveness CEX and witness trace
- **Unreachable cover debugging:** Highlight relevant differences between not-from-reset traces that both hit and miss the cover target

 © Cadence Design Systems, Inc. All rights reserved.

cadence

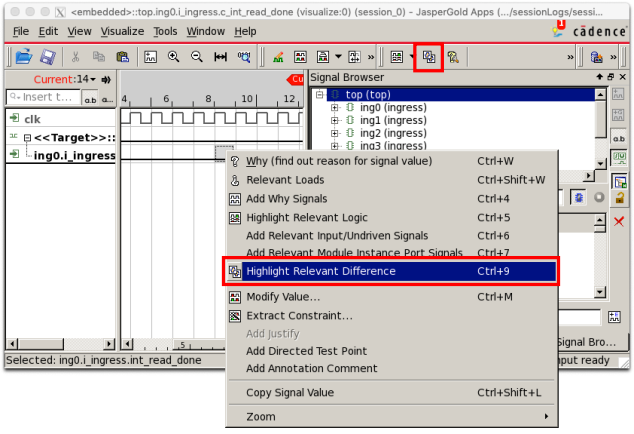
This page does not contain notes.

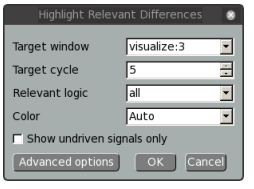
© Cadence Design Systems, Inc. All rights reserved.

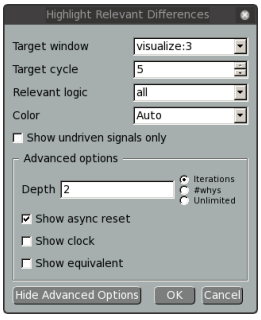
This document is for the sole use of Kun Wang of Seas

Highlight Relevant Difference GUI

- **Right**-click on a signal cycle
- Select **Highlight Relevant Differences**
- Select a target window and configure parameters







248 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

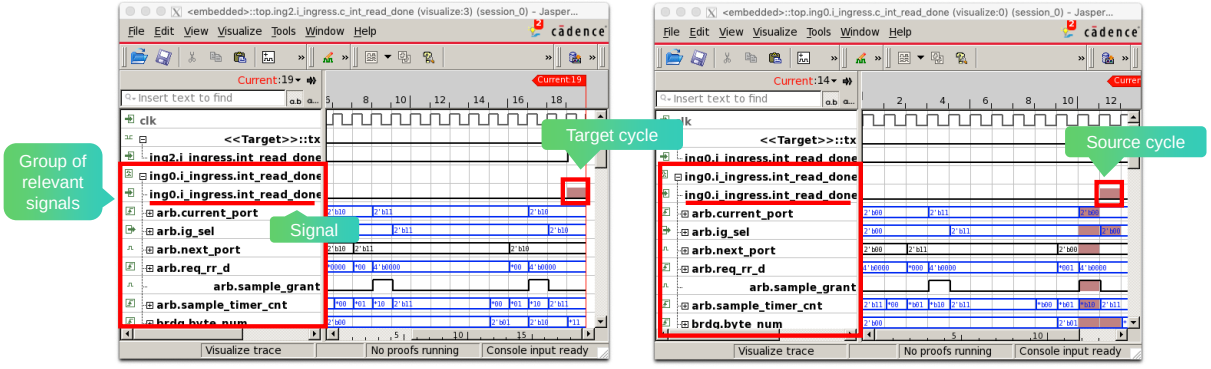
This document is for the sole use of Kun Wang of Seas

Highlight Relevant Difference Outcome

Both Visualize windows present the same relevant signals inside a group

Relevant signals have their relevant cycles highlighted if these cycles differ between the two windows

- Result example for signal “ing0.i_ingress.int_read_done” source cycle 14 and target cycle 19:



a) Target Visualize window “visualize:3”

b) Source Visualize window “visualize:0”

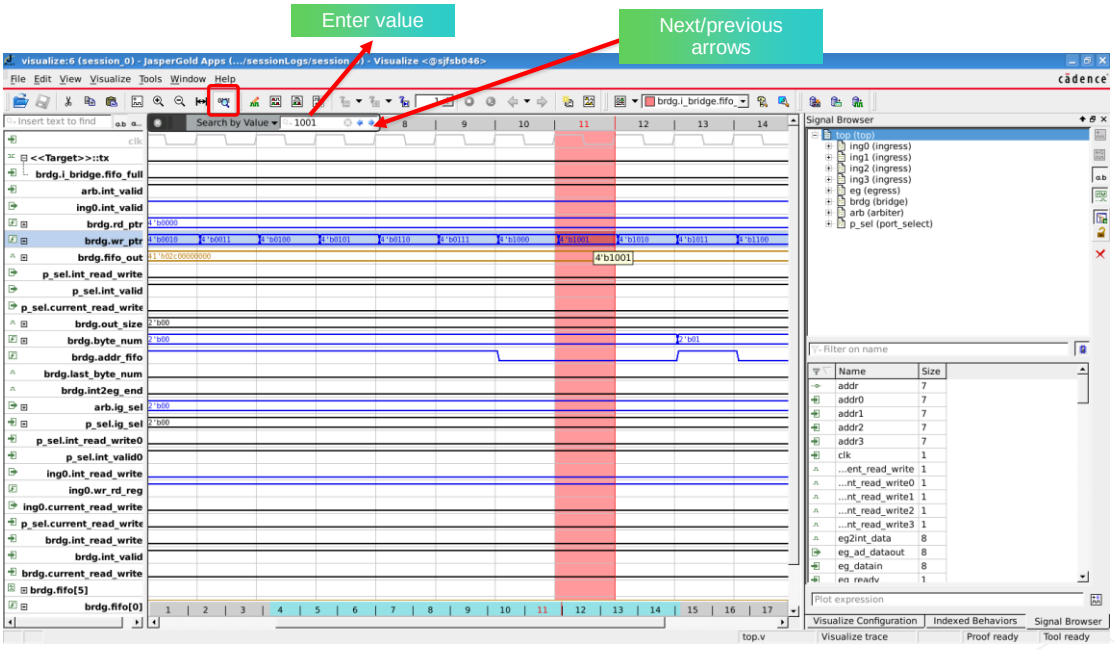
 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Features: Value Search



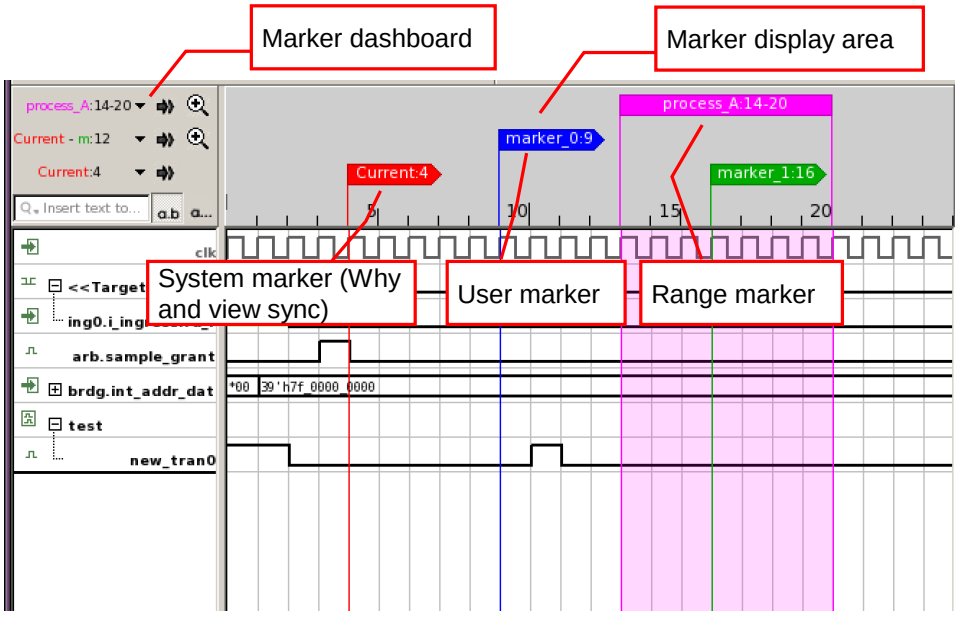
250 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Markers Overview



 © Cadence Design Systems, Inc. All rights reserved.

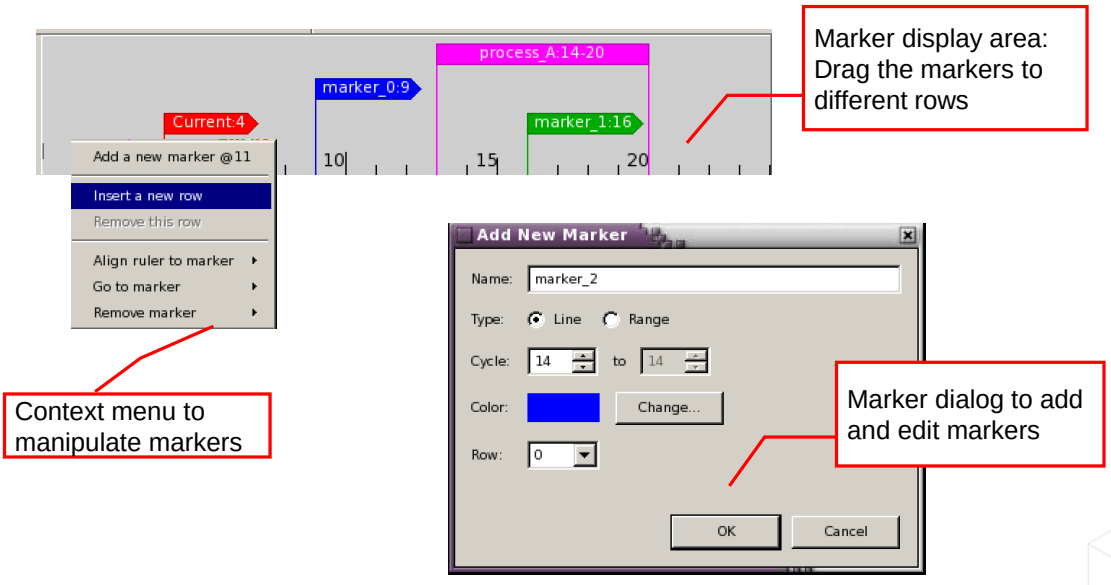
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Marker Display Area



 © Cadence Design Systems, Inc. All rights reserved.

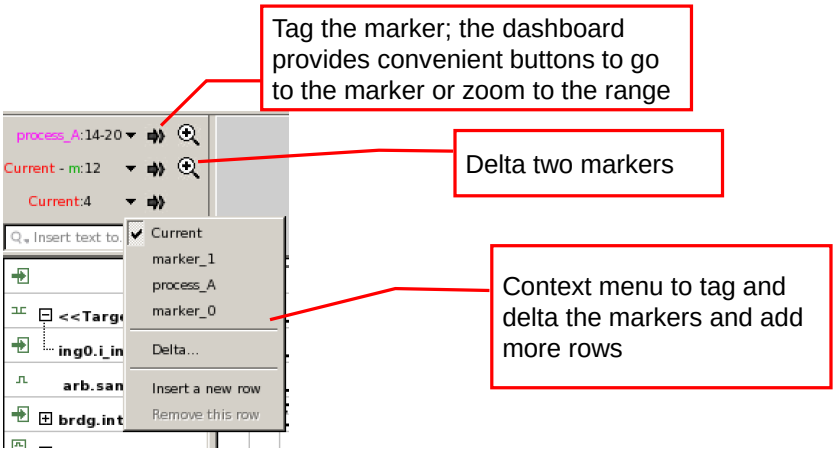
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Visualize Markers Dashboard



Custom Markers and Vertical Annotation video is available on Cadence® Learning and Support (http://support.cadence.com)

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

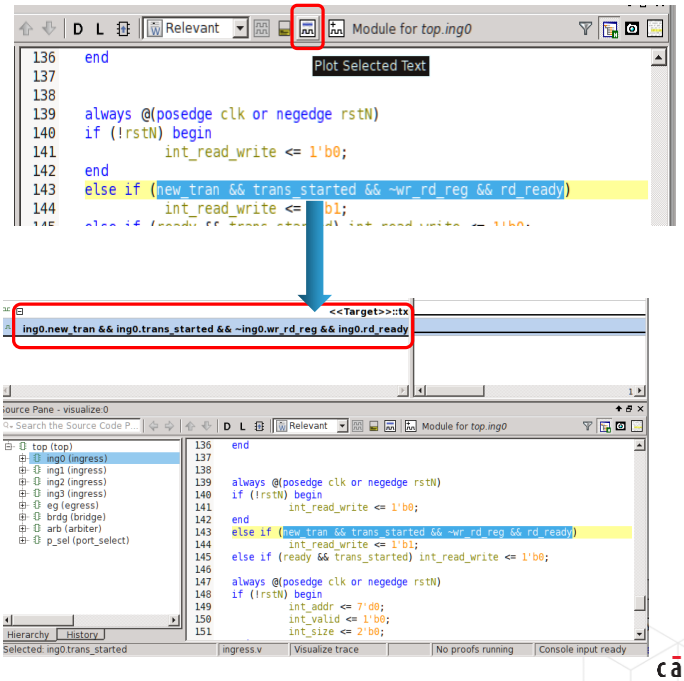
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Plot Selected Text As Expression

"Plot Selected Text" button (and menu item)

- Enabled when any text is selected
- Signals resolved according to the current instance
- Expression plotted in the waveform, if valid
- Error message if the selected text is an invalid expression, a constant, or a macro



© Cadence Design Systems, Inc. All rights reserved.

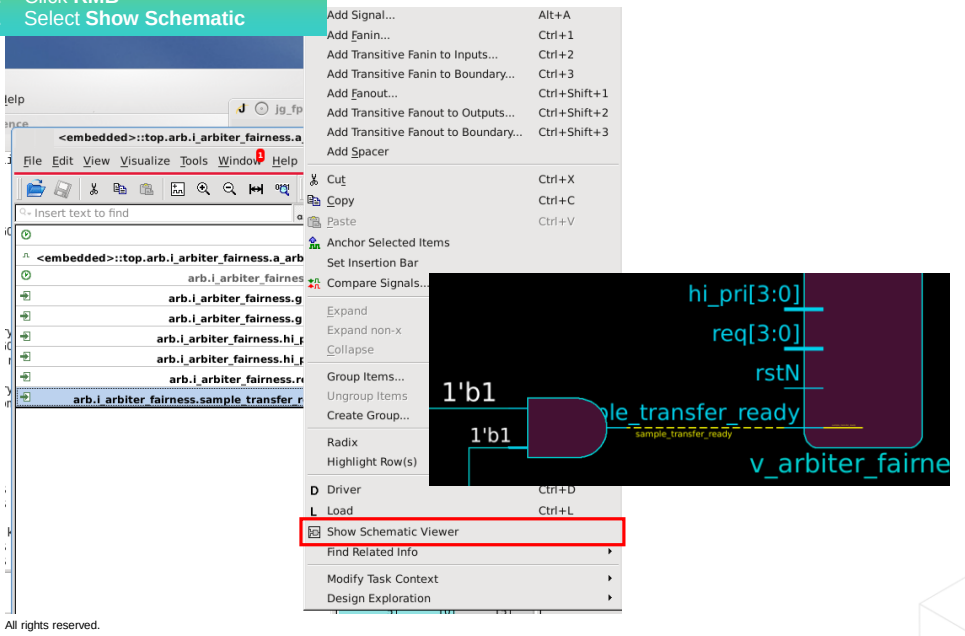
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Viewing Schematic from Visualize Window

1. Select **Signal** in Visualize
2. Click **RMB**
3. Select **Show Schematic**



255 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Summary

- Early design exploration without writing a property, a testbench, or requiring a complete design
- Fast root cause analysis with highlighting of relevant signals and cycles
- Analysis of other properties of existing waveforms
- Capture assertions and properties directly from the waveforms
- Easeir to debug CEX and cover
- Easeir to add constraints on the go-to check certain functionality
- QuietTrace can be useful for minimizing signal toggle while maintaining the same visualization target

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 8

Jasper™ Expert System

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Module Objectives

In this module, you will learn how to

- Jasper™ Expert System Introduction
- Use Jasper Expert System recommendations to improve the performance of your proofs

 © Cadence Design Systems, Inc. All rights reserved.

cadence

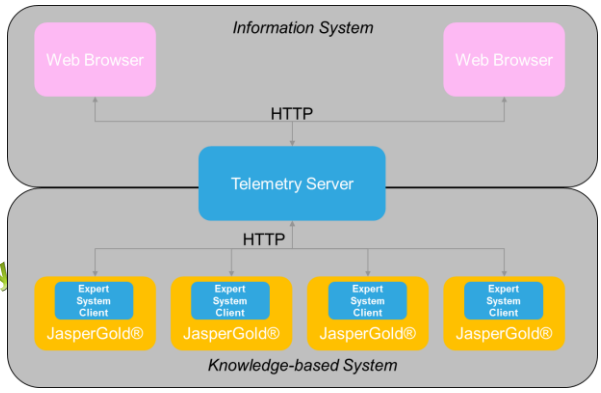
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Jasper Expert System

The Jasper Expert System consists of two components:



Knowledge-based System (client)

- Information regarding best practices and known solutions to common problems are automatically presented to the user
- Recommendations are in context to the current situation in JG
- Applicable to newbies and experts in formal and JG

Productivity

Information System (server)

- Tool usage information is aggregated from the clients for analysis by formal champions, engineering managers, etc.
- Visibility provided to formal champions

Visibility

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Knowledge-Based System

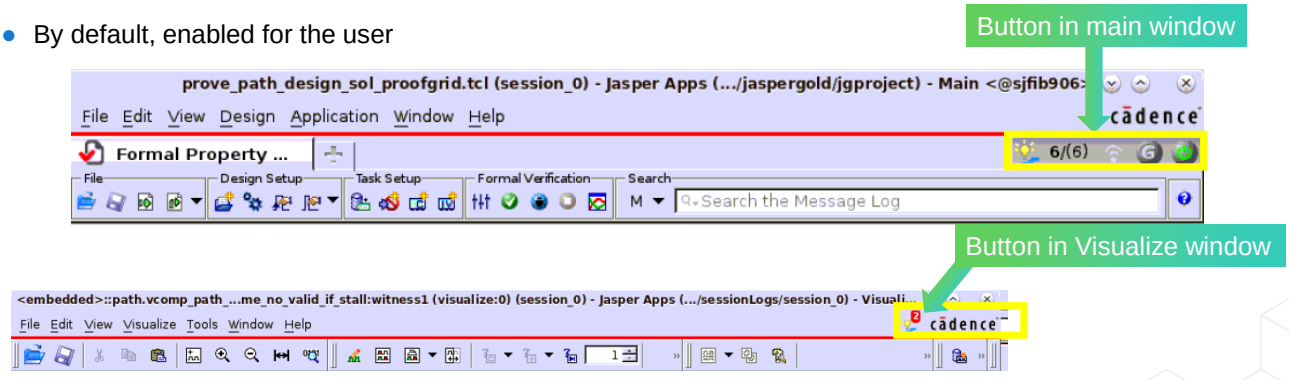
Recommends specific actions based on the user profile, task, and tool state

- The recommendations are “context-aware” and can range anywhere from overcoming analysis errors, and debugging to ways of possibly achieving proof convergence

Works in the background with minimal interference to Jasper’s usage

Part of the Jasper default installation

- By default, enabled for the user



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

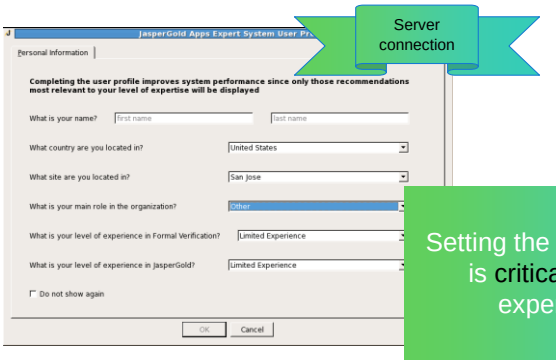
© Cadence Design Systems, Inc. All rights reserved.

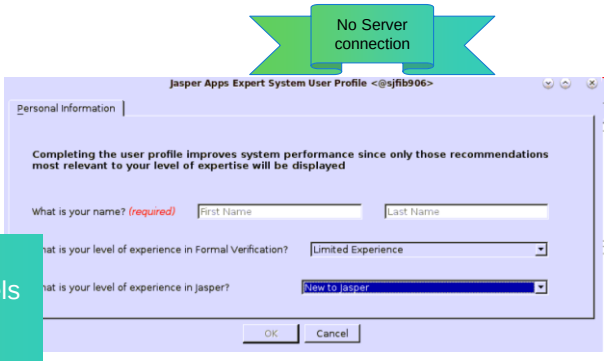
This document is for the sole use of Kun Wang of Seas

Knowledge-Based System Setup Information

At the initial login, the user is asked to create a user profile

- Dialog depends on the Information Server connection





Setting the proper levels is critical for user experience!

User profile can be changed later in the Expert System GUI

More information on the Knowledge-based System can be found in the following:

Help → Mini Guides → Jasper Expert System User Guide

 © Cadence Design Systems, Inc. All rights reserved.

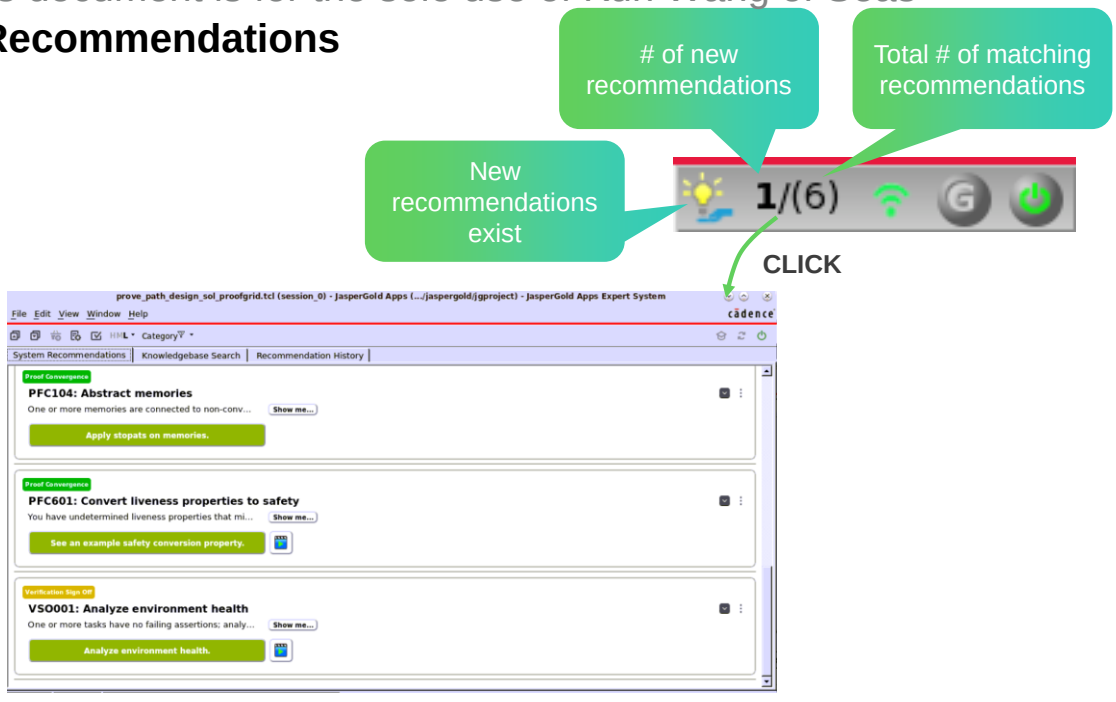
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Recommendations



 © Cadence Design Systems, Inc. All rights reserved.

cadence

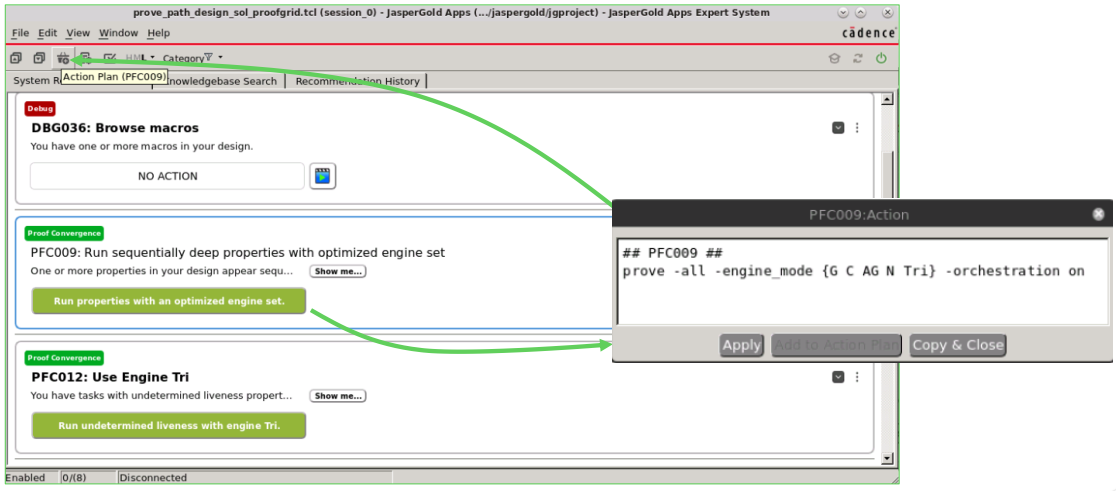
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Action Plan

- Apply recommendations 1 at a time
- Apply multiple recommendations → Action plan



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

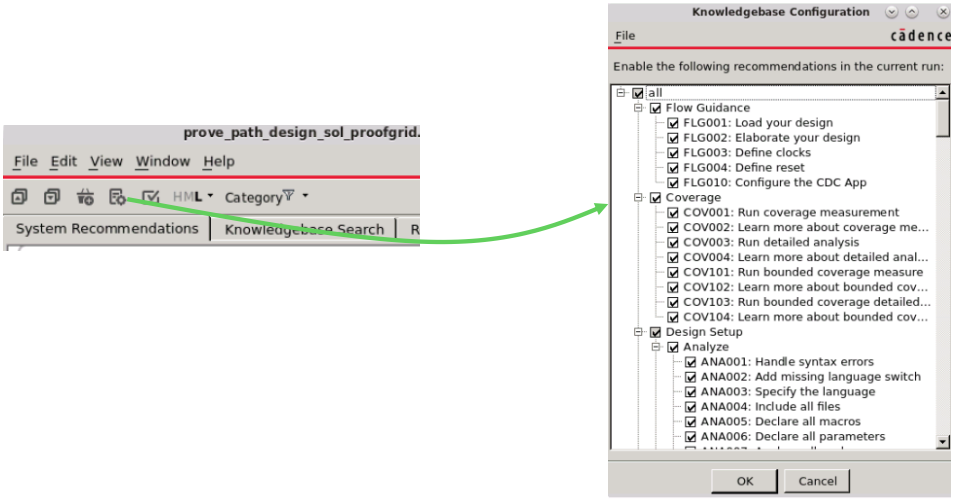
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Knowledgebase Configuration

User can control what recommendations are enabled/disabled

- Control can be for current run or for all runs



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

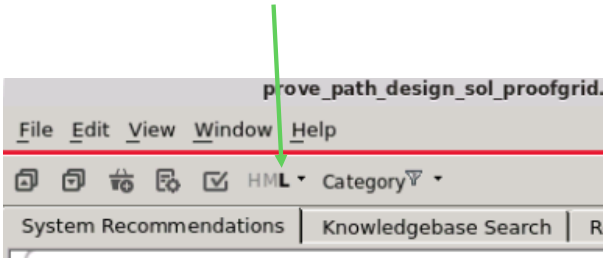
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Recommendation Verbosity

The Knowledge-based system was designed to minimize noise and attempts to provide a focused, highest-impact set of recommendations

Users can control the verbosity (H/M/L) of recommendations (default: low)



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

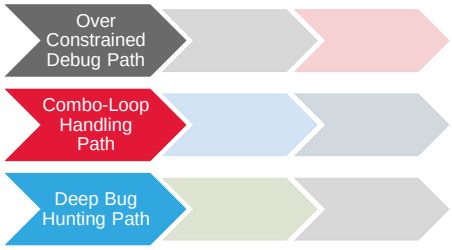
© Cadence Design Systems, Inc. All rights reserved.

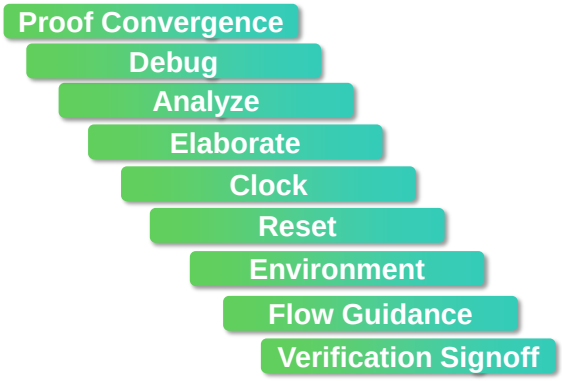
This document is for the sole use of Kun Wang of Seas

Summary: Jasper Expert System

Over 200 Recommendations and growing

Recommendation Paths





User-defined recommendations using RDE

- Specific to the user's environment
- Deploy certain Methodologies & BKMs

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 9

Interface Property Development

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Module Objectives

In this module, you will learn how to

- Follow a systematic process for deriving and implementing properties and covers
- Verify that a blackbox interface meets its functional specifications
- Apply techniques for effective interface verification

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Agenda

- Develop blackbox protocol properties
- Enumerate a finite set of fundamental behaviours for features extracted from the verification plan
- Enumerate a finite set of protocol categories with which standard protocols are associated
- Identify the subset of fundamental behaviors exhibited by a given protocol category
- Use these fundamental behaviors to drive the development of the protocol properties
- General versus Specific Properties
- Recommendations
- Lab description

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Considerations: Blackbox Protocol Property

Focus area identification

- Complex interfaces typically require a large investment in blackbox protocol property development and verification
- Correctness of the blackbox protocol properties is often a prerequisite for any other type of check
- It is essential that any focus area analysis include the complexity of the candidate block's interfaces

Complexity

- Blackbox protocol constraints can add to the overall complexity of the environment in which the design's assertions are evaluated:
  - Sequential constraints add complexity
  - Combinatorial constraints do not add complexity
- Careful consideration when making these properties efficient is necessary

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Problem of Infinite Protocol Diversity

The types of protocols that can be employed in designs today are infinite:

- Ranging from simple to exceedingly complex
- What approach does one follow for writing protocol properties?

Choice of approach can determine success or failure with respect to employing assertion-based verification on protocols.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Possible Approaches to Deal with Protocol Diversity

Unique protocol property approach

- Each check for a protocol feature requires developing a unique property for it

Protocol behavior approach

- Identify which subset of fundamental behaviors are exhibited by a given protocol
- Use these behaviors to drive the development of the protocol properties

Use of such behaviors greatly simplifies the development of blackbox protocol properties

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

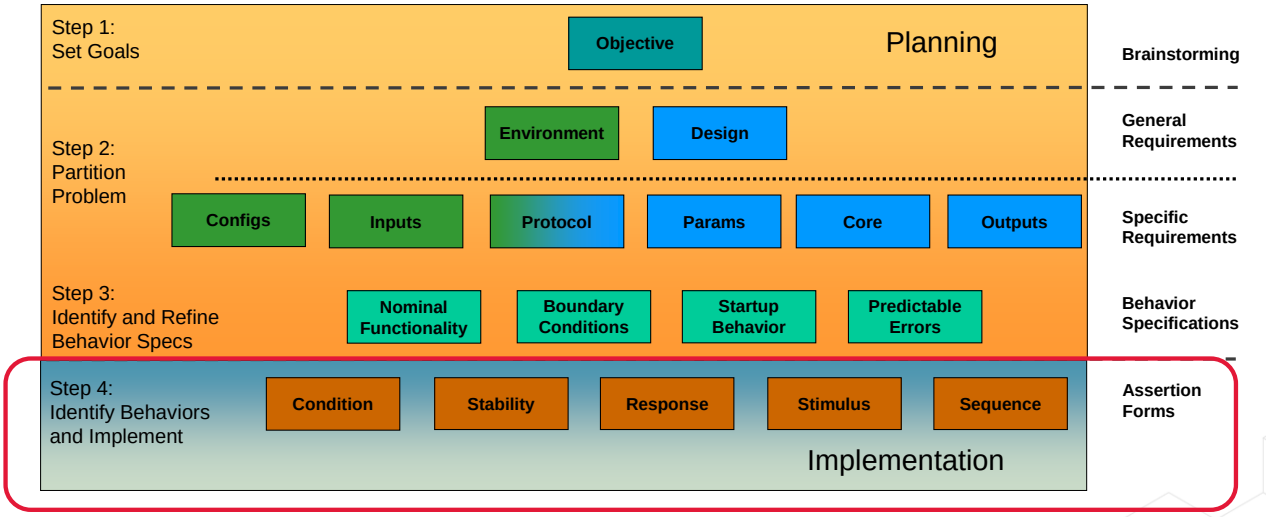
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Complete Assertion Development Process

This module is focused on step 4

We will talk about assertion forms



273 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Agenda

- Develop blackbox protocol properties
- Enumerate a finite set of fundamental behaviours for features extracted from the verification plan
- Enumerate a finite set of protocol categories with which common protocols are associated
- Identify the subset of fundamental behaviors exhibited by a given protocol category
- Use these fundamental behaviors to drive the development of the protocol properties
- General versus Specific Properties
- Recommendations
- Lab description

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Fundamental Behaviors

All features extracted in the plan can be broken down into a set of fundamental behaviors:

- Condition
- Stability
- Response
- Stimulus
- Sequence

Each of the behaviors can be mapped onto assertion forms

It is not important how long the list of behaviors is or what they are named

It is a starting point

- A mental checklist – Did I address this kind of behavior?

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Condition Behavior

Requirement that restricts values of signals or imposes relationship between them

- Also useful for parameter checking
- Typically, the easiest properties to write

Behavioral extent

- Signal can take on only certain values or fall within a certain range
- Boolean relationship between signals or slices of an array
- Conditional relationship between signals

Temporal extent

- Invariant: Independent of time

Assertion forms captured in:

- Simple Boolean expression
- Same Cycle (Overlapping) implication

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Examples: Condition

Data width can only be 8, 16, or 32 bits

```
P1: assert property
( DWIDTH==8 || DWIDTH==16 || DWIDTH==32 );
```

Device selection inputs are nonoverlapping; at most, one is high at any given time

```
P2: assert property ($onehot0(select[7:0]));
```

If writing to memory, then a chip select must also be active

```
P3: assert property (mem_wr |-> mem_cs);
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Stability Behavior

Signal must be stable during a given period

- Typically, the value of the signal does not matter

Behavioral extent after some event is observed

- Signal must remain stable
- Signal must remain stable for N cycles
- Signal must remain stable until some other event occurs

Temporal extent

- Period of stability is typically bounded by time or event

Assertion forms

- Periods defined by time typically use the `$stable()` methods within assertion languages
- Periods defined by events can leverage the until and throughout constructs

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Examples: Stability

If no PUSH, then the write pointer must not change

```
P1: assert property (!push |=> $stable(wr_ptr));
```

When a write starts, the address must be stable for the subsequent three cycles, i.e., the same address for 4 cycles

```
P2: assert property ($rose(wr)} |=> $stable(addr)[*3]);
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Response Behavior

Many assertions are of this type

Behavioral extent

- After some event is observed
  - Signal should be asserted or deasserted
  - Signal value should change
  - Then, a particular sequence should occur
  - Functional transaction should end
- Cause and effect relationship

Temporal extent

- Response can be immediate or within some time window

Assertion forms

- Can involve Booleans or sequences
- Typically includes next cycle (non-overlapping) implication

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Examples: Response

If grant, then request should be deasserted on the next cycle

```
P1: assert property (gnt |=> $fell(req));
```

If request, then grant should be asserted within five cycles

```
P2: assert property ($rose(req) |-> ##[1:5] gnt);
```

Anytime a write occurs, the address must change on the next cycle

```
P3: assert property ( wr |=> $changed(addr) );
```

If request, then eventually there should be a grant

```
P4: assert property ($rose(req) |=> s_eventually gnt);
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Response Behavior: Transactions

Some behaviors are described in terms of transactions

Two different types of transactions are considered

- Interface transaction
- Functional end-to-end transaction

Interface transaction

- A fundamental unit of transfer between two blocks
  - This unit of transfer has a unique meaning
  - This unique meaning is defined in the protocol's specification
  - The transfer is composed of a set of signals and their values over time
    - Can be a single signal at a single instance of time

Functional end-to-end transaction

- A particular sequence on a set of inputs that sometime later yields a particular behavior or sequence on a set of output signals
  - In this respect, most designs can be considered a transaction processor

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Response Behavior: Transactions Examples

Properties describing transactions can be unbounded or bounded

- Depending whether latency is known

Typical form for unbounded is shown here

```
sequence txn_input_sequence;
   A ##1 B[*1:3] ##1 D;
endsequence
sequence txn_output_sequence;
   SETUP ##1 (VALID && SEL)[*8];
endsequence

P5: assert property (
    txn_input_sequence) |=> s_eventually txn_output_sequence );
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Stimulus Behavior

Behavioral extent

- Given some observed behavior, there must have been a previous cause
  - For example, a specific input sequence
- An effect must have had a prior cause
  - This is the opposite of response behavior

Temporal extent

- Typically involves one or more cycles in the past

Assertion forms

- Typically use a combination of overlapping implications and functions that look at the past

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Examples: Stimulus

The data-ready signal is only asserted immediately following a read operation to this memory

```
P1: assert property (drdy |-> $past(mem_read) );
```

An acknowledge must only occur after a request that has not been acknowledged until now

```
reg req_without_ack; //Use as a flag
 always @(posedge clk or negedge rst_n)
   if (!rst_n)
     req_without_ack <= 1'b0; //Clear flag
   else begin
    if (req && !ack)
      req_without_ack <= 1'b1; //Set flag
    else if (ack)
      req_without_ack <= 1'b0; //Clear flag
   end

P2: assert property ( ack |-> req_without_ack );
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Sequence Behavior

Behavioral extent

- Sequences that should occur
  - Cover statements
- Sequences that should never occur
  - Assertion statements

Temporal extent

- Can be instantaneous Boolean or multicycle sequences

Assertion forms

- Sequence syntax and operators

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Examples: Sequence

Cover that a FIFO can go from empty to full and back to empty

```
C1:  cover sequence (empty ##[+] full ##[+] empty);
```

A 16 entry FIFO should not go from empty to full in less than 16 cycles

```
P1: assert property ( not(empty ##[0:15] full) );
```

```
P1_W_IMP: assert property (empty |-> !full[*16) );
```

Are these 2 properties equivalent?

Yes

What is the advantage of using `P1_W_IMP`?

Jasper™ can create an automated precondition cover for P1_W_IMP.
It is impossible to witness P1 by definition, as it's a "not" property.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Behavior Summary Table

Behavior	Description
Condition	Requirement that restricts the values of signals or imposes a relationship among them
Stability	Requirement that a signal must be stable during a given period
Response	Requirement that one or more signals must change in response to an event or that a relationship must hold following an event
Stimulus	Requirement that a given response must occur only following the appropriate stimulus
Sequence	Requirement regarding sequences of conditions or events occurring in the design

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Agenda

- Develop blackbox protocol properties
- Enumerate a finite set of fundamental behaviours for features extracted from the verification plan
- Enumerate a finite set of protocol categories with which common protocols are associated
- Identify the subset of fundamental behaviors exhibited by a given protocol category
- Use these fundamental behaviors to drive the development of the protocol properties
- General versus Specific Properties
- Recommendations
- Lab description

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Process for Blackbox Protocol Development

To be useful, this process must be repeatable:

- Identify a finite set of protocol categories with which common protocols can be associated
- Identify the subset of fundamental behaviors exhibited in each protocol category
- Use those behaviors to drive the protocol properties development

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Categories of Protocols

Most interface protocols belong to one of the following categories:

- Ready/valid
- Request/grant
- Bus protocols
- Packet-oriented protocols

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

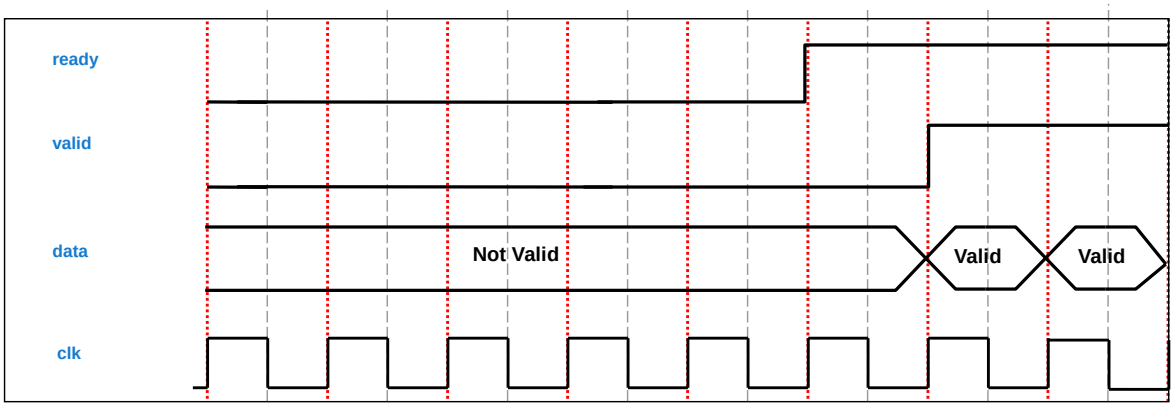
This document is for the sole use of Kun Wang of Seas

Ready/Valid Protocol Category

Data sink side of the interface asserts a *ready* signal indicating that it is able to accept data

After the data sink is ready, the data source is allowed to send data to the data sink side

The data is valid when the source asserts the *valid* signal



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

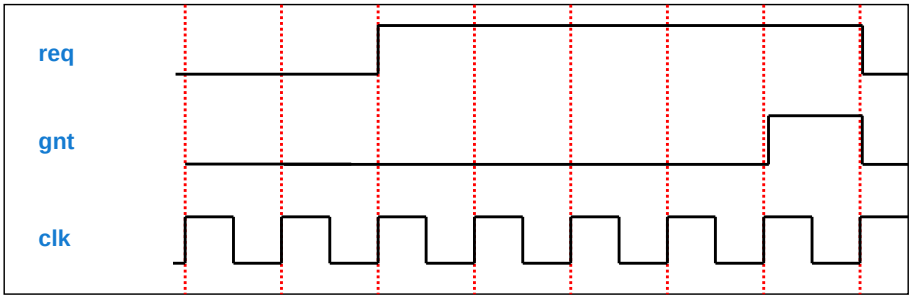
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Request/Grant Protocol Category

Master side asserts a request signal to send data or access some shared resource

Grant signal is asserted after the data is accepted or access to a shared resource is granted by the arbiter



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Bus Protocol Category

Various signals control the handshaking of data transfers on the bus

- Similar to request/grant protocols but more complex

Buses can also be pipelined so that several transactions can be in flight at any one time

Burst mode capabilities can also be supported so that more than one word of data can be transferred during a single transaction

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Agenda

- Develop blackbox protocol properties
- Enumerate a finite set of fundamental behaviours for features extracted from the verification plan
- Enumerate a finite set of protocol categories with which common protocols are associated
- Identify the subset of fundamental behaviors exhibited by a given protocol category
- Use these fundamental behaviors to drive the development of the protocol properties
- General versus Specific Properties
- Recommendations
- Lab description

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Behaviors Exhibited by Ready/Valid Protocol Category

	Stimulus	Stability	Response	Sequences	Condition
Ready/Valid	X	-	-	-	-
Request/Grant	X	X	X	-	-
Bus Protocols	X	X	X	X	X
Packet-Oriented Protocols	X	-	X	X	X

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

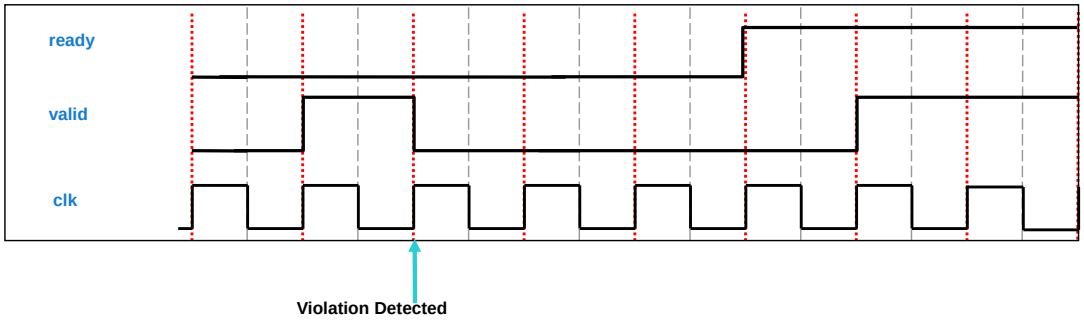
Ready/Valid Stimulus Behaviors

In a typical ready/valid protocol, the data sink side of the interface asserts a ready signal, indicating that it is able to accept data

After the data sink is ready, the data source is allowed to send data to the data sink side

The data is valid when the source asserts the valid signal

```
P1: assert property (valid |-> ready);
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Behaviors Exhibited by Request/Grant Protocol Category

	Stimulus	Stability	Response	Sequences	Condition
Ready/Valid	X	-	-	-	-
Request/Grant	X	X	X	-	-
Bus Protocols	X	X	X	X	X
Packet-Oriented Protocols	X	-	X	X	X

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

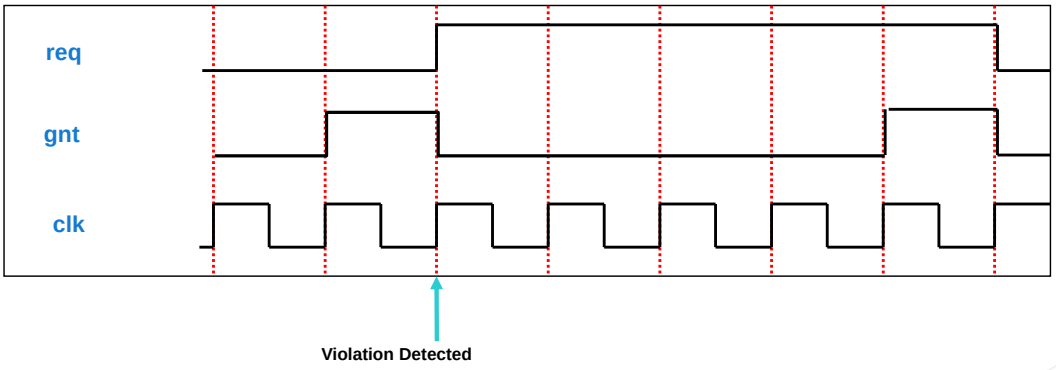
This document is for the sole use of Kun Wang of Seas

Request/Grant Stimulus Behaviors

In a request/grant protocol, the grant signal should be asserted only when there is a corresponding request

The responder should not respond without a reason

```
P1: assert property (gnt |-> req);
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

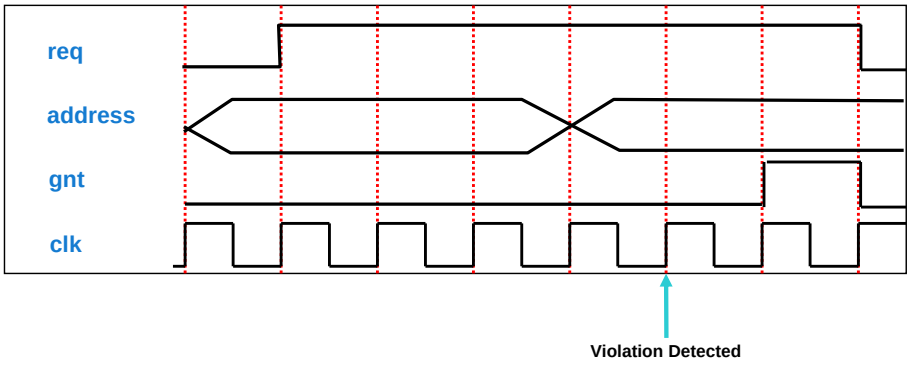
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Request/Grant Stability Behaviors

A fundamental requirement of a request/grant protocol is, after a request, the address must be held stable until the grant signal is asserted

```
P1: assert property ( (req && !gnt) |=> ($stable(address)) );
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Request/Grant Response Behaviors

When a request is initiated, something is expected to happen

Response may be guaranteed to happen:

- Within time bound
- Response expected at a specific time

```
P1: assert property ( req |=> !gnt[*4] ##1 gnt ) ;
```

- Response expected at an indeterminate time in the future

```
P2: assert property ( req |-> s_eventually gnt );
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Bus Protocol Behaviors

	Stimulus	Stability	Response	Sequences	Condition
Ready/Valid	X	-	-	-	-
Request/Grant	X	X	X	-	-
Bus Protocols	X	X	X	X	X
Packet-Oriented Protocols	X	-	X	X	X

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Bus Protocols Sequence Behaviors

Bus protocols may have a set of framing signals that indicate the duration of a transaction or a phase of a transaction

A bus that uses start-of-transfer (sot) and end-of-transfer (eot) signals to frame a transaction

```
reg pending_sop;

  always @(posedge clk or negedge rst_n)
    if (!rst_n)
      pending_sop <= 1'b0;
    else begin
      if (sop && !eop)
        pending_sop <= 1'b1;
      else if (eop_out)
        pending_sop <= 1'b0;
    end

NOT_TWO_SOPS_WO_EOP: assert property ( @(posedge clk) sop |-> !pending_sop);
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Identifying Behaviors and Protocols Category Summary

The types of protocols that can be employed in designs today are infinite

They range from simple to exceedingly complex

Choice of approach can determine success or failure

Observations:

- Most interface protocols belong to a finite set of categories
- All protocols exhibit a finite set of behaviors

Development of protocol properties can be made much simpler by following an approach that does the following:

- Identify the protocol category you are working with
- For the protocol category, identify its exhibited behaviors
- Capture each of the exhibited behaviors with the appropriate assertion

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Agenda

- Develop blackbox protocol properties
- Enumerate a finite set of fundamental behaviours for features extracted from the verification plan
- Enumerate a finite set of protocol categories with which common protocols are associated
- Identify the subset of fundamental behaviors exhibited by a given protocol category
- Use these fundamental behaviors to drive the development of the protocol properties
- General versus Specific Properties
- Recommendations
- Lab description

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

General Versus Specific Properties

Do not add unnecessary detail to properties

Bringing in too much detail into your properties runs the risk of:

- Overconstraining your system
- Increasing the complexity of your properties
- Writing an incorrect property

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

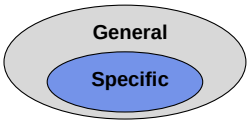
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Example: General Versus Specific Properties

If the general assertion passes, the specific assertion also passes

```
general:  assert property ( not(a && b)     );
specific: assert property ( not(a && b && c));
```



Of course, these properties are not equivalent

- But the general property may tell one all one needs to know

case	a	b	c	General a && b	Specific a && b && c
0	0	0	0	Pass	Pass
1	0	0	1	Pass	Pass
2	0	1	0	Pass	Pass
3	0	1	1	Pass	Pass
4	1	0	0	Pass	Pass
5	1	0	1	Pass	Pass
6	1	1	0	Fail	Pass
7	1	1	1	Fail	Fail

General pass implies specific pass

That may be “good enough”

Only combination of `a, b and c` where the result is different
We decide if that is important

307 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Considerations: General Versus Specific Properties

- Use general instead of specific properties
- This should decrease the complexity of your properties
- You must be sure that the general property does not miss any required behavior

```
general: assume property
     (@(posedge clk) ((d) |=> (e));

specific: assume property
     (@(posedge clk) ((a ##1 b[*5] ##1 c ##1 d) |=> (e));
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Why Specific Properties May Be Undesirable?

Let us analyze a different example to understand the reasons why

```
specific : assert property (  req && !gnt |=> C ##1 D );
general  : assert property (  req         |=> C ##1 D );
```

Clearly, these are not equivalent

However, if the COI for `gnt` is very large, then the general assertion may change the result from `Undetermined` to `PASS/FAIL`

If the general assertion passes, then so will the specific one

We may need to adapt our properties if:

- We have `Undetermined` results
- Have excessive runtimes
- Give too many false negatives

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Agenda

- Develop blackbox protocol properties
- Enumerate a finite set of fundamental behaviours for features extracted from the verification plan
- Enumerate a finite set of protocol categories with which common protocols are associated
- Identify the subset of fundamental behaviors exhibited by a given protocol category
- Use these fundamental behaviors to drive the development of the protocol properties
- General versus Specific Properties
- Recommendations
- Lab description

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Avoid Constraint Development Extremes

Constraint development extremes

- Start with zero constraints and add constraints one by one until you converge to 100% constraints
  - This means that you use the design RTL and your assertions to identify which constraints are missing based solely on false negatives
- Start with 100% constraints
  - This means that you attempt to define 100% of your constraints before testing your constraints based on your design RTL
  - Has the risk of over-constraining your environment

Both options require long development and testing times before converging to a correct set of constraints

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Recommended Constraint Development Process

Target something in between these extremes before beginning your checks

Implement the most obvious or meaningful constraints upfront

- Maybe 70-80%
  - Then, incrementally add additional constraints
- Maybe one or two at a time
  - By incrementally adding additional constraints, it is easier to recognize and debug overconstrained situations
  - Adding too many constraints at any one time can cause difficulty in finding the root of the overconstraint
  - To further aid in identifying and debugging an overconstrained situation, use cover statements to test the correctness of your environment

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Verification Components (Checkers)

Recommended to put blackbox protocol properties in a separate module

Verification components:

- Are generic modules that encapsulate properties and auxiliary code
- Provide an infrastructure reusable properties for commonly used interfaces and components
- Just as with UVCs in a UVM testbench, instance and `bind` to where they are needed

```
module fr_cnt_chkr (
        input logic  CLK,
        input logic  rst_n,
        input logic  CNT_ENB,
        input logic[2:0]  CNT_VAL
       );

  ast_cnt_ne_6 : assert property (
         @(posedge CLK) CNT_VAL != 6 );
endmodule
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Performance Considerations

Blackbox protocol constraints add to the complexity of the rest of your assertions

It is important to be cognizant of complexity issues when writing them

- Use general assertions when applicable
- Minimize the number of state bits in the properties
- Use auxiliary code, which can reduce the added complexity of the constraint
- Make sure that the auxiliary code is efficient, that is, minimize the use of:
  - Memories
  - Counters or upperbounds of counters
    - For example, don't use an integer (32 bits) if one only needs to count from 0 to 7
      Use a logic [2:0] instead

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Agenda

- Develop blackbox protocol properties
- Enumerate a finite set of fundamental behaviours for features extracted from the verification plan
- Enumerate a finite set of protocol categories with which common protocols are associated
- Identify the subset of fundamental behaviors exhibited by a given protocol category
- Use these fundamental behaviors to drive the development of the protocol properties
- General versus Specific Properties
- Recommendations
- Lab description

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Bus Protocol Lab Overview

The design is called a Transfer engine

It provides automated data transfers to and from various interfaces

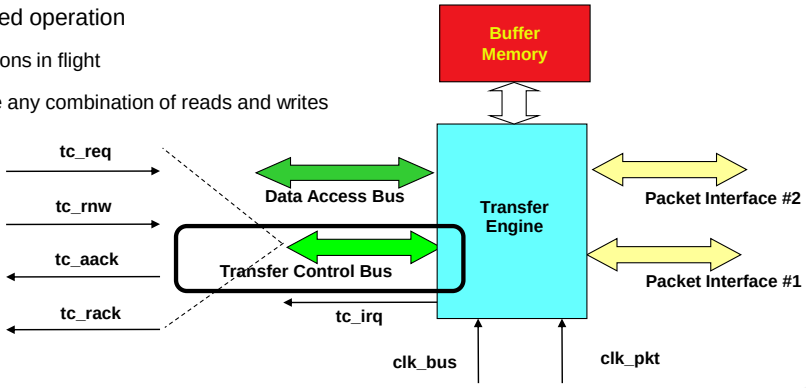
The bus protocol example comes from the Transfer Control Bus

Block transfers are initiated when a transfer request is written into the Transfer Control Bus

The Transfer Control Bus is also used to monitor the status of the transfers

This bus protocol supports pipelined operation

- Pipeline supports up to 4 transactions in flight
- These in-flight transactions can be any combination of reads and writes



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Bus Protocol Lab: Pin Description

Pin name	Width	Direction	Description
tc_req	1	In	• This is the request line. • Active high and indicates that either a read or a write request is on the bus. • These outstanding requests can be any mix of read and write requests. • Once *tc_req* goes high, it must stay high until acknowledged by *tc_aack.*
tc_rnw	1	In	• This is the read/write modifier. • When high, it implies that the requested transaction is a read transaction. • When low, it implies that the requested transaction is a write transaction. • tc*_rnw* should be driven when tc*_req* is asserted and stay stable up to and including the cycle that tc*_aack* is asserted.
tc_aack	1	Out	• This is the address acknowledge signal. • When this signal goes high, it indicates that a transaction has been acknowledged and the write data (if applicable) and address have been latched. • *tc_aack* can be asserted speculatively if it can be guaranteed that a new transaction will be accepted. • If there are four pending transactions, *tc_aack* should no longer be asserted until at least one of the pending transactions is completed.
tc_rack	1	Out	• This is the read data acknowledge. • This signal will go high when the read data bus has valid read data. • Read data is only valid for a single clock cycle. • There is a one-to-one correspondence between read requests and tc_rack so that if *tc_rack* is asserted, it implies that sometime in the past, there was a read request. • If *tc_rack* is asserted during two back-to-back clock cycles, it implies that there were two corresponding read requests (possibly to the same address).

 © Cadence Design Systems, Inc. All rights reserved.

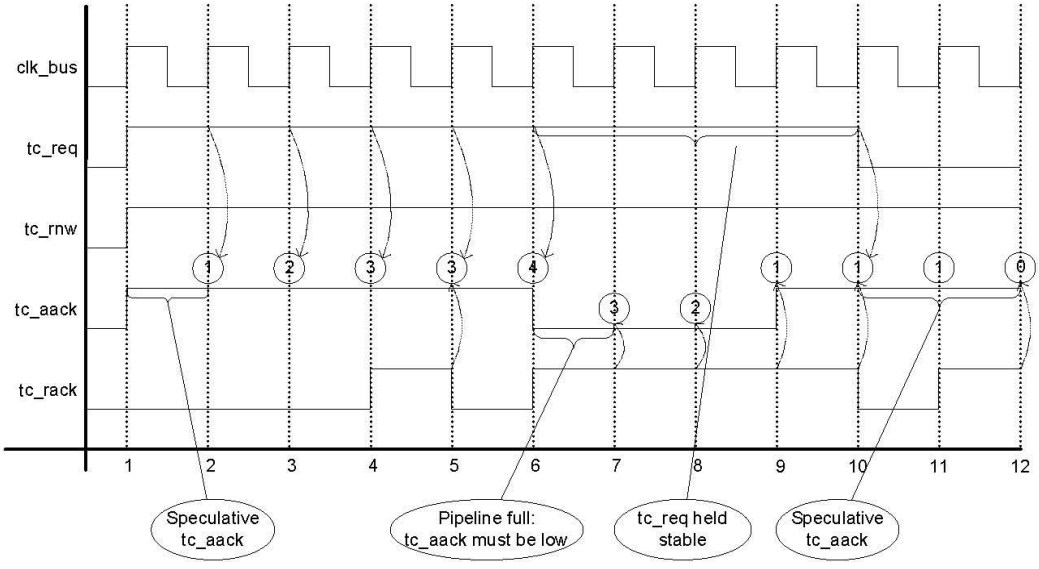
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Example: Waveform of Transfer Control Bus



318 © Cadence Design Systems, Inc. All rights reserved.

cadence

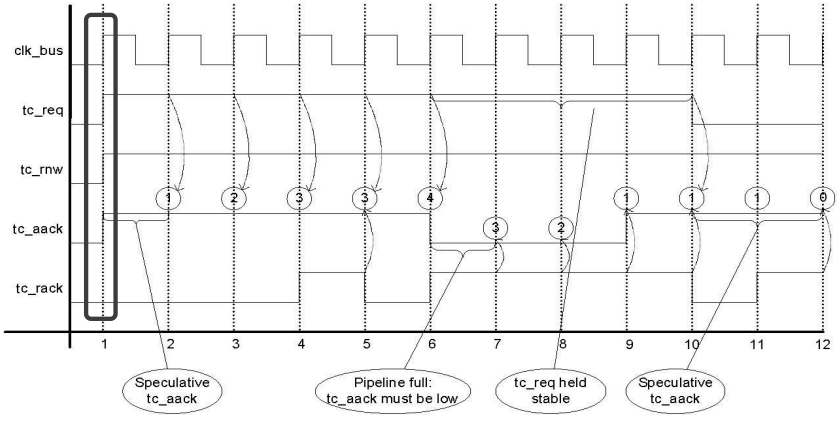
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Waveform of Transfer Control Bus: Clock 1

- *tc_req* is low at the rising edge of *clk_bus*
- Therefore, there is no activity on the bus
- There are also no pending transactions in the pipeline
- *tc_aack* is speculatively high, which means that it is asserted without a corresponding request



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Waveform of Transfer Control Bus: Clock 2

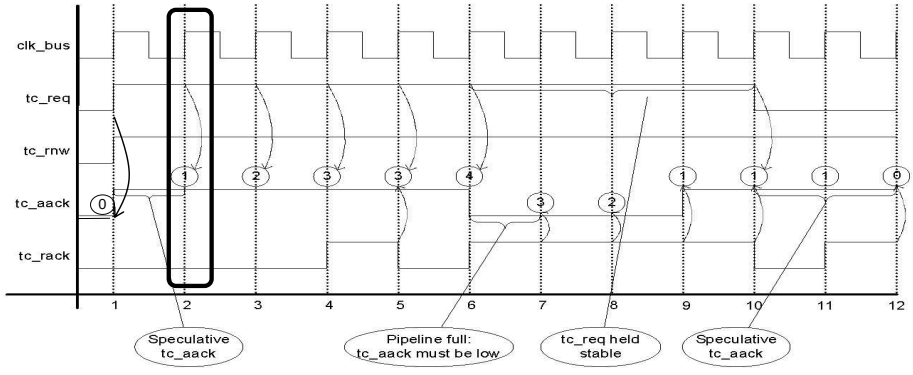
tc_req is asserted along with *tc_rnw,* which indicates a read transaction has been requested

This transaction is immediately acknowledged through the assertion of *tc_aack*

In this case, *tc_aack* is speculatively asserted

This speculative assertion of *tc_aack* is legal because there are less than four pending transactions

The *1* in the circle above *tc_aack* indicates that there is only 1 pending transaction



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

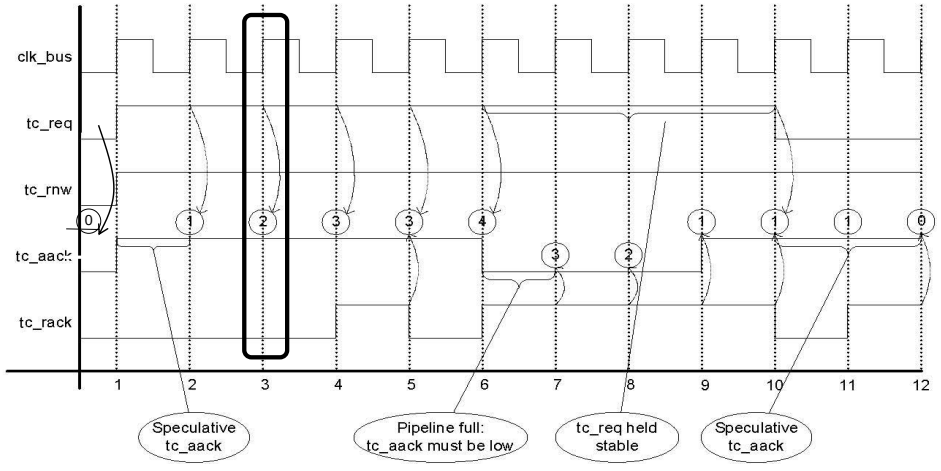
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Waveform of Transfer Control Bus: Clock 3

tc_req is asserted and immediately acknowledged

Because *tc_rack* has not yet been asserted, the number of outstanding transactions increases to 2



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

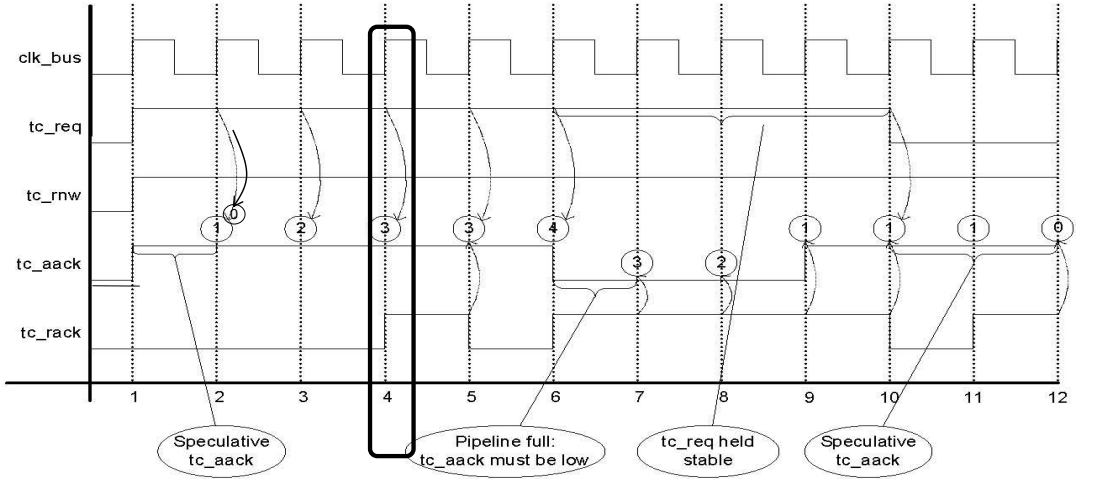
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Waveform of Transfer Control Bus: Clock 4

tc_req is asserted and immediately acknowledged

Because *tc_rack* has not yet been asserted, the number of outstanding transactions increases to 3



322 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

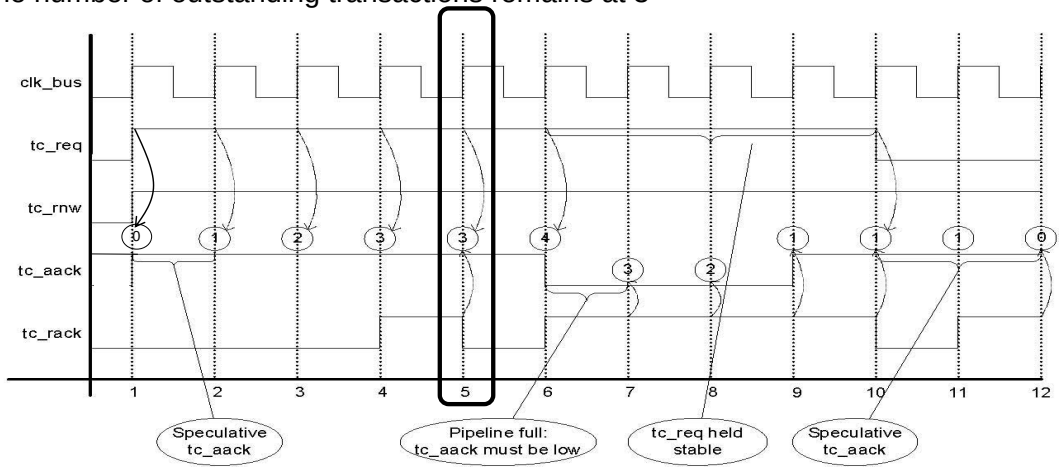
This document is for the sole use of Kun Wang of Seas

Waveform of Transfer Control Bus: Clock 5

tc_rack is asserted at the same time as *tc_aack*

One of the read transactions is completed, while another is added

The number of outstanding transactions remains at 3



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

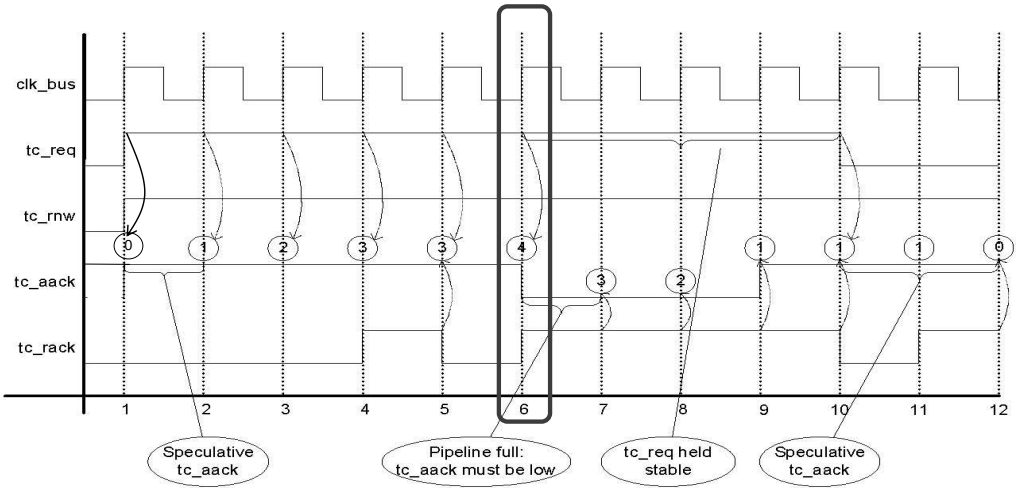
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Waveform of Transfer Control Bus: Clock 6

tc_req is asserted and immediately acknowledged

Because *tc_rack* is not asserted, the number of outstanding transactions increases to 4



324 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

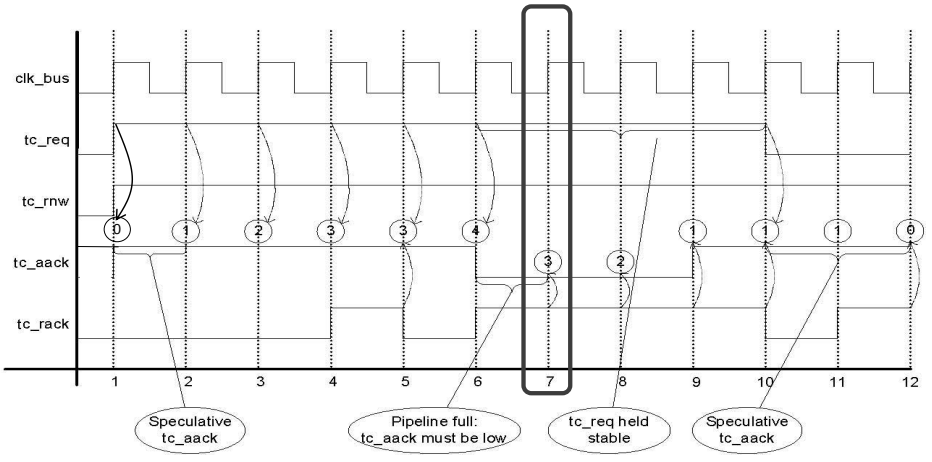
This document is for the sole use of Kun Wang of Seas

Waveform of Transfer Control Bus: Clock 7

tc_aack is deasserted because the pipeline is full

tc_rack is asserted without *tc_aack* being asserted

Therefore, a read transaction is completed, and the number of outstanding transactions decreases to 3



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

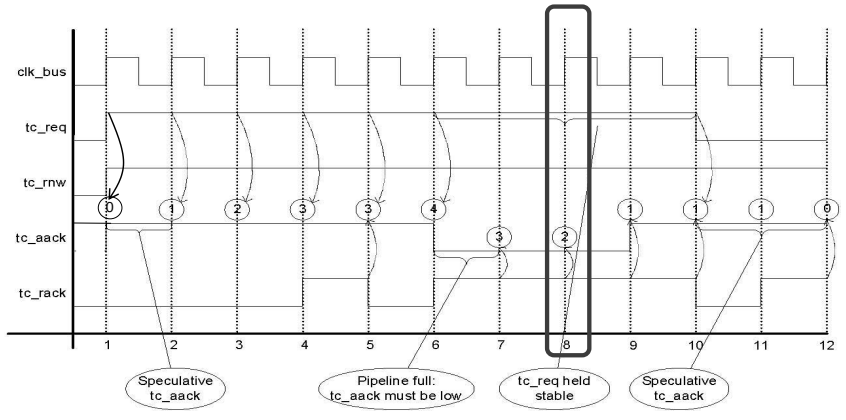
Waveform of Transfer Control Bus: Clock 8

tc_aack was not asserted in the previous cycle

The requestor must hold *tc_req, tc_rnw,* and *tc_addr* (not shown) stable during this cycle and during each of the next 2 cycles

In this cycle, *tc_rack* is asserted

Therefore, the number of outstanding transactions decreases to 2



326 © Cadence Design Systems, Inc. All rights reserved.

cadence

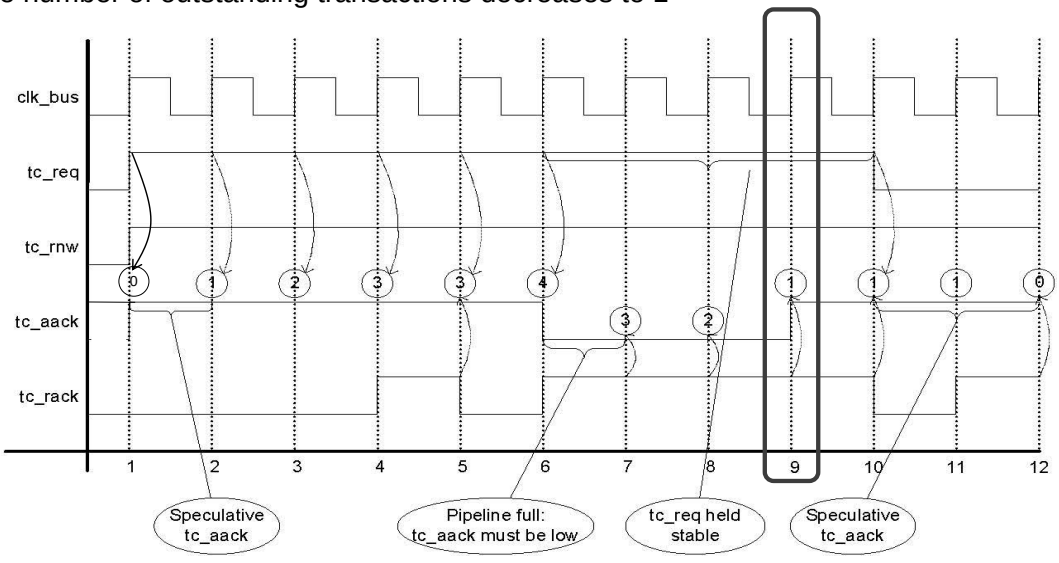
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Waveform of Transfer Control Bus: Clock 9

The number of outstanding transactions decreases to 1



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

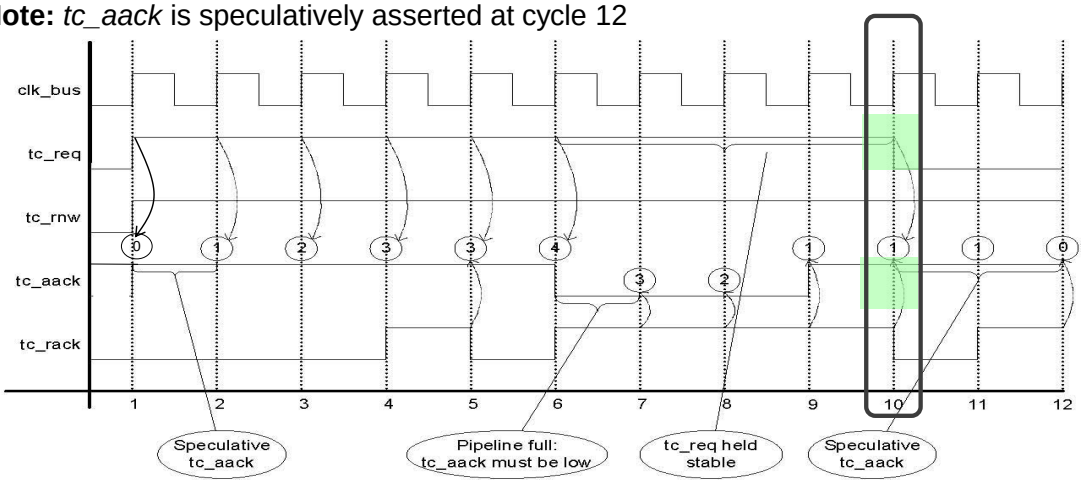
This document is for the sole use of Kun Wang of Seas

Waveform of Transfer Control Bus: Clock 10

tc_req is finally acknowledged

Because this happens when *tc_rack* is being asserted, the number of outstanding transactions stays the same

Note: *tc_aack* is speculatively asserted at cycle 12



328 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Recommendation of Approaching the Labs

We want to make the best use of time.

- Please ask questions if you are stuck or don't understand the instructions!

1. Take a maximum of 20 minutes to read and understand the spec:
   - The spec may or may not be unambiguous – ask if unsure.
   - Try to answer your own questions by using the tool.
     - For example, if you want to know whether a `tc_wack` can occur the same cycle as a `(tc_req && tc_aack && !tc_rnw)` then write a cover and run the tool.
2. Take a maximum of 30 minutes to write down in natural language on the worksheet provided the properties (assumes and asserts) that you think are required in the categories which we discussed.
3. Read the comments in the files specified in the lab book, which propose the properties which are required.
4. Check your description against the comments. Do your properties state the behavior that the comments suggest? Did you define any useful behavior that the comments did not state?
5. Convert the comments in the file provided into SVA + your own properties. If you think that they are useful, then try to prove the properties following the lab book instructions.
6. Repeat steps 2 to 5 but this time for cover statements instead of properties.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Summary

The Systematic development process for blackbox protocol properties can be summarised as follows:

- Develop blackbox protocol properties
- Enumerate a finite set of fundamental behaviors for features extracted from the verification plan
- Enumerate a finite set of protocol categories with which standard protocols are associated
- Identify the subset of fundamental behaviors exhibited by a given protocol category
- Use these fundamental behaviors to drive the development of the protocol properties

Endeavor to make your properties as general (less detailed) as possible

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 10

Basic Complexity Handling

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Module Objectives

In this module, you will be able to,

- Identify different types of complexity
- Identify factors that contribute to formal complexity
- Apply flowcharts systematically to simplify complex processes
- Identify SVA properties that add state to the proof
- Re-write SVA properties to reduce complexity
- Use tool features to help us understand where complexity exists

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Understanding the Complexity in Formal Verification

- An exponentially large number of behaviors are possible in systems
- Formal Verification solves problems that are mathematically challenging, known as "NP-complete“
  - This approach is contrasted with simulation, which primarily handles problems that are linear in nature
- Consequently, no Formal Verification tool can guarantee efficient outcomes for all designs due to these complexities
- Not a show-stopper, though it just means you must be aware of potential complexity & techniques to address it

 © Cadence Design Systems, Inc. All rights reserved.

cadence

In formal, Exponentially many behaviors are possible.

Formal Verification solves problems with mathematical calculation.

Contrast with simulation, which solves essentially linear problems.

This means no FV tool can ever guarantee efficient results on all designs.

it is Not a show-stopper though, it just means you must be aware of potential complexity & techniques to address it.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Defining Formal Complexity

Complexity for Formal Analysis is a function of:

- Size
- Diameter
- Secondary factors
  - Design structure, state machine transitions, design operations, etc.

The symptoms of high complexity are:

- Long run times
  - Many minutes or hours per assertion
- High memory consumption
  - Large amount of memory or running out of memory
- Undetermined results

Overcoming complexity means:

- Reducing runtime/memory consumption
- Turning Undetermined results into conclusive Proven or CEX results

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

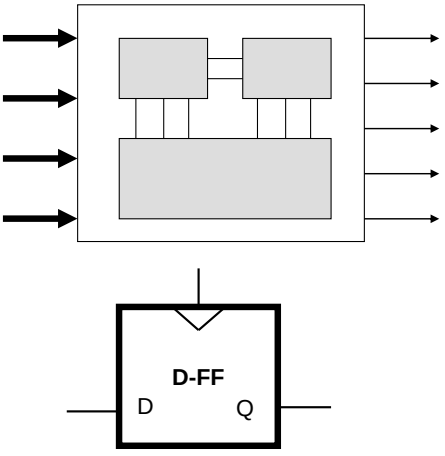
This document is for the sole use of Kun Wang of Seas

Factors Contributing to Complexity Size

Size not measured in terms of gates

Size measured in terms of:

- Number of primary inputs (PIs)
  - Excluding PIs constrained with fixed values
- Number of state bits
  - Excluding state bits that have constant values
  - Constraints and Auxiliary code can also add state bits



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

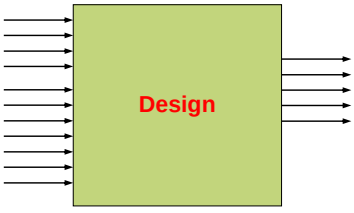
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Design Size Versus COI Size

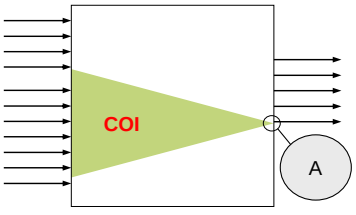
Design size

- Can be viewed as a worst-case scenario
- Usually, an individual assertion would not depend on everything in a design



COI size

- Normally, a fraction of the design size
- COI size is the relevant metric for Formal Analysis
- Not design size
  - It is the size relevant for an individual assertion



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

COI Size Classification

Performance is not predictable purely from size

Many large models can run fast

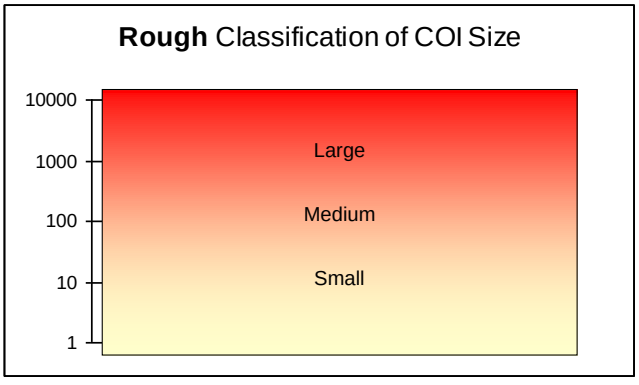
- Maybe complexity is local despite large COI
- Maybe logic is inherently simple

Some small models can hit complexity

- FPV is solving a mathematically hard problem

But in general:

- Larger COIs: more likely to hit complexity
- Smaller COIs: less likely to hit complexity



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

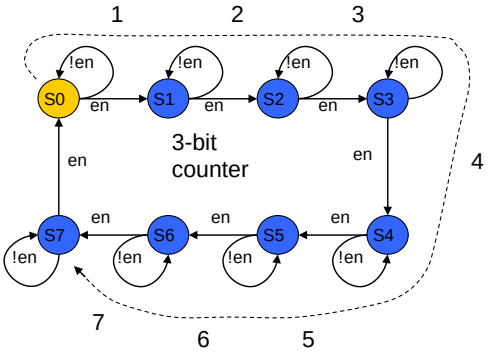
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

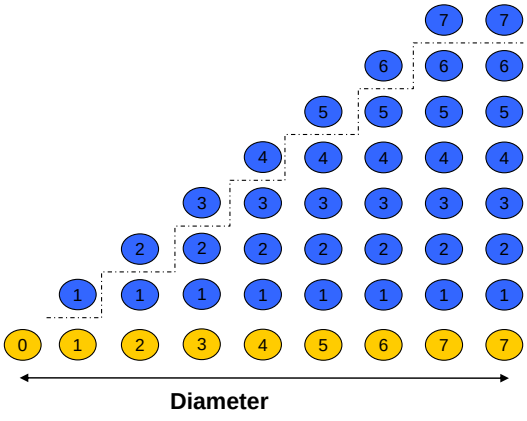
Diameter Definition

Number of steps it takes to get to all reachable states from the initial state

Diameter is dependent on the initial state and state transitions



Diameter is 7



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Shallow Diameters

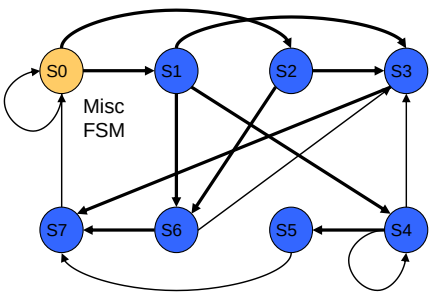
Based on the same number of state bits, the diameter can vary

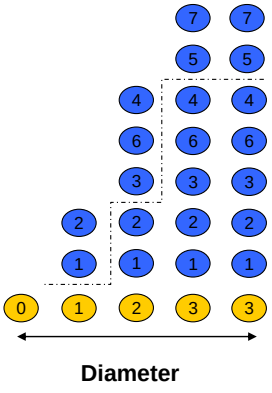
The range is 0 to 2 $^{\text{number of state bits}}$ -1

Standalone counter represents the maximum diameter of 2 $^{\text{number of state bits}}$ -1

If there are more branching forward transitions, the diameter is typically shallower

Destination from S0	Diameter	Shortest Path
S1	1	S1
S2	1	S2
S3	2	S1-S3
S4	2	S1-S4
S5	3	S1-S4-S5
S6	2	S1-S6
S7	3	S1-S6-S7





 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Diameter Classification

Not possible to predict performance solely based on the diameter

Sometimes, large diameters will run fast

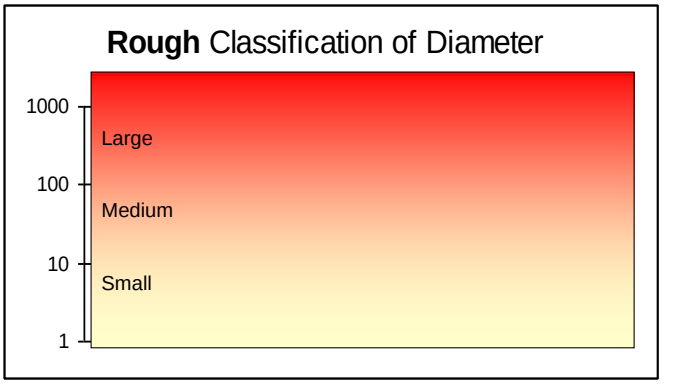
- E.g., standalone counters

Sometimes, small diameters will run slow

- E.g., counter with other logic
- This is due to many secondary complexity factors

Can classify diameter and associate the likely complexity

- Diameter assessed in combination with the size of the COI
- Small COIs can have large diameters without being complex
- Larger COIs are much more diameter sensitive



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Standalone Counter Versus Counter with Other Logic

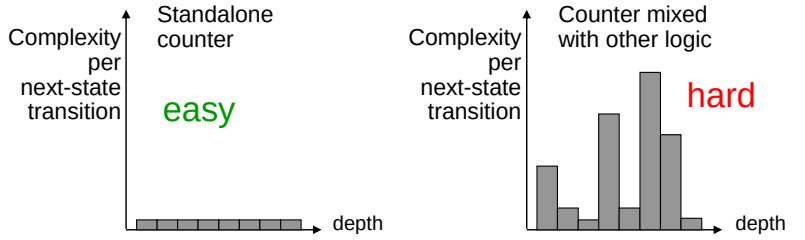
Formal analysis of a counter in isolation is not a hard problem

- Analysis of the counter by itself is easy
- Even to 1M+ diameter

Combining counters and other design operations creates complexity

- For a simple increment operation, the complexity is low
- If many operations are performed, the problem gets harder

Counters can have a multiplication effect on the complexity



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Properties and Complexity

Properties can significantly increase the complexity

They can increase size and diameter

Complexity can stem from

- Auxiliary code
- Implied state bits and diameter

Assertions can only add to the complexity

Constraints can either add or reduce complexity

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Assertions That Do Not Imply State Bits

- This assertion does not Imply complexity
- Assuming `rd_en` and `wr_en` are not aux code
- It is simply a Boolean which has to be true on every clock
- Hence, no knowledge of the past is required, and no obligations are placed on the future

```
assert_no_rd_and_wr : assert property
      (@(posedge clk) ( rd_en |-> !wr_en));
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Assertions That Imply State Bits

This assertion implies state via the next cycle implication operator

Auxiliary code can imply state in the same way

Perform the following thought experiment (i.e., we aren't recommending you change your code to look like this)

- When implementing an assertion as a Boolean expression, how many state bits would I need in my auxiliary code?

```
assert_req_until_gnt : assert property
  ( @(posedge clk) req && !gnt |=> req) ;
```

Equivalent

```
reg prev_req_and_no_gnt;      Verilog Aux Code
always @(posedge clk or negedge rst_n)
  if (!rst_n)
    prev_req_and_no_gnt <= 1'b0;
  else
    prev_req_and_no_gnt <= req && !gnt;
```

```
assert_req_until_gnt_equiv: assert property
 (@(posedge clk) prev_req_and_no_gnt |-> req));
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Temporal Functions Implying State Bits

How many state bits does `assert_port_stable_until_gnt` add?

```
assert_port_stable_until_gnt: assert property
      ( req && !gnt |=> $stable(port) );
```

Equivalent

```
logic prev_port [7:0] ;
logic prev_req_and_no_gnt;
...
always @(posedge clk or negedge rst_n)
  if (!rst_n)
    prev_req_and_no_gnt; <= 1'b0;
  else begin
    prev_req_and_no_gnt <= req && !gnt;
    prev_port <= port
  end
```

Verilog Aux Code

```
assert_port_stable_until_gnt_equiv: assert property (
                    prev_req_and_no_gnt |-> (port == prev_port) );
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Operators That Imply State Bits

Partial list of SVA operators and functions that Imply state bits and typically diameter

```
[*n], [=n], [->n]          Sequence repetition operators
|=>                        Sequence Non-Overlapping Implication
##n                        Cycle delays
$past, $fell, $rose, $stable     Temporal functions
```

If `n` is large or one uses `$past(expr, some_big_number)` then this is likely to be problematic

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

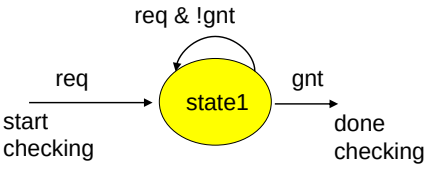
This document is for the sole use of Kun Wang of Seas

Implied State Bits: Repetition Operators

The number of Implied state bits scale with the size of the repetition count `n` (as in `*n`)

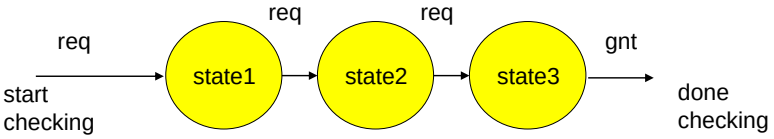
A simple `[*] or [1:$]` interval in SVA does not Imply many states

```
P1: assert property (
    @(posedge clk) req |=> req[*]##1 gnt );
```



This Implies a counter range 0 to 3

```
P2: assert property (
    @(posedge clk) req |=> req[*0:3] ##1 gnt );
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Reducing Input Permutations

- Formal Analysis does not always get simpler if input permutations are reduced
- During state exploration, the level of activity on an interface does not matter
- How much state space has to be explored matters
  - i.e., Design and Constraint and Assertions
- Attempts to reduce input permutation might increase the complexity
- To reduce complexity, constraints need to limit the exploration and affect the model of the DUV

```
//Do not have more than 2 active transactions in a row
assume property (@(posedge clk) (active_trans[*2]) |=> (idle_trans));
```

`active_trans` and `idle_trans` are named sequences defined elsewhere

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

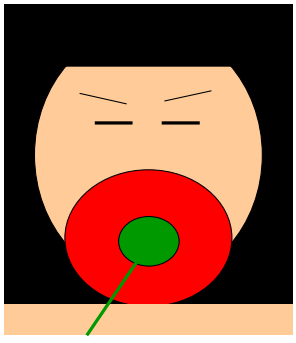
This document is for the sole use of Kun Wang of Seas

Abstraction

- Formal analysis tools perform abstractions to improve performance
- An abstraction is an optimization technique
- Abstractions are made to improve the performance of proven results
- The tool might analyze only a portion of a design and not the whole design







 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Local Assertions

Some assertions do not require the entire COI to pass

- For example, protocol properties are typically `local`

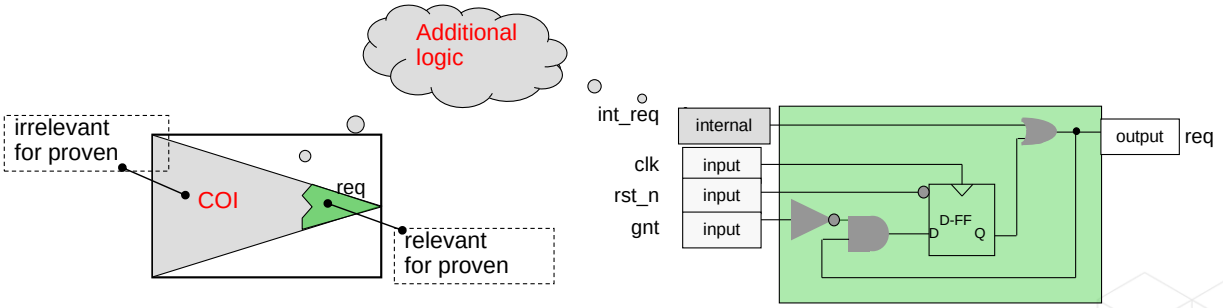
Relevant logic can be much smaller than actual COI

- This applies for a pass only

Abstractions can take advantage of `locality`

The assertion passes for any value of int_req. We can cut the circuit at int_req and still achieve a proven result.

```
assert_req_until_gnt: assert property
    @(posedge clk)   req && !gnt |=> req);
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Consequences of Assertions Locality for Coverage

If one made Jasper report elements in the COI of the assertion, then it would be both the grey and green parts

- i.e., the static fanin cone of all signals in the assertion

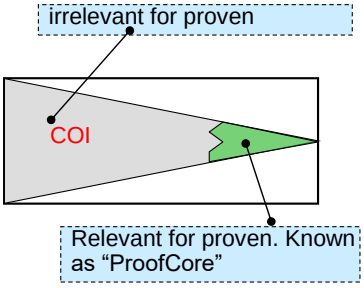
Does this mean we have covered/verified/exercised all elements in the COI of the property?

- There is the danger of over-optimism on coverage
- The grey part is irrelevant to the proof
- We learned nothing about the grey part

DESIGN

Design Coverage Verification App

- Provides formal coverage metrics
- Analyzes property set completeness
- Shows verification from bounded proofs



How do we solve this?

- Write good cover properties, exercising expected design behaviors
- With the use of Jasper™ Coverage App
  - It reports how much of the design was exercised in a meaningful way

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

End-to-End Assertions

Some assertions do require the entire COI to pass

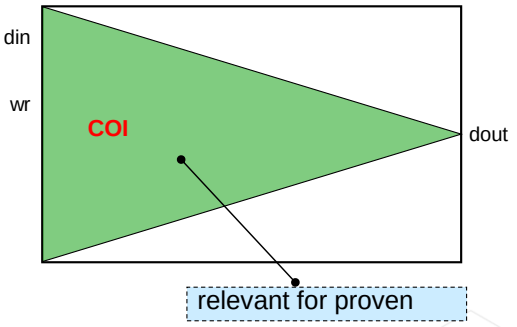
- For example, end-to-end checks are typically global

Relevant logic is the entire COI

Automatic abstractions can't take advantage of locality

If an end-to-end assertion is facing complexity issues, try to break it down into more atomic assertions on smaller regions of the logic

```
genvar I;
generate
for (I=0; i<=MAX_DATA_VAL; I=I+1)
 begin :genass
  data_in_comes_out: assert property (
   @(posedge clk) ( wr && (din == I)) |->
      s_eventually (rd && dout == I) );
 end
endgenerate
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Amenability Table

Multiple factors affect the complexity of formal analysis

Design Type	Sequential Depth	Interface Type	Block Specification	Criticality & Risk	Block Size
Control Logic	10s of cycles	Protocol Handshake	Well Documented	High	Small (<100k flops)
Data Transport	100s of cycles	Pipelined/Parallel	Partially Documented	Medium	Medium (100K-500K)
Data Transform	1000s of cycles	Serial	Minimally Documented	Low	Large (500K+)

FPV Easiest → FPV Hardest

Mainly Signoff, some Bug Hunting → Mainly Bug Hunting, some Signoff → Bug Hunting only

353 © Cadence Design Systems, Inc. All rights reserved.

cadence

- This Table Demonstrates the complexity of different design types while using formal Property verification.
- Control logic is best suited for FPV because of the less sequential depth, the focus here is on formal signoff.
- Some parts of data transport design can be signed off by FPV with assisting bug hunting.
- In data transform designs only bug hunting is recommended, because of complexity and data manipulation, there are other techniques in formal like C to RTL which can be used to verify data transform design.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Complexity Summary

We now understand contributing factors to complexity

Complexity means how hard it is for the tool to obtain a proof

- Not how many gates the design has

Bad set of Properties can cause Complexity

Writing simple properties can reduce Complexity

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Submodule 10.1

Complexity Flowchart

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Submodule Objective

In this module, you will learn how to

- Use a complexity Flow chart to approach complexity

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Why a Flowchart?

In training, we wanted to have a systematic approach how to solve a complexity problem

You can use it as a checklist

The order is a suggestion – not necessarily always the best recipe

- Depends on the design you want to achieve (bugs vs. proofs), goals, etc.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Complexity Flowchart



 © Cadence Design Systems, Inc. All rights reserved.

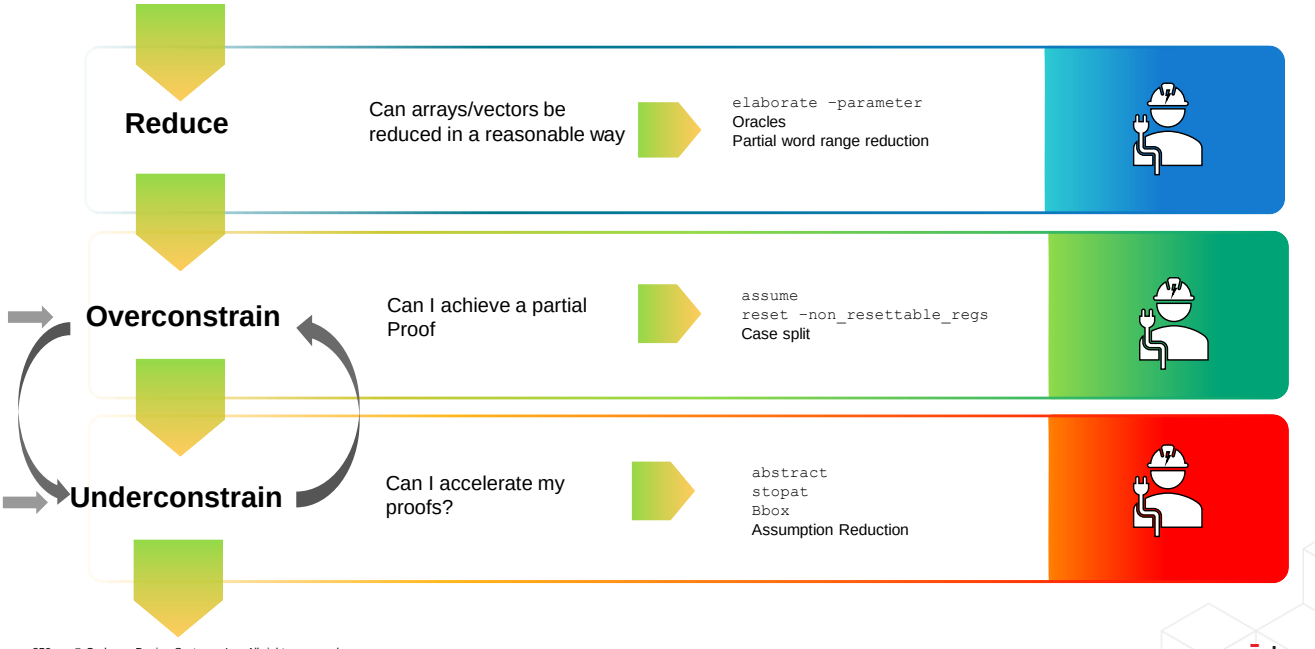
cadence

A recipe a user can follow.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Complexity Flowchart (continued)



 © Cadence Design Systems, Inc. All rights reserved.

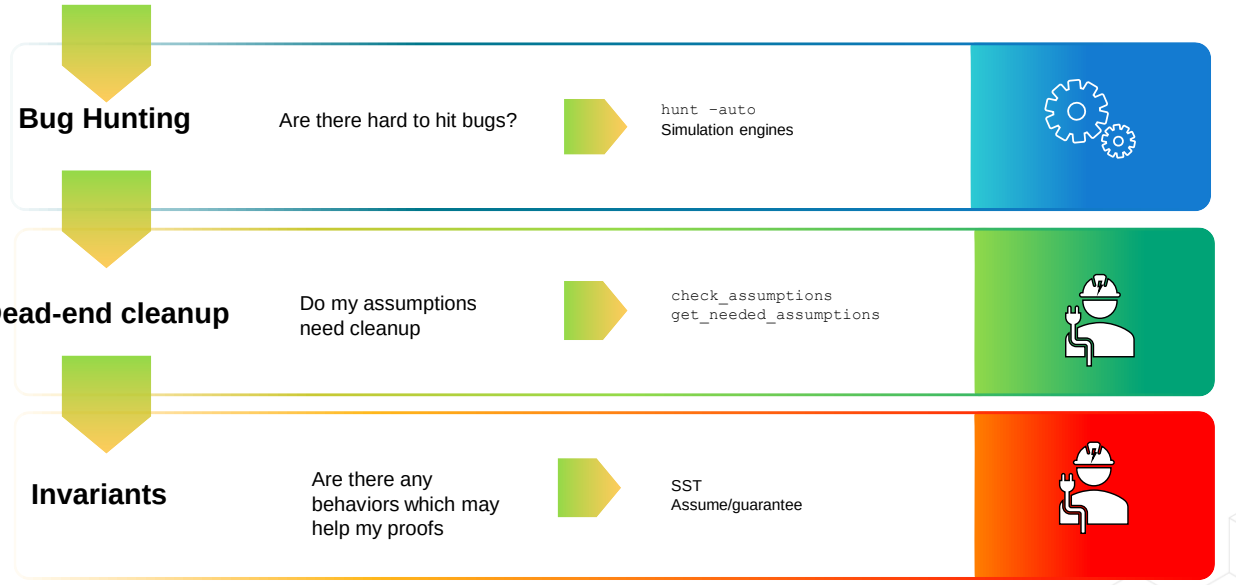
cadence

A recipe a user can follow.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Complexity Flowchart (continued)



 © Cadence Design Systems, Inc. All rights reserved.

cadence

A recipe a user can follow.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Consider Goal

Looking for a proof vs a CEX the course of action might be different

Don’t forget bug hunting strategies

Overconstrain	**TRACE** (**cex**/**covered**)	Any trace with overconstraints is still valid
Underconstrain	**PROOF** (**proven**/**unreachable**)	Any proof with underconstraints is still valid





 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Submodule 10.2

Generic Complexity Reduction Methods

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Submodule Objectives

In this module, you will be able to

- Identify complexity reduction techniques
- Analyze the effects of clock and reset mechanisms on complexity
- Apply strategies for simplifying the environment
- Use parametrization and blackboxing

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Looking at Clock and Reset

Check Clock and reset config

- Clock –list
- Reset –list

Simple issues to look for

- Clock/reset defined as you except
- Missing clocks
- Very high ratios between some clocks
  - High ratio == Lots of analysis for one slow-clock step
  - Can you FV with smaller ratio?
- **Complexities that belong in dedicated flows**
  - Clock gating: Lots of extra computation for FPV; verify in separate SEC run
  - Clock Domain Crossing (CDC): Keep clocks synchronous for FPV, separate CDC run

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Clock and Reset can cause complexity in some cases, we need to ensure that all clocks and rest are defined as per the specification.

We also need to make sure there is no missing clock.

A high ratio between clocks can cause much analysis for slow clock steps.

While verifying the design, using FPV clock gating can cause lots of extra computation. So, it is recommended that the clock is gated separately using sequential equivalence checking(S-E-C).

For Clock Domain crossing, run separate CDC run, and in FPV, keep the clock synchronous

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Reset and Reset State

Look at reset analysis line in console

```
[<sb_always_1_symbol>] % prove -property {<sb_always_1_symbol>::top.sb.sb1.genblk6.core.genblk8.genblk1.lite_check_always_symbol.*} -regexp
INFO (IPF036): Starting proof on task: "<sb_always_1_symbol>", 2 properties to prove with 0 already proven/unreachable
INFO (IRS029): Starting reset analysis: phase 1 of 4.
INFO (IRS030): Running reset analysis: phase 2 of 4.
INFO (IRS031): Running reset analysis: phase 3 of 4.
INFO (IRS020): Starting the reset analysis simulation with a limit of 100 iterations (phase 4 of 4).
INFO (IRS024): Reset iterations 0 to 4 analyzed.
INFO (IRS018): Reset analysis simulation executed for 3 iterations. Assigned values for 822 of 958 design flops, 0 of 0 design latches, 252 of 348 internal
elements.
INFO (IRS039): Reset analysis complete.
```

Check reset state with **`visualize -reset`**

- Check for enables to flop clocks, and so on
- Can also use **`get_reset_info -x_value`** in console

Non-reset state may be good or bad

- **Less reset:** Can reach arbitrary state sooner ➔ Help with sequential complexity
- **More reset:** Starting state more concrete ➔ May simplify analysis
- Need to decide based on the complexity type in your design

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Check reset state with visualize –reset command

We need to look for the non-reset state, which can be good or bad

Less reset can reach arbitrary state sooner, which helps with sequential complexity

More reset can help simplify the analysis

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Environment Simplification

Simplifying the environment typically simplifies the complexity

- It can be a very effective complexity reduction method

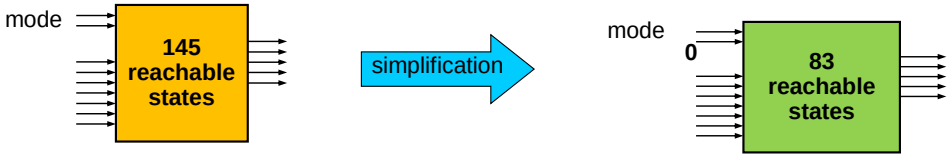
The environment can be simplified using constraints

- Typically, fixed value constraints are the most effective
  - Effectively input pin constraints

These constraints can make state bits constant

- This can reduce the `size` and the `diameter`

```
A1 : assume property (mode == 2'b00);
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Example: Environment Simplification

Limit values space of a control input

- length is the input for a packet-based design
- Do not allow long packets to come into the devices
- Limits the diameter of the design

```
input [7:0] length      RTL
```

```
A1 : assume property (length[7:4] == 4'b0000);
```

Limit the values space of a data input, for example:

- Only allow the MSB and LSB to be variable
- Makes data-path state bits constant
  - Reducing size

 © Cadence Design Systems, Inc. All rights reserved.

cadence

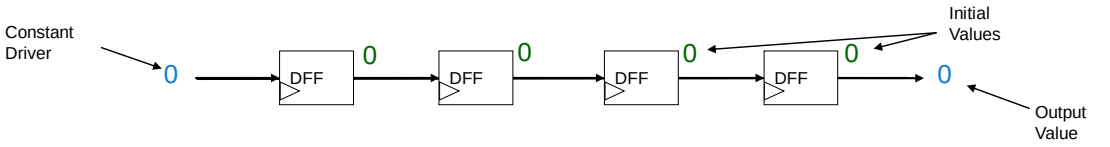
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

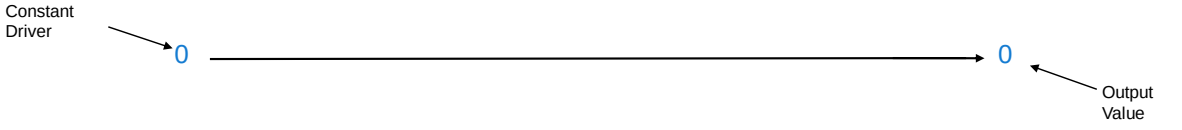
This document is for the sole use of Kun Wang of Seas

Constant Propagation: How Does It Work?

If the constant on the input of a flop is identical to the initial value, the flop can be replaced by the constant itself



If the constant on the input of a FF is identical to the initial value, the FF can be replaced by the constant itself



 © Cadence Design Systems, Inc. All rights reserved.

cadence

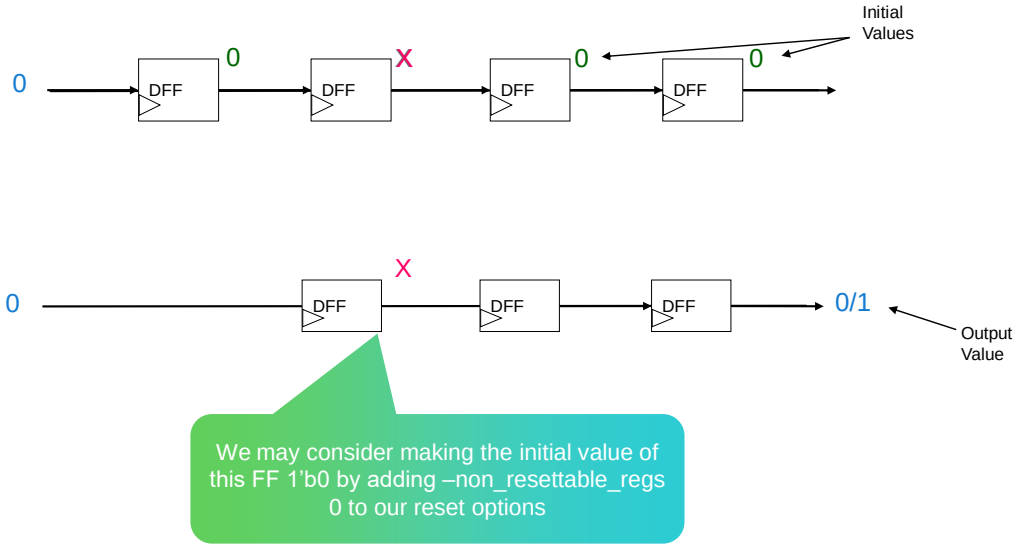
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Constant Propagation: How Does It Work? (continued)

If flops have different values or are un-initialized, no constant propagation happens beyond this point



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

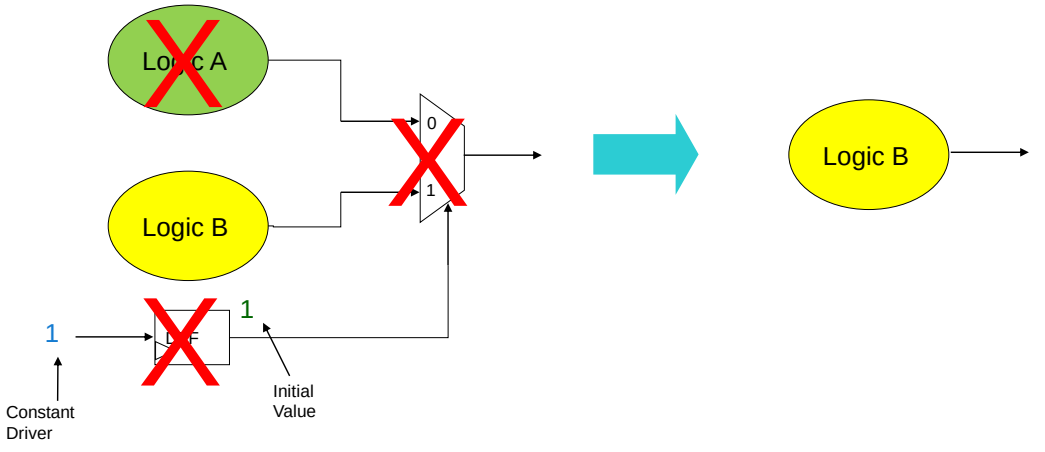
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Example: Constant Propagation

Impact of constant propagation on control logic

- Tying mux select to a constant value can remove a big portion of the design



 © Cadence Design Systems, Inc. All rights reserved.

cadence

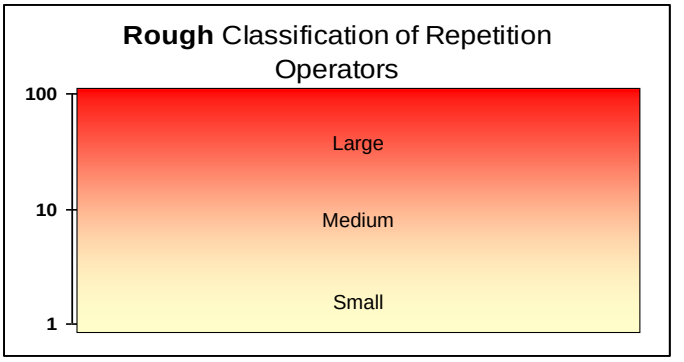
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Property Simplification: General

- Avoid ranges and high repeat counts
  - They imply state and diameter
  - Increasing diameter is particularly problematic
- Effectively they bring a counter into the analysis
- Small repetition values are not likely to be problematic
- Large repetition value likely to be problematic



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Assertion Simplification

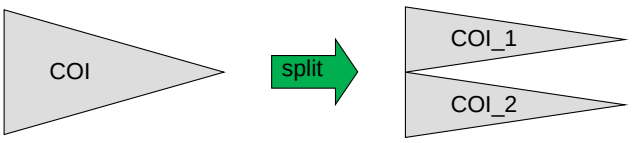
Splitting assertions can reduce complexity

- Smaller pieces have lower complexity
- The goal is to end up with smaller and less complex COIs
- This is a “divide and conquer” approach

As complexity tends to grow exponentially, splitting can be very effective

Splitting methods with different characteristics

- Functional split
- Bit-wise split



Goal is that:

```
Complexity(COI) >> Complexity(COI_1) + Complexity(COI_2)
```

COI ➔ assert property (a && b)

COI_1 ➔ assert property (a)

COI_2 ➔ assert property (b)

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Functional Split

Overall requirement for verification

- Request needs to be held high and enable needs to stay stable until they get acknowledged

Requirement can be split into two properties

When combined, these meet the overall requirement

- Request needs to be held high until it gets acknowledged
- Enable needs to stay stable until it gets acknowledged

```
req_high_ena_stable_until_ack:
  assert property (
   req && !ack |=> req && $stable(ena) );
```

≡

```
req_high_until_ack:
  assert property ( req && !ack |=> req );
```

```
assert_ena_stable_until_ack:
 assert property
      (req && !ack |=> $stable(ena) );
```

373 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Make All Assertions as Simple as Possible

Some common functional splits:

- a |-> b && c ==> a |-> b, a|->c
- ||b |-> c ==> a |-> c, b |-> c

Advantages:

- Complex proof items don't prevent covering easier ones
- Debug is much easier for more atomic assertions
- Can be effective for assertions
- Ensures you don’t try and say everything in one property

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Bit-Wise Split

Overall requirement for verification

- 8-bit vector addr needs to be stable during a request until an acknowledge

After the split, each bit of addr will be evaluated separately

Are the original property and the split ones equivalent?

```
addr_stable_until_ack: assert property
( @(posedge clk)
   req && !ack |=> $stable(addr) );
```

Equivalent?

```
genvar I;
generate
for (I=0; I<=ADDR_WIDTH; I=I+1)
 begin :genass
  addr_stable_until_ack: assert property
   ( @(posedge clk) req && !ack |=> $stable(addr[I]) );
 end
endgenerate
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Bit-Wise Splits: Pros and Cons

Advantages:

- Can be very effective for assertions
- Easy to implement

Disadvantages:

- Generate many assertions
- Result of a bit-wise split might not be equivalent to the intent of the original assertion
  - Word-wise consistency is not being checked

Can solve with free variables

The problem here is that the range is variable

Timing diagram

```
din[3:0]  0100000
          0100000
          0100000
          0100000

dout[3:0]
          0000100
          0000001
          0000100
          0000100
```

```
logic [3:0] J; //Undriven wire
J_STBL: assume property($stable(J));

din_out_in_1_to_5 : assert property(
   (wr && (din ==J)) |=>
      ##[0:4] (dout == J) );
```

!≡

```
logic [3:0] J; //Undriven wire
J_STBL: assume property($stable(J)); //NDC
genvar I;
 generate
  for (I=0; I<=D_WIDTH; I=I+1)
   begin :genass
    din_out_in_1_to_5: assert property
      ( (wr && (din[I] == J[I])) |=> ##[0:4] (dout[I] == J[I]) );
   end
endgenerate
```

Choice of 0,1,2,3,4 may differ for different bits

376 © Cadence Design Systems, Inc. All rights reserved.

cadence

As the delay has a variable range ##[0:4] then word wise consistency is not maintained in the generate loop properties for each bit of the word.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Property Equivalence Checking

When remodeling a property, check for equivalence

Process

- Put both properties in an empty dummy module
- Turn PS1_1 into an assertion and the PS2_1 into an assumption
- Run the proof and observe the results
- Turn PS2_1 into an assertion and PS1_1 into an assumption
- Run the proof and observe the results
- If there are no failures, properties are equivalent

It is important to prove it both ways

- Overconstraining could hide a problem

Empty Dummy Module

```
module B2B_CHK(
   input clk, input A,
   input C, input D );
```

Define Properties

```
property C_BEFORE_D;
   (  A |=> strong(!D[*] ##1 C) );
endproperty

property NOT_D_UNTIL_C;
   (  A |=> (!D s_until C ) );
endproperty
```

Verification Directives

```
PS1_1 : assert property ( C_BEFORE_D );
PS2_1 : assume property (NOT_D_UNTIL_C);
```

Swap

```
PS1_1 : assume property ( C_BEFORE_D );
PS2_1 : assert property (NOT_D_UNTIL_C);
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

OC means overconstraining

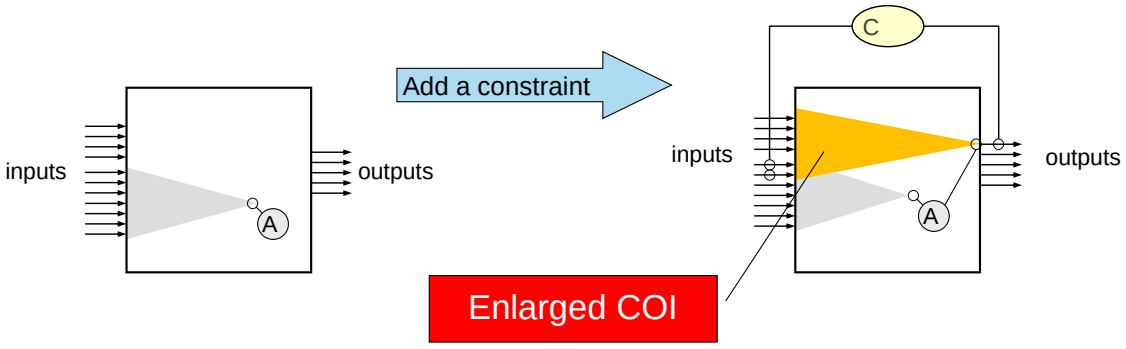
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Constraints

Simplifying constraints generally higher ROI than simplifying assertions

- Constraints apply to many assertions
- Constraints can Imply state bits and increase the COI
  - By pulling in additional logic



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Avoid Unnecessary Constraints

Only define constraints needed for assertions

Jasper will find any way in which the assertion can be made to fail

You will never come up with the right constraints in advance!

- Implement major conditions you know about (turn off scan, eliminate unused modes, etc)
- Don't try to get every detail ahead of time

"Wiggling" is an important first phase of FPV

- Examine initial (wrong) FPV counterexamples/cover traces, identify assumptions needed to rule out
- Iterate after adding a few assumptions, repeat as needed
- Every real-life FPV effort begins with Wiggling

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Parameterization

An effective complexity reduction method

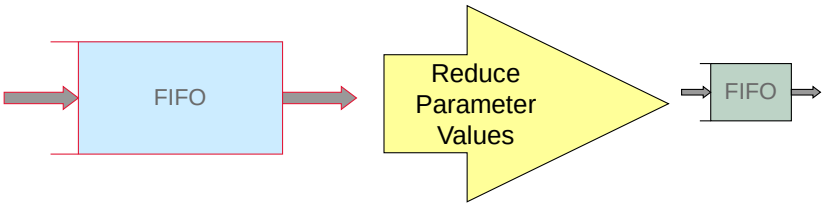
- In most of cases, this may be the most effective method

If parameters exist in your design, set them to the smallest meaningful value

- Most failures can be detected with small parameter values
- For signoff, parameters may need to be set to the intended size
  - Conclusive results might not be achievable

Adding parameters to an existing design is very costly

- ROI considerations are crucial
- Best practice to implement parameters unless it makes no sense



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Dangers of Parameterization

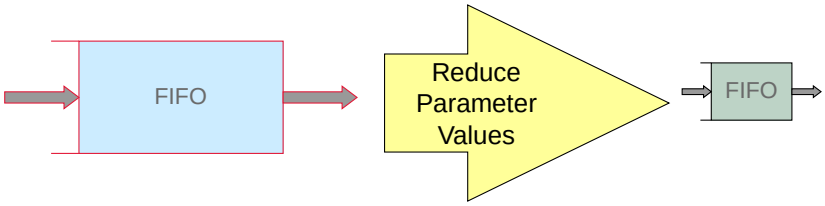
When we make the parameter values smaller, this is known as a “Reduction” in Formal

All original design behaviors may no longer be possible

Therefore, the reduction in parameters might hide bugs

Some common examples:

- Too few operations to create corner case
- Numerical conditions (error on multiple of 64, etc.) no longer met
- Downstream queue can't fill



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Parameterized FIFO Example

Things don't go wrong because the number of FIFO words is 4096 or 8192

- Or data width is 32 bits

They go wrong at corner cases

- Read/write pointers at the same location
- Read/write pointers wrap around
- Read/write pointers at adjacent locations

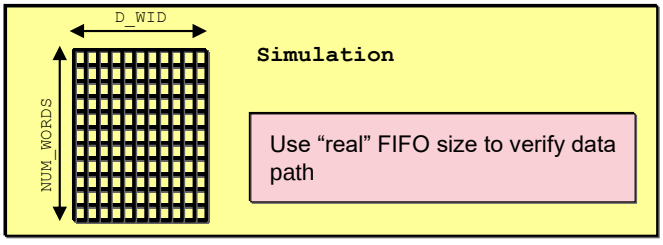
Size Problem

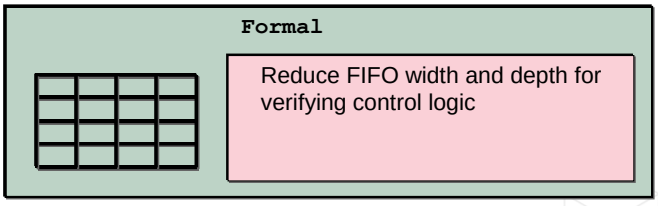
- Memories have, by definition, lots of states

Could blackbox the memory

- Control logic is what you want to verify
- Partition RTL for verification

```
module FIFO_CTRL #(D_WID, NUM_WORDS) (
input logic  [D_WID-1:0] din;
output logic [D_WID-1:0] dout;
....
```





 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

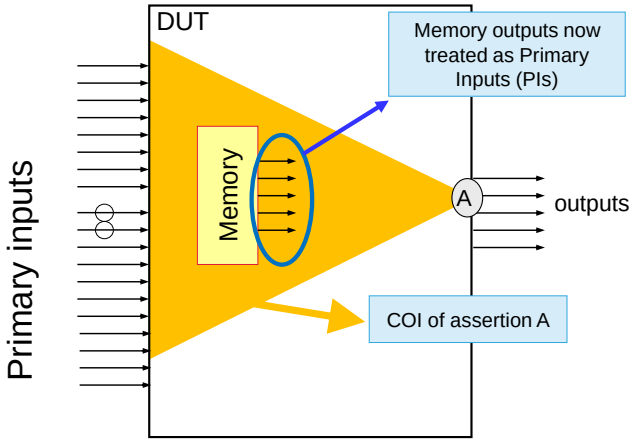
Blackboxing

We cut out parts of the design which add complexity

- For example, memories are a typical target for blackboxing

```
elaborate … -bbox_m my_memory …
```

The outputs of the blackbox become primary inputs that the tool can drive



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Consequences of Blackboxing

If the assertion is proven, can we conclude that the proven result is “safe”?

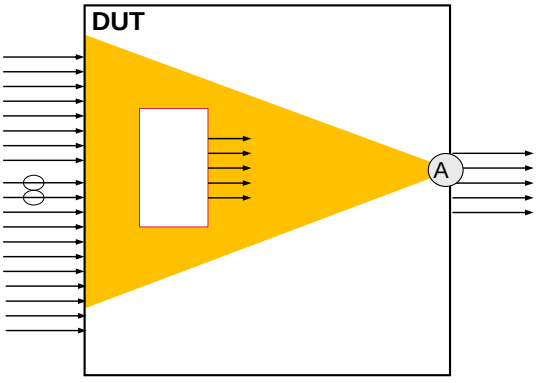
- Yes. If the assertion passes, then there is no value the memory outputs could ever take which would cause it to fail

What if the assertion fails?

- Then we don’t know if the assertion would pass when the memory was not blackboxed

I blackboxed the memory because the assertion would not converge; what now?

- Abstract the memory or property:
  - Reduce the memory size via parameterization
  - Replace the memory with an abstract model, e.g., Aux code or a Proof Accelerator



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Submodule 10.3

Complexity Evaluation Tools

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Submodule Objective

In this module, you will learn how to

- Use tool features to help us understand where complexity exists

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Tool Options for Evaluating Formal Complexity

- The methods discussed previously are applicable generically to any formal tool
- There are also tool specific methods to evaluate the complexity
- We can also use abstraction techniques to reduce complexity

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Reporting Design Structure

The tool can provide us insight into formal complexity

For the entire design: get_design_info

```
[<embedded>] % get_design_info
Statistics [for instance "counter"]
-------------------------
# Flops:         1 (35) (26 property flop bits)
# Latches:       0 (0)
# Gates:         88 (738)
# Nets:          101
# Ports:         6
# RTL Lines:     37
# RTL Instances: 1
# Embedded Assumptions: 0
# Embedded Assertions:  2
# Embedded Covers:      2
```

For a specific type of design structure:

```
% get_design_info -list counter
List of counters: 2 (8)
-----------------------
  fifo_inst.rd_ptr (4)
  fifo_inst.wr_ptr (4)
fifo_inst.rd_ptr fifo_inst.wr_ptr
```

There are lots of options: The most important one?

```
% help get_design_info
```

388 © Cadence Design Systems, Inc. All rights reserved.

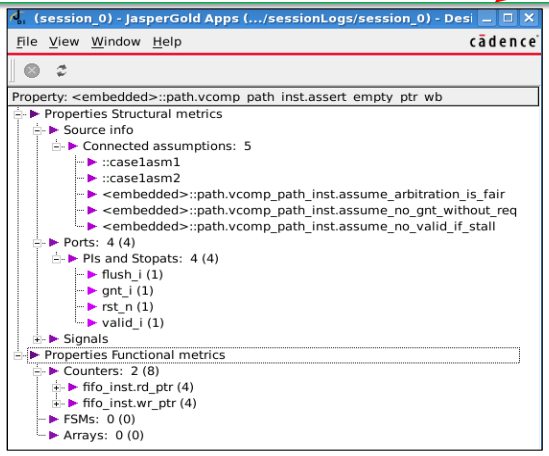
Certain kinds of design structures are known to cause formal complexity, e.g. large arrays, memories, counters, FSMs and so on.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

`get_design_info` for Specific Property

```
%  get_design_info -property <embedded>::path.vcomp_path_inst.assert_empty_ptr_wb -gui
Statistics [for <embedded>::path.vcomp_path_inst.assert_empty_ptr_wb]
# Flops:     5 (11)  (0 property flop bits)
# Latches:   0 (0)
# Gates:     73 (1849)
# PI bits:   4
# Stopats:   0
# Connected assumptions: -
```



You can drag this from the property table, rather than typing the long-qualified property name

389 © Cadence Design Systems, Inc. All rights reserved.

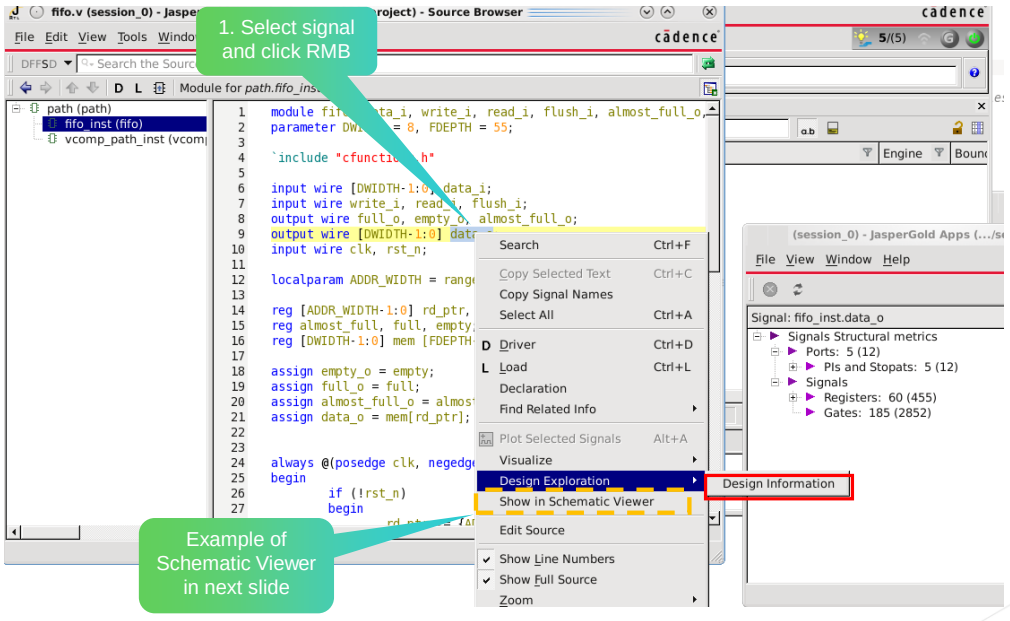
cadence

With these tool features there is often more than one way to get to the same information.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

`get_design_info` for Specific Signal



390 © Cadence Design Systems, Inc. All rights reserved.

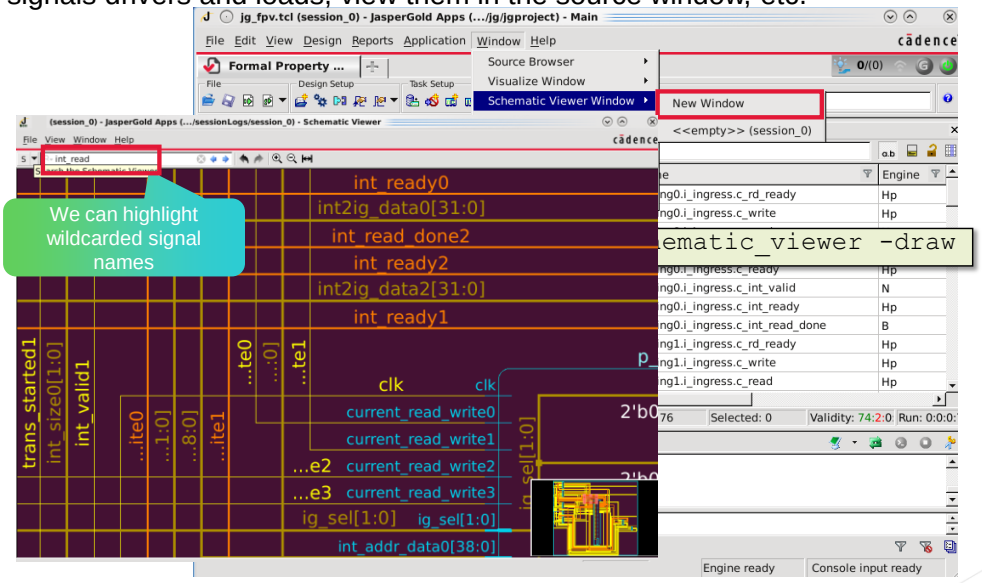
With these tool features there is often more than one way to get to the same information.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Schematic Viewer

We can trace signals drivers and loads, view them in the source window, etc.



391 © Cadence Design Systems, Inc. All rights reserved.

With these tool features there is often more than one way to get to the same information.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

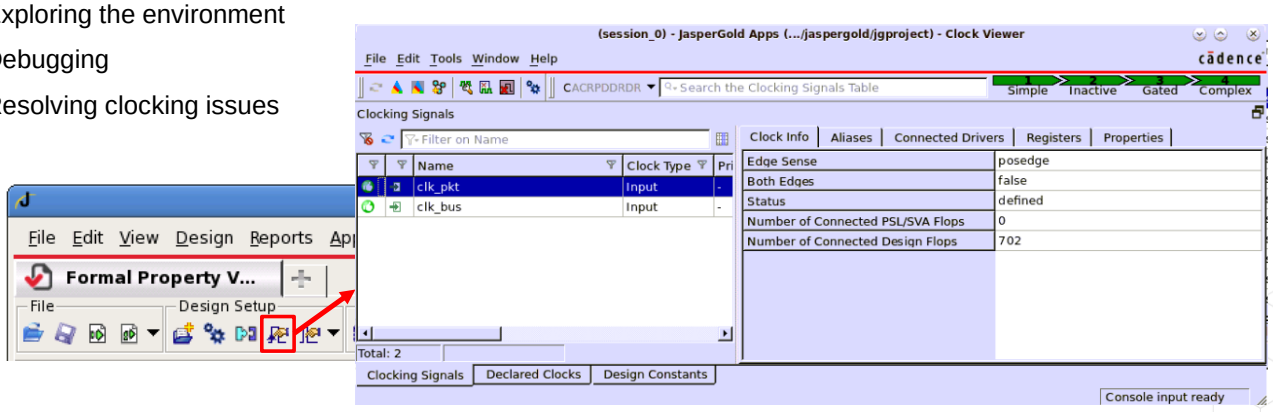
Clock Viewer GUI

The Clock Viewer provides a centralized clock environment information

Options for:

```
% get_clock_info -gui
```

- Efficiently declaring clocks
- Exploring the environment
- Debugging
- Resolving clocking issues



392 © Cadence Design Systems, Inc. All rights reserved.

cadence

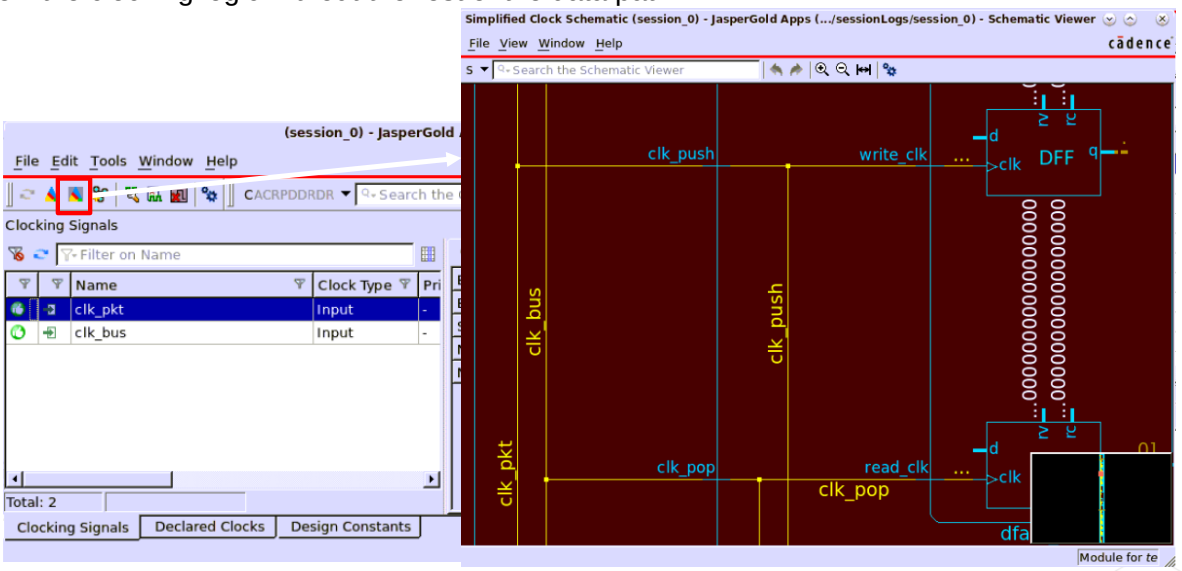
With these tool features there is often more than one way to get to the same information.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Clock Schematic Viewer

View the clocking logic without the rest of the data path



 © Cadence Design Systems, Inc. All rights reserved.

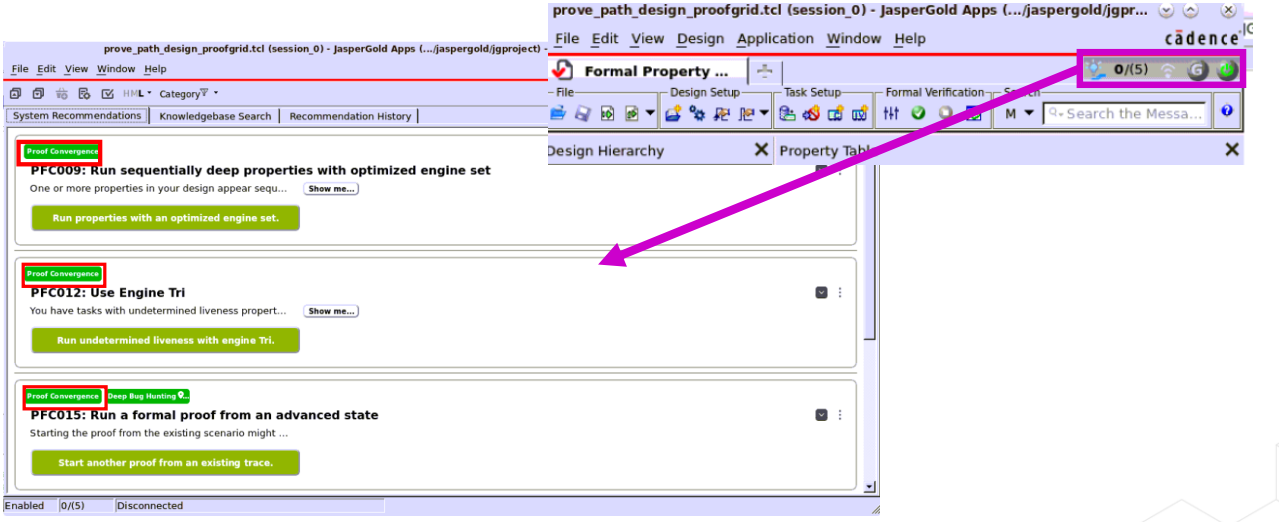
With these tool features there is often more than one way to get to the same information.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

JG Expert System Recommendations

Here we would look for the “Proof Convergence” category for recommendations on reducing complexity



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Complexity Manager

Show statistics for the entire design, or COI of property/task

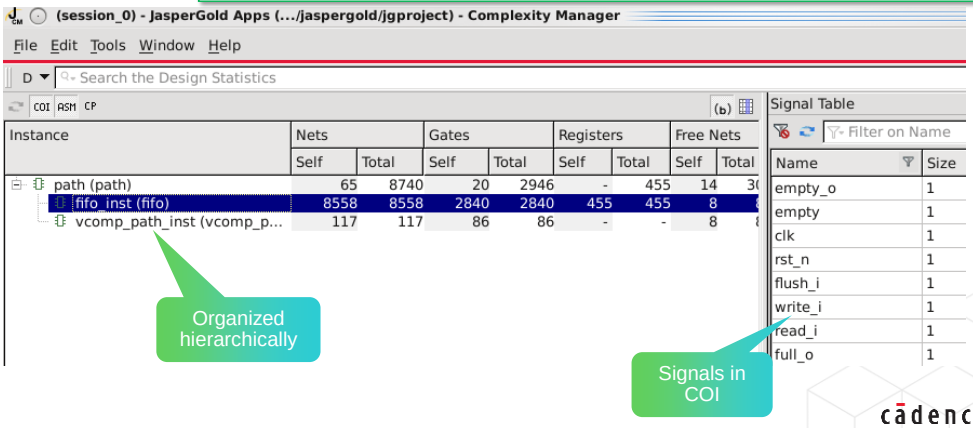
Option to include/exclude logic:

- Disabled by propagated constants
- In targets COI
- In assumptions, COI

```
assert_data_good_bb: assert property (
 @(posedge clk)
  (valid_i && data_i==data |->
    s_eventually gnt_i && data_o==data));
```

```
% complexity_manager -property \

<embedded>::path.vcomp_path_i.assert_data_good_bb
```



395 © Cadence Design Systems, Inc. All rights reserved.

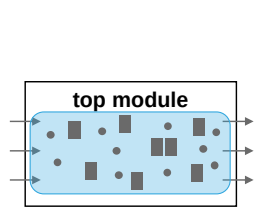
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

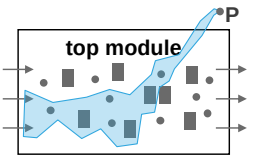
This document is for the sole use of Kun Wang of Seas

Complexity Manager Invocation



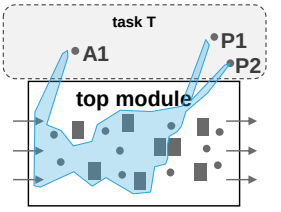

```
% complexity_manager
```

Considers all design logic



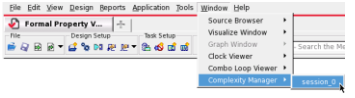

```
% complexity_manager
    -property P
```

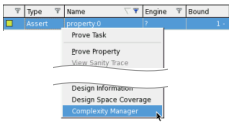
Considers only COI of property

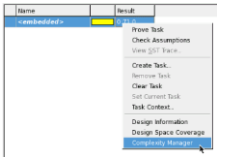



```
% complexity_manager
    -task T
```

Considers combined COI of all properties in task







Refer to the Jasper™ User Guide for more details

- Help → User Guide (Design Exploration Tools chapter → Jasper Complexity Manager)

396 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Improving Proof Throughput

This is an advanced topic not covered in detail in this course

However, it is mentioned now because it has a large effect on overall verification throughput

Therefore, you may wish to investigate this after this course.

Search http://support.cadence.com/Jasper for ProofMaster

In the next few slides, we will show a brief overview of the concepts of ProofMaster

 © Cadence Design Systems, Inc. All rights reserved.

cadence

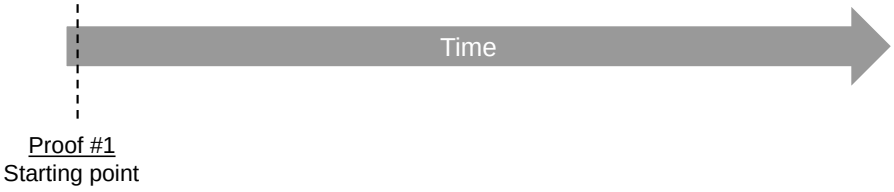
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

ProofMaster

Using historical data



 © Cadence Design Systems, Inc. All rights reserved.

cadence

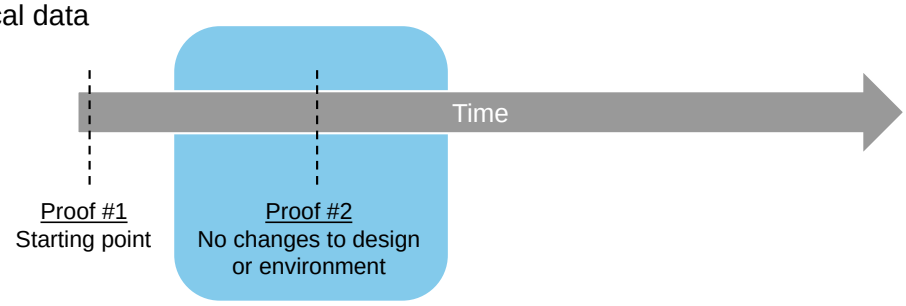
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

ProofMaster (continued)

Using historical data



Proof results caching restores all converged results stored traces and bounds for undetermined properties based on property signature matching.

No further effort is required from the engines to reproduce previous tool results.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

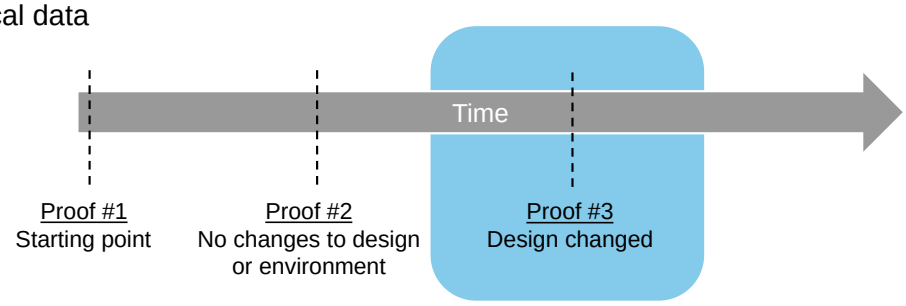
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

ProofMaster (continued)

Using historical data



Proof results caching is resilient against small design/environment modifications, so it can partially retrieve results for properties unaffected by changes.

Proof profiling database applies proof strategies that worked well in the past for your properties, which are likely to perform well again.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

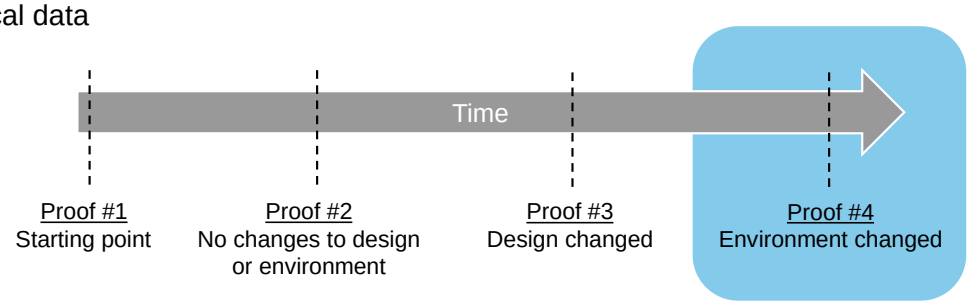
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

ProofMaster (continued)

Using historical data



Proof results caching is resilient against small design/environment modifications, so it can partially retrieve results for properties unaffected by changes.

Proof profiling database applies proof strategies that worked well in the past for your properties, which are likely to perform well again.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

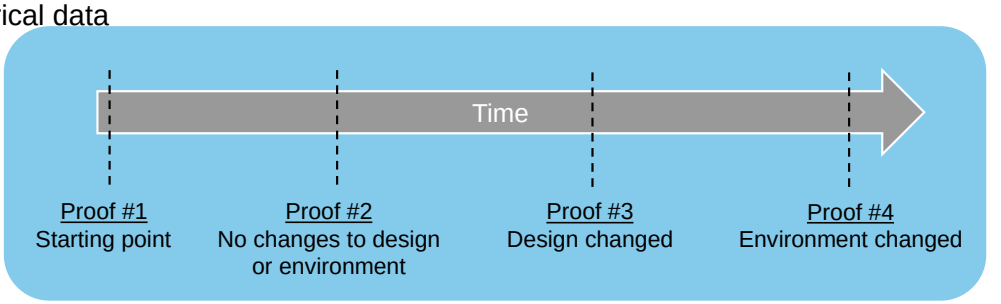
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

ProofMaster (continued)

Using historical data



During all proofs, proof orchestration dynamically adjusts engine selection, time limits, license usage, and others via reinforcement learning.

It respects the resources configured by the user: number of licenses, max jobs, and time limits.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

ProofMaster Example Tcl Commands

See the Tcl help for more commands and details

Command	Argument	Default	Description
set_proofmaster	on | off	off	Enable or disable ProofMaster.
set_proofmaster_dir	proofmaster_dir	Project directory. ProofMaster files are not deleted when overwriting it.	Configure where to store the ProofMaster database outside the project directory of the Jasper run.
set_proofmaster_initial_dir	initial_dir	None	Import ProofMaster data from an existing directory. Useful to leverage existing ProofMaster data when starting your own runs from scratch. If the proofmaster dir is already populated, data from initial dir is ignored. No data is written back to the initial directory.
set_proofmaster_max_data_age	N	0	Configure the tool to delete proof cache data not matched by the tool in the last N days.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Profiler Motivation

- Verification Engineers always seek optimal proof results
- Static analysis of bottlenecks using Complexity Manager has already been done
- But a proof is stuck or slowly progressing
  - … where is the engine struggling in bound n?
  - … what are the bottlenecks in the design?
  - … what parts of the design are being used? … not used?
- Formal Profiler uses information from the **engines** to answer those questions
- Formal Profiler would typically be used **after** trying:
  - JG Expert System Suggestions
  - Reviewing information reported by Complexity Manager

 © Cadence Design Systems, Inc. All rights reserved.

cadence

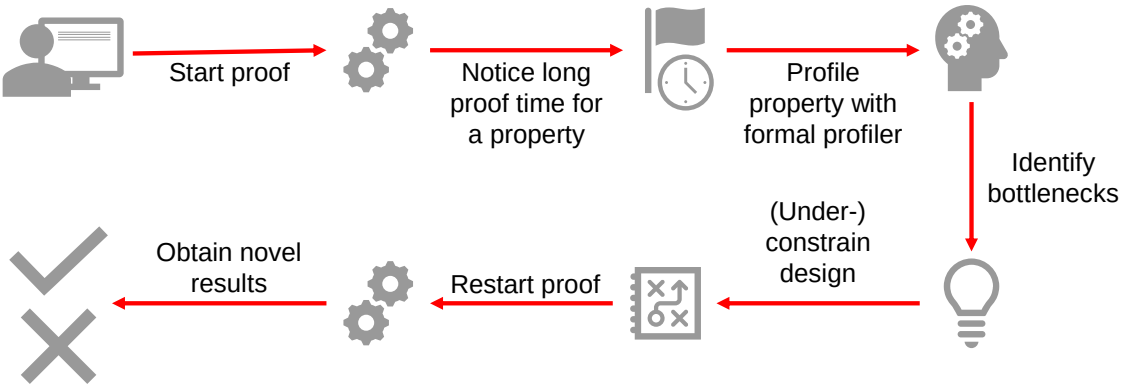
In order to run the formal_profiler you will need an additional license feature:

JGFVADVP : Jasper Advanced Platform Option to JGFVBASE : jasper_advp

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Profiler Use Model High-Level Overview



405 © Cadence Design Systems, Inc. All rights reserved.

In order to run the formal_profiler you will need an additional license feature:

JGFVADVP : Jasper Advanced Platform Option to JGFVBASE : jasper_advp

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Profiler: Tcl API

Full documentation can be found on TclHelp

Launch Formal Profiler:

```
formal_profiler -property <prop_name>[-engine_mode (B | M)]
                                     [-initial_bound <bound_num>]
                                     [-time_limit <time_lim>]
                                     [-silent][-viewer]
```

```
[<embedded>] % formal_profiler -property <embedded>::path.vcomp_path_inst.assert_data_good_bb -viewer
```

Stop the profiler:

```
formal_profiler -stop
```

Reports:

```
formal_profiler -report (-instance <instance_list>
                        |-signal <signal_list>
                        |-property <property_list>)
                        [-bound <bound_num>]
```

List bounds:

```
formal_profiler -list (bound)
```

406 © Cadence Design Systems, Inc. All rights reserved.

cadence

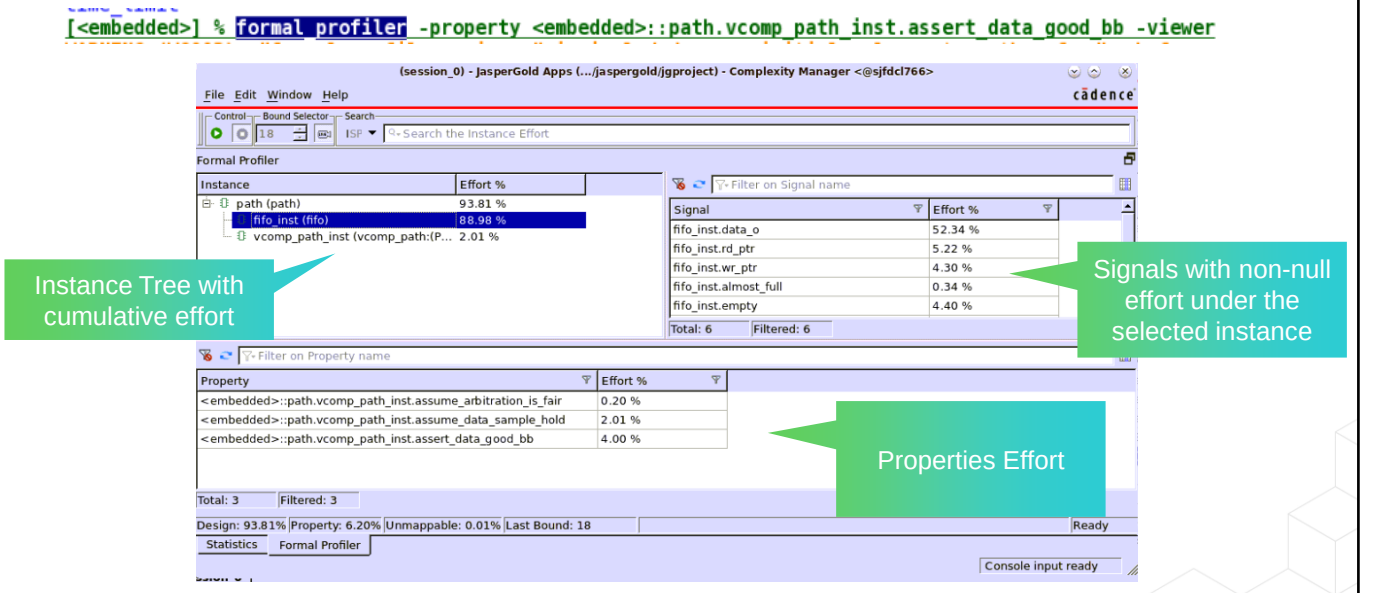
In order to run the formal_profiler you will need an additional license feature:

JGFVADVP : Jasper Advanced Platform Option to JGFVBASE : jasper_advp

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Profiler: GUI



407 © Cadence Design Systems, Inc. All rights reserved.

In order to run the formal_profiler you will need an additional license feature:

JGFVADVP : Jasper Advanced Platform Option to JGFVBASE : jasper_advp

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Profiler: Possible Remedial Actions

- Identify more abstraction candidates by identifying high activity
  - Structures (counters, FSMs …)
  - Instances
  - Signals
- Blackbox/stopat high activity instances outside verification scope
- Add stopats on “unused” signals
- Disable assumptions that are not used (as reported by the profiler)
- Recode complex constraints / Aux code

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Summary

There are generic SVA coding techniques that can help the tool

How “hard” a problem is for the tool has many complex factors

We don’t always need the whole design, or even the whole COI, to get a pass result

Don’t fix things that are not broken

- If you get conclusive results within an acceptable time, then don’t spend time trying to make it quicker

Jasper provides tools to help us understand complexity visually or in text form

- Tcl commands and options allow users to create powerful scripts to aid understanding of the complexity

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 11

Complexity Reduction Methods

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Module Objectives

This module is mainly focused on reduction, overconstraint, and abstraction

In this module, you will be able to,

- Compare abstraction, overconstraint, and reduction
- Reduction
  - Parameterization
  - Difference between Reduction and abstraction
- Overconstraints
  - Use Overconstraints in a good way to gain confidence in design
  - Apply Overconstraints
- Abstractions(Underconstraint)
  - Use abstraction mechanisms to greatly reduce the time taken for a formal proof
  - Apply IVAs to choose arbitrary configurations
  - Apply Reset Value Abstractions (RVAs)
  - Apply Counter Abstractions

 © Cadence Design Systems, Inc. All rights reserved.

cadence

We need to know what reduction, overconstraints, and abstraction mean and why we do use them.

We need to understand what is reduction and the difference between reduction and abstraction.

We need to understand use of overconstraints in good way and how to apply overconstraint.

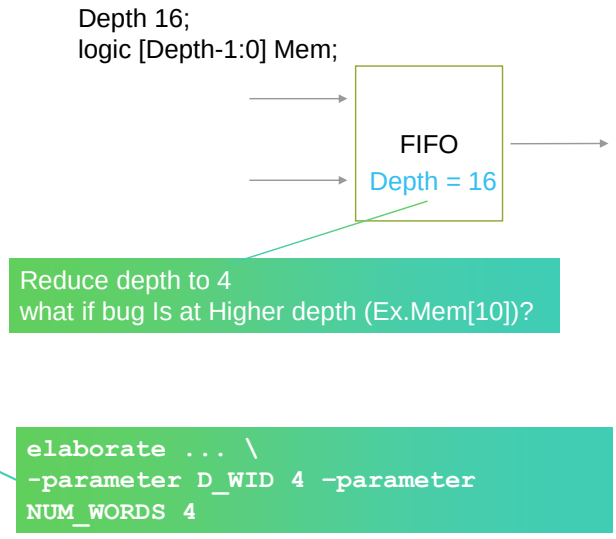
We need to understand what an abstraction is and what it is not. why we use abstractions, and the different kinds. So we're going to talk about I-V-A initial value abstractions, reset value abstractions, and counter abstractions.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Reduction

- **Large data or delays common in designs**
  - Memories, Fifos, Wide data bus
  - For large, symmetric designs, why not simplify?
- **Many common problems are not size-dependent**
  - Read/write pointers at the same location
  - Read/write pointers wrap-around
  - Read/write pointers at adjacent locations
- **Easy to reduce if RTL is well-parameterized**
- **But be careful**
  - Need to keep reductions self-consistent
  - Can distort behavior and add false failures
  - Not true overconstraint-changes signal relationships
  - Can hide real bugs



412 © Cadence Design Systems, Inc. All rights reserved.

cadence

Let’s start with the simple complexity reduction method called Reduction.

Many large memories or Fifo depth can be reduced, minimizing depth space, and making it easier to run the design on formal engines.

If the design is well parameterized, it is easier to use reduction.

It is good to gain confidence at the initial stages of verification, but there is a risk of hiding bugs. Sometimes it can distort behavior and add false failures.

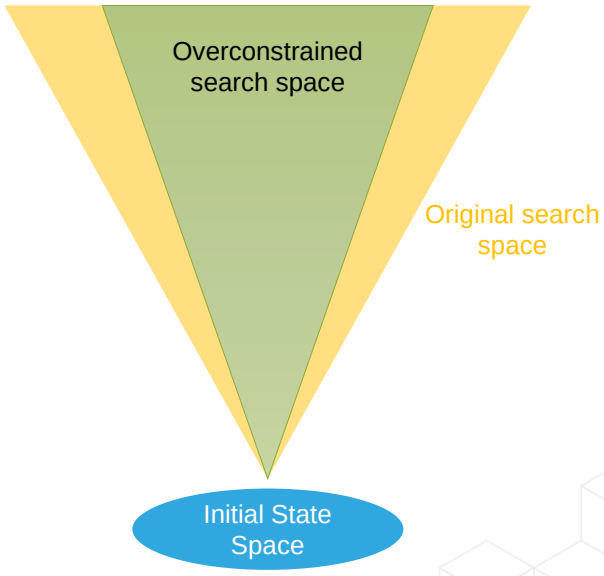
For example, if FIFO has an actual depth of 16 and we reduce the depth of FIFO to 4 from 16, there is a chance of bugs at a deeper depth .

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Overconstraining the Proof

- **Overconstrain = Limit search space**
  - Fewer possible values or behaviors
  - Usually more general than reduction
- **Common methods**
  - SVA assumptions
  - Tcl commands (*reset –non_resettable_regs0)*
  - Case Split: analyze one value/mode/op at a time
- **False Failures not possible**
  - Verifying strict subset of original proof space
  - Any cex / trace generated is valid
- **But proofs are not fully reliable**
  - Hidden bug possible in eliminated space
  - Need to check coverage carefully
  - May need to shift strategy for validation closure



413 © Cadence Design Systems, Inc. All rights reserved.

cadence

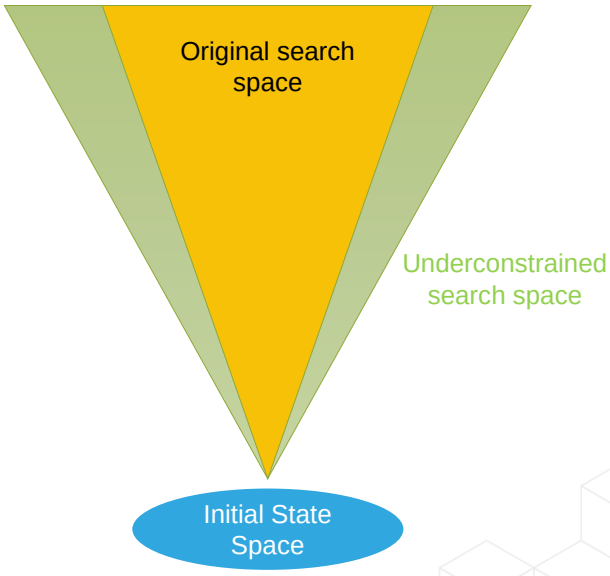
- In Overconstraining, we are reducing the state space to verify design-specific features or logic behavior.
- Overconstraining can be done by simply adding SVA Assumptions
- As we are reducing state space, there is no possibility for false failures, but legal behavior can be missed.
  - Any counterexample is valid
- While Overconstraining design, we must be careful that it can hide bugs.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Underconstraining the Proof

- **Expand but Simplify Proof Space**
  - More possible behaviors
  - Stopats, Black Boxes, Abstractions
  - Simplifies formal analysis in many cases
  - “Fast Forward”-events possible in fewer cycle
- **False Failures are possible**
  - Need to be aware when debugging
  - Poor choice of underconstraint ➔ extra noise, effort
    - May need to add companion assumptions
  - Examine cover traces carefully
- **Proofs are reliable**
  - All original behaviors can happen… and more
  - Can signoff on proof completed during underconstraint



414 © Cadence Design Systems, Inc. All rights reserved.

cadence

- In underconstraining, we relax state space to verify design specific features or logic behavior.
- As we are relaxing state space, there is a possibility for false failures, but no chances of missing in legal behavior.
- Abstracting complex logic can ease the proof, but we can see many false failures.
- Proofs are reliable as all original behaviors can happen

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Reduction, Overconstraint and Undercontraint

	Reduction	Overconstraint	Underconstraint (Abstraction)
Are all original behaviors possible?	**No** Design behaves differently now, and some behaviors may no longer possible	**No** Design behaves differently now, and some behaviors may no longer possible	**Yes** Can still exhibit the original design behavior – and more
Can all bugs in the original design still be caught?	**No** Bugs caused by the removed behavior will not be caught – but ones still allowed may be caught sooner	**No** Bugs caused by the removed behavior will not be caught – but ones still allowed may be caught sooner	**Yes** All the original behaviors are still possible, so any bugs caused by them can still be caught
Could it cause any false failure (bogus cex)?	**Yes (sometimes)** Reductions usually keep the design functioning in a way close to the original, but they can distort some logic	**No** An overconstraint strictly reduces the space of possible behaviors, so any counterexample is real	**Yes (often)** We're expanding the space of possible behaviors, often adding some bogus ones
Effort	**Low (typically)** If design is well-parameterized, easy to reduce	**Low-Medium** Need some design understanding to choose good overconstraint	**Low-High** Simple bboxes/stopats are easy; finding abstraction with minimal false failures can be tricky

 © Cadence Design Systems, Inc. All rights reserved.

cadence

- Some key factors must be considered when applying these three complexity reduction methods. This table represents the difference between Reduction, Overconstraint, and underconstraint.
- In Underconstraint all original behaviors are possible, while in reduction and overconstraint some behaviors are not possible.
- All bugs can be caught in underconstraint because all behaviors are possible, but chances of false failures are also there, on the other hand, there is no chance of false failure in reduction and overconstraint but it can hide bugs.
- In terms of effort needed reduction requires low effort if the design is well parameterized, for overconstraining and underconstraint some design understanding is needed to apply properly.
- At last, it depends upon the need which reduction techniques need to apply.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Submodule 11.1

Abstraction (Underconstraint)

cadence

This page does not contain notes.

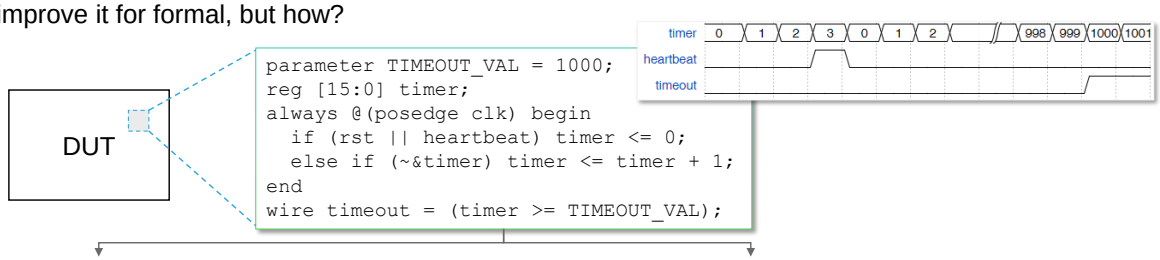
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Abstraction Versus Reduction

Let's start with an example:

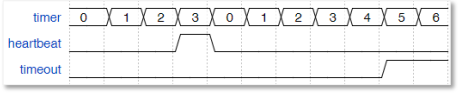
- DUT has a timeout mechanism which takes too long to hit
- Need to improve it for formal, but how?



Reduction

Change timeout value

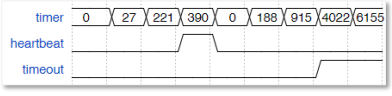
```
elaborate ... \
  -parameter TIMEOUT_VAL 5
```



Abstraction

Allow timer to fast forward

```
stopat timer
assume {rst || heartbeat |=> timer == 0}
assume {!rst && !heartbeat && &timer |=> &timer}
assume {!rst && !heartbeat && ~&timer |=> (timer > $past(timer))}
```



Assumptions model timer behavior, but increment is allowed to be greater than 1

417 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

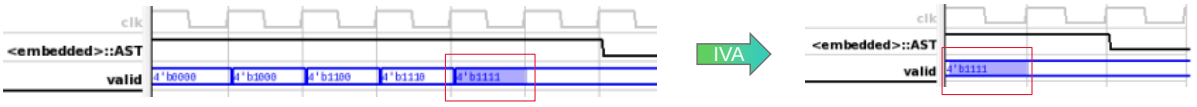
Initial Value Abstraction (IVA) Introduction

Formal Analysis starts from an initial state

- The initial state is typically the state after the reset sequence has finished

Often difficult scenarios are only hit when a part of the design has reached a certain state

- Reaching this state can take many cycles from the initial state – i.e., the tool has to compute many cycles
- Can we shorten the number of cycles that are necessary from the initial state to hit the scenarios?



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Example: IVA

- A cache enable bit is set in a configuration register
  - Many interesting things happen when the cache is active
- Reset value of the mode bit is 0; functionality to be verified is switched off out of reset
- It takes several cycles to write to the configuration register and change its mode
- In order to hit the interesting scenarios, formal has to go through the `init` sequence and enable the cache
- Solution: **Free** the initial value of the cache mode after reset
- Now formal verification includes all scenarios with the enabled cache from the initial state!

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Typical Scenarios Where IVAs Are Used

- Bypass initialization sequence
  - Long initialization sequence that happens after reset
- Configuration registers (“architectural registers”)
  - Registers are user-writeable and do have a reset value
  - Specify valid range and possible relationships with other registers
- Transaction look-a-side buffer
  - Start with the TLB initialized (random but non-empty)
- Arbiter priority
  - Start with any priority
- Many more examples…

- IVAs are unique to formal verification and a powerful technique
- Sometimes, IVA only removes a few cycles from traces
  - This can be the difference between finding/not-finding a trace

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

IVA Example

```
1   module test(clk,rstn);
2
3      input clk, rstn;
4
5      reg   a, b, rstn_delayed;
6      wire  free;
7
8      always @(posedge clk or negedge rstn) begin
9         if (!rstn) begin
10           a <= 0;
11           rstn_delayed <= 0;
12        end else begin
13           a <= free;
14           rstn_delayed <= rstn;
15        end
16     end
17
18     always @(posedge clk or negedge rstn_delayed) begin
19        if (!rstn_delayed)
20          b <= 0;
21        else
22          b <= free;
23     end
24
25  endmodule // test
26
```

a has a reset value of 0

b has a reset value of 0 and reset is delayed for b

 © Cadence Design Systems, Inc. All rights reserved.

cadence

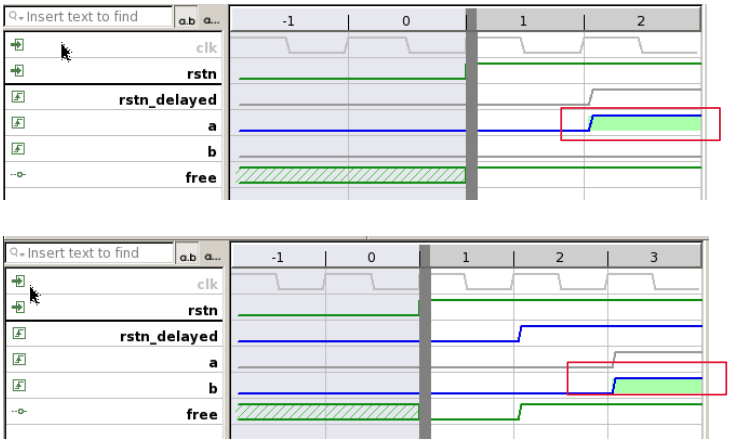
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

IVA Example (continued)

Shortest traces without IVA to make a == 1 and b == 1 respectively




```
module test(clk,rstn);

   input clk, rstn;

   reg   a, b, rstn_delayed;
   wire  free;

   always @(posedge clk or negedge rstn) begin
      if (!rstn) begin
         a <= 0;
         rstn_delayed <= 0;
      end else begin
         a <= free;
         rstn_delayed <= rstn;
      end
   end

   always @(posedge clk or negedge rstn_delayed) begin
      if (!rstn_delayed)
        b <= 0;
      else
        b <= free;
   end

endmodule // test
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

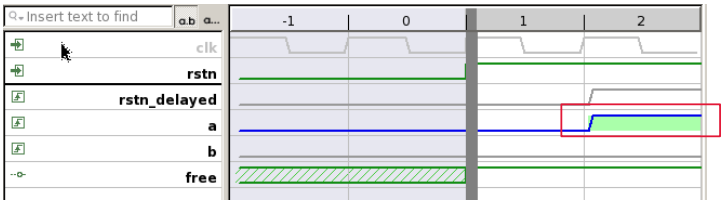
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

IVA Example (continued)

Original trace




```
module test(clk,rstn);

   input clk, rstn;

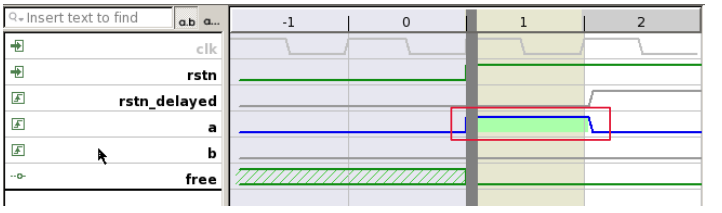
   reg   a, b, rstn_delayed;
   wire  free;

   always @(posedge clk or negedge rstn) begin
      if (!rstn) begin
         a <= 0;
         rstn_delayed <= 0;
      end else begin
         a <= free;
         rstn_delayed <= rstn;
      end
   end

   always @(posedge clk or negedge rstn_delayed) begin
      if (!rstn_delayed)
        b <= 0;
      else
        b <= free;
   end

endmodule // test
```

```
abstract -init_value a
```



423 © Cadence Design Systems, Inc. All rights reserved.

cadence

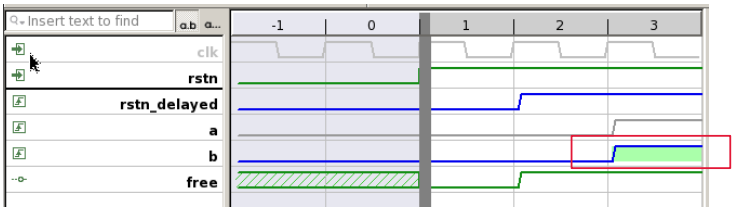
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

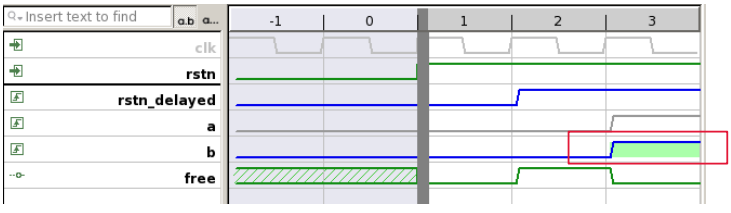
This document is for the sole use of Kun Wang of Seas

IVA Example (continued)

Original trace



`abstract -init_value b` does not have the required effect!




```
module test(clk,rstn);

   input clk, rstn;

   reg   a, b, rstn_delayed;
   wire  free;

   always @(posedge clk or negedge rstn) begin
      if (!rstn) begin
         a <= 0;
         rstn_delayed <= 0;
      end else begin
         a <= free;
         rstn_delayed <= rstn;
      end
   end

   always @(posedge clk or negedge rstn_delayed) begin
      if (!rstn_delayed)
        b <= 0;
      else
        b <= free;
   end

endmodule // test
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

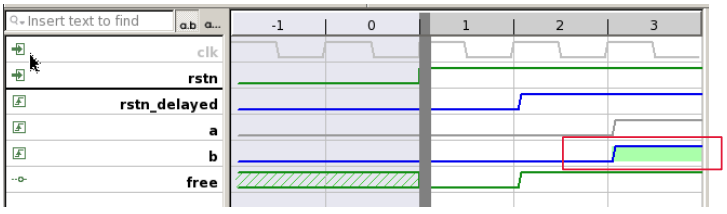
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

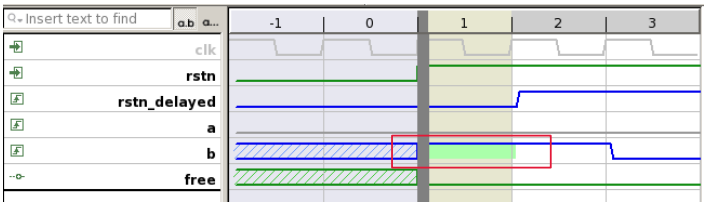
This document is for the sole use of Kun Wang of Seas

IVA Example (continued)

Original trace




```
abstract -reset_value b
```




```
module test(clk,rstn);

   input clk, rstn;

   reg   a, b, rstn_delayed;
   wire  free;

   always @(posedge clk or negedge rstn) begin
      if (!rstn) begin
         a <= 0;
         rstn_delayed <= 0;
      end else begin
         a <= free;
         rstn_delayed <= rstn;
      end
   end

   always @(posedge clk or negedge rstn_delayed) begin
      if (!rstn_delayed)
        b <= 0;
      else
        b <= free;
   end

endmodule // test
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

IVA `abstract` Command Basics

`% abstract –init_value <register>`

- Free value of the target register in the **first cycle** of Formal Analysis
- Driver for logic is retained by JG after the first cycle
- Apply this command if the register is reset during reset analysis

`% assume –bound N <initial_value expression>`

- Specify legal set of initial values for register
  - Applied in tandem with the abstract –init_value command
  - –bound N means assume N cycles after the defined reset

`% abstract –reset_value <register>`

- Free value for the target register **while reset pin of register is active**
  - **This is known as a Reset Value Abstraction (RVA)**
- Apply this if the register is reset during Formal Analysis
- Apply this in cases of, for example, a delayed reset

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Example: Test Multiple Configurations – Description

Imagine a DUT with 3 single bit configuration registers programmed via an external bus

Those config regs will have deterministic reset values

It may take many cycles to program those config regs to specific values via the XBUS

Verification requirement: Know if an assertion passes under every valid configuration

- There would be a maximum of 8 possible configurations

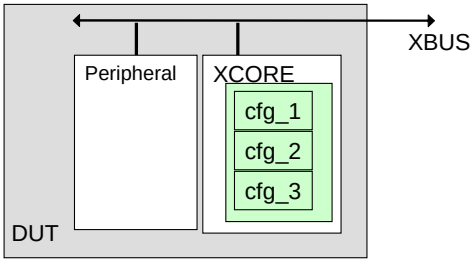
We can do this using IVA

- Allow the config regs to have any value at reset
- If there is a config for which the assertion fails, then we see the CEX

To reduce the state space, we may choose to prevent XBUS writes using constraints

What do we need to do if not all 8 possible configs are valid for this DUT? For example, `cfg1` and `cfg2` cannot both be `1'b1` at the same time.

How many different valid configs are there now?



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Example: Test Multiple Configurations – Assumptions

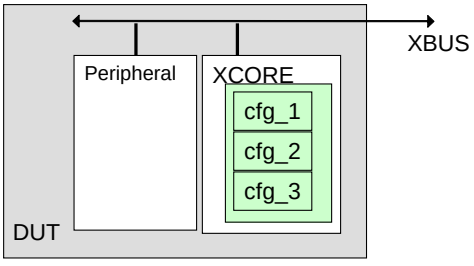
We write assumptions to exclude invalid configs

Those assumptions define which configs are legal

We can convert these assumptions to assertions in other verification contexts, for example

- A different formal task where bus writes are allowed

With an IVA we can verify the assertion for all 6 configurations in a single FPV run




```
cfg1and2_not_both_1: assume property (!(cfg1 && cfg2));
cfg1and2_not_both_1: assert property (!(cfg1 && cfg2));
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Counter Abstractions

Counters are particularly problematic as they infer a large diameter

- e.g., it takes 2^{32} clocks to see all states of a 32-bit counter

If interesting design behaviors only occur when a 32-bit counter is all 1's, then it's unfeasible to observe that behaviour without abstractions

At least 3 possible options:

1. Allow the counters initial value to be abstracted using an IVA

```
%abstract -init_value U1.U2.cntr
```

2. Place a `stopat` (cutpoint) on the counter output

```
%stopat -task <embedded> U1.U2.cntr
```

3. Use a counter abstraction

```
%abstract -counter -task <embedded> U1.U2.cntr
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Results of Abstracting Counters – None and IVA

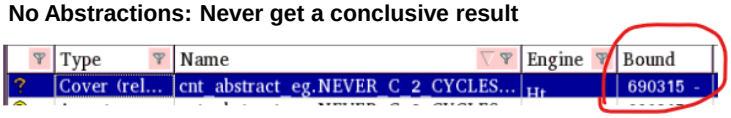
Clearly the assertion will fail as `C` gets stuck at `1'b1` unless reset

But it will take 2^{32} clocks to get that result without abstractions

```
logic [31:0] cntr; //Giant counter
wire  cntr_full =  &cntr;
 always@(posedge clk or negedge rst_n)
  begin
   if (~rst_n) begin
      cntr <= 0; A <= 1'b0;
      B <= 1'b0; C <= 1'b0;
                end
   else begin
      cntr <= cntr + 1;
      if ( cntr == ((2**23) - 19346))
        A <= 1'b1;
      if ( cntr == ((2**28) - 27311))
        B <= 1'b1;
      if (cntr_full)
        C <= 1'b1;
     end
   end

NEVER_C_2_CYCLES :
   assert property (C |=> !C);
```

No Abstractions: Never get a conclusive result



IVA: Counter value all 1's in *first cycle only*



430 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

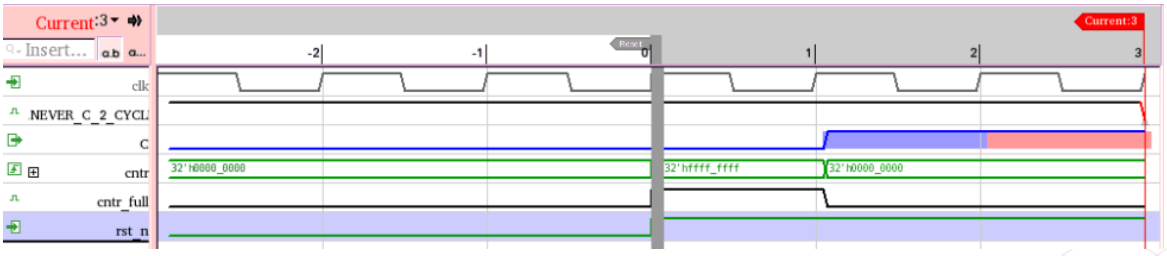
This document is for the sole use of Kun Wang of Seas

Results of Abstracting Counters – `stopat`

```
logic [31:0] cntr; //Giant counter
wire  cntr_full =  &cntr;
 always@(posedge clk or negedge rst_n)
  begin
   if (~rst_n) begin
      cntr <= 0; A <= 1'b0;
      B <= 1'b0; C <= 1'b0;
                end
   else begin
      cntr <= cntr + 1;
      if ( cntr == ((2**23) - 19346))
        A <= 1'b1;
      if ( cntr == ((2**28) - 27311))
        B <= 1'b1;
      if (cntr_full)
        C <= 1'b1;
        end
   end

NEVER_C_2_CYCLES : assert property (C |=> !C);
```

stopat : Counter can be *any* value on *any* cycle



 © Cadence Design Systems, Inc. All rights reserved.

cadence

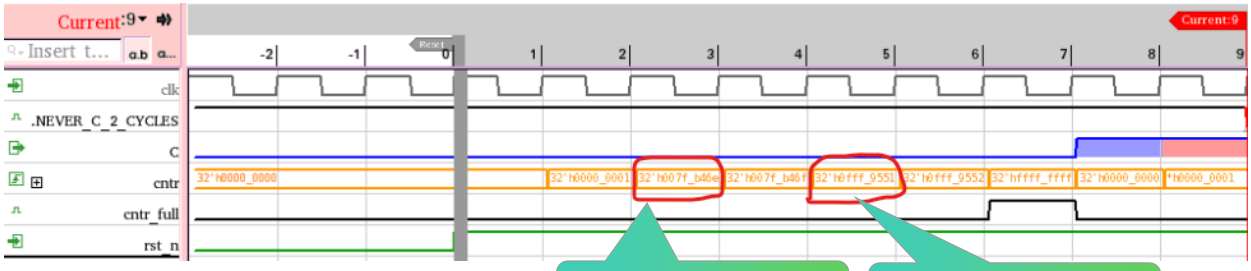
((223) - 19346) in Hex is 007fb46e**

((228) - 27311) in Hex is 0fff9551**

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Results of Abstracting Counters – `abstract`




```
logic [31:0] cntr; //Giant counter
wire  cntr_full =  &cntr;
 always@(posedge clk or negedge rst_n)
  begin
   if (~rst_n) begin
      cntr <= 0; A <= 1'b0;
      B <= 1'b0; C <= 1'b0;
                end
   else begin
      cntr <= cntr + 1;
      if ( cntr == ((2**23) - 19346))
        A <= 1'b1;
      if ( cntr == ((2**28) - 27311))
        B <= 1'b1;
      if (cntr_full)
        C <= 1'b1;
        end
   end

NEVER_C_2_CYCLES : assert property (C |=> !C);
```

((2**23) - 19346)) = Hex 007fb46e

((2**28) - 27311)) = Hex 0fff9551

abstract –counter: Counter value goes through "important values" in the sequence which the real design would

The important values are auto-detected

432 © Cadence Design Systems, Inc. All rights reserved.

cadence

((223) - 19346) in Hex is 007fb46e**

((228) - 27311) in Hex is 0fff9551**

We can use the abstract command to find counters in the design and their important values and their disable condition:

abstract –counter –find

The Important can be given via –values switch but it is strongly not recommended to use –values as some important value might get missed -> chances of missing bugs

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Summary

The Jasper™ Tcl command `abstract` allows user to define Initial value abstraction(IVA), Reset value abstraction(RVA), and counter abstractions

Abstractions allow us to greatly reduce the time to achieve a formal proof

It is essential that we understand the consequences of the abstraction we are making

- Otherwise, we may be hiding problems

Designers are often the people with the greatest insight into the validity of an abstraction

Assumptions may be required when using abstractions

- Those assumptions need to be validated

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Submodule 11.2

Overconstraint

cadence

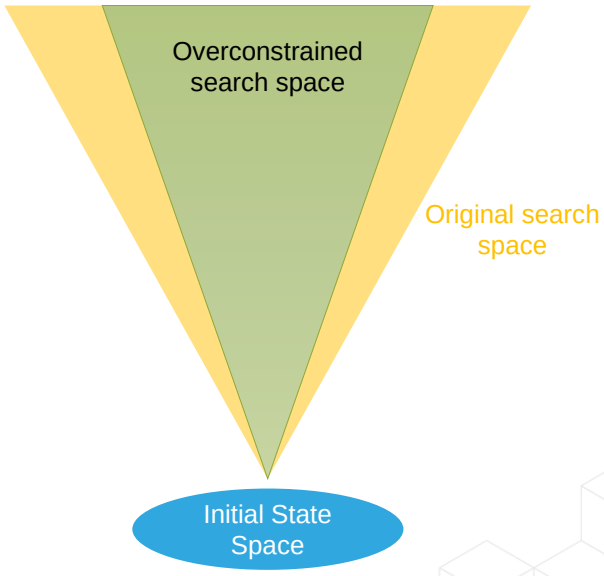
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Overconstraining the Proof

- **Overconstrain = Limit search space**
  - Fewer possible values or behaviors
  - Usually more general than reduction
- **Common methods**
  - SVA assumptions
  - Tcl commands (*reset –non_resettable_regs0)*
  - Case Split: analyze one value/mode/op at a time
- **False Failures not possible**
  - Verifying strict subset of original proof space
  - Any cex / trace generated is valid
- **But proofs are not fully reliable**
  - Hidden bug possible in eliminated space
  - Need to check coverage carefully
  - May need to shift strategy for validation closure



435 © Cadence Design Systems, Inc. All rights reserved.

In Overconstraining, we are reducing the state space to verify design specific features or logic behavior. As we are reducing state space, there is no possibility for false failures, but legal behavior can be missed.
While Overconstraining design, we need to be careful that it can hide bugs, to gain confidence at first. Overconstraining is good.
After overconstraining, it is good to run coverage and check

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Preload Value Using Stopat and Assume

- Also possible to load values just at the beginning
- Useful to preload values in fifos, memories, etc.
  - `% stopat -reset {active_transactions[0].valid}`
  - `% stopat -reset {active_transactions[0].opcode}`
  - `% stopat -reset {active_transactions[0].data}`

  - `% assume -reset {active_transactions[0].valid  == 1'b1}`
  - `% assume -reset {active_transactions[0].opcode == OP_INV}`
  - `% assume -reset {active_transactions[0].data   == 8'h5a}`
- Fill array/memory by scripting above commands in loop
- Can enable “interesting” activity much earlier than normal run
  - But maybe harder to reach case you didn’t anticipate

 © Cadence Design Systems, Inc. All rights reserved.

cadence

- In particular conditions or scenarios, if we want to check a specific logic combination, then we can preload the value with the help of stopat and assume.
- Look at this example here. A specific combination needs to be checked. So, stopat is added to the signals so the tool will stop driving these signals from its driving logic, and the tool can give any random values. So, constraints are added to them. Now, this will allow the tool to drive only these particular values.
- We are deliberately giving some value to the signal; because of these assumptions, no other combination is possible.
- The result of this tool can give you interesting scenarios at the early run stage, which is more challenging to reach without this

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Reset State Overconstraining

- Uninitialized flops can cause complexity: engines check all combos
  - Check if your design can benefit from this technique

```
% llength [get_reset_info -silent]
100   <-- Flops with concrete values
% llength [get_reset_info -x_value -silent]
36    <-- Flops with X-values
```

- Use the command **`reset -non_resettable_regs <0|1>`**
  - Forcing initial state all X flops to 0/1
  - Will greatly reduce spatial complexity on cycle-1
- Modifying the **`reset`** command will impact all tasks
  - Can do this in the separate task by adding bound-1 assumes to all uninitialized flops:
    - `foreach f [get_reset_info -x_value-silent] { lappendxlist[lindex$f 0] }`
    - `foreach f $xlist{ assume -name oc_over_nr__$f-bound 1 "~|$f" }`
- *Check results–sometimes overconstraining reset **increases** complexity*

 © Cadence Design Systems, Inc. All rights reserved.

cadence

In formal, engine checks for all combinations possible in the design which include Flops with concrete values and x-values.
Uninitialized flops can cause complexity. In such cases, reset state overconstraining can be used; it will force all X flops to take 0 or 1. which significantly reduces spatial complexity from cycle one itself.

Always check the result. Sometimes, overconstraining reset increases the complexity.

© Cadence Design Systems, Inc. All rights reserved.

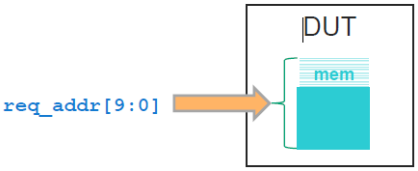
This document is for the sole use of Kun Wang of Seas

Design Reductions Using Overconstraint

- **The simplest reduction method is to reduce parameters**
  - Reduce data width, FIFO size, memory size, etc.
  - `elaborate -parameter` can change parameters of inner modules
  - Be careful, though: not true overconstraint
  - Can distort relations between signals/modules
- **What if RTL is not well-parameterized, or logic distorted?**
- **Use overconstraint to reduce without touching parameters**
  - Blackbox or `stopat` unwanted logic
  - Overconstrain environment to avoid access to removed logic

```
parameter MEM_SZ = 1024;
…
logic [MEM_SZ-1:0] mem;
…
// What happens below
// if MEM_SZ changed?
logic [9:0] addr;
…
assign foo = mem[addr];
```

Example: `stopat` on memory indexes 4-1023 + overconstraint to prevent access to 4-1023




```
stopat mem[4:1023]
assume {req_addr < 4}
```

438 © Cadence Design Systems, Inc. All rights reserved.

cadence

As we discussed about the reduction in the earlier module, if the design is well parameterized, we can reduce the design to reduce its state space, which will reduce the complexity.

But what if the design is not well parameterized or has distorted logic? In such cases, overcosntraint can be used to reduce the design size.

Let's take an example to understand, stopat is added on memory indexes 4-1023, and overconstraint is added to prevent access to 4-1023, resulting in only four valid req_addr instead of 1024. so now the tool has to check for only four req_addr which will reduce complexity.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Assumptions Based Overconstraining

- Remember to add assumptions in separate tasks
  - So you don't lose track of the real environment
  - Also add *oc_*prefix to overconstraining assumptions

```
# create dedicated task for INDEX over-constraining
task -create <oc_index> -source_task <baseline> -copy_all -set
assume -name oc_over_index_0 {address[7:4] == 4'b0}

# create dedicated task for TAG over-constraining
task -create <oc_tag> -source_task <baseline> -copy_all -set
assume -name oc_over_tag_0 {address[19:10] == 10'b0}
```

- Usually don't want **`assume -env`**: applies to all tasks at once
  - Appropriate to tie-off non-function features like BIST / scan / DFT
  - Clears proof results for **all tasks** when changing a single **`-env`** assumption

 © Cadence Design Systems, Inc. All rights reserved.

cadence

We can add assumptions to block certain combinations, which helps tools reduce complexity.

But to be more specific, as in overconstraint, there is a chance that it will hide bugs.

For example, if address is 32 bit and we want certain bits to force to some specific value

Address[4:0] equal to zero and Address[19:10] to zero then by adding simple assumptions on this signal, we can do the job

It is recommended to use the task function in Jasper tool to add over constraint so it will not affect the actual design run

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Assumptions Based Overconstraining: Good Candidates

- Ignore highly disruptive events: interrupts, errors, flush, bad packets
  - These scenarios greatly impact persistent state
  - Complexity may prevent FV engines from filling caches / branch predictors / queues
- Limit to a subset of the ISA
  - Instructions, packet types, opcodes
- Limit the number of active interfaces
- Limit the number of pending requests
- Remove backpressure events
  - Semi-formal engines will often take the easiest route and might spend less time on hard cases

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Let’s see what the excellent candidates for Assumptions Based Overconstraining

We must ignore interrupts, errors, and flush signals because this will significantly impact the persistent state.

We can limit a subset of instructions packet types and opcodes as sometimes we want to send a specific combination .

we can limit the number of active interfaces, which will help to find bugs in one particular path, and it will be easier for the tool to run properties faster.

In some cases, if we check certain request grant logic, we can reduce the number of pending requests to check for proper grants for each request and forward progress in the design.

Simple assumptions can be used to remove back-pressure events that cause complexity.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Overconstraint Example: Interface Limiting

Limit the number of active interfaces (NOC)

- Can be handled by a bit-vector in conjunction with $countones(vector)
- Allow traffic from only N interfaces -> remaining interfaces are idle

```
// SVA
logic [9:0]           enable_traffic;
integer               used_cnt;
always @(posedge clk) used_cnt <= used_cnt;
oc_used_cnt: assume property (used_cnt >= 1 && used_cnt <= 10);
oc_used_if:  assume property ($countones(enable_traffic)==used_cnt);
for (int i=0; i<=9; i++) begin
oc_disable_if:   assume property (~enable_traffic[i]|-> ~if[i].valid);
end
```

- Many error/deadlock scenarios require specific timing on 2-3 interfaces

 © Cadence Design Systems, Inc. All rights reserved.

Let us understand with an example of interface limiting

In this example, we want to enable traffic on certain valid signals: if[i]:Valid

Used_cnt signal is used as an un-deterministic constant, which means it can take any value, but the value will be stable for each trace. However, because of constraint oc_used_cnt, it can take value only between 1 to 10.

Use of oc_used_if is to enable traffic only on signal which is given by used_cnt: if used_cnt is 3, then on only three inputs valid will be High

Oc_disable_if is used to constraint valid apart form value given by used_cnt: if used_cnt is 33 than rest 7 valid will be low

Now, we are only enabling transactions on specific valid signals with the help of this logic and constraint, which will affect the complexity and reduce complexity.

This will help to find any deadlock and errors in small amount of time and window.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Overconstraint Example: Transaction Limiting

Limit the number of transactions sent on a specific interface

- Can some high-value property be proven for less than N transactions?
- After N transactions, the interface will be IDLE forever

```
task -create <limit_reqs_1> -source_task <baseline> -copy_all -set
task -create <limit_reqs_2> -source_task <baseline> -copy_all -set
task -create <limit_reqs_3> -source_task <baseline> -copy_all -set
task -create <limit_reqs_4> -source_task <baseline> -copy_all -set
assume -name <limit_reqs_1>::oc_over_limit_reqs{ar_valid && ar_ready)[=1] |=> always(~ar_valid)}
assume -name <limit_reqs_2>::oc_over_limit_reqs{ar_valid && ar_ready)[=2] |=> always(~ar_valid)}
assume -name <limit_reqs_3>::oc_over_limit_reqs{ar_valid && ar_ready)[=3] |=> always(~ar_valid)}
assume -name <limit_reqs_4>::oc_over_limit_reqs{ar_valid && ar_ready)[=4] |=> always(~ar_valid)}
```

- Will yield a transaction bound based on proven results by task
- Use COV to determine how much can be exercised by N requests

	Type	Name	Engine	Bound	Time	Task
✔	Assert	nbdcache.b_chk.ast_check_bit0	N (16)	Infinite	0.2	<xlimit_1>
✔	Assert	nbdcache.b_chk.ast_check_bit0	N (32)	Infinite	7.6	<xlimit_2>
✔	Assert	nbdcache.b_chk.ast_check_bit0	N (42)	Infinite	13.7	<xlimit_3>
✔	Assert	nbdcache.b_chk.ast_check_bit0	AM (85)	Infinite	1903.3	<xlimit_4>

 © Cadence Design Systems, Inc. All rights reserved.

Lets understand overconstraining with transaction limiting example

Transaction can be limit on specific signals; in this example, four different tasks are used with four different over-constraint to limit the transaction from one to four per transaction, which will help the tool to prove properties in a short amount of time, as we can see in properties table for each task each taking a different time, it concludes that for less number of transaction, less no of time is needed for the tool to prove the property.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Leveraging Overconstraint: Risky But Worth It

- **Overconstraints == major productivity booster early…**
  - Initial “wiggling” phase / design bring-up: rapidly iterate bug finds
  - But can be a long-term hazard
  - What if you (or the future owner) forget details?
- **Remember to isolate in a separate task**
  - Try to prove undetermined properties in overconstraint task

```
set ulist [get_property_list -include {type assert status undetermined}]
task -create <undet> -copy_assumes -copy_stopats -copy_abstractions all -copy $ulist -set
foreach f [get_reset_info -x_value -silent] { lappend xlist [lindex $f 0] }
foreach f $xlist { assume -name oc_over_nr__$f-bound 1 "~|$f"}
```

  - Any CEX is still valid
  - Any proof == chance to prove, with more effort/time in main task
    - Switch focus back to main task after overconstrained proof clean

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Overconstraint is risky but worth the effort; use over constraint at early stages to boost verification.

At the initial design bring-up stage, bugs can be found easily by using overconstraint.

We need to make sure for long time, it can become a hazard if you or a future owner forget the details, so it will be difficult to fully verify the design.

It is best to use the task to isolate the overconstraint so it will not affect the actual task; once the proof is clean or you have reached the expected result, then the overconstraint can be relaxed so the focus can be switched back to the main task.

At last, if overconstraint can bring productivity boost in the early stages, but some risk of hiding bugs there if you are planning to apply overconstraint, make sure that you follow all the necessary steps and keep track of overconstraints so it will not affect the actual set verification goals

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Summary

- Overconstraining reduces problem space to focus efforts, but it can hide bugs
- No chances of false failures
- Assumptions can be used to add overconstraint
- Best to isolate overconstraints to specific tasks
- Great for early bug-finding

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 12

The Verification Completeness Problem

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Module Objectives

In this module, you will be able to,

- Review verification methodologies
- Examine the case study highlighting the verification completeness problem
- Identify/List out techniques for minimizing risk of verification holes
- Evaluate the merits of each verification technique

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Scale of the Functional Verification Problem

The complex of IC designs may have, for example, 500 thousand flips-flops (FFs)

Those FFs could be in any of 2^{500000} different states at any particular time

The actual number of states will be much less because:

- By design, not all states will be possible
- There will be many states we don't particularly care about

However, the number of states is still absolutely massive

It is essential that you accept this truth

You can't possibly test everything

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

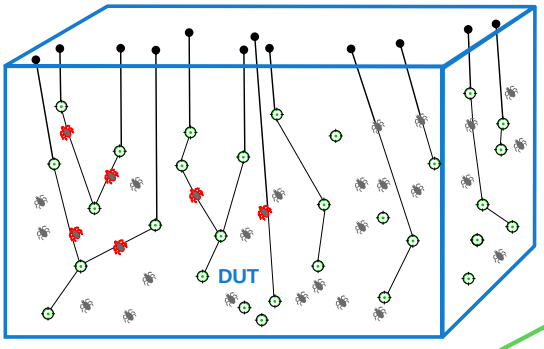
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Most Basic Approach to Verification: Directed Tests

Verification engineer:

- Defines DUT states to be tested based on specified behavior and corner cases
- Writes directed test to visit and verify each item in the test plan



Redo if design changes

Automation — Significant manual effort to write all the tests

Automation — Work required to verify each goal was reached

Completeness — Poor coverage of non-goal scenarios … especially the cases that you didn't "think of"

448 © Cadence Design Systems, Inc. All rights reserved.

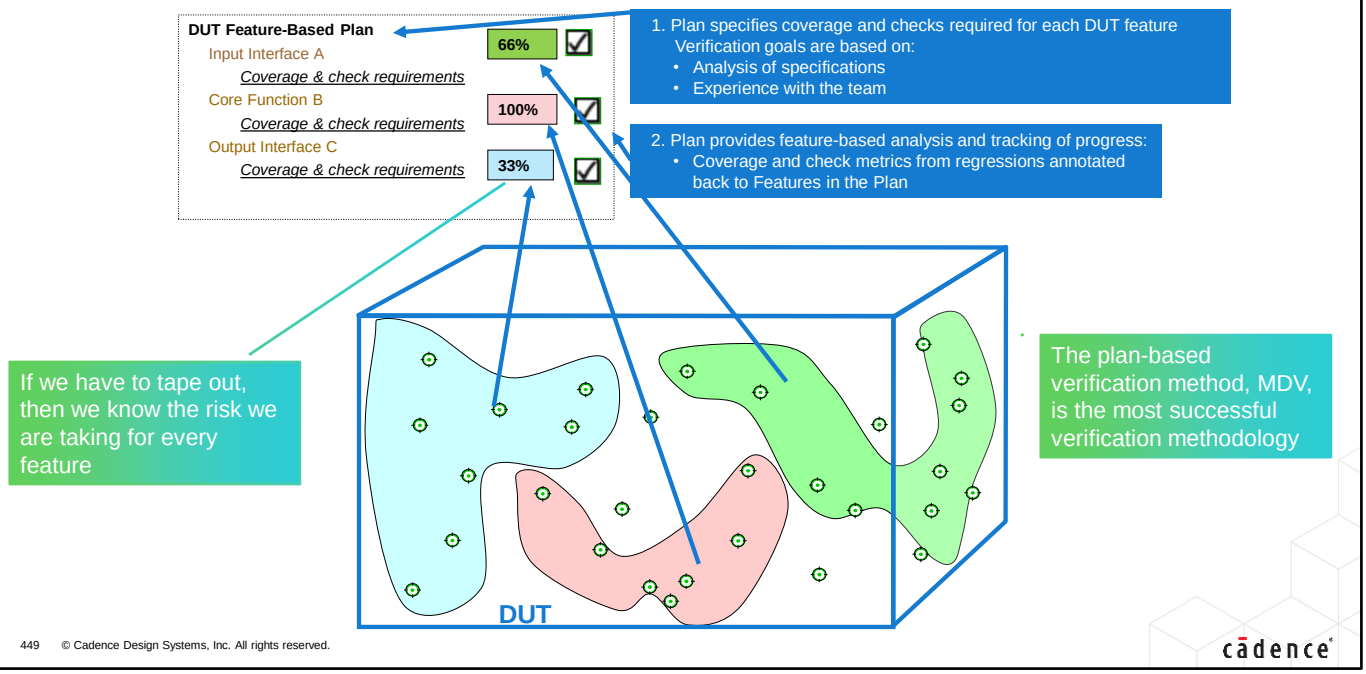
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Metric Driven Verification (MDV)



449 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Concise and Simplistic Summary of MDV

Define a hierarchical verification plan that:

- States the features the design is supposed to have
- Specifies the checks to be done in order to test those features
- Specifies the coverage required in order to be confident that we tested every important scenario the design is expected to work under

We know we have finished verification when we have reached 100% coverage in the context of no checks failing

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

“The Best Laid Plans of Mice and Men”

As the saying goes, “If we fail to plan, then we plan to fail”

However, having a plan does not guarantee success

The plan could be wrong

The plan might not be implemented correctly

To show how this can happen for even the simplest of designs, we will follow a case study

The schematic we will show defines the functional specification
The intent is to verify every feature of the design by defining SVA properties
We will verify these properties using Jasper™ Formal Verification

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Completeness of Property Sets

We obviously define what we think are sufficient properties:

- As identified in the Verification plan(Vplan)

This is not just limited to properties or formal verification

- Same argument for a constrained random coverage-driven simulation testbench:
  - Did we specify enough coverage?

How can we be certain they cover all required functionality?

A formal tool might tell us that all I/Os and FFs are covered by assertions

This does not mean that we have written properties to cover all functionality

- In the same sense that code coverage does not tell us we have complete functional coverage

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Example: Verification [in]Completeness

Shown is a schematic of the functional requirements that are implemented in RTL:

- We write blackbox properties, which we think completely check all desired functionality.
- Human error or oversight might mean that we don't specify all desired behaviour.

Tool might tell us

- All I/Os and all FFs are included in some assertion:
- There is no dead code
- All checks pass
- All cover items pass

Should we conclude that verification is complete?

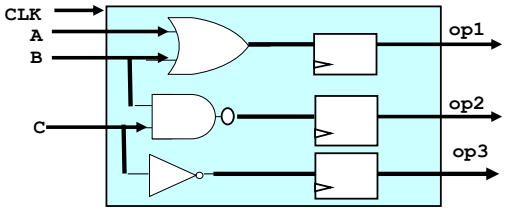
- No way of knowing that we specified enough properties to check all functionality.
- Human error or oversight might escape detection in the verification process.

At first glance these properties seem sensible, however ….

```
P1: assert property ( (A || B) |=> op1 ); ☑
P2: assert property( (!A && !B) |=> op2 );
P3: assert property( !C |=> op3 );
```

Human error. A has no effect upon op2 but property still holds

a	b	c	op1`	op2`	op3`
0	0	0	0	1	1
0	0	1	0	1	0
0	1	0	1	1	1
0	1	1	1	0	0
1	0	0	1	1	1
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	1	0	0



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

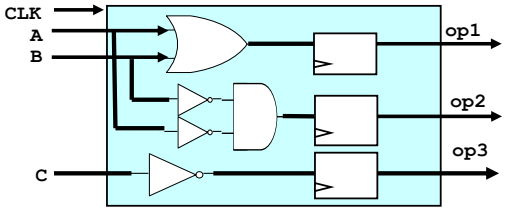
Design Mutation

- Shown is a schematic whose behaviour satisfies our properties
- This is a mutant of the desired design
- We can see that our verification environment fails to detect this mutation of our desired design
- Hence, we did not capture a property set that completely and uniquely defines desired behaviour

Change from desired truth table

a	b	c	op1`	op2`	op3`
0	0	0	0	1	1
0	0	1	0	1	0
0	1	0	1	0	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	0	0
1	1	0	1	0	1
1	1	1	1	0	0

```
P1: assert property ( (A || B) |=> op1 ); ☑
P2: assert property( (!A && !B) |=> op2 );
P3: assert property( !C |=> op3 );
```



454 © Cadence Design Systems, Inc. All rights reserved.

In this example, we can (if the properties are sufficient) detect the mutant design.

In practice, the mutation might not propagate an observable difference from the expected behavior to somewhere it could be detected.

This indicates a problem with complete stimulus, that is, we did not provide the correct stimulus to observe the difference in behavior caused by the mutation.

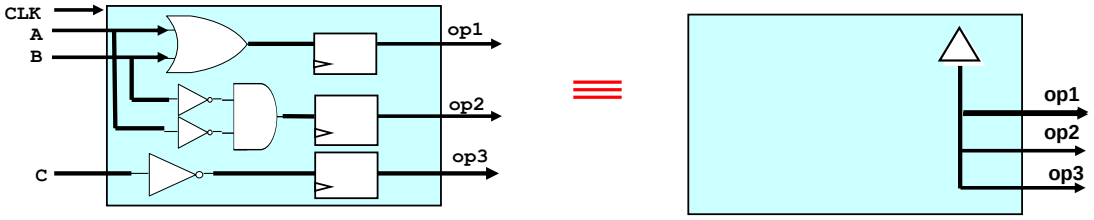
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

We Don’t Even Have the Fundamentals Correct

All these properties pass with the outputs tied to the power rail

```
P1: assert property ( (A || B) |=> op1 );  ☑
P2: assert property( (!A && !B) |=> op2 ); ☑
P3: assert property( !C |=> op3 );         ☑
```



The correct assertion to check OP1 is not particularly intuitive

```
P1: assert property( op1 == $past(A || B)); ☑
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

How Can We Catch That Problem?

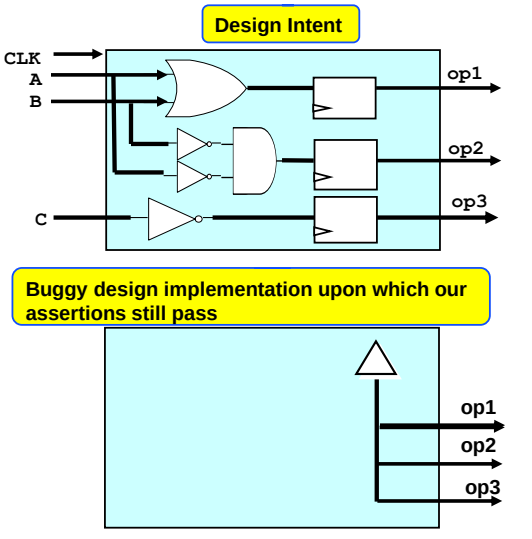
Clearly, our assertions didn't check what we intended

Probably, given our design example, we didn't think of any meaningful user-defined covers

How can we lower the probability of this bug escaping verification?

We have already seen Jasper features which would find these problems

- Superlint Auto Formal Toggle check
- Jasper Coverage App
  - None of the designs would be in the proof core




```
P1: assert property ( (A || B) |=> op1 );  ☑
P2: assert property( (!A && !B) |=> op2 ); ☑
P3: assert property( !C |=> op3 );         ☑
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Locality and the Dangers of Overoptimism

If one made Jasper report elements in the COI of the assertion, then it would be both the grey and green parts

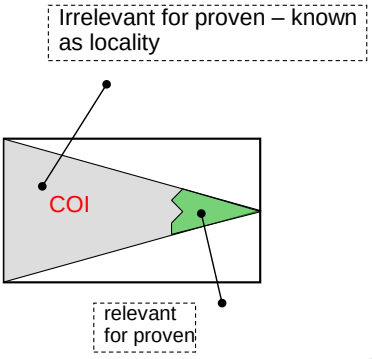
- i.e., the static fan-in cone of all signals in the assertion

Question: Does this mean we have covered/verified/exercised all elements in the COI of the property?

There is the danger of overoptimism on coverage

The grey part is irrelevant to the proof

We learned nothing about the grey part



457 © Cadence Design Systems, Inc. All rights reserved.

cadence

Knowing how much of the design was meaningfully exercised by Jasper is a very important metric for just about every customer.

The coverage app can collect design metrics on:

- Stimuli coverage
- COI coverage
- Proof coverage
- Bound coverage
- Code Coverage
  - Branch
  - Statement
- Functional Coverage
  - SVA/PSL covers

Contact your local AE to discuss the Jasper Coverage App.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Can We Achieve Completeness?

Is it impossible for a tool to read our minds?

Probably, but we can get closer using tool features

No current tools can tell you when you have a complete property set

There is no substitute for:

- Understanding the requirements
- Intelligent planning
- Attention to detail
- Hard work

No EDA tool can tell you that your verification is complete.
All the EDA tools can tell us is if we didn't test all of the things that we said we would, namely if we are incomplete.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Assessing Completeness

Defining your verification objectives upfront is the best way to assess when you are done

Beyond this, there are other ways to assess incompleteness:

- Do all the outputs of the block have assertions describing their behavior?
- Have you reviewed your specification documents for statements that might be translated to an SVA property?
- Have you written good cover properties for expected behaviors, and are they covered?
- Jasper Coverage App can help assess what portion of the design your assertions are exercising.
- Cannot tell you when you are done, but when you are not done.
- There are no tools that can tell you when you have done enough verification at present

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Verification Execution Completeness

This is something that we can know

The goal is the same in both dynamic (simulation) and formal:

All check items pass/proved in the context of coverage items *equals* verification complete

Execution Method	Execution Item	Completeness
Dynamic	Check	All checks are executed with no violation (checks pass)
	Coverage	All coverage items are reached (pass); **driver** + DUV dependent

Execution Method	Execution Item	Completeness
Formal	Check	All checks are proven with no violation (checks proved)
	Coverage	All coverage items are reachable; **constraints** + DUV dependent

460 © Cadence Design Systems, Inc. All rights reserved.

cadence

We can know for sure that all checks we have defined have passed and we have reached all cover items.

The holy grail of verification is knowing whether those checks and cover items defined all behaviours which we want to verify.

This might be possible in the future but not at the present time.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

What Can Be Automated?

Its probably more important to ask the question in reverse, what can't be automated:

- Writing the functional specification
- Writing the Verification Plan
- Interpreting the verification plan to implement the checks and coverage.
- Interpreting the results from verification runs
- Understanding the limitations of the verification technique we use

EDA is about automating all that can be; for example,

- Managing huge amounts of data
- Compilation/recompilation
- Running and rerunning verification tools after bug fixes
- Analysing which tests are the most valuable in terms of coverage achieved
- Tracking bug rates/coverage accumulation rates

 © Cadence Design Systems, Inc. All rights reserved.

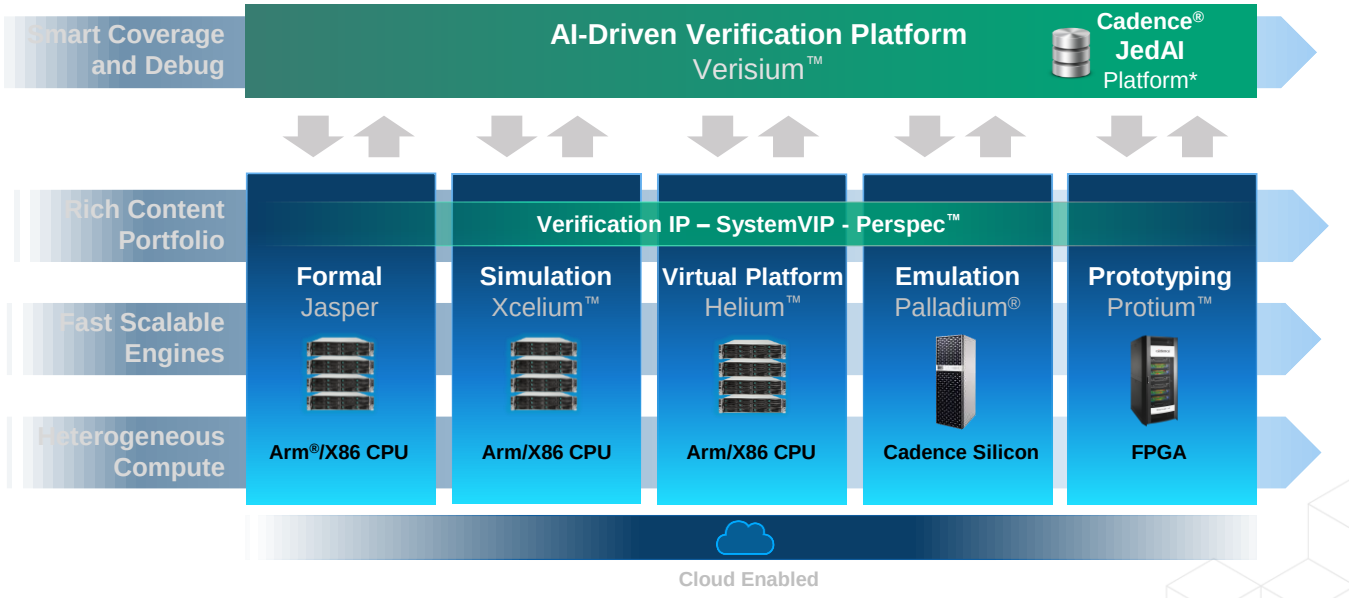
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Cadence Verification Solution



 © Cadence Design Systems, Inc. All rights reserved.

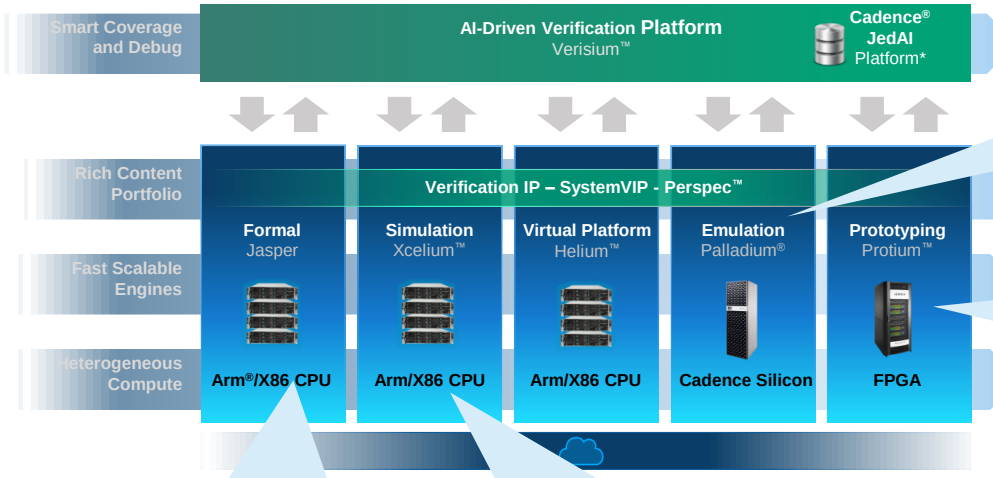
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Where Each Technique Apply



- Execute and debug RTL and software on custom processor hardware 1000's time faster than software simulator Jeda Xcelium

- Execute and debug RTL and software OS and Apps on FPGA's 10,000's time faster than software simulator Xcelium

- Mathematically proves properties
- Automatically exercises all input combinations without a testbench
- *Helps verification to "shift left"*
- *Historical limitation: scalability beyond* block/IP level

- Tests based on dynamic stimulus and checkers
- Testbench quality controls verification effectiveness
- Some directed tests early, full testbench later
- Scales well to block/IP and integration tests

463 © Cadence Design Systems, Inc. All rights reserved.

cadence

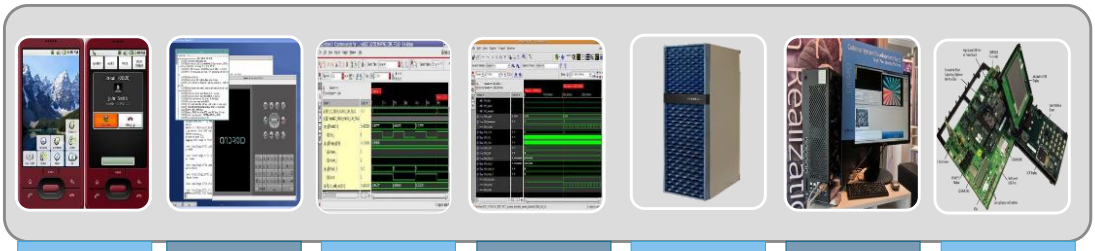
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

There Is No "One Size Fits All"

Verification and Software Platforms Need to Interoperate



SDK OS Simulation	Virtual Platform	Formal Analysis	HDL Simulation	Acceleration Emulation	FPGA Prototype	Prototyping Board
•Highest speed •Earliest in the flow •Ignore hardware	•Almost at speed •Less accurate (or slower) •Before RTL •Great to debug (but less detail) •Easy replication	•Non-scalable •Exhaustive •Early RTL •Great for IP •No SW execution	•KHz range •Accurate •Excellent HW debug •Broadly available • Mixed-abstractions •Limited SW execution	•MHz Range •RTL accurate •After RTL is available •Good to debug with full detail •Expensive to replicate	•10's of MHz •RTL accurate •After stable RTL is available •OK to debug •More expensive than software to replicate	•Real time speed •Fully accurate •Post Silicon •Difficult to debug •Sometimes hard to replicate

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Summary

- Functional Verification is about risk management not verifying everything
- We need to minimise the chances of a critical bug escaping the verification process
- There are many tools and methodologies to help us do this
- We cannot abdicate the responsibility for thorough verification to automation
- Automation only helps with manging data, predictable repetitive task and computing
- Functional Verification is still a human process vulnerable to human error
- There is no single verification methodology which worst best for every circumstance all of the time

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 13

Coverage in Formal

cadence

In this module we are going to cover Coverage in Formal verification.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Module Objectives

In this module, you will be able to

- Identify the importance of Coverage
- Define the concepts of stimuli, checker, and formal coverage
- Analyze the Cone of influence (COI) and Proof core measurement
- Describe Mutation
- Demonstrate an overview of Jasper™ Coverage app

 © Cadence Design Systems, Inc. All rights reserved.

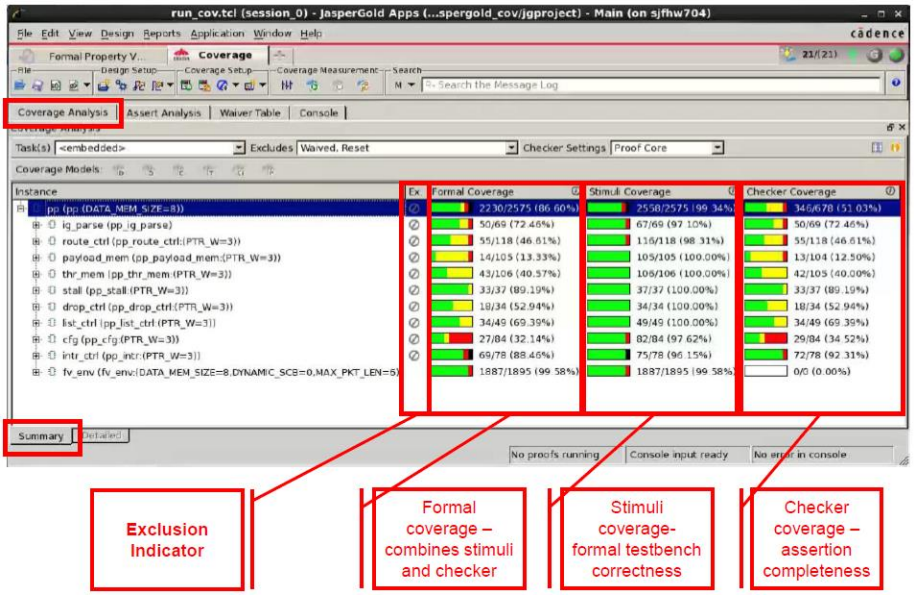
cadence

- In this Module We will learn about, why coverage is important.
- What are stimuli, checker, and formal coverage.
- What is the Cone of influence, and Proof core measurement and what is their difference?
- What is Mutation measurement?
- overview of Jasper coverage app.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Jasper Coverage App GUI



 © Cadence Design Systems, Inc. All rights reserved.

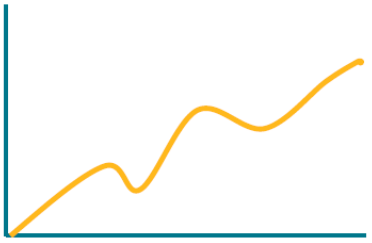
This is GUI representation of Jasper coverage app.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Importance of Coverage

- Coverage serves as a reliable metric for tracking progress and enabling sign-off
  - Often measured against a verification plan
- Provides confidence as to what is being verified
  - It highlights areas where verification efforts are still needed
- Coverage offers valuable feedback on the effectiveness of verification efforts
  - It directs attention towards areas of higher risk
- Just like in simulation, coverage is crucial in formal verification processes



 © Cadence Design Systems, Inc. All rights reserved.

cadence

Why Coverage is required?

- With the help of coverage, we can identify incompleteness in verification
- Have you written good properties that is covering all the scenarios in design ?
- Have you reached the verification goal that is set?
- This kind of question comes up after verification that can be solved by using coverage measurement
- In every verification methodology coverage has essential role, we can use formal techniques to find out the coverage

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

What Is Formal Coverage?

What formal-specific questions need to be answered?

- **What code or functionality has been "exercised" by the formal testbench?**
  - During classic formal or bug hunting
  - Covers may be reached, undetermined, unreachable
- **Is my formal checking complete?**
  - Properties may be proven, but how much of the design do they cover?
  - Complete checks, but not fully proven – have the engines achieved sufficient design coverage to detect a bug?
- **Use the Jasper Coverage App in conjunction with FPV (or other apps) to answer these questions!**

 © Cadence Design Systems, Inc. All rights reserved.

What formal-specific questions need to be answered?

During classic formal or bug hunting, what code or functionality has been exercised by the formal testbench?

Properties may be proven, but how much of the design do they cover?
Is my formal checking complete?

have the engines achieved sufficient design coverage to detect bugs?

This kind of question can be answered by Using the Jasper Coverage App in conjunction with FPV (or other apps)

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

What Is Formal Coverage? (continued)

- Code coverage models and functional coverage
  - Representative subset (approximation) of design states to verify
- Supported code coverage models – automatically generated covers
  - Branch
  - Statement
  - Expression
  - Toggle
- Functional coverage – user-defined
  - Property (SVA/PSL/Tcl/Assertion-related)
  - Covergroup
- Enabling models for coverage reporting
  - `check_cov -init -model {branch statement expression functional toggle}`
  - Equivalent: `check_cov -init -model all`
  - Default: `check_cov -init (equivalent to {branch statement expression functional})`

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Formal Coverages include code coverage and functional coverage

Code coverage is automatically generated covers; Code coverage focuses on design code, such as blocks, expressions, and signals

It supports Branch, Statement, Expression, and Toggle coverage in code coverage models.

Functional coverage is defined by the user and includes written property and covergroups; functional coverage focuses on design functions, such as data values and control sequences.

In the jasper tool coverage can be invoked by adding the check_cov –init command in the TCl

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Code Coverage Versus Functional Coverage

- Code coverage
  - **Easy to generate (automatic)**
  - **Guaranteed to be *structurally complete* –not prone to human error**
  - **Standard models capture much of the meaningful behavior of the design**
  - May not capture all meaningful design *functionality*
  - Can be noisy – large number of covers, some may not be important
- Functional coverage
  - **Possible to represent all meaningful design functionality**
  - **Implements the verification plan –what needs to be verified**
  - **Noise-free –created deliberately**
  - Requires planning, coding, and debug
  - May be incomplete due to human error
  - Coverage app can help to find some gaps

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Difference between code and functional coverage.

Code coverage is system generated, so the generated coverage is structurally complete as no human error is present. standard model captures much of the meaningful behavior of design but may not capture all meaningful design functionality.

Because it is auto-generated, it can capture many covers, some of which may not be necessary.

On the other hand, Functional coverage needs human intervention to add properties and cover points. Possible to cover all design functionality. No meaningless covers are present, as they are added deliberately to cover the functionality of the design.
Due to human error, coverage may be incomplete, and it requires planning, coding, and debugging.
To implement functional coverage, you must be familiar with the design.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Code Coverage Models: Branch Model

Branch Model

- Enumerates all branches in the code as cover items
- Constructs: if/else, case, Verilog “?:”(turnery operator)
- Optionally creates cover items for empty else

Example

```
if (!rstN)                          ├ Branch
       trans_started<= 1'b0;
else if (new_trans)                 ├ Branch
       trans_started<= 1'b1;
else                                ├ Branch
       trans_started<= 1'b0;
```

Three branches cover items -2 “then” and 1 “else

 © Cadence Design Systems, Inc. All rights reserved.

If branch is enabled in the coverage app for code coverage , then it will generate cover item on all branches, which include if-else, case, and turnery operators.

Look at the example : 3 branches are present, i.e., if, else-if, and else, so three cover items will be generated.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Code Coverage Models: Statement Model

Statement Model

- Enumerates all statements in code as cover items
- Includes assignments and conditional expressions

Example

```
if (!rstN)
       trans_started<= 1'b0;      Statement
else if (new_trans)
       trans_started<= 1'b1;      Statement
else
       trans_started<= 1'b0;      Statement
```

 © Cadence Design Systems, Inc. All rights reserved.

If a statement is enabled in the coverage app for code coverage, then it will generate cover item for all statements, which include assignments and conditional expressions

Look at the example : 3 statements are present, for each statement cover item will be generated as they are in different branches

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Code Coverage Models: Branch/Statement

Example 1

```
        if (new_trans) begin
Branch          trans_started = 1'b1;      Statement
                header = din;              Statement
        end
        else
Branch          trans_started<= 1'b0;      Statement
```

Example 2

```
        case (a)                           Statement
Branch  0: b = ZERO;                       Statement
Branch  1: b = ONE;                        Statement
Branch  2: b = TWO;                        Statement
Branch  3: b = THREE;                      Statement
        endcase
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

These are the examples of what a branch is and what a statement is. For each branch, an statements mentioned here, the tool will generate a cover item for them.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Code Coverage Models: Expression Model

Expression model decomposes logical expressions into multiple covers

- Control – checks if each input has controlled the output value of the expression

Example 1

```
assign C = A && B;
```

A	B	Result
0	1	0
1	0	0
1	1	1

Expression
Expression
Expression

Control model considers A=0 and B=0 redundant with these

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Expression model decomposes logical expressions into multiple covers
If Expression is enabled in the coverage app for code coverage, then it will generate cover item on all expressions

In this example, C has assigned the values A and B, for each combination of A and B, cover item will be generated

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Code Coverage Models: Expression Model (continued)

Example 2

```
assign A = B && (C | D);
```

decomposed into two levels →

Level	Operand1	Operand 2	Operator
1	B	(C|D)	&
2	C	D	|

Table for Level 1

B	(C|D)	Result	
0	1	0	Expression
1	0	0	Expression
1	1	1	Expression

Table for Level 2

C	D	Result	
0	1	0	Expression
1	0	0	Expression
1	1	1	Expression

477 © Cadence Design Systems, Inc. All rights reserved.

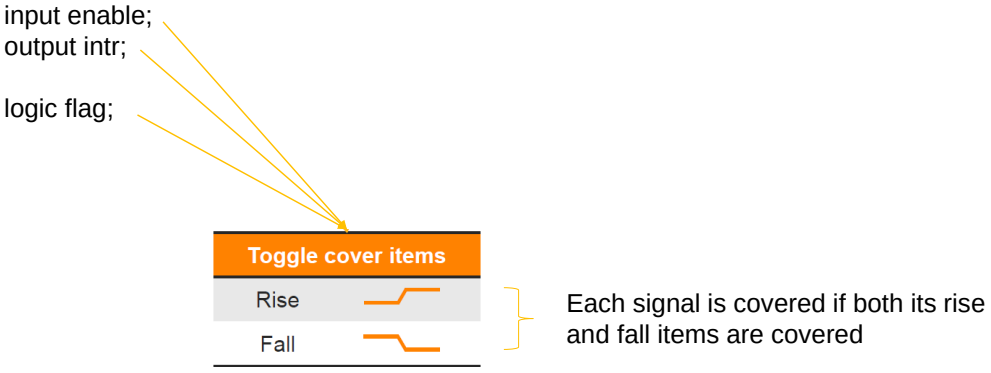
In this example, A has assigned the values B and (C or D); for each combination of B and (C or D), the cover item will be generated, and for C or D again, the cover item will be generated.
This demonstrates that for each expression, the combination will break down and the cover item will be added

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Code Coverage Models: Toggle Model

Toggle model represents the ports and internal signals changing values.



 © Cadence Design Systems, Inc. All rights reserved.

cadence

The toggle model represents the ports and internal signals changing values.
If toggle is enabled in the coverage app for code coverage, then for each rising and falling edge cover item will be generated

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Properties of Coverage Models

- SVA, PSL, Tcl cover properties – user-defined
  - Describe specific states, conditions, or sequences to be verified
  - Part of <embedded> or other tasks
- Assertion-related precondition/late_precondition/witness
  - Auto-generated by tool
  - Part of <embedded> or other tasks

Example

Cover item will be created for this property

```
A_init_size:  assert property (init_valid  |->  init_avail):
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

When coverage model property is enabled, the Tool will generate covers on property present in the design, it will also generate cover items on precondition and witness for the written property.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Coverage Models: Covergroup

Coverpoint bins – user-defined

- Convenient for creating comprehensive coverage involving variable values/transitions/crosses

```
module cov;
  logic clk;

  struct {
    // Physical fields
    logic [7:0] header;
    enum logic {TYPE0, TYPE1} packet_type;
    logic [7:0] data [15:0];
    logic [7:0] crc;
    // Control fields
    logic [7:0] data_length;
    logic has_crc;
  } my_packet;

  covergroup cg @(posedge clk);
    packet_type: coverpoint my_packet.packet_type;
    data_length: coverpoint my_packet.data_length;
    has_crc: coverpoint my_packet.has_crc;
  endgroup

  cg cg1 = new();
endmodule
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

When the coverage model property is enabled, the Tool will generate coverpoint from each covergroups. It is convenient to create comprehensive coverage involving variable values/transitions/crosses.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Measuring Coverage: Types

- **Stimuli Coverage:** What code or functionality is reachable by the formal testbench?
  - Collected during classic formal or bug hunting
  - Gives confidence that formal testbench can exercise all behavior that could yield a bug
- **Checker Coverage:** Is my formal checking complete?
  - Determines how much of the design is checked by assertions
  - Gives confidence that formal checking is complete enough to detect a bug
- **Formal Coverage:** Consolidation of Stimuli and Checker Coverage results
  - Cover item is marked “covered” if ***both its*** Stimuli and Checker results are “covered”
  - Provides a single-metric view of formal verification coverage

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Stimuli Coverage determines what code or functionality is reachable in the design under a set of constraints. Choose your proof environment to check the constrainedness of the environment.

Checker Coverage Determines how much of the design is checked by assertions.

It will give confidence that formal checking is complete enough to detect a bug.

Formal Coverage is a combination of both stimuli and checker coverage results. It provides a single-metric view of formal verification coverage.

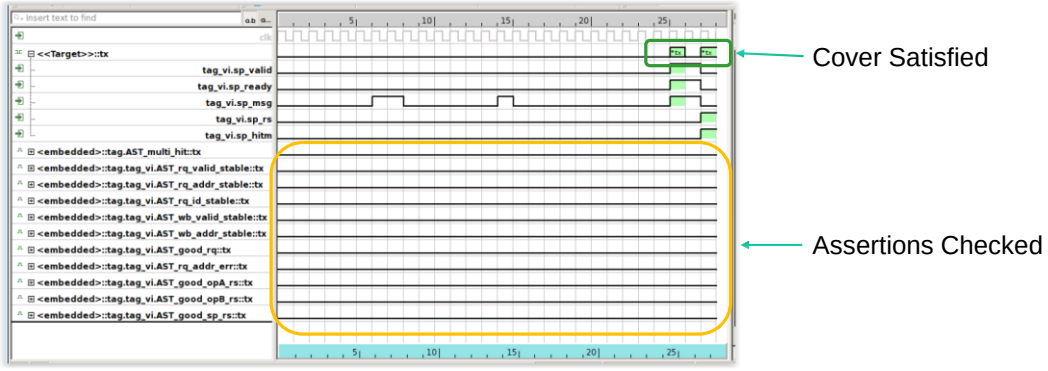
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

1. Stimuli Coverage

What code or functionality is reachable by the formal testbench?

- Formal engines attempt to find the stimuli necessary to “hit” a cover
- Result is **Covered**, **Uncovered**, **Unreachable**, or **Deadcode**
- **Covered → Engines check trace for failures (Use `evaluate_in_trace`)**



482 © Cadence Design Systems, Inc. All rights reserved.

Stimuli Coverage determines what code or functionality is reachable in the design under a set of constraints.
Formal engine attempt to find out the stimuli necessary to hit cover.

If stimuli is covered mean cover is hit

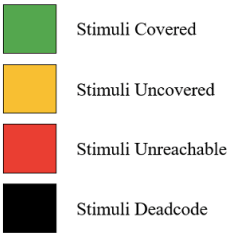
Result is either **Covered**, **Uncovered**, **Unreachable**, or **Deadcode.**

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Stimuli Coverage Status

- Reachability analysis applies to all coverage models
  - Statement, branch, expression, covergroup, property (SVA/PSL/Tcl/Assertion-related covers)
- Stimuli Coverage Status indicators for cover items
  - **Unreachable** status
    - Formal engines have determined that cover is unreachable **with** constraints (assumes) applied
  - **Deadcode** status
    - Formal engines have determined that cover is unreachable **without** constraints (assumes) applied
    - Any covers initially found **unreachable** are automatically re-run with constraints disabled to determine if they are **deadcode**



483 © Cadence Design Systems, Inc. All rights reserved.

cadence

Lest understand the coverage status of stimuli
stimuli is covered mean cover is hit.
Stimuli is uncovered, which means tool is still trying to hit the cover
In the uncovered state it is still not determined that it will get covered or not, the result can be any possibility either covered or uncovered.

Unreachable state means Formal engines have determined that cover is unreachable **with** constraints applied to design.

Deadcode state means Formal engines have determined that cover is unreachable **without** constraints applied. Any covers initially found unreachable are automatically re-run without constraints to determine if they are deadcode.

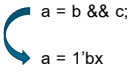
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

2. Checker Coverage

Is my formal checking complete?

- Measures completeness of checking by the assertion set
  - Provides useful metrics when the goal is to cover an entire design
- Tool determines what design code impacts assertion pass/fail status
  - Identifies gaps where checks are insufficient to detect a bug
- Uncovered portion of the design represents inconsequential logic
  - Can be black-boxed without any resulting assertion failures
  - Black-boxing represents any possible design mutation
  - Uncovered lines of code could be replaced with a stop-at or x-assignment without impact



 © Cadence Design Systems, Inc. All rights reserved.

Checker Coverage Determines how much of the design is checked by assertions.
Provides useful metrics when the goal is to cover an entire design.
Identifies gaps where checks are insufficient to detect a bug.
Uncovered portion of the design represents inconsequential logic that can be black-boxed without any resulting assertion failures.

Checker coverage will give confidence that formal checking is complete enough to detect a bug.

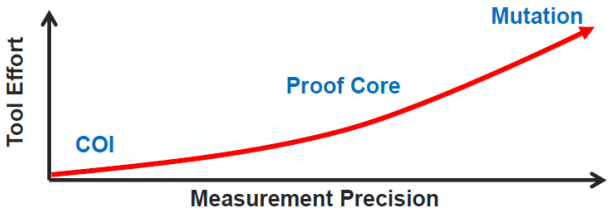
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Checker Coverage: How It's Measured?

- Measurement options:
  - **Cone Of Influence (COI)** computation – structural fan-in analysis
  - **Proof Core** –formal engine-based abstraction (blackboxing)
    - Normal Precision – flop-level (default)
    - High Precision – code-level
  - **Mutation Mode** – engine-based single point mutation

- Increasing ability to detect unchecked code/measurement precision
  - At the cost of higher tool effort/runtime



485 © Cadence Design Systems, Inc. All rights reserved.

The checker coverage includes Cone of Influence (COI), Proof Core, and Mutation use models.

Cone of Influence coverage use model, uses computation –structural fan-in analysis.

The Proof Core coverage use model complements the COI coverage use model by finding the proof core region it uses the formal engine to run

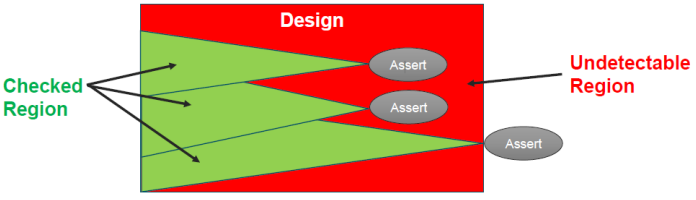
The Mutation coverage model measures checker coverage with a greater level of precision than the COI or Proof Core coverage by mutating the design. It is also formal engine based.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Cone-Of-Influence (COI) Measurement

- Design fan-in is computed starting from assertion, traversing back to inputs
- The union of COIs from all assertions is reported
- Anything outside the COI region *cannot* influence assertion status
- Anything inside the COI region *may* influence assertion status
- Fast measurement – no formal engines are run



486 © Cadence Design Systems, Inc. All rights reserved.

With the Cone of Influence coverage use model, you write a set of properties or targets to determine (before

running the proof) how good your properties are, that is, how completely they cover the

design.
Design fan-in is computed starting from assertion and traversing back to inputs.

Anything outside the COI region *cannot* influence assertion status.

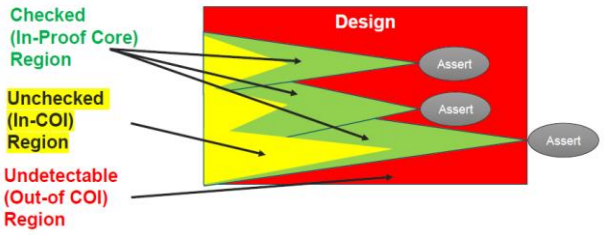
Anything inside the COI region *may* influence the assertion status.
COI measurement is Faster, no formal engines are run.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Proof Core Measurement

- Identify more unchecked code than COI measurement, with greater tool effort
- The union of proof cores from all asserts is reported
- Anything outside the Proof Core region *cannot* influence assertion status
- Anything inside the Proof Core region *may* influence the assertion status
- Slower measurement than COI – requires running formal engines



 © Cadence Design Systems, Inc. All rights reserved.

The Proof Core coverage use model complements the COI coverage use model, by finding the proof core region, which is the proof-based abstraction region for each target, and Intersecting this region with the coverage model.

it reports the union of the intersections, that is, the cover items that are in any proof core region of any target.

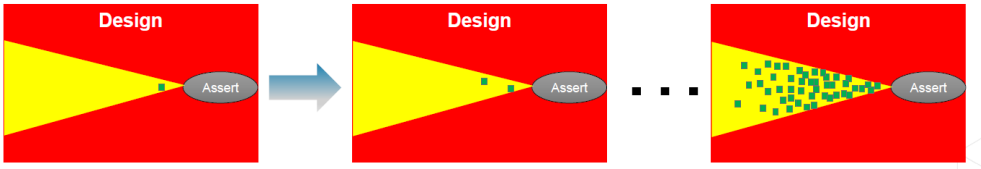
Proofcore is slower in measurement than COI and requires running formal engines.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Checker Coverage Measurement: Mutation Mode

- Tool must check each point in the design corresponding to a cover item for mutation failure
  - Each assert failure detected for a mutation point marks the cover item checked
- Provides the highest level of precision, but requires more tool effort than Proof Core
  - Number of cover items (mutations) to evaluate depends on the design
  - Single point “steps” require more iterations to process than Proof Core’s region-based “jumps”



488 © Cadence Design Systems, Inc. All rights reserved.

The Mutation coverage model measures checker coverage with a greater level of precision than the COI or Proof Core coverage by mutating the design to determine precisely which elements are being checked by the assertions.

Provides highest level of precision, but requires more tool effort than Proof Core.

Number of cover items to be evaluate depends on design

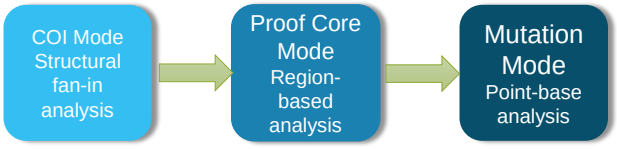
It is Single point require more iterations to process than Proof Core’s region-based “jumps.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Checker Coverage Modes: Recommended Usage

- Use a progression of modes throughout project for the highest ROI
  - Move to the stricter mode when Checker Coverage reaches 100% with waivers
  - Increase measurement precision with additional tool effort
  - Final mode dependent on design applicability
    - Size
    - Complexity
    - Criticality



 © Cadence Design Systems, Inc. All rights reserved.

The checker coverage includes Cone of Influence (COI), Proof Core, and Mutation use models. Use a progression of modes throughout project for HigherROI,

Keep in mind that higher levels of precision place higher demands on performance.

Thus, we recommend moving to a higher precision level only after exhausting the current mode. That is, begin with COI, move to Proof Core coverage with normal precision, then Proof Core coverage with high precision, and finally, Mutation.

Finally it is depends upon size, complexity and criticality of design which modes is to used .

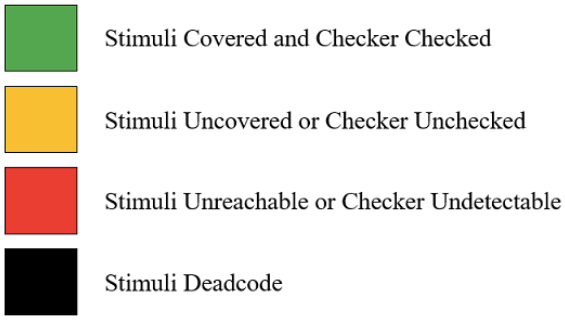
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Formal Coverage Metric

Consolidation of Stimuli Coverage and Checker Coverage results

- Cover item is marked ***green*** if both **Covered** and **Checked**
  - For functional cover items, Checker Coverage is not measured – Formal Coverage = Stimuli Coverage Status
- Formal Coverage Status color coding



 © Cadence Design Systems, Inc. All rights reserved.

cadence

Formal Coverage is a combination of both stimuli and checker coverage results. It provides a single-metric view of formal verification coverage.

This diagram shows coverage matric for formal coverage

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Summary

- Coverage will identify where verification is incomplete
- Properties may be proven, but how much of the design they cover can be determined by formal coverage
- Formal coverage can easily find dead code
  - Also, it can differentiate between dead code and unreachable
- For detailed Coverage app usage check introduction to Jasper Design Coverage (COV) Verification App on support.cadence.com

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Summaries

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 14

Course Conclusions

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Takeaways from This Course

- Formal is a completely different paradigm to simulation
- Fundamental difference between formal and simulation
  - Simulation: One guesses what **may** be a good stimulus
    - Non-exhaustive (limited by testbench creativity) and input is driven in nature
  - Formal: You state what should/should not occur (assertions) or what should occur (covers)
    - Exhaustive and Output driven
- One must take care to understand the context of the results
  - For example, looking at a PROVEN result is not enough
- Formal is not limited to exhaustively proving every feature of the DUT
  - There are many other use models, bug hunting, SoC connectivity, etc.
- When starting with formal, a systematic approach helps overcome blank page syndrome
  - 1st blackbox protocol properties, 2nd whitebox properties, 3rd end-to-end blackbox checks

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Dispensing Myths

Don’t use formal on blocks over X number of FFs

- Fact: Each assertion has a different COI and is evaluated in isolation

I need to learn SVA and formal in-depth to get any value

- Fact: Many apps require no SVA or Formal knowledge, e.g., Xprop, Superlint, Connectivity, etc.

Formal cannot verify datapaths

- Fact: Can be done, but it needs some really inventive and abstract thinking

Formal is only for expert Verification PhDs

- Fact: It’s a mainstream verification technology that many designers and verification engineers use to help verify their designs

Formal is additional work to the simulation that I already need

- Fact: Formal can save much simulation work, and the check and coverage results can be integrated with simulation check and coverage results using vManager™

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Why Do I Get Undetermined Results?

Ensure that you:

- Set your goals before you start
- Choose appropriate focus areas
- Have realistic expectations about what can be achieved

Don’t use complex SVA assertions – instead, use:

- Simple Aux code
- Simple assertions

Use multiple engines, engine orchestration, JGES suggestions, and Proofgrid

Use ABVIPs and PAs

- It would be extremely difficult and unlikely to outperform these

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Next Steps: Suggestions

Try techniques learned in this course on your own designs

Find out if you have a formal expert or group in your company

Find out who your local Jasper™ AE is and organize a site visit

Discuss what you expect to achieve using Formal and the best course of action

- Jasper Apps that you can use to solve your verification needs
- Applicability/ROI on using formal with your targeted block/design

If Formal is in a nascent stage in your company:

- Make a commitment to use Formal where applicable
- Try overcoming the initial barrier of using it
  - First time is always the hardest
- There will always be schedule pressures; resist the temptation to take the easy way out (simulation)

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 15

Next Steps

cadence

This page does not contain notes.

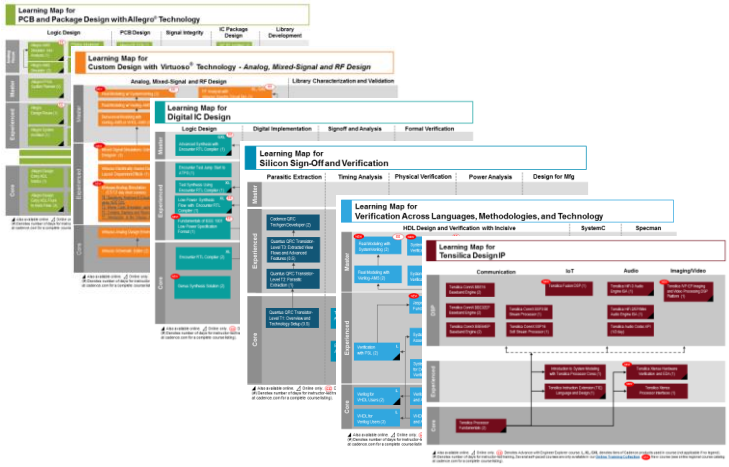
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Learning Maps

Cadence® Training Services learning maps provide a comprehensive visual overview of the learning opportunities for Cadence customers.

Click here to see all our courses in each technology area and the recommended order in which to take them.



498 © Cadence Design Systems, Inc. All rights reserved.

cadence

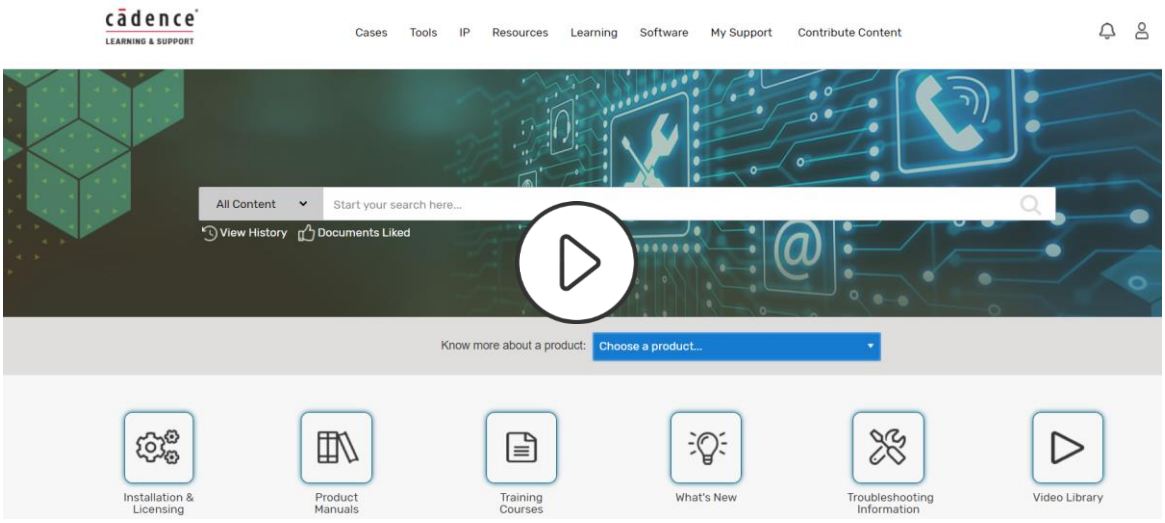
Go here to view the learning maps:

http://www.cadence.com/Training/Pages/learning_maps.aspx

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Cadence Learning and Support



Customer Support now includes over 2000 product/language/methodology videos ("Training Bytes")!

499 © Cadence Design Systems, Inc. All rights reserved.

Click the play button in the figure on this slide to view the demo of Cadence Learning and Support.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Wrap Up

- Complete Post Assessment, if provided
- Complete the Course Evaluation
- Get a Certificate of Course Completion and a Digital Badge

Thank you!

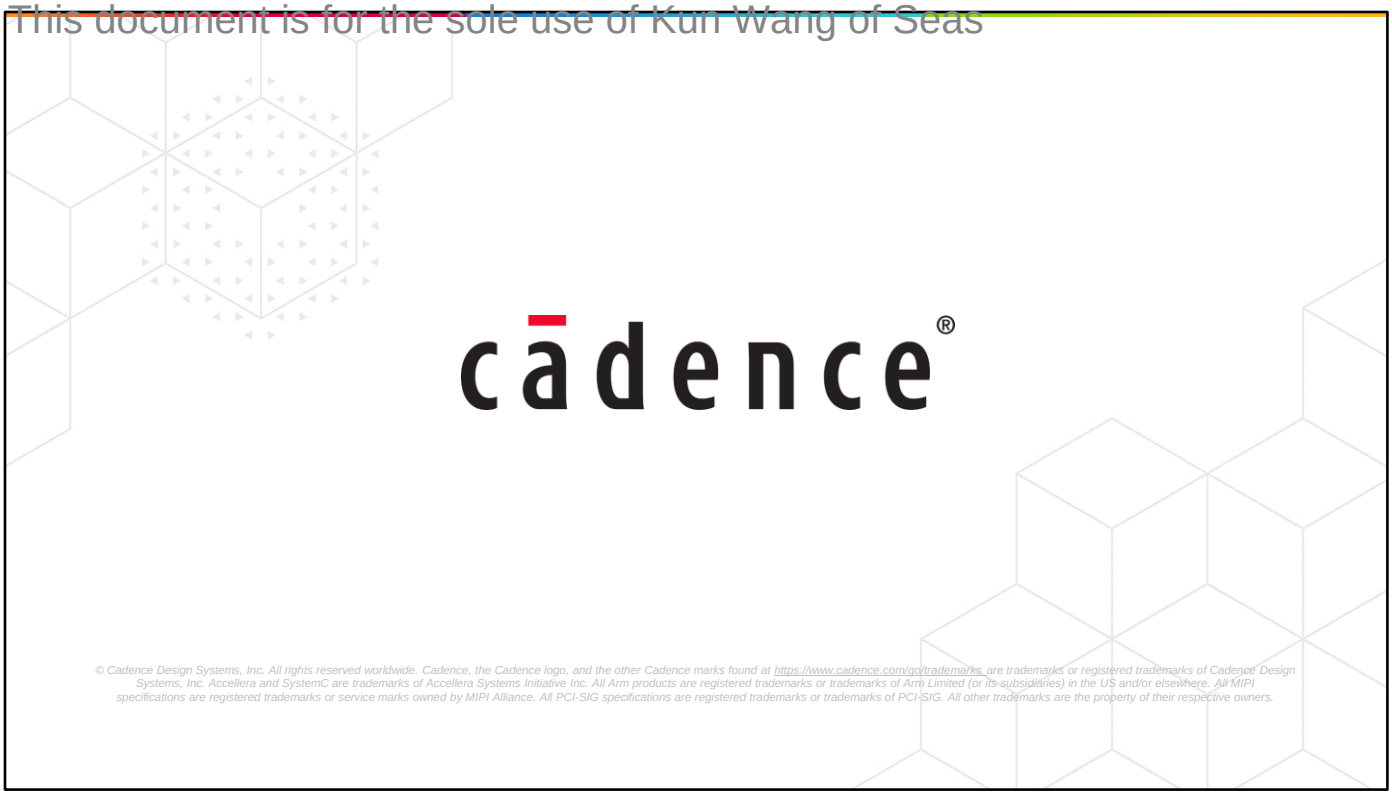
 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.